



Elementos de Automação Industrial



CAPÍTULO 01

Nos dias de hoje, praticamente para todo lado que olhamos vemos exemplos de sistemas que foram automatizados ou que precisam de automação para ganhar em padronização, organização e melhoria no desempenho de suas funções.

Para exemplificar, podemos pegar papel e caneta e fazer o trajeto normal do nosso cotidiano, com um pouco mais de atenção. Então vamos considerar alguns cenários.

Hipótese: Sou estudante. Assim, acordo bem cedo, com a ajuda de um despertador, e levanto. Há quem tome um banho para despertar, e aqueles que não dispensam uma xícara de café; eu sou desses; e para ajudar, tenho uma cafeteira que programo para fazer meu café às 6h da manhã. Ao sair, ando apressado para não perder o ônibus. Subo, utilizo o cartão magnético para pagar a passagem, e logo depois chego à escola/faculdade. Controlo a estação em que devo descer pela indicação luminosa do ônibus. Lá na escola/faculdade, toca o sinal, aliás, ele sempre é pontual. Almoço com os colegas, em um shopping próximo à escola/faculdade. À tarde, sempre ajudo meu pai, em sua loja. Como trabalhamos com vendas, temos um sistema de senhas que nos ajuda e muito na organização e satisfação dos clientes. Ao final do dia, retorno com meu pai para casa. Chegando lá, abrimos o portão eletrônico, guardamos o carro, ligamos o alarme e vamos nos preparar para descansar para um novo dia.

Nesse cenário, podemos identificar diversos processos de automação, e vou ajudá-lo a enumerá-los:

1. No café da manhã, tenho uma cafeteira, com um sistema inteligente que inicia um processo (passar o café) em uma hora programada.
2. Ao subir no ônibus, pago a passagem com o cartão magnético, ou seja, há um sistema inteligente, que é capaz de identificar o meu cartão, receber uma informação de créditos no cartão e subtrair a passagem que estou utilizando naquele momento. Esse sistema ainda se comunica com uma central, que faz a gestão de todo o processo.
3. Ainda no ônibus, há um sistema que informa o próximo ponto. Esse sistema precisa ter a informação de todos os pontos e controlar quando os ultrapassa.
4. Na escola/faculdade, há indicação de sinal, ou

seja, há um sistema de controle de hora, com ações programadas (sinais).

5. No almoço, vou com os colegas ao shopping. Lá, existem sistemas de controle de atendimento, que nos permitem fazer o pedido, esperá-lo ficar pronto e sermos chamados por uma senha.
6. Na loja do meu pai, temos um sistema de atendimento semelhante, onde o cliente chega, solicita uma senha em um botão na entrada, imprime-a e aguarda até ser chamado. Paralelamente, há um sistema paralelo no qual é identificado o último cliente chamado, e que podemos chamar o próximo com nosso controle.
7. Ao final do dia, temos a oportunidade de saber quantos clientes realmente atendemos, e inclusive se conseguimos atender todos ou se estamos deixando-os esperar demais, afinal a satisfação do cliente é nosso melhor termômetro.
8. Ao chegar em casa, abrimos o portão eletrônico. Esse sistema conta com um controle remoto que envia um sinal de rádio a um controlador de motor responsável pela movimentação do motor.
9. Ao entrarmos na garagem, ligamos o alarme do automóvel. O alarme nada mais é do que um sistema de monitoramento de sensores que atua em sirenes caso o campo dos sensores seja violado.

Esse cenário representa justamente inúmeros sistemas que precisam monitorar sensores para poder atuar, segundo as regras de um controlador.

Outros cenários serão brevemente exemplificados a seguir:

- simples processos de automação, em que um controlador monitora a presença de peças e atua no movimento de esteiras;
- processos de automação residencial, que, além de monitorar os diferentes sensores em portas, janelas ou de presença, têm uma função de alarme;
- ainda em automação residencial, também podem interagir em funções mais avançadas,



como o controle de câmeras de vídeo, controles a distância de luzes e ar-condicionado, controle de tetos móveis através de sensores de chuva e de luminosidade, além do próprio controle multimídia interativo por controle remoto ou smartphones;

- os próprios sistemas de alarme, tanto residencial quanto veicular, são bons exemplos de cenários de automação;
- jogos eletrônicos infantis como Simon (Genius), ou Tetris também podem ser bons exemplos, em que os comandos por botões interagem com o controlador do jogo;
- podemos citar os carrinhos de controle remoto, que respondem aos comandos do usuário a distância, ou ainda carrinhos mais inteligentes, com sensores de obstáculos. Desses últimos, já foram desenvolvidos carrinhos de limpeza (aspiradores de pó) autônomos;
- por fim, podemos exemplificar os robôs, que estão presentes nos brinquedos, mas estão também nas diferentes indústrias.

Enfim, sistemas de automação estão por toda nossa volta, e com um pouco de atenção podemos identificá-los ou até mesmo imaginá-los com certa facilidade.

O mais importante, nessa identificação, é percebermos que sistemas automatizados são compostos basicamente por três elementos: sensores, controlador e atuadores.

Os sensores, grosso modo, são dispositivos responsáveis por “sentir” grandezas físicas e convertê-las em um sinal eletrônico proporcional à grandeza monitorada. Os controladores, por sua vez, são os elementos responsáveis por receber o sinal eletrônico já adequado por um circuito eletrônico intermediário, de condicionamento de sinais, e enquadrá-lo sob uma lógica de controle do sistema em questão. Em função dessa lógica e dos níveis dos sinais de entrada, o sistema interage com atuadores que interferem no sistema, seja para informar, seja para efetivamente interagir com o sistema.

A essa característica de monitorar ou sentir grandezas físicas e interagir com o meio damos o nome instrumentação.

O processo de instrumentação lembra, em um primeiro momento, o fato de que empresas precisam de instrumentos específicos para monitorar uma grandeza

física e, em função da alteração dela, tomar algumas decisões. Para exemplificar isso, podemos citar uma caldeira, cujo nível de temperatura o processo de instrumentação monitora, através de termômetros industriais. Um sistema de controle recebe os sinais desse termômetro, e, abaixo de certo valor, aumenta o fluxo de gás da caldeira, ao passo que acima de outro determinado valor baixa o fluxo de gás da caldeira, buscando manter uma temperatura desejada constante.

Entretanto, o processo de instrumentação ocorre também fora de indústrias, não utilizando instrumentos específicos de monitoramento de grandezas físicas, mas sim dispositivos específicos para tal, chamados sensores. Para exemplificar, podemos ter um controle semelhante ao industrial citado, em um simples sistema de temperatura de uma estufa de flores caseira, onde um ar-condicionado normal faz o controle em função de um valor de temperatura programado. Ainda, podemos também exemplificar um sistema em arcado automotivo, de controle de temperatura, o qual, ao ter uma temperatura interior programada, controla a intensidade de funcionamento do compressor automotivo do ar-condicionado.

Nas indústrias, a maior parte do processo de instrumentação tem como controladores dispositivos específicos conhecidos como Controladores Lógicos Programáveis (CLP), que possuem requisitos e robustez específicos para essas aplicações.

Os CLPs são sistemas digitais que permitem que rotinas lógicas sejam programadas, monitorando botões e sensores como dispositivos de entrada e controlando motores, chaves, sinais luminosos, entre outros, como dispositivos de saída ou atuadores. Basicamente, são sistemas controladores eletrônicos robustos, com requisitos de temperatura, sujeira, oscilações elétricas e interferência eletromagnética diferenciada para prover confiança em suas aplicações na indústria.

Da mesma forma, dependendo do ambiente, são necessários controladores específicos, em função dos requisitos da aplicação e do ambiente de utilização. É o caso dos controladores automotivos, presentes nas centrais automotivas que são definidas por requisitos específicos de operação.

Percebe-se que são os requisitos da aplicação que definirão o tipo de controlador a ser utilizado.

Os sistemas microcontrolados de baixo custo, na maioria dos casos, são uma alternativa para o desenvolvimento de controladores mais simples, como por exemplo controles



de acesso, controles de senhas e atendimento ao público, sistemas de alarmes, controles de processos simples em pequenas indústrias, entre outros.

Um dos pontos que diferenciam um CLP de um sistema microcontrolado é justamente a apresentação final.

Um CLP é um dispositivo fechado, com as interfaces de entrada (sensores) e saída (atuadores) oferecidas por conexões simples (bornes com parafusos para fixação). Independentemente da aplicação, basta que sejam identificadas as entradas e saídas, conectadas e programadas para que o CLP esteja pronto para operar.

Já um sistema microcontrolado baseia-se em um sistema eletrônico que precisa de eletrônica a sua volta, para condicionamento de sinais e adaptações específicas. Em outras palavras, na maioria das vezes, baseia-se em um projeto específico para cada aplicação, a qual depende também de um projeto eletrônico específico para que as entradas e saídas sejam conectadas e o microcontrolador, programado, possa operar. Em bora existam projetos de microcontroladores operando como simples CLPs, vale a pena ressaltar que já são resultado de projetos específicos, cujo objetivo é ser uma simples CLP, o que resulta em um sistema final não realmente otimizado para a aplicação final.

Assim sendo, sabe-se que existem diferentes microcontroladores no mercado, e muitos requisitos que diferenciam famílias e aplicações. Entretanto, um dos pontos que fazem com que o projeto final de um sistema de automação (ou de instrumentação) seja mais demorado é justamente o projeto eletrônico intermediário aos sensores, atuadores e microcontrolador.

Visando reduzir justamente o tempo de projeto de sistemas microcontrolados, tem-se buscado o desenvolvimento de plataformas padronizadas, que permitam a utilização simples de dispositivos de entrada e saída, através de simples conexões. Como exemplo dessas padronizações podemos citar as placas de inicialização da Texas Instruments (MP430 e Solaris) e, amplamente mais difundida, a plataforma ARDUINO.

Arduino, portanto, é uma plataforma open-source de prototipagem eletrônica baseada em flexibilidade, na qual o hardware e software são fáceis de serem usados e adaptados aos mais diferentes cenários e aplicações.

Devido à sua simplicidade física de conexões e de expansão dos circuitos eletrônicos, ganhou um caráter hobbista, pois possibilita que qualquer pessoa interessada em criar objetos ou ambientes interativos possa desenvolver seus

protótipos de forma rápida.

Criado em 2005 na Itália, compõe um protótipo de hardware e software, de fonte aberta (open source), que significa que qualquer pessoa pode criar projetos sem precisar pagar direitos autorais aos criadores, sendo possível obter informações e detalhes gratuitamente na internet ou em livros com pessoas que fornecem projetos criados.

Com base nisso, esta obra tem como objetivo apresentar os diferentes conceitos envolvidos com processos de automação e apresentar diferentes projetos de instrumentação aplicada utilizando a plataforma Arduino como controlador.

Além de abordar o método de funcionamento dos diferentes sensores, será tratado o condicionamento de sinais para adaptar o funcionamento dos sensores ao controlador Arduino. Posteriormente, serão tratadas algumas técnicas de condicionamento do sinal de saída para o controle dos diferentes atuadores, e serão apresentados diversos exemplos de aplicações.

Pretende-se que esta obra seja uma referência para estudantes das diferentes áreas, nas quais suas aplicações e experimentos precisam de instrumentação, e também para profissionais em suas aplicações nos diferentes ambientes, residenciais, comerciais ou industriais.

Neste capítulo introduziremos a teoria referente aos diferentes sensores e também atuadores, de modo simplificado e ao mesmo tempo didático, com o objetivo de dar suporte às aplicações de instrumentação que serão à frente abordadas.

Primeiramente, é preciso definir o que é um sensor.

Sensor é um dispositivo capaz de sentir a variação de uma grandeza física qual quer, entretanto a capacidade de correlacionar essa variação com alguma outra grandeza o caracteriza como um transdutor.

Um termômetro de mercúrio caseiro é um exemplo clássico de sensor transdutor. Nesse caso, o mercúrio sofre uma dependência direta de expansão volumétrica (pressão) em função da temperatura. Entretanto, para que ele seja útil, primeiramente é preciso ter uma câmara de expansão graduada, de modo a que percebamos a variação volumétrica em função de uma escala. Entretanto, o monitoramento de temperatura ainda se faz em função da averiguação visual da escala.



Outros transdutores podem tornar-se mais interessantes se converterem o sinal de variação (de entrada) em um sinal elétrico (de saída) correlacionado com a grandeza física. É o caso de outros sensores de temperatura que têm sua característica de resistividade atrelada à temperatura.

Entretanto, na maioria dos casos, circuitos eletrônicos são necessários para adequar os níveis e características dos sinais elétricos em sinais que possam ser melhores manipuladores por sistemas de controle. Esses circuitos já estão intrínsecos na maioria dos dispositivos “sensores” comercialmente disponíveis, que entregam sinais em níveis adequados para que CLPs e microcontroladores os analisem. Para os casos em que o sinal elétrico ainda é bruto, esses circuitos de condicionamento de sinais são necessários.

Os termos sensores e transdutores ainda são corriqueiramente confundidos, principalmente na esfera comercial, não trazendo prejuízo às nossas considerações à frente, e justamente por isso utilizaremos apenas o termo sensor.

CLASSIFICAÇÃO DE SENSORES

Existem diferentes formas de classificação de sensores:

- **Quanto ao campo de aplicação:** sensores para biomedicina, meteorologia, consumo, automação etc.
- **Quanto à aplicação, função que realizam ou que medem:** sensores de pressão, aceleração, campo magnético, temperatura, capacidade térmica etc.
- **Quanto ao princípio físico de funcionamento:** sensores de transdução resistiva, transdução capacitiva, transdução indutiva, transdução piezoelétrica, transdução piezoresistiva, transdução fotovoltaica, transdução termoelétrica e
- **Quanto à forma de energia do sinal que convertem:** sensores de energia mecânica, magnética, radiante, térmica e elétrica (não há conversão da forma energia do sinal).

Desses, abordaremos efetivamente os principais conceitos

físicos utilizados sensores, e posteriormente suas aplicações, buscando abordar as principais grandezas que possam precisar ser analisadas. Assim sendo, utilizaremos a seguinte subclassificação organizacional:

- **Quanto à tecnologia:** sensores indutivos, sensores capacitivos, sensores fotoelétricos, sensores resistivos e extensômetros, sensores piezoelétricos, sensores magnéticos e sensores ultrassônicos.
- **Quanto à aplicação:** sensores de posição/deslocamento, sensores de nível, sensores térmicos, sensores de umidade, sensores de pressão, sensores de gás, sensores de tensão elétrica, sensores de corrente elétrica, sensor de luz e sensores de som.

QUANTO À TECNOLOGIA

Quanto à tecnologia, vamos tratar dos sensores em função das principais grandezas mensuradas, independentemente da aplicação.

SENSORES INDUTIVOS

Sensores indutivos são aqueles que operam segundo o princípio da alteração das características de campo magnético devido à presença de algum elemento metálico. Basicamente, são bobinas (conjunto de n espiras) ou indutores pelos quais circula uma corrente variável no tempo, a qual produz um campo magnético também variável no tempo. Esse indutor pode ser enrolado sobre ar, ou ter como núcleo um material que concentre as linhas de campo magnético em seu interior devido à permeabilidade magnética do material (μ), alterando sua autoindutância (L). Associado a esse conceito, o efeito que se opõe ao campo magnético é chamado de relutância magnética (R).

A Figura 1.1 representa uma bobina de n espiras, para a qual podemos relacionar autoindutância L com a relutância R por (2.1). Considerando A a área da seção transversal da bobina, e comprimento l , a relutância pode ser escrita por (2.2), e, consequentemente, a autoindutância por (2.3), em que μ_0 é de $4\pi \cdot 10^{-7}$ H/m.



$$L = \frac{N^2}{R} \quad (2.1)$$

$$R = \frac{1}{\mu} \frac{\ell}{A} \quad (2.2)$$

$$L = \frac{\mu_0 N^2 A}{\ell} \quad (2.3)$$

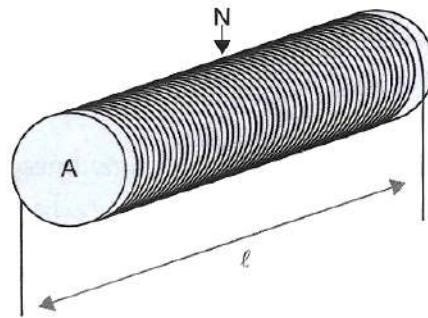


Figura 1.1 - Bobina de n espiras, comprimento ℓ e área de seção transversal.

Dessa forma, analisa-se que a autoindutância depende da permeabilidade magnética do material (μ) que influencie o campo magnético e das características geométricas da bobina (área da seção transversal, número de espiras e comprimento).

Os sensores indutivos podem ser baseados em uma única bobina, ou entre duas ou três, utilizando a lei de indução de Faraday como princípio fundamental de detecção. Dessa forma, o campo magnético produzido por uma bobina pode induzir uma corrente elétrica em outra bobina, através da indutância mútua M existente entre elas, com maior ou menor intensidade dependendo

da distância e da presença ou não de elementos com alta permissividade magnética. Portanto, a indutância mútua dependerá das formas geométricas das bobinas envolvidas, da distância entre elas e dos meios em que estão inseridas, além dos materiais que tem como núcleo.

Para um caso específico em que os dois solenoides são enrolados um sobre o outro, como representado pela Figura 1.2, considerando a mesma seção transversal e comprimento do núcleo, podemos equacionar a indutância mútua em função do número de espiras por (1.4).

$$M = \frac{\mu_0 N_1 N_2 A}{\ell} \quad (2.4)$$

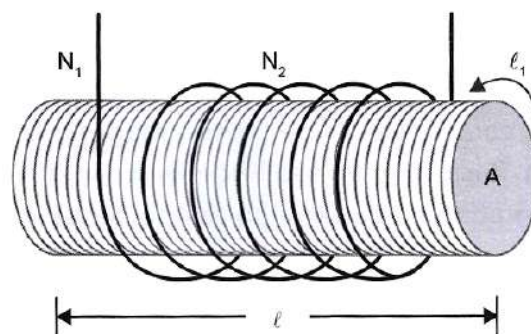


Figura 1.1 - Dois solenoides sobrepostos sobre o mesmo núcleo, com número de espiras diferentes.

Dessa forma, podemos utilizar a alteração da autoindutância através do deslocamento do núcleo dentro da bobina, ou, considerando duas ou mais bobinas, utilizar a indutância mútua para, através do deslocamento do núcleo comum, como sensores indutivos.

Para utilização desse tipo de sensor, aplica-se uma tensão elétrica alternada de frequência tipicamente de poucos kHz (para minimizar as perdas magnéticas e correntes de Foucault).

Basicamente, podemos classificar em dois grupos:

sensores indutivos de núcleo móvel (que modifica m 1-l e que provocam uma variação de M) e os sensores de entre ferro variável (I variável que resultam em uma alteração na relutância magnética e que provocam uma variação de L).

Dentre os sensores indutivos de núcleo móvel, aquele que mais se destaca pela precisão e linearidade é o Transformador Diferencial Linear Variável (LVDT).

O LVDT é um tipo especial de Sensoriamento indutivo, clássico na medida de deslocamentos, com alta linearidade,



alta sensibilidade e repetibilidade. São formados por um enrolamento primário excitado por uma tensão alternada e dois secundários idênticos, conectados em série, mas com condutores enrolados em sentidos opostos. No interior desses enrolamentos, um núcleo móvel desloca-se livremente, como sugerido pela Figura 1.3a.

Verificamos que quando o núcleo está deslocado para a esquerda ele induz maior tensão no enrolamento

secundário 1 (V_{s1}) e menor no enrolamento secundário 2 (V_{s2}), como sugerido pela Figura 1.3b. Como a saída é uma subtração das ondas induzidas ($V_{saída} = V_{s2} - V_{s1}$), ao deslocarmos o núcleo para o lado direito, o secundário 2 (V_{s2}) terá maior indução do que o secundário 1 (V_{s1}) (Figura 1.3d). Por conclusão, quando o núcleo estiver centralizado, ambas as induções terão mesmas amplitude, e, por consequência, a tensão de saída será nula (Figura 1.3c).

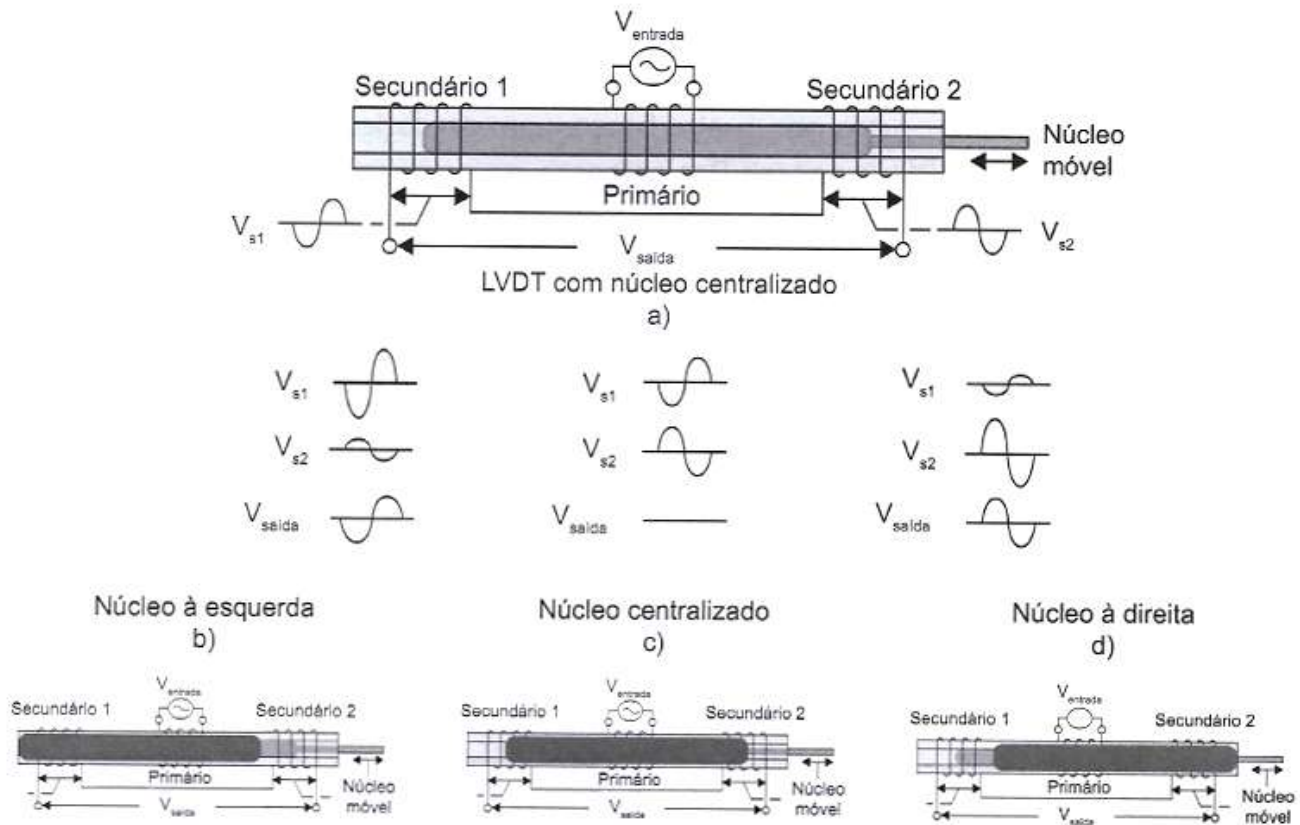


Figura 1.3 - a) Representação de um LVDT e seus enrolamentos; b) formas de onda para núcleo deslocado para a esquerda; c) formas de onda para núcleo centralizado; d) formas de onda para núcleo deslocado para a direita.

Dentro de uma certa faixa de operação, os LVDT possuem comportamento linear, conforme é ilustrado pela Figura 1.4. O primeiro gráfico dessa figura apresenta a amplitude de pico (magnitude) do sinal de saída do sensor. Já o segundo gráfico apresenta a fase desse sinal, apontando

que, para um lado, tem-se o aumento da amplitude, e para o outro, simetricamente, temos o aumento, mas com fase oposta. Já no terceiro gráfico, combinando os dois primeiros, temos uma região positiva e negativa, obtida através do circuito de condicionamento do sinal.

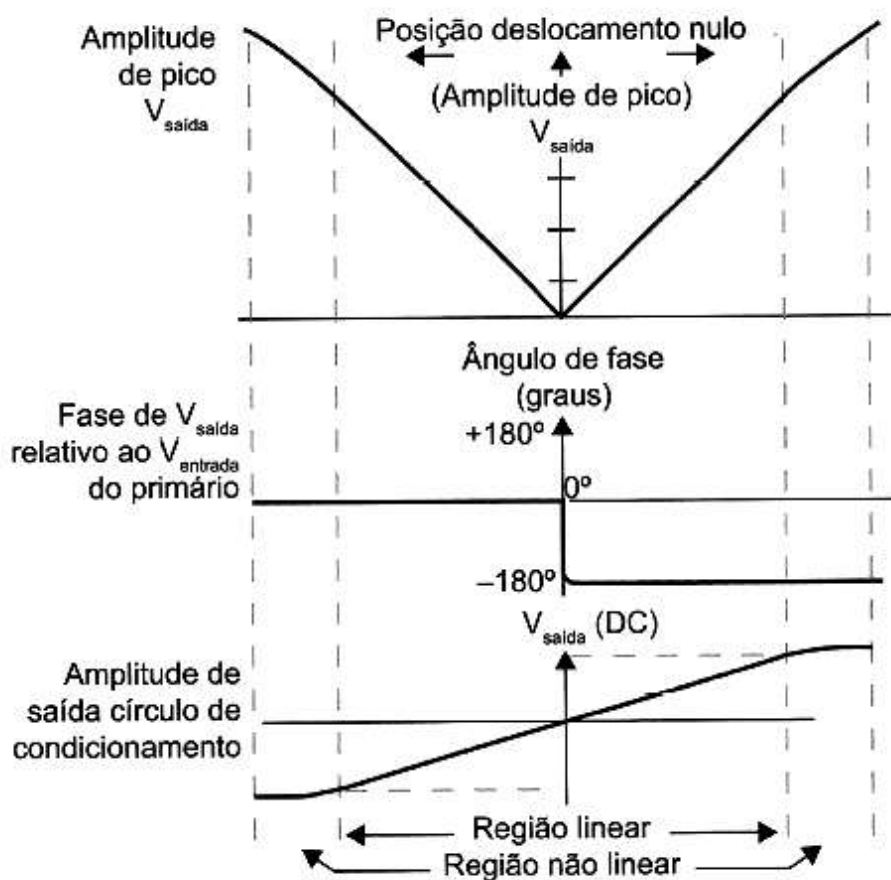


Figura 1.4 - Sinais de magnitude e ângulo de fase, na região de operação do sensor (dentro da região linear).

Os sensores indutivos industriais comercialmente disponíveis são normalmente utilizados para a identificação de presença de materiais metálicos, ferrosos ou não ferrosos. Nesse caso, são compostos por quatro

blocos: um oscilador, um demodulador, um sistema de disparo e um bloco sensor. Basicamente, na presença de um alvo metálico o oscilador para de oscilar, conforme sugerido pela Figura 1.5.

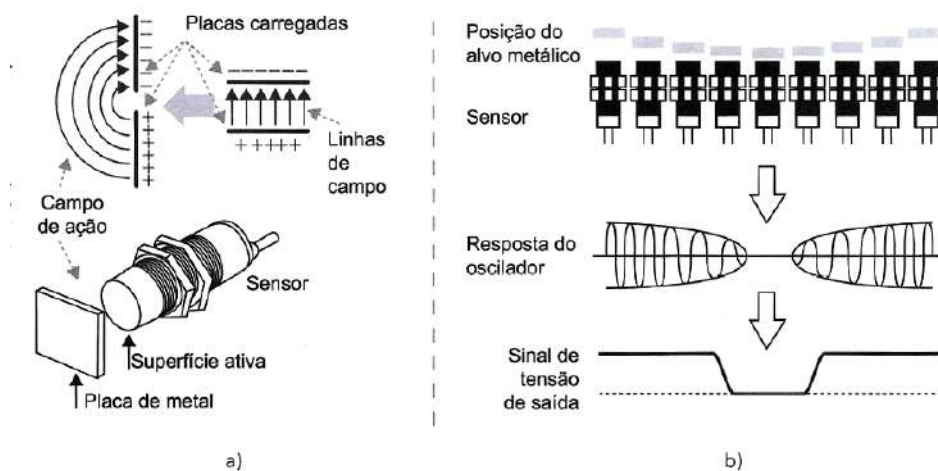


Figura 1.5 - a) Representação de um sensor indutivo comercial e as linhas de campo entre as placas; b) sinal de saída e oscilador, em função da aproximação de um alvo metálico.



Ainda sobre sensores indutivos comerciais, algumas definições comumente encontradas nas folhas descritivas (datasheets) são:

- Distância sensora real (S_r)

É o valor influenciado pela industrialização, especificado em temperatura ambiente (20 °C) e tensão nominal, com desvio de 10%.

$$0,9 S_n \leq S_r \leq 1,1 S_n$$

- Distância sensora efetiva (S_u)

É o valor influenciado pela temperatura de operação. Possui um desvio máximo de 10% sobre a distância sensora real.

$$0,81 S_n \leq S_u \leq 1,21 S_n$$

- Distância operacional (S_a)

É a distância em que seguramente se pode operar o sensor, considerando todas as variações de industrialização, temperatura e tensão de alimentação.

$$0 \leq S_a \leq 0,81 S_n$$

- Fator de redução ou correção

É um fator de redução usado para determinar o alcance quando se quer detectar outros materiais metálicos diferentes do aço doce padrão. Caso o material do objeto alvo seja de um outro material metálico, deve-se multiplicar a distância sensora informada por um fator de redução (segundo a Tabela 1.1), para se determinar o alcance específico para aquele alvo.

Tabela 1.1 - (Alcance Específico) = (Fator de Correção) x (Alcance Nominal)

Material do Objeto	Fator de Redução
Aço Doce	1,00
Aço Inoxidável	0,85
Latão	0,50
Alumínio	0,45
Cobre	0,40

$$(\text{Alcance Específico}) = (\text{Fator de Correção}) \times (\text{Alcance Nominal})$$

EXEMPLO:

Um sensor indutivo possui distância de detecção nominal de 8 mm. Qual seria o alcance específico para um alvo de cobre com as mesmas dimensões que um alvo- padrão?

RESOLUÇÃO

$$A_{\text{cobre}} = \text{Fator}_{\text{cobre}} \times A_{\text{nominal}}$$

$$A_{\text{cobre}} = 0,40 \times 8 \text{ mm} = 3,2 \text{ mm}$$

CONCLUSÃO

Se usarmos um alvo de cobre, ele somente será detectado a 3,2 mm de distância do sensor.

SENSORES CAPACITIVOS

Sensores capacitivos baseiam-se no princípio do armazenamento de energia em tre duas placas paralelas quando estas estão submetidas a uma diferença de potencial elétrico. A capacidade de armazenamento de energia damos o nome capacitância. De modo genérico, a capacitância depende da área comum das superfícies metálicas, da distância entre elas e do elemento dielétrico que as separa. Assim, a variação de qual quer um desses parâmetros que aconteça em função de uma grandeza elétrica pode resultar em um transdutor. A Figura 1.6 representa um capacitor variável básico de placas paralelas, com a indicação dos três parâmetros de variação.



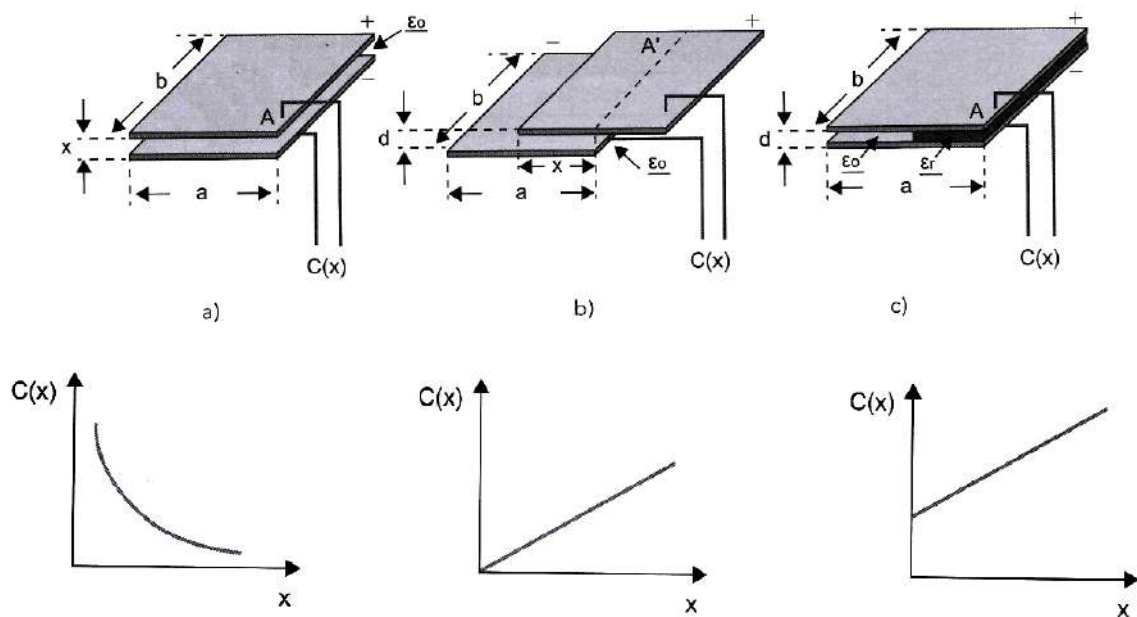


Figura 1.6 - Ilustração de um sensor capacitivo de placas paralelas, em que: a) indica uma possível variação da distância entre as placas; b) indica uma possível variação da superfície comum; c) indica a a alteração do elemento dielétrico.

$$C(x) = \frac{\epsilon_0 A}{x} \quad (2.5)$$

$$C(x) = \frac{\epsilon_0 (b \cdot x)}{d} \quad (2.6)$$

$$C(x) = \frac{\epsilon_0 A}{d} \left[1 + \frac{x}{a} \cdot (\epsilon_r - 1) \right] \quad (2.7)$$

Outra forma clássica de sensor capacitivo é a de cilindros concêntricos. A Figura 1.7 ilustra um capacitor desse formato, cuja aplicação principal como sensor é a identificação do tipo de dielétrico, seja pelo preenchimento por um dielétrico qualquer e sua identificação, seja pelo preenchimento parcial de um dielétrico, e a identificação da proporção de ocupação.

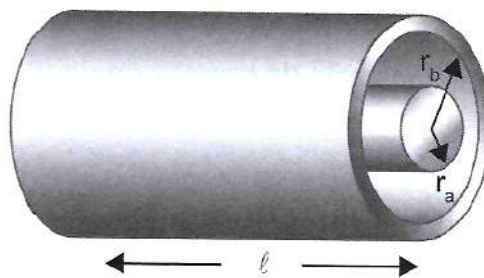


Figura 1.7 - Sensor capacitivo cilíndrico.

$$C = 2\pi\epsilon_0\epsilon_r \frac{\ell}{\ln(r_b/r_a)} \quad (2.8)$$

O valor da capacitância depende diretamente do valor da

constante dielétrica do material, e, nesse caso, quanto maior essa constante, maior será a capacitância. A Tabela 1.2 apresenta a constante dielétrica relativa de diversos materiais (ϵ_r).



Tabela 1.2 - Constante dielétrica relativa (ϵ_r) de diversos materiais

Material	ϵ_r	Material	ϵ_r	Material	ϵ_r
Acetona	19,5	Cimento em pó	4	Papel	1,6-2,6
Água (a 27 °C)	81	Etanol	24	Polietileno	2,3
Amônia	15-25	Etilenoglicol	38,7	Porcelana	4,4-7
Anilina	6,9	Gasolina	2,2	Sal	6
Ar	1,0002	Glicerina	47	Soluções aquosas	50-80
Baquelite	3,6	Mica	5,7-6,7	Teflon	2
Borracha	2,5-3,5	Náilon	4-5	Vaselina	2,2-2,9
Cereal	3-5	Óleo de soja	2,9-3,5	Vidro	3,7-10

A Figura 1.8 apresenta outras duas técnicas de Sensoriamento indutivo, utilizadas para detecção de toque. Na primeira, uma película, ao ser pressionada, altera a separação das placas metálicas, alterando a

capacitância; e na segunda, o dedo humano tem sua própria capacitância, que, em contato com outra (de um circuito sensor), se associa, alterando a capacitância local e identificando a presença do dedo.



Figura 1.8 Duas formas de detecção de pressão de dedo, por técnicas capacitivas.

Os sensores capacitivos industriais comerciais são esteticamente muito parecidos com os sensores indutivos, como apresentados na Figura 1.9a. São utilizados para a detecção de alvos metálicos ou não metálicos, e são compostos basicamente por quatro blocos: eletrodos sensores, um circuito de oscilação, circuito de condicionamento de sinais e disparo e um estágio de saída, de sinalização (Figura 2.9b).

Podem contar com um elemento de ajuste de sensibilidade (potenciômetro controlado por um parafuso externo) que

possibilita a calibração para um determinado alvo.

A Figura 1.9c mostra uma representação de identificação de um alvo, em função de sua aproximação. Nota-se que, ao identificar a presença de um anteparo, o circuito oscilador entra em ressonância e um circuito de disparo sinaliza esse evento. Vale a pena salientar que esse é o comportamento contrário dos sensores indutivos: quando estes encontram um anteparo metálico, o oscilador para de operar.

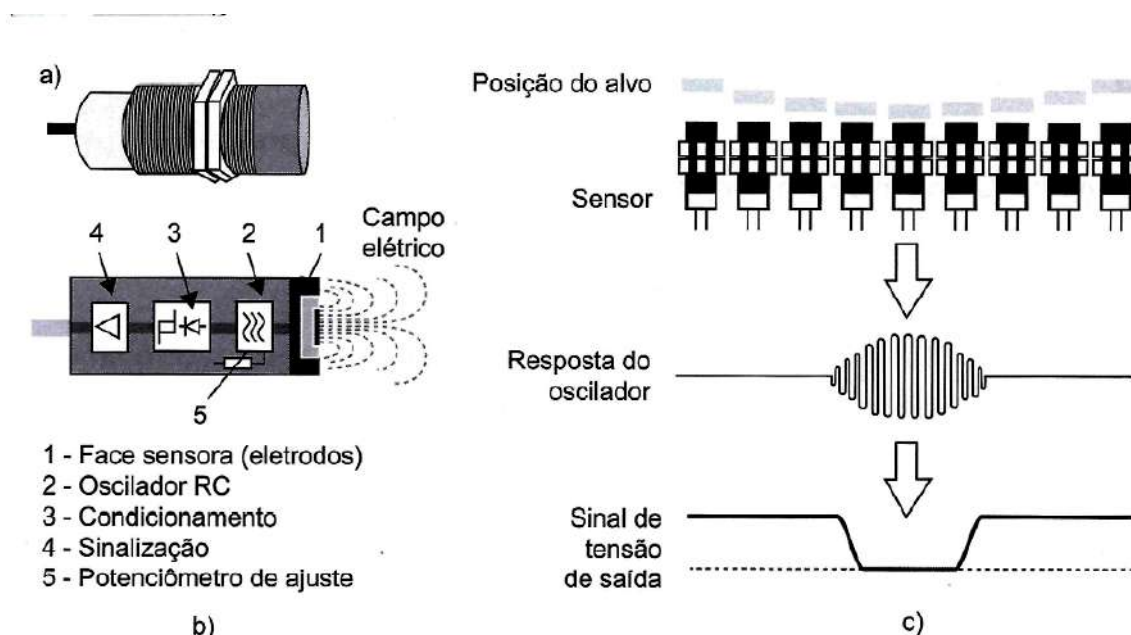
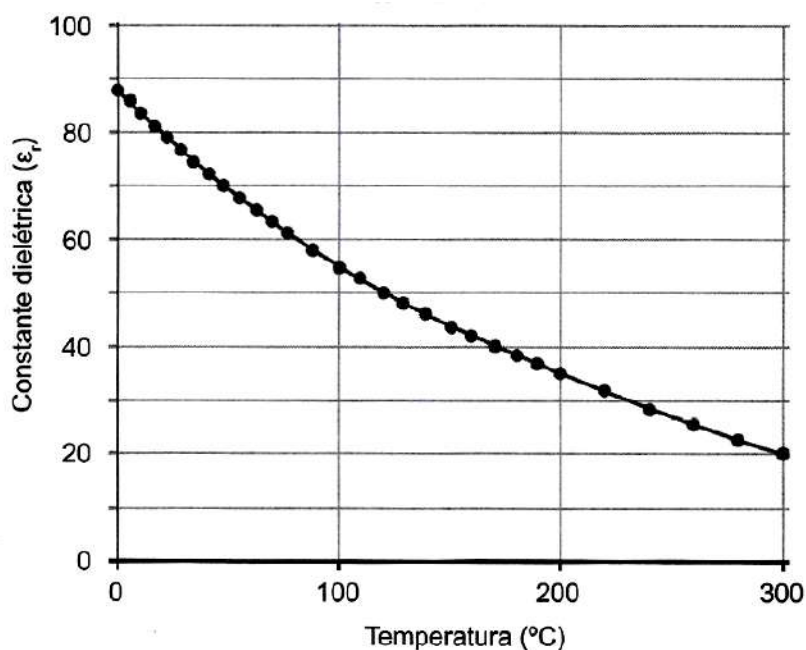


Figura 1.9 a) Exemplo de um sensor capacitivo industrial; b) diagrama de funcionamento interno; c) diagrama de funcionamento, em função da aproximação de um alvo e sua sinalização.

Até aqui, foi apresentado o comportamento de um sensor para a alteração ou mistura de algum material (dielétrico). Entretanto, vale ressaltar que todos os materiais dependem de temperatura, pressão etc. Logo, se considerarmos as compensações de dilatação, entre outros, podemos caracterizar sensores em função do

comportamento do material em função da temperatura ou de outro parâmetro de ambiente. O Gráfico 1.1 ilustra a dependência da constante dielétrica da água em relação à temperatura. Vemos que, a 20 °C, a $E_{r\text{ água}} = 81$, e que a 100 °C, $E_{r\text{ água}} = 55$, e que a 300 °C, cai para $E_{r\text{ água}} = 20$.

Gráfico 2.1 - Constante dielétrica relativa da água em função da temperatura



SENSORES FOTOELÉTRICOS

Os sensores fotoelétricos são utilizados para identificar presença, proximidade, deslocamento. Baseiam-se em conjunto Emissor-Receptor, e podem operar identificando o sinal do emissor, ou identificando sua ausência.

Basicamente, são de três tipos: barreira, difuso e fotorreflexivo.

Os sensores fotoelétricos do tipo barreira possuem transmissor e emissor em dispositivos separados. Normalmente, são separados por uma grande distância. Operam indicando o recebimento contínuo do sinal do transmissor, até que um alvo se aproxima e corta o feixe óptico, interrompendo a transmissão, conforme ilustrado pela Figura 1.10.

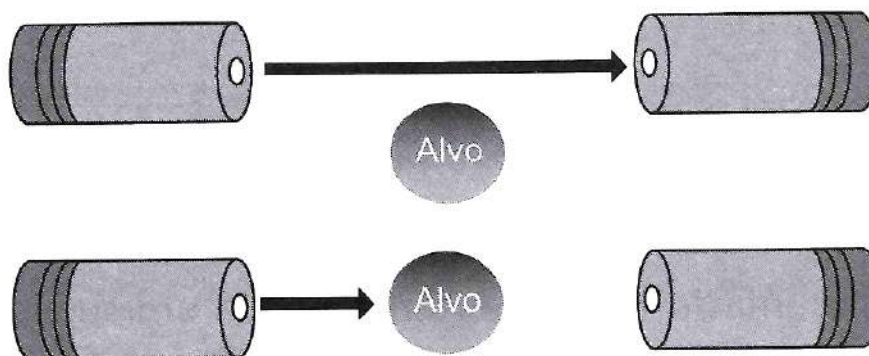


Figura 1.10 - Sensor fotoelétrico tipo barreira.

Os sensores do tipo difuso têm o receptor e o emissor no mesmo dispositivo, entretanto o receptor só recebe o sinal do transmissor quando um alvo/anteparo se aproxima o suficiente para que sua superfície reflita o sinal. Percebe-se que a distância sensora, nesse caso, depende do material do alvo a ser identificado. Assim sendo, a distância sensora depende de um fator de correção (F_{DSE}), determinado pelo tipo de cor (F_c) e pelo tipo do material (F_m), como

exemplificado pela Tabela 1.3.

Nesse caso, a distância sensora efetiva (DSE) pode ser obtida através da multiplicação do fator de correção pela distância nominal de um alvo padrão ($D_{nominal}$):

$$F_{DSE} = 0,81 \times F_c \times F_m$$

$$DSE = F_{DSE} \times D_{nominal}$$

Tabela 1.3 - Fatores de correção de cor (F_c) e de material (F_m)

Cor	F_c
Branco	0,95 a 1,0
Amarelo	0,9 a 0,95
Verde	0,8 a 0,9
Vermelho	0,7 a 0,8
Azul-claro	0,6 a 0,7
Violeta	0,5 a 0,6
Preto	0,2 a 0,5

Material	F_m
Metal polido	1,2 a 1,8
Metal usinado	0,95 a 1,0
Papéis	0,95 a 1,0
Madeira	0,7 a 0,8
Borracha	0,4 a 0,7
Papelão	0,5 a 0,6
Pano	0,5 a 0,6

EXEMPLO

Se um sensor tem uma distância nominal para um alvo padrão de 60 cm, qual seria a distância sensora efetiva para um material de pano preto?

RESOLUÇÃO

$$F_{DSE} = 0,81 \times F_c \times F_m$$

$$DSE = 0,81 \times 0,5 \times 0,5 \times 60 \text{ cm} = 12,15 \text{ cm}$$

CONCLUSÃO

A distância sensora para um objeto de pano de cor preta reduz em aproximadamente 1/5 a distância sensora para um material branco de metal polido.

Já o terceiro tipo, sensor retrorreflexivo, utiliza-se de um espelho prismático para refletir o sinal do transmissor para o receptor. Um objeto é identificado quando ele

corta o feixe de luz entre o sensor e o espelho, acusando deslocamento do alvo.

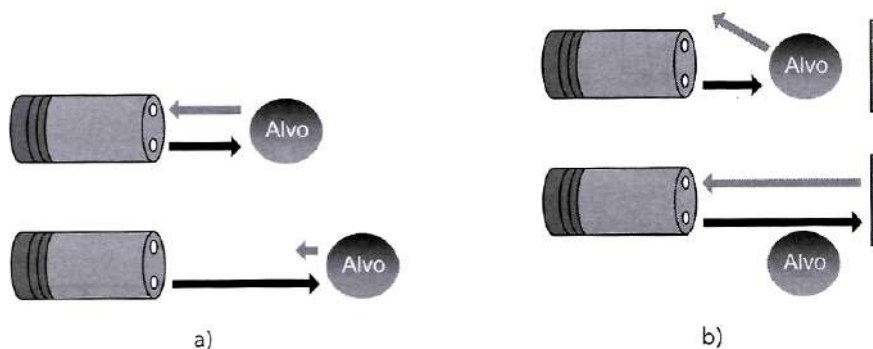


Figura 1.11 - a) Sensor fotoelétrico difuso; b) sensor fotoelétrico retrorreflexivo

Outras apresentações podem ser utilizadas para indicação de presença ou interceptação de feixe de luz, por isso esse dispositivo também é conhecido como opto interruptor.

são a distância de sensibilidade muito maior que a dos sensores indutivos e capacitivos e a indiferença quanto ao tipo do material. Entretanto, têm como desvantagem, principalmente no meio industrial, a sujeira das lentes.

As grandes vantagens dos sensores de presença ópticos

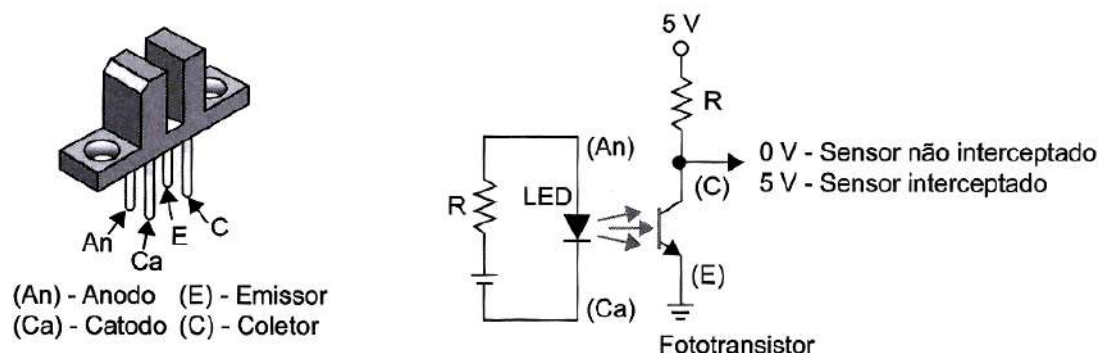


Figura 1.12 - Sensor óptico de proximidade, tipo interruptor.



SENSORES RESISTIVOS E EXTENSÔMETRO

A primeira classe de sensores resistivos são os potenciométricos. Eles nada mais são que sensores que

convertem um deslocamento físico linear ou angular em uma variação resistiva proporcional ao deslocamento. A Figura 1.13 exemplifica esses dois potenciômetros.

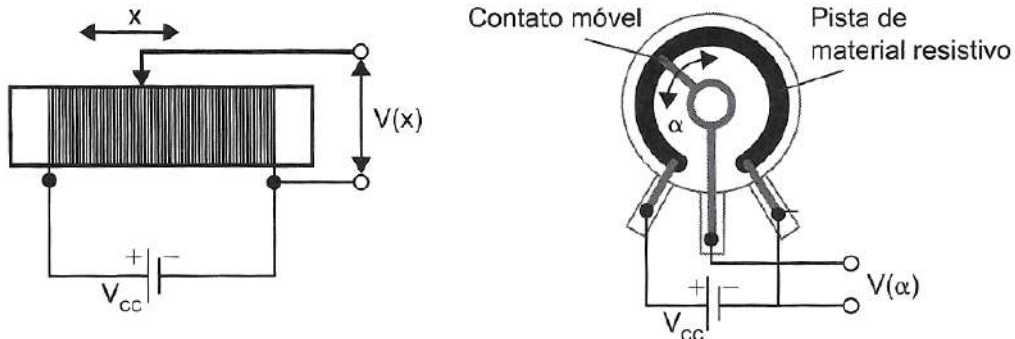


Figura 1.13 - Exemplos de sensores resistivos potenciométricos.

Já a Figura 1.14 apresenta um exemplo prático da aplicação de sensores potenciômetros, que é a correlação entre nível de tanque com valor de resistência.

Dependendo da impedância do sistema de controle a que um sensor potenciométrico se reporta, ou ainda da distância em que este se separa do controle, a sua característica de linearidade deve ser avaliada. A Figura 1.15 mostra justamente a comparação entre um sensor potenciométrico sem carga e outro com carga. Percebe-se que a presença da carga (ou a impedância de entrada compatível com as resistências do sensor potenciométrico)

resulta em uma resposta não linear do sensor. Portanto, para se garantir uma resposta linear, a impedância de carga deve ser muito menor que a resistência do sensor.

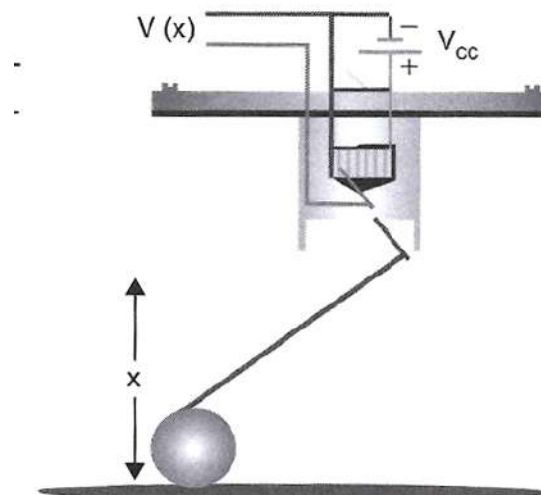


Figura 1.14 Exemplo de sensor potenciométrico - boia de nível.

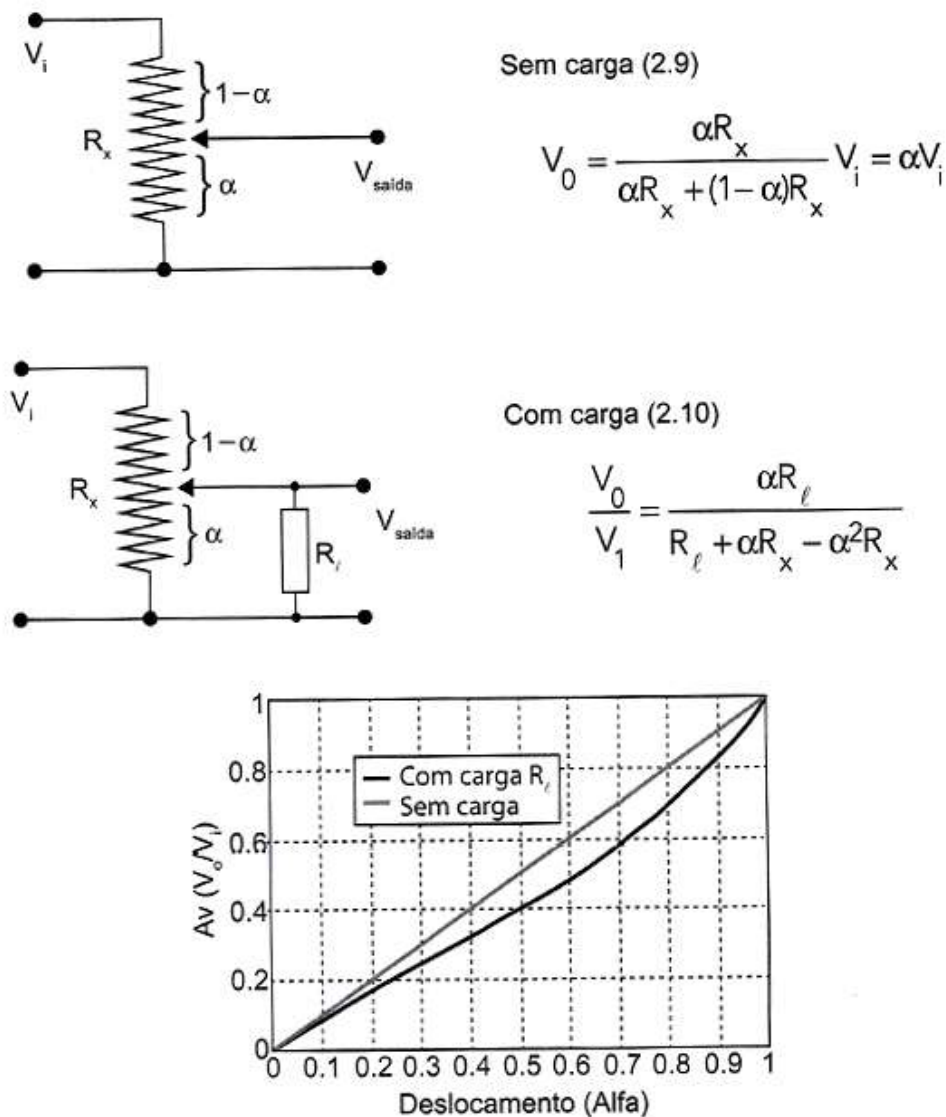


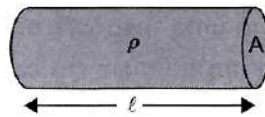
Figura 1.15 - Sensores potenciométricos com carga e sem carga. Análise comparativa de linearidade.

Paralelamente aos potenciométricos, a análise de propriedades físicas do sensor que influenciam diretamente na resistividade do material pode resultar em características a serem mensuradas.

No caso de um condutor elétrico, ilustrado na Figura 2.16, sua resistência depende de suas características geométricas como área da seção transversal (A) e comprimento (l), além das características intrínsecas do material (ρ) e pode ser calculada de forma genérica

pela Equação 2.11. Entretanto, percebe-se que existe dependência do tipo de material em relação a outras grandezas, como a temperatura, o que possibilita que essa grandeza (temperatura) seja monitorada pela resistência do material utilizado como sensor. A Equação 2.12 ilustra a dependência da resistência em função da temperatura, através do coeficiente térmico do material, (α); enquanto a Equação 2.14 apresenta a relação da variação térmica sobre a resistividade (ρ).





$$R = \rho \cdot \frac{\ell}{A} \quad (2.11)$$

$$R = R_0 (1 + \alpha \cdot (t - t_0)) \quad (2.12)$$

Exprimindo-se a resistividade em função da variação de temperatura, tem-se:

$$\rho = \rho_0 \left[1 + \alpha(t - t_0)^2 \right] \quad (2.13)$$

Figura 1.16 - Trecho de fio condutor, de resistividade ρ , área da seção transversal A , comprimento C .

Alguns dos materiais que variam sua resistência em função da temperatura são chamados de termistores e materiais condutores. A Tabela 1.4 apresenta a resistividade para diversos materiais condutores. serão tratados mais à frente em capítulo específico.

Tabela 2.4 - Resistividade e condutibilidade de alguns materiais metálicos

Material	Resistividade [$\Omega \cdot m$] a 20 °C ρ	Coeficiente de temperatura [$^{\circ}C^{-1}$] α
Prata	1.59×10^{-8}	0.0038
Cobre	1.72×10^{-8}	0.0039
Ouro	2.44×10^{-8}	0.0034
Alumínio	2.82×10^{-8}	0.0039
Tungstênio	5.60×10^{-8}	0.0045
Níquel	6.99×10^{-8}	0.0047
Latão	0.8×10^{-7}	0.0015
Ferro	1.0×10^{-7}	0.005
Estanho	1.09×10^{-7}	0.0045
Platina	1.1×10^{-7}	0.00392
Chumbo	2.2×10^{-7}	0.0039
Manganin	4.82×10^{-7}	0.000002
Constantan	4.9×10^{-7}	0.00001
Mercúrio	9.8×10^{-7}	0.0009
Nicromo	1.10×10^{-6}	0.0004
Carbono	3.5×10^{-5}	-0.0005
Germânio	4.6×10^{-1}	-0.048
Silício	6.40×10^2	-0.075

EXEMPLO

Para exemplificar, o cálculo da resistência de um fio de cobre, com 30 metros e 2 mm² de secção transversal, sabendo que a resistividade do cobre é igual a 0,0172 Ωmm²/m, é dado por:

$$R = \rho \cdot \frac{\ell}{A} = 0,0172 \cdot \frac{30}{2} = 0,25\Omega$$

Assim, a resistividade para alguns materiais é dependente da temperatura, e pode ser escrita pela Equação 2.12. Continuando o exemplo, agora em função da alteração da temperatura, se considerarmos uma variação de 50 °C, a resistência do material aquecido passaria para:

$$R = \rho \cdot \frac{\ell}{A} = (0,0172 \cdot [1 + 0,0039 \cdot (50)^2]) \cdot \frac{30}{2} = 2,7735\Omega$$

Outro grupo de sensores que dependem das características físicas são extensômetros.

Extensômetros são dispositivos capazes de medir deformação mecânica aplica como tensão e compressão sob a forma de variação de resistência. Strain gauges são extensômetros largamente utilizados para monitoramento

destas grandezas mecânica. Basicamente, quando fabricados de materiais metálicos, são três as variáveis física comprimento, área e resistividade, que podem ser relacionadas pela Equação 2.13. Essas variáveis físicas são afetadas por tensões mecânicas (extensão e/ou compressão) temperatura, conforme indicado na Figura 1.17.

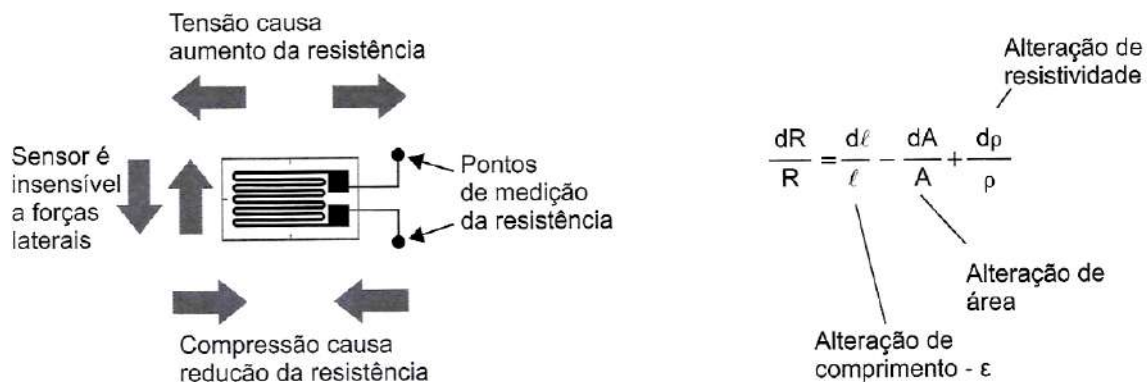


Figura 1.17 - Exemplo de extensômetro.

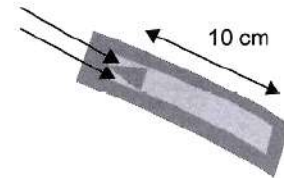
Define-se como uma das principais características de um strain gauge o fator de sensibilidade, também conhecido como fator G, que descreve a razão da alteração fracionai

da resistência e da alteração fracionai de tensão mecânica (ϵ). Para materiais metálicos, o fator G é tipicamente em torno de 2.



EXEMPLO

Seja um strain gauge de 10 mm de comprimento, com parâmetros $G = 2$ e $R = 120 \, \Omega$. Calcule a variação de resistência para uma tensão que cause a variação de comprimento de $1 \, \mu\text{m}$.

**SOLUÇÃO**

$$G = 2$$

$$R = 120 \, \Omega$$

$$\ell = 10 \, \text{mm}$$

$$d\ell = 1 \, \mu\text{m}$$

$$G = dR / R \cdot d\ell / \ell$$

$$dR = G \cdot R \cdot d\ell / \ell$$

$$dR = 2 \cdot 120 \cdot 1 \cdot \frac{10^{-6}}{0,01}$$

$$dR = 0,024 \, \Omega$$

Como visto no exemplo anterior, os valores de variação de resistência são muito pequenos, o que reforça a necessidade de circuitos adaptativos para leitura dessa variação. Para tal, utilizam-se pontes resistivas (pontes de Wheatstone), as quais serão tratadas no capítulo de condicionamento de sinais.

Ao contrário dos extensômetros de base metálica, os extensômetros semicondutores possuem uma relação não linear entre a variação relativa da resistência (dR/R) e a deformação axial relativa ($d\ell/\ell$), podendo ser aproximada pela seguinte expressão:

$$\frac{dR}{R} = k_1 \cdot \frac{d\ell}{\ell} + k_2 \cdot \left(\frac{d\ell}{\ell} \right)^2 \quad (2.15)$$

em que os coeficientes k_1 e k_2 dependem das características do semicondutor, como por exemplo seu grau de dopagem. Apesar da não linearidade, extensômetros semicondutores possuem maior sensibilidade, o que lhes permite medir deformações de maior amplitude.

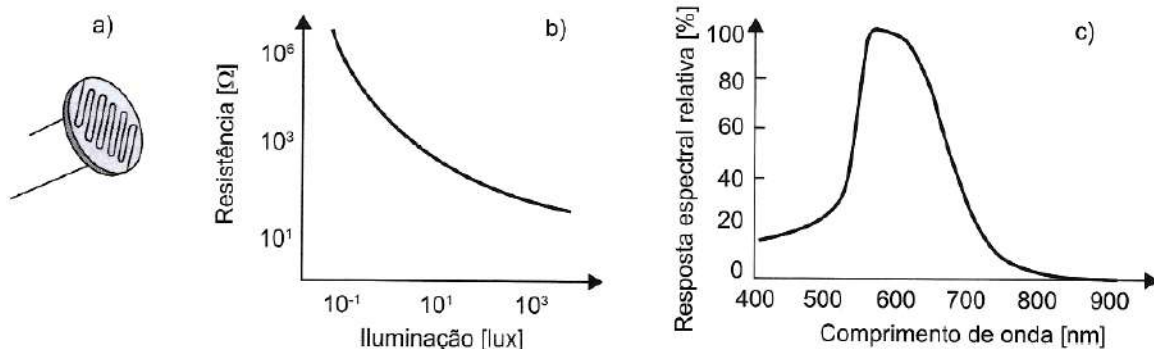


Figura 1.18 - Sensores fotorresistivos (LDR): a) forma comercial; b) curva característica de resistência x iluminação; c) resposta espectral relativa do LDR.

Tipicamente, apresenta resistência muito alta (na ordem de dezenas a centenas de $k\Omega$) na falta de luz. Quando há iluminação em sua face, sua resistência cai drasticamente para a ordem de centenas de Ω , caracterizando seu comportamento inversamente proporcional entre resistência e iluminação, conforme sugerido pela Equação

2.16 e ilustrado pela curva característica da Figura 1.18b. O valor 500 na equação refere-se a uma constante construtiva do sensor, e a iluminação é dada em lux.

$$R_{LDR} = \frac{500}{\text{Iluminação [klux]}} \quad (2.16)$$

A Figura 1.18c apresenta o comportamento espectral, mostrando que na região entre 550 nm e 650 nm ocorre a maior absorção da luz pelo material.

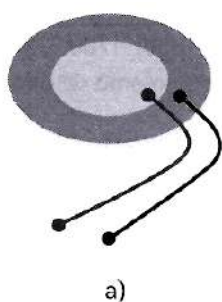
SENSORES PIEZOELÉTRICOS

O fenômeno piezoelétrico é aquele no qual um material tem a capacidade de converter energia mecânica (como pressão ou vibração) em energia elétrica. Quando esses materiais sofrem uma pressão mecânica, ocorre um fenômeno de polarização provocado em determinados cristais através de contrações mecânicas. Os efeitos de dipolos somados resultam em uma diferença de potencial no material, que dependem basicamente de três fatores: geometria, propriedades físicas do material e excitação mecânica.

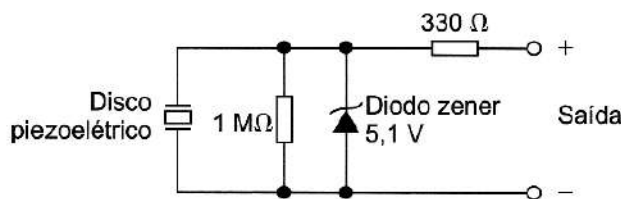
Comercialmente, são encontrados como discos, conforme ilustrado na Figura 2.19 Quando o disco sofre alguma

intervenção física, como um toque físico ou pressão, ele irá produzir uma tensão elétrica nos terminais. Essa tensão é proporcional à quantidade de força exercida sobre o disco. Assim, se for aplicada uma pequena pressão ou toque, será gerada uma pequena tensão elétrica; se for aplicada uma pressão mais forte, será gerada uma tensão elétrica maior.

O material piezoelétrico em discos cerâmicos piezoelétricos é tão eficiente que até mesmo uma força moderada sobre o disco vai produzir em excesso de 5 ou 10 Volts. Essa característica tem seu lado positivo, pois torna mais fácil fazer a interface dos discos a um circuito, uma vez que normalmente não há necessidade de amplificar o sinal; por outro lado, a tensão e corrente a partir do disco podem facilmente exceder as entradas máximas do outro dispositivo eletrônico no qual ele está conectado, como por exemplo um microcontrolador.



a)



b)

Figura 1.19 a) Sensor piezoelétrico (em disco); b) circuito para grameamento de tensão máxima e controle de corrente máxima fornecidos pelo disco.

A Figura 1.19b apresenta uma hipótese de proteção para circuitos de controle, em que um diodo zener de 5,1 V e a resistência de 330 Ω são incluídos para limitar a tensão máxima fornecida pelo disco em 5,1 V, adicionalmente limitando a corrente produzida pelo disco. Na prática, a maioria dos microcontroladores modernos, incluindo o Arduino, já tem um circuito de proteção embutido, de modo que os componentes citados poderiam ser suprimidos. No entanto, como eles não alteram o funcionamento do circuito, a adição deles pode garantir a proteção de sobressinais.

SENSORES MAGNÉTICOS

Sensores magnéticos são dispositivos responsáveis por perceber a presença de campo magnético. Podem, em função da presença de campo magnético, fechar o contato de um circuito, literalmente como uma chave magnética,

ou simplesmente indicar a presença do campo magnético. Os dois principais tipos são: chave magnética do tipo reed switch e sensores de efeito hall.

CHAVE MAGNÉTICA OU REED SWITCH

As chaves magnéticas são invólucros de vidro que, na presença de um campo magnético, fecham um contato elétrico. Portanto, são chaves com o comportamento Normalmente Aberto (NA), quando não expostas ao campo magnético. Outra opção é quando possuem duas opções de saída: uma Normalmente Aberta (NA) se inicialmente a saída estiver no contato B, ou Normalmente Fechada (NF) se inicialmente a estiver no contato A, e quando não exposta ao campo magnético; e outra, alterando a condição anterior, quando exposta ao campo magnético (Figura 1.20).



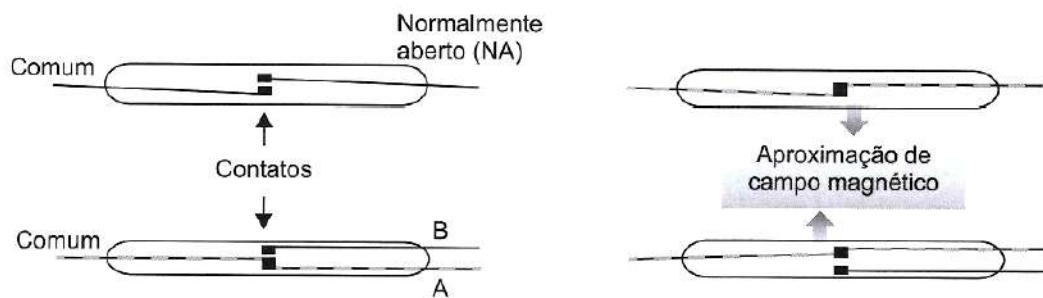


Figura 1.20 - Chaves metálicas (reed switch) de uma ou duas saídas, e seu comportamento quando da aproximação de um campo magnético.

Com relação às aplicações, são vulgarmente utilizadas para acionar, magneticamente, dispositivos eletroeletrônicos como alarmes, trancas elétricas, portas, circuitos eletrônicos de partida, entre outros.

SENSOR DE EFEITO HALL

Sensores de efeito Hall são dispositivos sensores magnéticos de estado sólido usado tanto como sensores

magnéticos ou para medir campos magnéticos. Os campos magnéticos são caracterizados pela densidade de fluxo (B) e pela polaridade (polo norte e polo sul). Uma vez alimentado, o dispositivo faz circular por si uma corrente elétrica I. Quando a densidade de fluxo magnético excede um certo limiar, o sensor gera uma tensão de saída, chamada de tensão de Hall (V_H). Esse funcionamento é ilustrado na Figura 1.21.

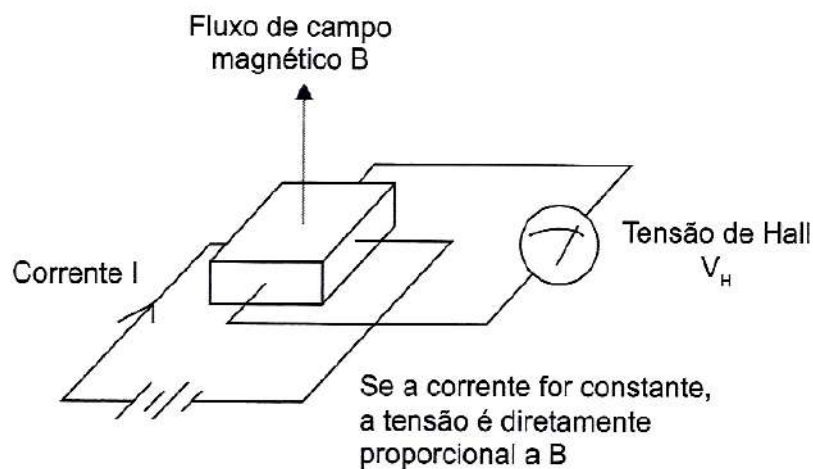


Figura 1.21 - Funcionamento de um sensor de efeito Hall.

Com relação ao tipo de aplicação, podem operar como chave magnética através do efeito Hall ou sensor analógico de campo magnético.

INTERRUPTOR DE EFEITO HALL

O dispositivo irá apresentar saída em nível lógico alto, na presença de campo magnético SUL em sua face, ou NORTE do lado oposto. Analogamente, se esses campos forem invertidos, o dispositivo apresentará nível lógico baixo em sua saída.

SENSORES DE EFEITO HALL ANALÓGICO

Sensores analógicos do tipo Hall apresentam uma saída proporcional ao ângulo do campo magnético referente à seção transversal do sensor.

Esse comportamento pode ser descrito por uma equação empírica, que descreve seu funcionamento:

$$V_H = \alpha \cdot I \cdot B \cdot \sin(\theta) \quad (2.17)$$

em que V_H é a tensão de saída de Hall, em função da corrente I e do fluxo de campo magnético B, α é uma constante de proporcionalidade referente à geometria da temperatura e da deformação do dispositivo, e θ é o

ângulo entre as linhas de campo magnético e a corrente I .

A Figura 1.22 ilustra o comportamento da tensão V_H de saída em função da interação do fluxo magnético com a

corrente ($B \cdot \sin(\Theta)$). Verifica-se que há uma região linear de operação, depois, dependendo da intensidade do fluxo magnético B , pode haver saturação, portanto, uma região de não linearidade.

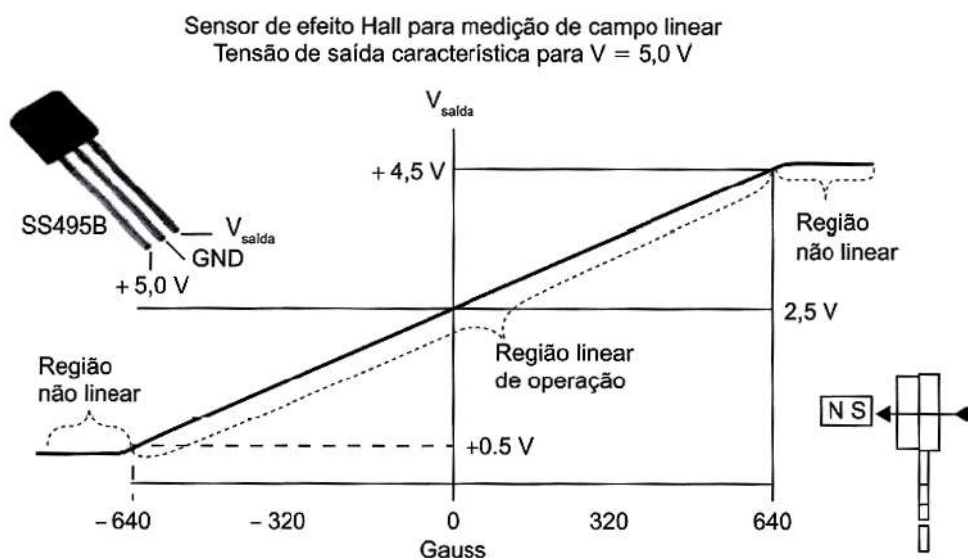


Figura 1.22 - Comportamento de um sensor de efeito Hall analógico.

Outro dispositivo (Texas Instruments DRV5053) fornece uma tensão analógica variável de 0 a 2 V em função de um fluxo magnético. A faixa de tensão de operação varia de 2,5 V a 38 V. O dispositivo tem várias versões e é oferecido com opções de sensibilidade de -11 a $+45 \text{ mV/mT}$.

Usando o HE3503 ou o A1302, temos a saída sem interação de campo $V_H = V_{H,0} = 2,5 \text{ V}$. Na presença de um campo com polaridade SUL, essa tensão V_H aumenta; aproximando um polo NORTE, V_H diminui. Para esses dois

dispositivos, a sensibilidade do dispositivo de Hall é de $2,50 \text{ mV/G}$, informação facilmente encontrada em gauss (G) datasheet do dispositivo. Assim, o campo magnético pode ser calculado em gauss, por:

$$B = \frac{10^3 \cdot (V_{H,0} - V_H)}{k} \quad (2.18)$$

A Figura 1.23 apresenta outras opções de sensores de efeito Hall, bem com pinagem e ligações recomendadas.

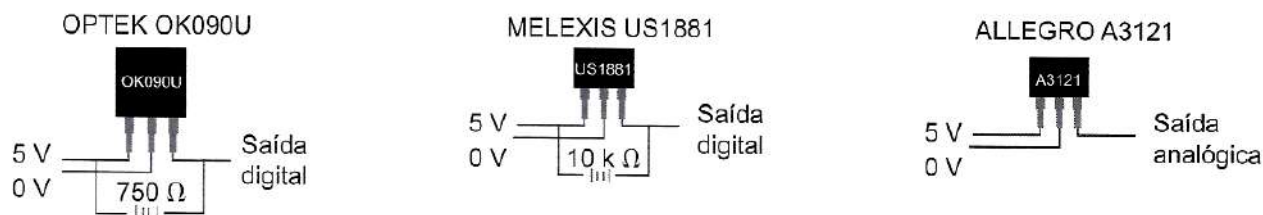


Figura 1.23 - Outros sensores de efeito Hall.



EXEMPLO 1

Suponha que você mediu uma tensão de saída do sensor de Hall $V_{H,0} = 2,48 \text{ V}$ e posteriormente, ao aplicar um campo magnético, obteve $V_H = 1,32 \text{ V}$. Descubra qual o polo magnético e a intensidade do campo aplicado no sensor.

RESOLUÇÃO

$$B = \frac{10^3 \cdot (V_{H,0} - V_H)}{k}$$

$$B = \frac{10^3 \cdot (2,48 - 1,32)}{2,50} = + 464 \text{ gauss}$$

Como o valor de B obtido é positivo, o polo aproximado é o polo NORTE.

EXEMPLO 2

Suponha, para o exemplo anterior, que você mediu $V_H = 4,56 \text{ V}$. Calcule o fluxo magnético e descubra o polo magnético aproximado.

RESOLUÇÃO

$$B = \frac{10^3 \cdot (V_{H,0} - V_H)}{k}$$

$$B = \frac{10^3 \cdot (2,48 - 4,56)}{2,50} = -832 \text{ gauss}$$

Como o valor de B obtido é negativo, o polo aproximado é o polo SUL.

SENSORES ULTRASSÔNICOS

São aqueles que permitem uma forma de medida que não invade o meio, sendo utilizados em campos como o controle de processos ou a eletromedicina (análise de imagens por ultrassom).

Como principais benefícios, os sensores ultrassônicos eliminam o risco de contaminação dos elementos que são alvo da medida, como por exemplo produtos alimentares, e também não interferem com o meio.

O sensor ultrassônico HC-SR04 (Figura 1.24) funciona

como um detector de objetos e permite medir distâncias mínimas de 2 centímetros, podendo chegar a distâncias máximas de até 450 centímetros com uma precisão de 3 milímetros.

Para ativação, necessita de um gatilho (trigger) de pelo menos $10 \mu\text{s}$ de nível alto. O módulo envia automaticamente 8 pulsos de 40 kHz e automaticamente detecta se houve retorno de algum pulso. Se houver um sinal de retorno a partir dos disparos, o nível de saída é modificado.

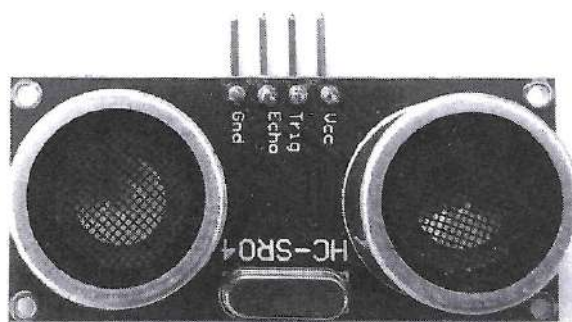


Figura 1.24 - Sensor HC-SR04.

O HC-SR04 tem como principais características:

- **Tensão de funcionamento:** 5 V_{DC}.
- **Corrente:** menor que 2 mA.
- **Saída de Sinal:** sinal pulsado entre 0 e 5 V, proporcional à distância detectada.
- **Ângulo de sensor:** < 15°.
- **Distância de detecção:** de 2 em a 450 cm.
- **Precisão:** acima de 3 mm.

A Figura 1.25 ilustra o modo de funcionamento desse módulo.

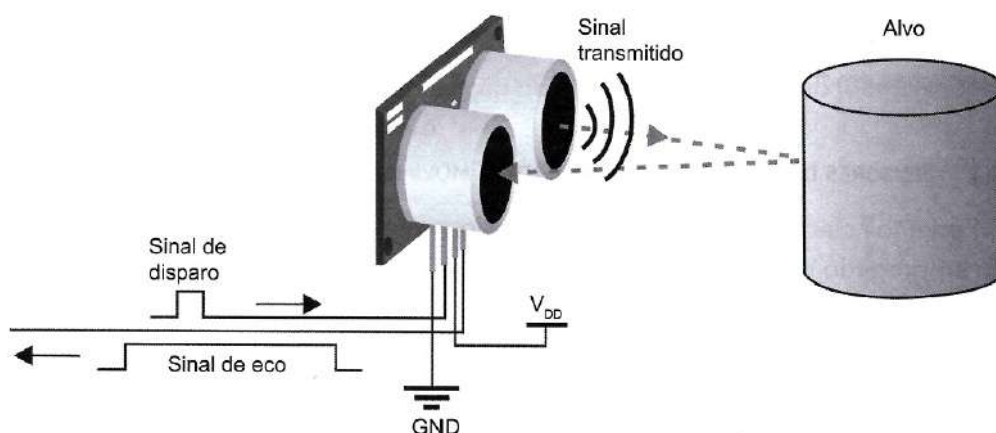


Figura 1.25 - Sensor ultrassônico - princípio de funcionamento.

Em relação ao modo de operação, podem ser encontrados em diferentes topologias e, assim, trabalhar de modo similar aos modos dos sensores fotoelétricos. Dois modos

são mais comuns: modo difuso e modo direto, ilustrados na Figura 1.26.

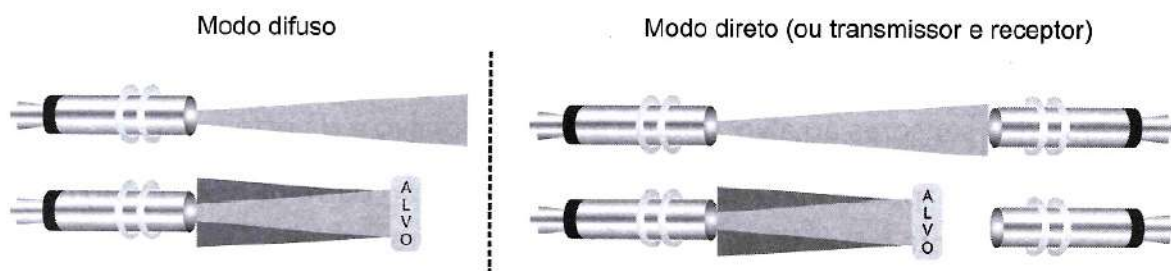


Figura 1.26 - Modos de operação comuns de sensores ultrassônicos.

Além do modo difuso e do direto, o sensor ultrassônico também pode operar no modo retrorreflexivo. Nesse caso, utiliza-se um anteparo reflexivo posicionado a uma distância fixa. Esse anteparo equivale a um tempo de detecção fixo. Qualquer objeto que intercepte os sinais sonoros antes do anteparo fará com que uma onda sonora seja percebida em um tempo menor.

Comercialmente, encontramos sensores para atender as diferentes aplicações industriais como nível e altura (com alcance de até 10 metros), ou para substituir sensores indutivos e capacitivos, para evitar colisão entre alvo e sensores em algumas linhas de produção.

QUANTO À APLICAÇÃO

Já abordamos algum dos diferentes sensores, classificados em função do seu modo de operação e das características físicas envolvidas. Agora, vamos apresentar alguns sensores em função de sua aplicação.

SENSORES DE POSIÇÃO / DESLOCAMENTO / MOVIMENTO

Podemos classificar os sensores de deslocamento em função do tipo de movimento empregado no sensor: linear ou rotacional. Diversas são as tecnologias para detectar ou monitorar posição ou deslocamento, e, por isso, vamos tratar desses sensores com exemplos de aplicações.

POSIÇÃO

As perguntas a serem respondidas por esse sensor, normalmente, são: O alvo está passando (neste instante) por esta posição? Ele está presente?

E as possíveis respostas são sempre sim ou não, ou seja, respostas binárias.

IDENTIFICAÇÃO DE POSIÇÃO DE PEÇAS EM UMA ESTEIRA

Podemos utilizar diferentes sensores para identificação

de peças em esteiras, mas temos que considerar as características da planta, para verificarmos as distâncias mínima e máxima dos sensores ao alvo, tipo de material do alvo e da esteira etc.

- **Hipótese 1:** posicionar sensores fotoelétricos tipo barreira, se o objeto não for transparente o suficiente para o feixe de luz atravessá-lo.
- **Hipótese 2:** posicionar sensores capacitivos, a uma distância de alguns milímetros da lateral do alvo.
- **Hipótese 3:** se o alvo for metálico, podemos posicionar sensores indutivos a alguns milímetros da lateral do alvo.

Entretanto, as hipóteses 2 e 3 exigem que o alvo esteja bem posicionado na esteira, o suficiente para que ele respeite a distância máxima de detecção, ainda não evite contato com o sensor.

- **Hipótese 4:** o sensor capacitivo é posicionado embaixo da esteira, detectando passagem do alvo sobre a esteira.
- **Hipótese 5:** utilizarmos um sensor mecânico (uma chave) que é acionado pela passagem do alvo. Esse tipo de sensor é muito barato, apresenta grande repetibilidade, mas sofre de desgastes mecânicos e problemas com sujeira com o passar do tempo.
- **Hipótese 6:** utilizar células de carga, como um sensor piezoelétrico, para detectar a passagem do alvo (se ele tiver peso suficiente para que a célula de carga perceba o peso sob a esteira, ou o eixo de rolamento da esteira).

A Figura 1.27 ilustra a aplicação de alguns sensores como identificadores de alvo, ou conteúdo de alvo.



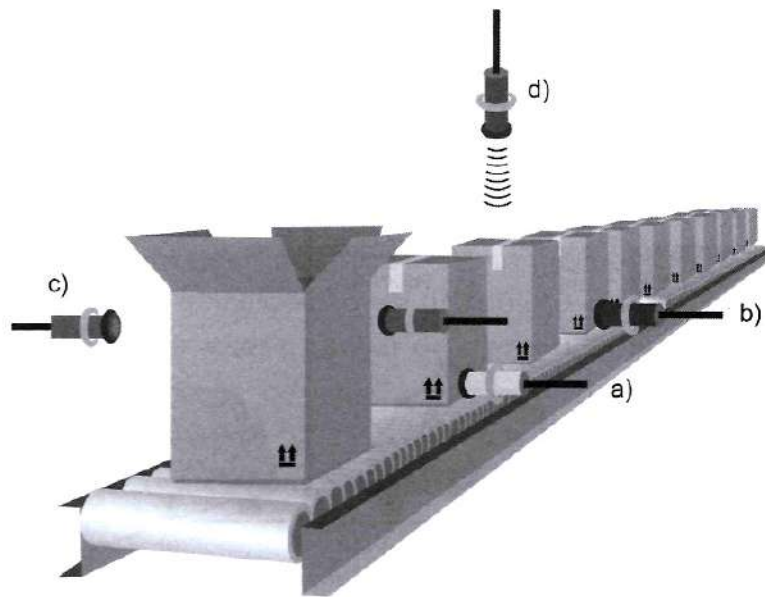


Figura 1.27 Exemplos de aplicação de sensores de posição (em uma esteira): a) um sensor indutivo, buscando o conteúdo metálico no interior da caixa; b) um sensor capacitivo, identificando a caixa; c) um sensor fotoelétrico de barreira, identificando a caixa; d) um sensor ultrassônico indicando a presença da caixa.

CONTROLE DE ACESSO EM AMBIENTE

Vamos imaginar um ambiente cujo acesso de pessoas se deseja controlar.

- **Hipótese 1:** sensores fotoelétricos, difusos, ou seja, o receptor consegue receber parte do sinal refletido pelo alvo.
- **Hipótese 2:** sensores ultrassônicos: controlam sinais recebidos a uma distância menor do que o limiar programado.
- **Hipótese 3:** sensores infravermelhos (IR): se estivermos controlando o acesso de pessoas a um ambiente, a utilização de sensores IR pode verificar a presença de calor em um certo raio de ação.

IDENTIFICAÇÃO DE PASSAGEM/POSIÇÃO

- Hipótese 1: sensor fotoelétrico, utilizado para identificação de presença ou interceptação de feixe. Nesse caso, a aproximação de uma gaveta, por porta de um equipamento, pode ser indicada, quando fechada, pela interceptação do feixe do sensor. Outro exemplo de aplicação é o posicionamento de um disquete em um drive de computador, em que a indicação de obstrução do sensor indica permissão ou não de escrita do disquete.
- Hipótese 2: veículos podem ser identificados (posição ultrapassada, ou passagem) em função de sensores indutivos instalados no chão, comumente encontrados em radares ou lombadas eletrônicas de trânsito; outra possibilidade é a utilização de sensores ultrassônicos, para indicar a posição de veículos, comumente encontrados em estacionamentos inteligentes, ou controle de acesso. Essas hipóteses estão ilustradas na Figura 1.28.

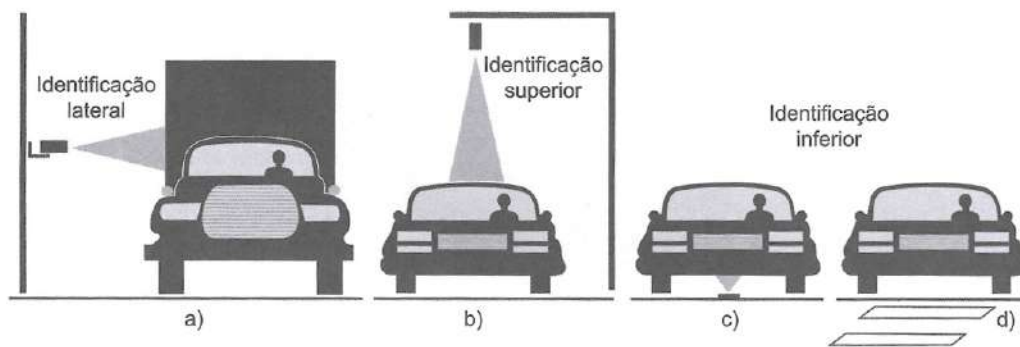


Figura 1.28 - Aplicações de identificação de posição de veículos: a), b) e c) com sensores ultrassônicos; d) com sensores indutivos.

DESLOCAMENTO/MOVIMENTO

Para medir um deslocamento linear, podemos lançar mão de sensores de posição, ou sensores que permitam medidas analógicas (contínuas).

As medidas de deslocamento ou movimento podem ser relativas, quando utilizamos um conjunto de sensores para avaliar se houve o deslocamento ou movimento; ou absolutas, quando, com um único sensor, efetivamente detectamos o deslocamento do alvo.

MEDIDAS RELATIVAS

- **Hipótese 1:** ao utilizar dois sensores indutivos, podemos identificar o deslocamento de um alvo metálico, e ainda, se cronometrarmos o tempo de passagem entre os sensores, podemos identificar a velocidade média do alvo. É o que se faz em alguns radares de trânsito (Figura 1.28 d).
- **Hipótese 2:** podemos utilizar sensores capacitivos, fotoelétricos ou até mesmo as chaves mecânicas, na mesma configuração da hipótese anterior, para diferentes tipos de alvos.

MEDIDAS ABSOLUTAS

- **Hipótese 1:** podemos utilizar um sensor indutivo para detectar pequenos deslocamentos do seu núcleo, por exemplo, resultando em uma régua eletrônica.
- **Hipótese 2:** sensores capacitivos rotativos podem ser utilizados para detecção de movimentos angulares (medição de ângulos propriamente dito) e também para aplicações correlatas, como por exemplo sintonia de rádios analógicos. Nesse último caso, a sintonia é feita pela variação da frequência de oscilação da portadora local, atrelada à alteração do valor do capacitor variável, nesse caso um varicap, que faz o papel de sensor de estação de rádio, como ilustrado na Figura 1.29.

Sensores resistivos, do tipo potenciométricos, também podem ser utilizados para detecção movimento angular. Nesse caso, podemos utilizar o movimento relativo de um pêndulo preso ao potenciômetro para correlacionar outra grandeza, como a aceleração de um objeto maior. Outro sensor para esse mesmo fim poderia ser o sensor de efeito Hall, através do movimento rotacional de um ímã.

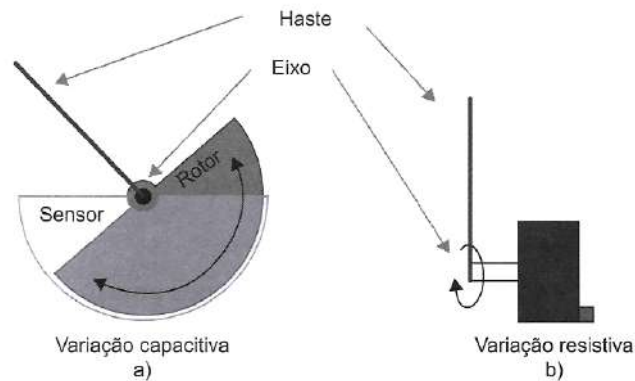


Figura 2.29 - Exemplos de sensores para aplicações de medidas relativas angulares: a) sensor capacitivo (variação da área comum entre as placas capacitivas); b) sensor resistivo (variação da resistência potenciométrica pelo movimento do eixo).

SENSORES DE NÍVEL

A detecção de nível depende efetivamente do tipo de material cujo nível se deseja conhecer, assim como do tipo de tanque ou silo, da condutividade do material, das condições de sujidade do ambiente, das características de alcance do sensor etc.

- **Hipótese 1: Detecção de nível de fluidos**

Para a detecção de fluidos, podemos utilizar sensores capacitivos e analisar a alteração de capacitância devido à alteração do dielétrico, ou combinação de parcelas de dielétrico (fluido e ar).

Outra possibilidade é fazer excursionar com a superfície

do líquido uma esfera metálica que é identificada por diferentes sensores indutivos posicionados verticalmente ao longo da coluna de líquido. Quando a baia metálica passa por um sensor, temos estimado o nível do líquido.

Uma terceira possibilidade seria utilizar sensores ultrassônicos e monitorar o eco do sensor. O tempo de chegada desse eco é diretamente proporcional ao nível do silo.

Ainda, se for posicionado um sensor de pressão na base, podemos medir o “peso” da coluna de líquido e assim teremos uma relação entre essa pressão e o nível ocupado.

A Figura 1.30 ilustra algumas dessas hipóteses.

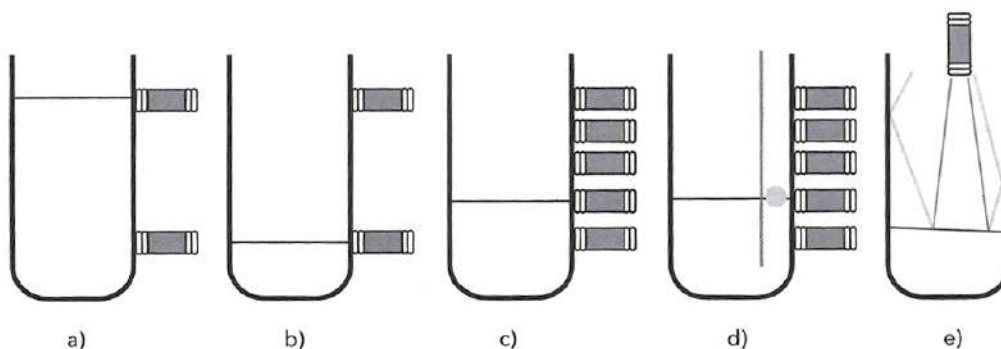


Figura 1.30 - Exemplos de detecção de nível em um tanque: a) e b) com sensores capacitivos, formato binário; c) com sensores capacitivos, modo com vários níveis; d) com sensores indutivos, com vários níveis; e) com sensores ultrassônicos.

- **Hipótese 2: Detecção de nível de tanques**

A maior parte dos tanques de automóveis ainda possui um transdutor resistivo, controlado por uma baia para apontar o nível aproximado do reservatório.

Outra possibilidade, mais atual, para a mesma aplicação é utilizar um sensor magnético para, em função da movimentação relativa da baia, movimentar um ímã que por sua vez altera a leitura de fluxo magnético em um



sensor de efeito Ha/1, como visto na Figura 1.31.

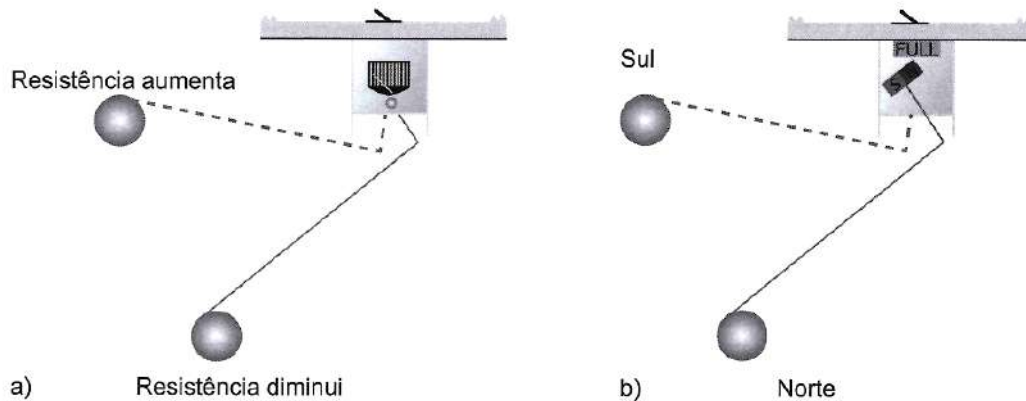


Figura 1.31 a) Senso r de nível de tanque, resistivo; b) um sensor magnético para o mesmo fim.

Uma terceira hipótese para medição de nível de líquidos ou granulados em silos é a utilização de medição capacitiva. O nível do elemento que se quer medir resulta em um sensor capacitivo de constante dielétrica definida pelo material, enquanto a outra parte do silo se comporta como um outro capacitor com ar como dielétrico, em paralelo com o anterior, como ilustrado na Figura 1.32 .

Nesse caso, a altura (h) pode ser calculada por:

$$h = \frac{C \cdot \ln(b/a) - 2\pi\epsilon_0}{2\pi\epsilon_0 \cdot (\epsilon_r - 1)}$$

onde ϵ_0 é a constante dielétrica relativa do ar, ϵ_r é a constante relativa do líquido, b é o raio do cilindro maior, a é o raio do cilindro menor, e C é a capacitância obtida da associação (ar, líquido).

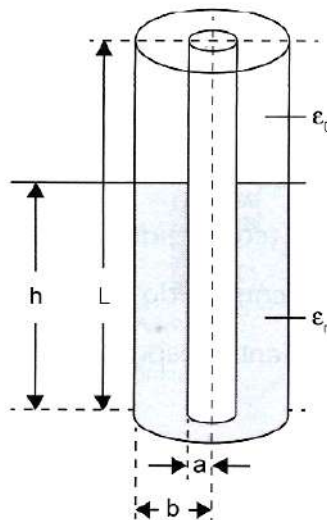


Figura 1.32 Sensor capacitivo para líquidos o granulados como sensor de nível em silo.

Dessa forma, resolvendo a associação capacitiva, e isolando a altura do material do silo, obtém-se, em função da capacitância total, a altura do material ocupante, uma incerteza típica em torno de 2%.

CAPÍTULO 02

SENSORES TÉRMICOS

Boa parte das grandezas que medimos no nosso cotidiano está relacionada a temperatura, seja para o controle de temperatura de um refrigerador, para mantermos a qualidade de nossos alimentos, seja para o controle da temperatura para manter o nível desejado em uma caldeira industrial em certo processo químico. Além disso, o controle de outras grandezas pode estar diretamente relacionado com a temperatura, como por exemplo quando monitoramos a resistência de um sensor capacitivo, em que devemos considerar as variações térmicas tanto do sensor quanto no meio para compensações das medições.

Podemos separar os sensores térmicos em quatro grupos:

- Termorresistência ou RTD (resistance temperature detectors).
- Termistores.
- Sensores semicondutores.
- Pares termoelétricas.

TERMORRESISTÊNCIA OU RTD

São dispositivos que se baseiam na variação da resistividade de um material em função da variação da temperatura. A maior parte dos RTD é feita de platina, mas também podem ser construídos de outros materiais, como por exemplo o níquel.

Comercialmente, utilizam uma nomenclatura que indica o material do qual são feitos e a resistência-base, encontrada em seus terminais quando o dispositivo é submetido a 0 °C.

Exemplos:

PT-100 (construído de platina e possui uma resistência de 100 na 0 °C);

PT-1000 (construído de platina e possui uma resistência de 1000 na 0 °C);

Ni-500 (construído de níquel e possui uma resistência de 500 na 0 °C).

Basicamente, respondem à equação:

$$R_T = R_0 \cdot [1 + \gamma \cdot (T - T_0)] \quad (2.20)$$

em que R_T é a resistência na temperatura T ; R_0 é a resistência a 0 °C; e γ é o coeficiente de temperatura do material do qual o termorresistor é construído.

Possuem faixa de operação normal de -200 °C a 850 °C.

As termorresistências têm como principais vantagens: a sua elevada exatidão, uma vasta gama de medida, uma curva de resistência x temperatura mais linear que outros tipos de sensores, dispensam a utilização de fios especiais e não possuem limitação para distância de operação. Quanto às desvantagens, podemos citar a sua fragilidade, seu elevado tempo de resposta se comparado ao termopar, bem como o fato de a sua utilização requerer alguns cuidados para evitar o aparecimento de efeitos indesejados, como autoaquecimento e descompensação por resistências parasitas.

O tipo mais comum é o PT100, que tem resistência de 100 Ω a 0 °C. Para essa termorresistência, a sensibilidade é de 0,384 $\Omega/^\circ\text{C}$, ou seja, cada 1 °C causa um incremento de 0,384 Ω em sua resistência. Assim, quando ele for aquecido a 100 °C terá uma resistência de 138,4 Ω .

Com relação aos métodos de utilização, são encontrados em três opções: 2, 3 ou 4 fios. A Figura 2.1 ilustra os métodos de utilização.

- **Método de medição a 2 fios (sem compensação):** a resistência dos cabos de ligação vai gerar um erro de leitura; no entanto, esse erro é normalmente desprezível para cabos com comprimento inferior a 2 metros (com seção padrão de 0,22 mm²). Em alguns equipamentos, a compensação ocorre automaticamente.
- **Método de medição a 3 fios (ou método de compensação simples):** é adicionado um fio que permite determinar com alguma exatidão a resistência dos fios de ligação, e o equipamento de medição faz a compensação da resistência minimizando os erros. Esse é o método mais comum na indústria.
- **Método de medição a 4 fios (compensação dupla):** nesse caso, são adicionados 2 fios que permitem fazer a medição dos terminais da RTD



(2 fios para injeção de sinal e 2 para leitura). Essa é a montagem com maior exatidão para as termorresistências, porém não é muito utilizada industrialmente, somente nos casos em que se

pretendem leituras muito exatas. Sua aplicação mais comum é em laboratórios de calibração devido aos padrões exigidos.

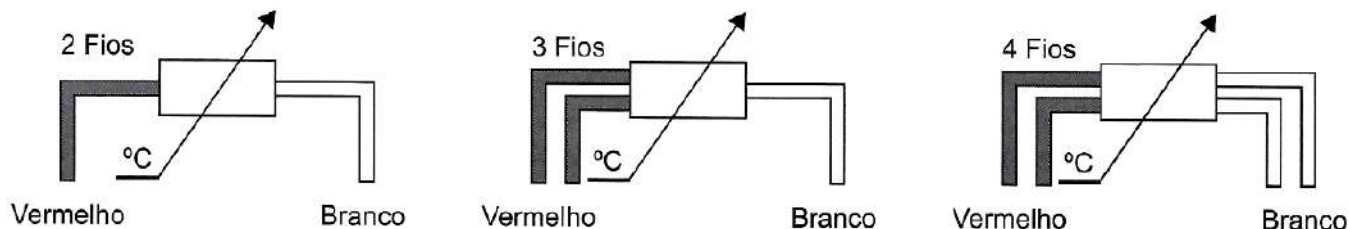


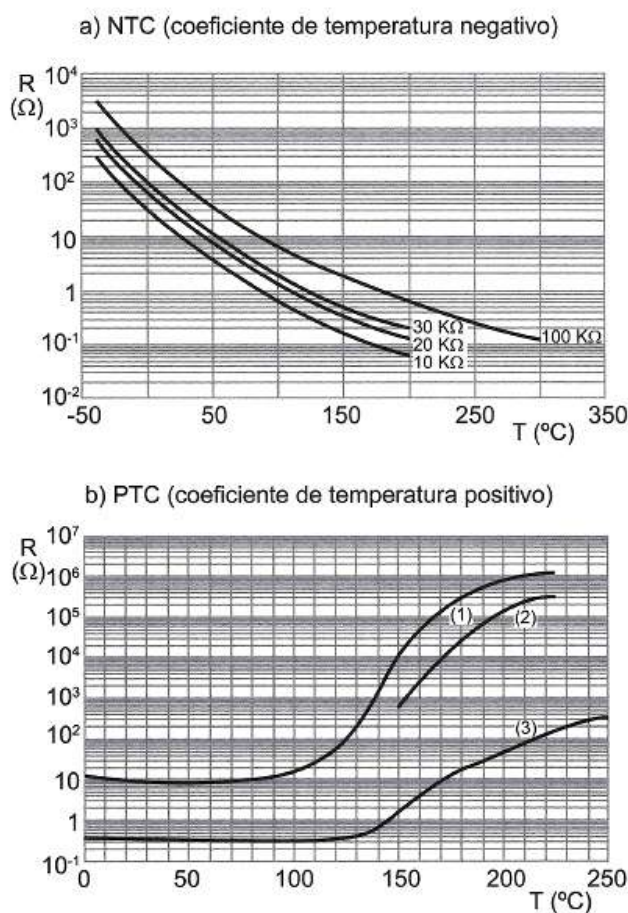
Figura 2.1 - Métodos de utilização de termorresistências.

TERMISTORES

São dispositivos em que, da mesma forma que os RTD, a resistência é dependente da temperatura. Entretanto, são fabricados de materiais cerâmicos semicondutores, o que significa que apresentam uma resistência muito mais alta. Dependendo do óxido semicondutor, podem apresentar coeficiente de temperatura positivo (PTC) ou

negativo (NTC). Um termistor PTC tem sua resistência aumentada com o aumento da temperatura; por outro lado, um termistor NTC tem sua resistência reduzida pelo aumento de temperatura. Esse comportamento pode ser observado através das diferentes curvas características obtidas de diferentes termistores, como nos Gráficos 2.1a e b, que ressaltam a resposta não linear do dispositivo.

Gráfico 2.1 - Curvas características de resistência versus temperatura para os termistores NTC e PTC



Esses sensores têm uma sensibilidade mais elevada do que a dos RTDs, o que lhes permite detectar variações ínfimas de temperatura; no entanto, não são tão estáveis, têm uma saída não linear e operam em uma faixa mais

reduzida, normalmente entre -100 a 450 °C.

A Figura 1.2 ilustra as apresentações comuns de um termistor.

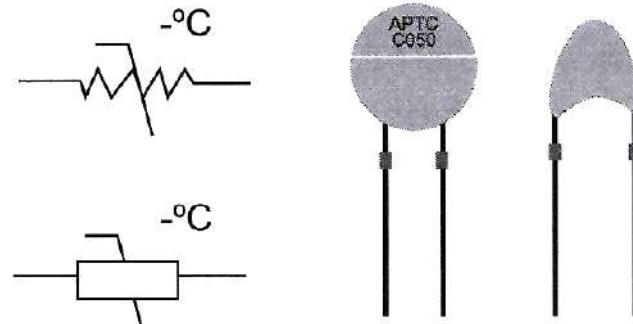


Figura 2.2 - Exemplos de termistor.

Um NTC tem seu comportamento modelado por uma equação de aproximação, a qual é conhecida como equação de Steinhart-Hart. Basicamente, ela relaciona a temperatura com a resistência, em função de três coeficientes adaptados para cada termistor (A, B, C), obtido experimentalmente.

$$\frac{1}{T} = A + B \cdot \ln(R) + C \cdot (\ln(R))^3 \quad (2.21)$$

Uma outra aproximação pode ser obtida através de:

$$\frac{1}{T} = \frac{1}{T_0} + \frac{1}{B} \cdot \ln\left(\frac{R}{R_0}\right) \therefore R = R_0 \cdot e^{B \cdot \left(\frac{1}{T} - \frac{1}{T_0}\right)} \quad (2.22)$$

onde, para obter a relação entre T (em kelvin) e R, precisamos de T0 e de B, sendo T0 a temperatura a 25 °C (= 298 K), B um coeficiente do termistor (que varia de 2000 e 5500) e R0 a resistência para T0 (exemplos > R0 = 10 kΩ).

Como a resistência desse dispositivo varia com a temperatura, sua aplicação baseia-se diretamente no monitoramento de um divisor resistivo, no qual a corrente do divisor é dependente da resistência do termistor, como sugerido pela Figura 2.3. Nesse exemplo, a tensão de saída do divisor de tensão é dada por:

$$V_{\text{saída}} = \frac{R}{R + R_S} V_{\text{ref}} \quad (2.23)$$

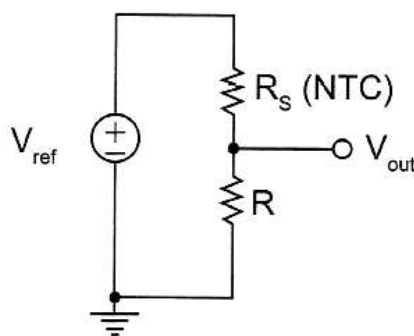


Figura 2.3 - Exemplo de aplicação direta de um termistor NTC.

Nesse caso, como RS diminui com a temperatura (T), Vout aumenta com o aumento de T.

SENSORES DE TEMPERATURA EM CIRCUITO IMPRESSO

A evolução da termometria remete aos transdutores de temperatura em circuitos integrados. Esses transdutores estão disponíveis como saída tanto em tensão elétrica quanto em corrente elétrica e normalmente operam com uma sensibilidade de 1 µA/K e 10 mV/K. Alguns



transdutores entregam um sinal digital de saída, pronto para ser lido diretamente por um microcontrolador, por exemplo.

Como principal vantagem, esse tipo de transdutor apresenta uma grande linearidade; entretanto, apresenta todas as outras desvantagens semelhantes às de um termistor.

A seguir, alguns exemplos de circuitos disponíveis comercialmente.

SENSORES DE TEMPERATURA KELVIN LM135, LM235, LM335

São CI transdutores que proporcionam uma tensão de saída proporcional à temperatura absoluta, com um coeficiente de temperatura de 10 mV/K. A tensão de saída nominal é de 2,73 V para 0 °C e 3,73 V para 100 °C.

SENSORES DE TEMPERATURA CELSIUS LM35, LM45

São os transdutores mais comumente utilizados. Possuem tensão de saída proporcional ao °C, a uma sensibilidade de 10 mV/°C. Apresentam tensão de saída de 250 mV para 25 °C e de 1,0 V para 100 °C. A Figura 2.4 ilustra as diferentes apresentações comerciais do LM35, onde $V_s = 5V_{DC}$.

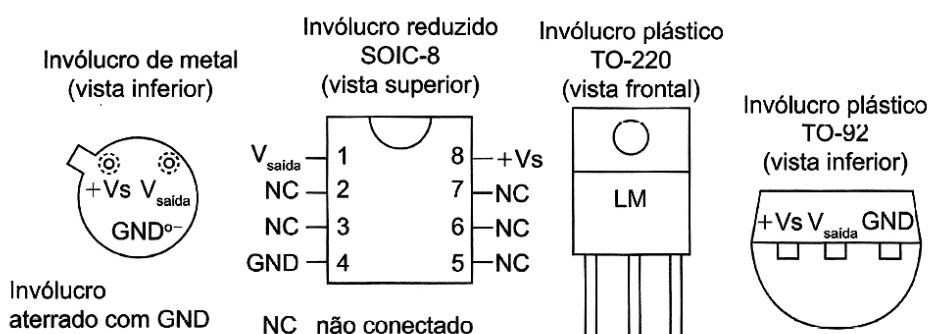


Figura 2.4 - Diversas apresentações comerciais do LM35.

O LM35 apresenta três variações (A, C e D). O LM35A opera entre -55 °C e +150 °C; o LM35C opera entre -40 °C e +110 °C, enquanto o LM35D opera entre 0 °C e +100 °C. A Figura 3.36 ilustra os modos básicos de conexão do

LM35, à esquerda, para temperaturas positivas apenas, e à direita, para temperaturas negativas também, em que, para tal, precisamos de uma tensão de referência negativa para operar, como indicado na Figura 2.5.

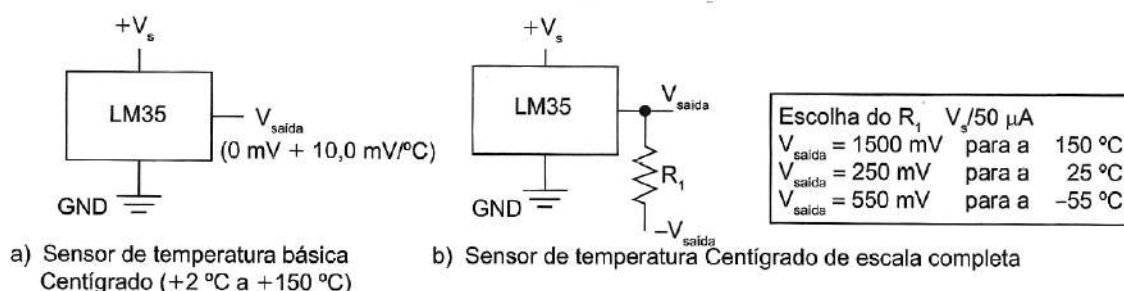


Figura 2.37 - Conexões básicas do LM35: a) para valores positivos de temperatura; b) considerando também os valores negativos de temperatura.

SENSORES ANALÓGICOS DE SAÍDA EM CORRENTE ELÉTRICA - LM134, LM234 E LM334

opera com alimentação típica de 1,2 V e tem uma janela de medição de -55 °C a +125 °C.

São transdutores de temperatura em função de uma corrente elétrica proporcional, para a temperatura absoluta. Tipicamente, possuem sensibilidade entre 1 $\mu\text{A}/^{\circ}\text{C}$ e 3 $\mu\text{A}/^{\circ}\text{C}$, ajustada por um resistor externo. O LM 134

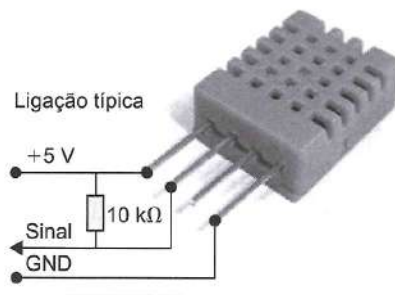
SENSORES DE TEMPERATURA COM SAÍDA DIGITAL - LM75

O LM75 é um exemplo de um CI transdutor de temperatura,

com um conversor analógico-digital (sigma-delta) interno e com comunicação serial I²C. Apresenta o valor de temperatura com 9 bits (1 para sinal e 8 para magnitude), com resolução de 0,5 °C.

SENSOR DE TEMPERATURA {E UMIDADE} DIGITAL - DHT11

Uma outra possibilidade de sensor de temperatura digital é o DHT11, dispositivo que utiliza um termistor do tipo NTC e o sensor de umidade é do tipo HR202, todos calibrados de fábrica com coeficientes armazenados em memória interna. Internamente, converte o valor para digital com 8 bits, o que propicia uma resolução máxima de 0,1% (com uma precisão de ± 2 °C), com um tempo de resposta de 2 s.



Para umidade, opera entre 20 e 90% de umidade relativa (UR) e possui uma precisão de $\pm 5,0\%$ UR.

O seu consumo de corrente é de 500 μ A a 2,5 mA, e a faixa de alimentação é de 3 a 5 V, conforme ilustra a Figura 2.38.

Este sensor utiliza um protocolo de comunicação pré-definido (em suas bibliotecas) de modo a transmitir um conjunto de bits, onde o bit 1 é definido por um pulso largo (80 μ s de duração) e o bit 0 é definido por um bit mais estreito (em torno de 24 μ s); o protocolo concentra 5 bytes de informação, sendo, após a identificação de transmissão e início de pacote:

- 1) parte inteira da umidade relativa;
- 2) parte decimal da umidade relativa;
- 3) parte inteira da temperatura;
- 4) parte decimal da temperatura;
- 5) checksum (avaliação/validação do pacote).

Exemplo :

Umidade: 0b00101100.0b00000001 = 44,1%.

Temperatura: 0b00010111.0b00000101 = 23,5 °C.



TERMOPARES

São dispositivos formados pela junção de dois metais diferentes que, quando submetidos a temperaturas diferentes, geram uma tensão (da ordem de mV) proporcional a essa diferença de temperatura. Esse princípio, conhecido como efeito Seebeck, propiciou a utilização de termopares para medição de temperatura. A Figura 2.7 ilustra a junção de duas ligas metálicas, formando um termopar.

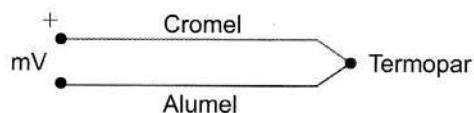
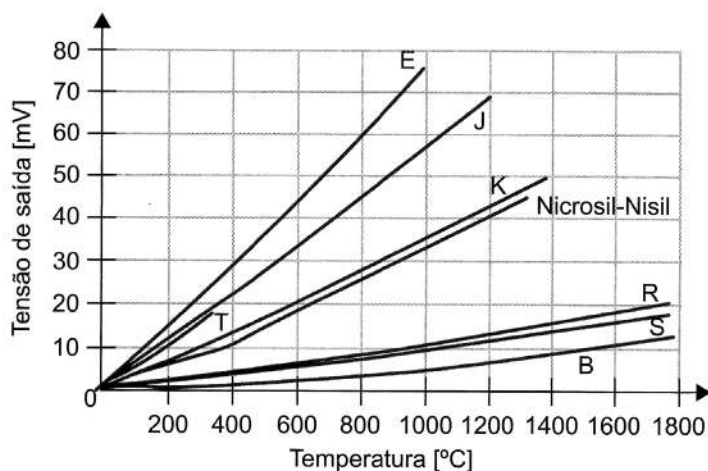


Figura 2.7 Termopar (junção das ligas Cromel e Alumel).

Um termopar ou par termométrico consiste em dois condutores metálicos de natureza distinta, na forma de metais puros ou ligas homogêneas. Os fios são soldados em um extremo ao qual se dá o nome de junção de medição; a outra extremidade, junção de referência, é levada ao instrumento medidor por onde flui a corrente gerada. Logo, para saber a tensão de um termopar para uma certa temperatura, é preciso que uma das junções esteja a uma temperatura conhecida (preferencialmente 0 °C) e que haja uma compensação com relação a ela quando for diferente de 0 °C.

Um termopar possui resposta não linear, e, dependendo dos metais utilizados na junção, segue comportamentos aproximadamente polinomiais de 5 a 9 ordens, para cada caso. O Gráfico 2.2 ilustra os diferentes polinômios para as diferentes junções de termopares comerciais.

Gráfico 2.2 Funções características dos diferentes termopares

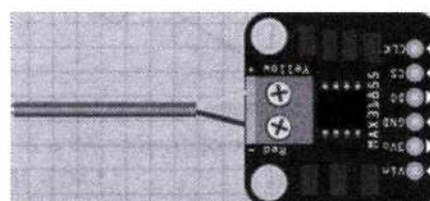


A Tabela 2.1 apresenta algumas junções metálicas e a faixa de operação, bem como a sensibilidade dos termopares resultantes. Percebe-se que os termopares possuem uma faixa de operação extensa, podendo operar entre -200 e 1800 °C.

Tabela 2.1 Pares de junções mais comuns em termopares

Referência	Material	Gama (°C)	$\mu\text{V}/^{\circ}\text{C}$
B	Platina 30% Ródio/Platina 6% Ródio	[0,1800]	3
E	Cromel®/Constantan®	[-200,1000]	63
J	Ferro/Constantan®	[-200,900]	53
K	Cromel®/Alumel®	[-200,1300]	41
N	Nirosil®/Nisil®	[-200,1300]	28
R	Platina/Platina 13% Ródio	[0,1400]	6
S	Platina/Platina 10% Ródio	[0,1400]	6
T	Cobre/Constantan®	[-200,400]	43

Para resolver o problema de compensação de temperatura de referência, utilizam-se normalmente circuitos integrados já desenvolvidos para esse fim. O CI MAX31855 é um exemplo comercial de CI com essa função. Quando conectado um termopar do tipo K (-270 °C a 1372 °C), esse CI, através de comunicação serial SPI, informa o valor de temperatura, com uma resolução de 0,25 °C. A Figura 2.7 apresenta um circuito baseado nesse CI, para interface do termopar tipo K com um microcontrolador.

**Figura 2.8** Shield de interface com termopar tipo K, baseado no MAX31855.

SENSORES DE UMIDADE

DHT11

Um dos sensores de umidade que íamos abordar aqui já foi tratado no capítulo de sensores de temperatura (DHT11), por conter internamente a medição de temperatura e umidade, obtidos e transmitidos de forma digital.

DHT22

Outro sensor semelhante, de medição de umidade e temperatura, que podemos abordar aqui é o RHT03 (também conhecido por DHT22). Semelhantemente ao DHT11, também tem interface digital e também é calibrado de fábrica, com as diferenças de ranges de temperatura (-40 °C a +80 °C, com uma precisão de 0,5 °C) e de umidade relativa (0 a 100% RH, com uma precisão de $\pm 2\%$).

HS1101

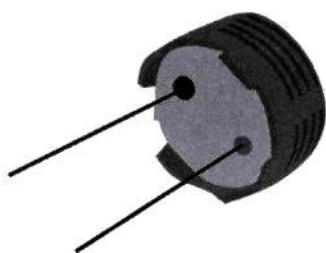
Um outro sensor é o HS1101. Esse é um sensor capacitivo e apresenta-se como tal, proporcionando comunicação analógica. Entretanto, para utilizá-lo, é preciso utilizar um circuito de transdução, para que se possa correlacionar a capacitância de saída com alguma outra variável elétrica. Para tal, pode-se utilizar um oscilador e relacionar a



frequência de oscilação, que é dependente da capacitância variável desse sensor, conforme ilustrado na Figura 2.9.

Como referência, esse sensor apresenta a capacitância típica de 180 pF para 55% RH.

HS1101 – Sensor de umidade relativa



- Faixa de umidade relativa: 1 a 99% RH.
- Temperatura de operação: -40 a 212 °F.
(-40 a 100 °C).
- Tensão de alimentação: 5 a 10 V_{DC}.

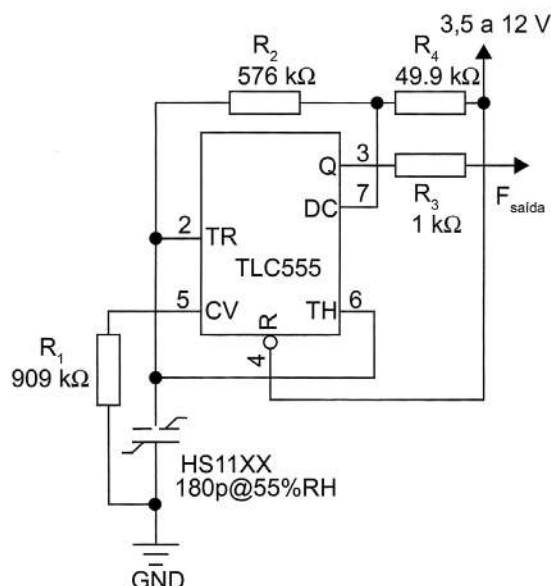


Figura 2.9 Sensor HS1101 e exemplo de circuito de medição.

Para o circuito proposto da Figura 2.10, integrando o sinal periódico da saída, temos uma tensão de média proporcional à umidade relativa, conforme tabela a seguir.

Tabela 2.2 Valores de RH (umidade relativa) exemplificados para um HS1101

RH [%]	0	10	20	30	40	50	60	70	80	90	100
Tensão [V]	-	1,41	1,65	1,89	2,12	2,36	2,60	2,83	3,07	3,31	3,55

OUTROS SENSORES DE UMIDADE

Além dos sensores de umidade relativa, podemos citar sensores de umidade do solo e de detecção de chuva.

SENSOR DE UMIDADE DO SOLO

Boa parte dos sensores de umidade de solo mede a condutividade da terra. Normalmente são compostos por duas sondas para passar a corrente através do solo, o qual se comporta como uma resistência em função da umidade (Figura 2.11). Quanto mais água no solo, este se torna mais condutor de eletricidade (menor resistência), enquanto o solo mais seco conduz menos eletricidade (mais resistência).



Figura 2.10 Sensor sonda de umidade de solo.

SENSOR DE CHUVA

Sensores de chuva podem ser baseados em placas de circuito impresso com trilhas condutoras, em que gotículas de água podem servir de condutoras de eletricidade quando caem entre duas trilhas de potenciais diferentes. Exemplos de sensores de chuva apresentam resistência de 100 k Ω quando molhados e de até 2 M Ω quando secos.

A Figura 2.11 ilustra um sensor de chuva baseado nesse princípio.

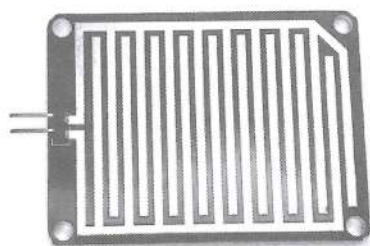


Figura 2.11 Sensor de chuva baseado na condutividade elétrica da água.

SENSORES DE PRESSÃO

Os sensores de pressão, na maioria dos casos, funcionam com o deslocamento de uma membrana, que é monitorado através de sensores strain gauges ou pela alteração capacitiva, conforme ilustrado pela Figura 2.12.

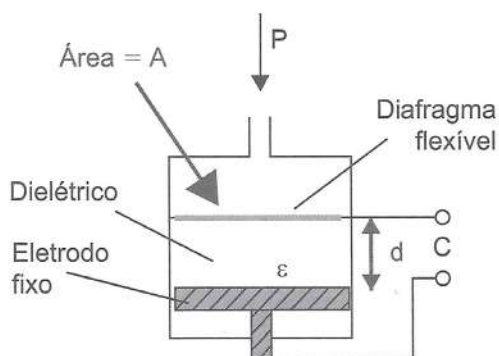


Figura 2.12 Sensor de pressão.

PRESSÃO ATMOSFÉRICA (BARÔMETRO)

Um exemplo de um módulo de medição de pressão atmosférica (mas mede também temperatura) é o que utiliza o sensor BMP085. Esse dispositivo baseia-se em um cristal de quartzo microscópico que é acoplado a uma membrana exposta (através de um orifício) à pressão atmosférica (ver Figura 2.13), apresentando robustez, alta precisão e boa linearidade. Ele opera com tensões entre 1,8 e 3,5 V, por isso os módulos já integram um regulador de tensão, normalmente de 3,3 V.



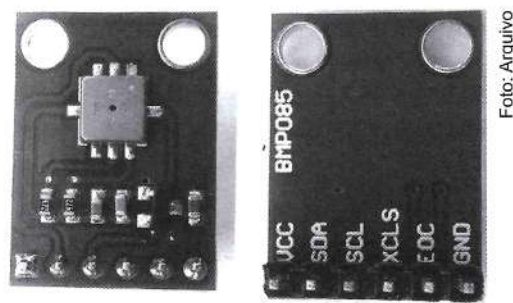


Figura 2.13 Sensor barométrico BMP085.

Como a pressão atmosférica está diretamente relacionada à coluna de ar em relação ao nível do mar, esse sensor também pode ser utilizado como um sensor de altitude.

Principais características:

- **Tensão de alimentação:** 3 a 5 V_{DC}.
- **Faixa de pressão:** 300-1100 hPa (9000 m a -500 m acima do nível do mar).
- **Resolução:** acima de 0,03 hPa / 0,25 m.
- **Faixa de operação:** -40 °C a +85 °C.
- **Faixa de medição:** 0 °C a 64 °C, com uma sensibilidade ±2 °C.
- **Comunicação:** digital, serial protocolo I²C (pinos SDA para dados e SCL para clock).

Além dos pinos de alimentação ($V_{CC} = 5\text{ V}$ e $GND = 0\text{ V}$), possui um pino de aviso de final de conversão (EOC) sinalizado em nível lógico alto e um pino de reset do sensor (XCLR), ativo em nível lógico baixo.

PRESSÃO DE FLUIDOS

São sensores utilizados para medição de qualquer fluido, como por exemplo água, óleo, ar, entre outros.

Exemplos de sensores de pressão de líquidos: Freescale MPX5010 (10 kPa - 1 m), MPX5050 (50 kPa - 5 m) e MPX5700 (700 kPa - 70 m), que são sensores que conseguem perceber uma variação de 1 em 1 litro, 5 em 5 litros e até 70 litros em 70 litros, para grandes volumes, ou seja, desses três, o primeiro é o mais sensível. A Figura 2.14 ilustra o invólucro desses sensores.

Sensores exemplificados e conversão de unidades de pressão:

- MPX5010 (10 kPa - 1 m de coluna de água - 1,45 psi - 0,1 bar).
- MPX5050 (50 kPa - 5 m de coluna de água - 7,25 psi - 0,5 bar).
- MPX5700 (700 kPa - 70 m de coluna de água - 101,5 psi - 7,0 bar).

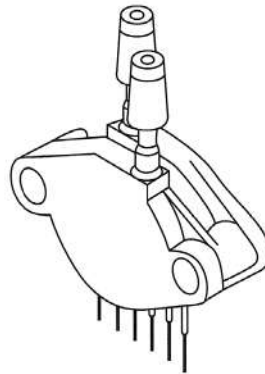


Figura 2.14 Sensor de pressão de fluido.

A saída é analógica, de 0 V a 5 V_{CC}, proporcional à pressão aplicada. Em um sistema microcontrolado (Arduino), basta alimentar, conectar a uma entrada analógica. Para traduzir o valor lido em litros, é preciso calcular como a faixa de leitura do sensor (0-1023) mapeia para o volume do seu reservatório.

Também é possível medir vácuo, usando a outra entrada do sensor. Inclusive é possível medir velocidade de fluxo do fluido, usando-se as duas entradas.

Esse dispositivo apresenta seis terminais no seu padrão de encapsulamento, porém são usados apenas três.

SENSOR DE VAZÃO

Uma das técnicas de medição de vazão comumente utilizada é o uso de rotor interno a um cano e um sensor magnético como por exemplo o de efeito de Hall para quantificar a movimentação do rotor. Quando a água passa pelo rotor, são gerados pulsos proporcionais à velocidade do rotor.

Conhecendo-se as medidas dos canos de conexão, é possível estimar o volume que passa pelo sensor e portanto controlar a vazão do fluido.

Um exemplo de um medidor de vazão comercial é o YF-S201 (ver Figura 1.13), o qual trabalha com a faixa de vazão de 1 a 30 litros por minuto e a tensão de operação é de 5 a 18 V_{DC}, sendo a tensão nominal de 5 V_{DC}.

Quando alimentado a 5 V, os pulsos de saída são de 4,5 V quando em nível alto e de 0,5 V quando em nível baixo.

Principais características:

- **Possui 3 fios:** vermelho (5 V), preto (GND), amarelo (sinal de saída).
- **Material resistente:** náilon.
- **Rosca:** 1/2".
- **Sinal de saída:** pulsos conforme a velocidade do rotor interno.



- **Faixa de vazão:** 1 a 30 litros por minuto (L/min).
- **Pressão de operação:** $\leq 1,75$ MPa.

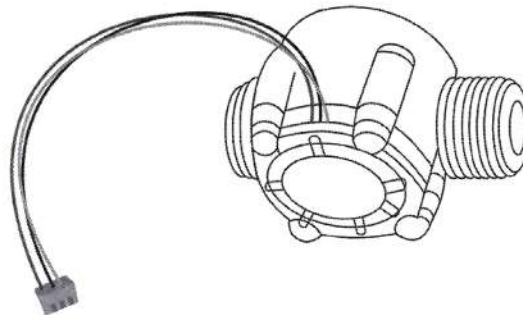


Figura 2.15 Medidor de vazão YF-S201.

Outro método para medir fluxo baseia-se no conhecimento das características físicas de uma mola. Ele funciona de acordo com o princípio do pistão apoiado por mola: o pistão situado na base de válvula de uma caixa é levantado contra a elasticidade da mola pelo meio em fluxo. A consulta da posição do pistão é efetuada por um sensor de campo magnético apresentando na saída um sinal analógico. Para exemplificar, o sensor SBT633 opera por esse princípio mecatrônico, e foi desenvolvido para medir fluxos entre 0,3 e 25 L/min, apresentando a medição de forma analógica através de uma corrente entre 4 e 20 mA.

SENSORES DE GÁS

Existem diversos sensores de gás, cada um otimizado para o tipo de gás. A Tabela 2.3 exemplifica a aplicação de diferentes sensores e suas propriedades.

Em especial, vamos detalhar o sensor MQ-6, destinado à detecção de gases, apresentando alta sensibilidade para gás de petróleo liquefeito (LPG), isobutano e propano e baixa sensibilidade para álcool e fumaça.

O seu modo de utilização está ilustrado na Figura 2.16. Três pinos são colocados em nível lógico alto (5 V), um outro é aterrado e um é colocado como referência em função da calibração do gás a ser detectado, baseando-se na escolha do resistor R_1 (entre 0 e 50 k Ω). O último pino apresenta uma saída analógica que pode ser analisada por um sistema microcontrolador qualquer. O sensor apresenta uma resistência R_S , a qual é colocada em série com R_1 , resultando na resistência de carga R_1 (que terá sempre valor entre 10 e 60 k Ω). A curva característica desse sensor é dada pela razão entre a resistência do sensor (R_S) e a resistência apresentada por ele quando submetido a 1000 ppm de LPG.

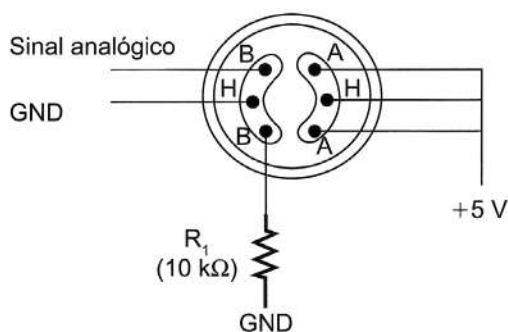
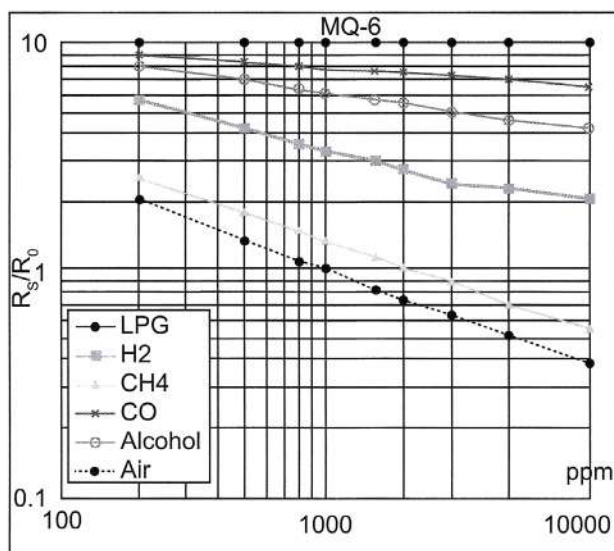


Figura 2.16 Conexão comum dos sensores de gás MQ-6.

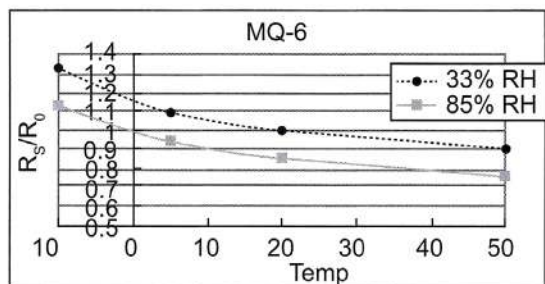
Gráfico 2.3 a) Curvas características do sensor de gás MQ-6;
b) sua dependência com temperatura e umidade relativa



a)

Características de sensibilidade do MQ-6 para diversos gases

- Umidade relativa (RH): 65% – Temp: 20 °C.
- Concentração de O_2 : 21% – RI 20 kΩ.
- R_s : Resistência do sensor para várias concentrações de gás.
- R_0 : Resistência do sensor a 1000 ppm de gás LPG.



b)

Dependência do MQ-6 da temperatura e umidade (RH)

- R_s para 1000 ppm de LPG em diferentes % RH e temperaturas.
- R_0 para 1000 ppm de LPG e a 33% RH e a 20 °C.

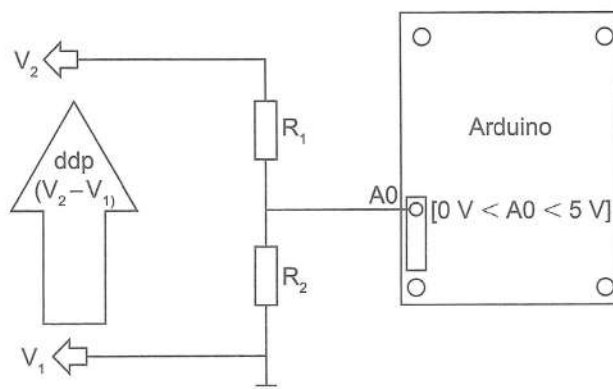


Tabela 2.3 Propriedades de sensores de gás

	MG811	MQ-3	MQ-303	MQ-6 Sensor	MQ-7 Sensor
Aplicação	Sensor de Dióxido de Carbono – CO ₂	Sensor de Álcool	Sensor de Álcool de alta precisão	Sensor de gás LPG	Sensor de Monóxido de Carbono – CO
Detecção	0 – 10.000 ppm CO ₂	10 – 1000 ppm Álcool	0 – 1000 ppm Álcool	200 – 10.000 ppm de gás isobutano propano	20 – 2000 ppm CO
Tempo de resposta	<60 s	<10 s	<20 s	<10 s	<150 s
Informações adicionais			Larga faixa de operação de temperatura. Sensor com faixa de operação de temperatura e umidade com pouco ou nenhum ajuste necessário.	Sensor pode identificar também isobutano, propano, LNG e fumaça de cigarro.	

SENSORES DE TENSÃO ELÉTRICA

Inúmeras vezes, precisamos medir um nível de tensão elétrica. Para realizarmos essa medida, normalmente montamos um divisor resistivo para que possamos adequar o valor máximo de tensão elétrica amostrada ao nível necessário em função do sistema de leitura (por exemplo, nunca ultrapassar os 5 V de um microcontrolador, conforme sugerido na Figura 2.17).

**Figura 2.17** Exemplo de divisor resistivo como sensor de tensão elétrica.

EXEMPLO

Imagine que temos uma tensão de até 55 V para ser monitorada. Se utilizarmos um divisor resistivo com R_1 (100 k Ω) e R_2 (10 k Ω), teremos uma tensão amostrada sobre R_1 de $55\text{ V} / 11 = 5\text{ V}$. Assim, qualquer variação da tensão de entrada resultará em uma tensão proporcional de no máximo 5 V, a ser monitorada por qualquer porta analógica de um microcontrolador.

MQ-5 Sensor	MQ131	MQ135	MQ-4 Sensor	MQ-8 Sensor
Sensor de Gás Natural	Sensor de Ozônio – O ₃	Sensor de controle de qualidade do ar	Sensor de gás Metano	Sensor de gás Hidrogênio
300 – 50.000 ppm Gás Natural	10 ppb – 2 ppm Gás O ₃	10 – 300 ppm NH ₃ , 10 – 1000 ppm Benzeno, 10 – 300 Álcool	300 – 10.000 ppm CH ₄ (gás Metano)	100 – 10.000 ppm H ₂ (gás Hidrogênio)
<10 s				
	Resistência de sensibilidade entre 100 kΩ – 200 kΩ	Sensor para qualidade de ar com detecção de diversos gases, incluindo NH ₃ , NO _x , álcool, benzeno, fumaça e CO ₂		

SENSORES DE CORRENTE ELÉTRICA

Para realizarmos medições de corrente elétrica contínua temos diversos métodos. Em especial, exemplificaremos: método do resistor shunt; método do transformador de corrente; método através do efeito Hall; método da bobina de Rogowski; e circuitos integrados semicondutores.

MÉTODO DO RESISTOR SHUNT OU DE DERIVAÇÃO

Medição de corrente elétrica pelo método do resistor shunt resume-se a utilizar como sensor de impedância muito inferior à do ponto do circuito em que se deseja medir. Dessa forma, uma amostragem da corrente principal pode ser utilizada para se determinar a corrente sobre a carga. Quando se emprega o resistor de shunt em série com a carga, utiliza-se um valor muito pequeno para que se obtenha uma queda de tensão irrelevante sobre o mesmo, a qual precisará ser amplificada por um amplificador operacional. A Figura 2.18 apresenta as duas configurações básicas de medição: a) lado alto, com tensão de modo comum (VMC) de terra; e b) lado baixo, com tensão de modo comum (VMC) de tensão da fonte. A Figura 2.18c ilustra o amplificador operacional INA138 utilizado como medidor de corrente DC, utilizando o resistor shunt (R_S).



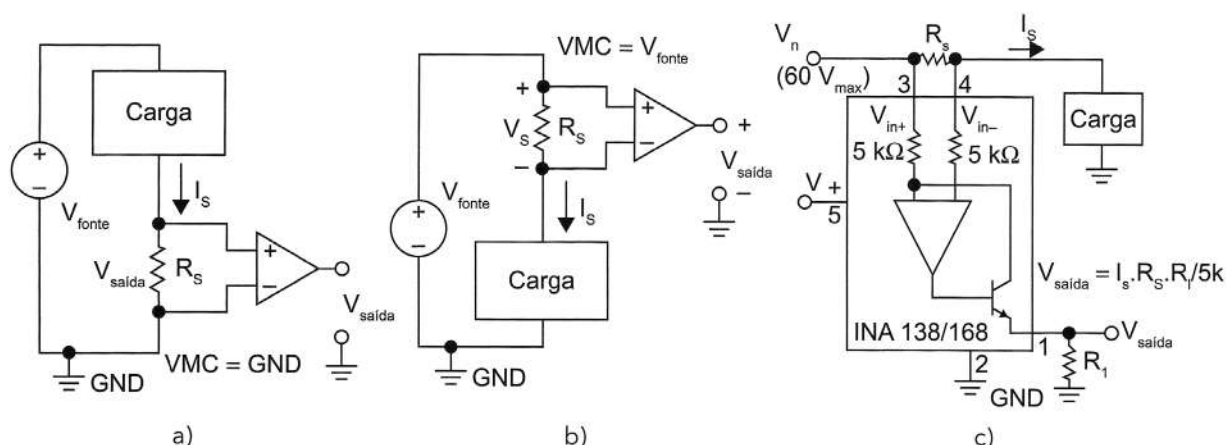


Figura 2.18 Medição de corrente por resistor shunt: a) configuração lado alto; b) configuração lado baixo; c) INA138 como medidor de corrente.

TRANSFORMADOR DE CORRENTE (TC)

Outro método de medição de corrente é o transformador de corrente, exemplificado na Figura 2.19.

Os transformadores de corrente utilizam o princípio da 1.ª Lei Elementar de Laplace ou Lei de Biot-Savart, que descreve o surgimento de um campo magnético perpendicular à direção da corrente elétrica que flui por um condutor ativo. O sentido é determinado pela regra do saca-rolha, também chamada regra da mão direita. Como é um método indireto, trata-se de uma forma segura de medição de corrente, pois não interrompe os condutores do circuito para sua medição.

Para conectar um sensor de corrente do tipo TC em um sistema microcontrolador, é preciso passar por um circuito de condicionamento, mesmo que simples, para controlar os níveis máximos de tensão de entrada analógica.

Para uma entrada de corrente alternada, é preciso passar por duas etapas: a) redução de magnitude de corrente induzida e b) circuito de condicionamento para enquadramento do sinal ao sistema de controle (exemplo: Arduino).

A Figura 2.19 ilustra dois tipos de sensores de corrente: a) SCT-013, com abertura da cabeça sensora para envolver o condutor e b) TA12-200, com enrolamento fixo, no qual o condutor deve passar ao centro. Já a Figura 2.20 ilustra um sensor com enrolamento do tipo toroidal.

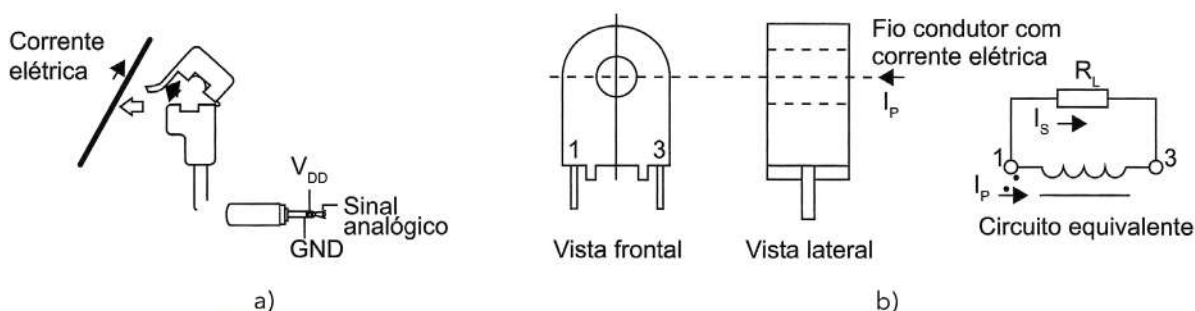


Figura 2.19 Exemplos de TC: a) sensor de corrente: SCT-013 (100 A Max); b) transformador de corrente TA12-200.

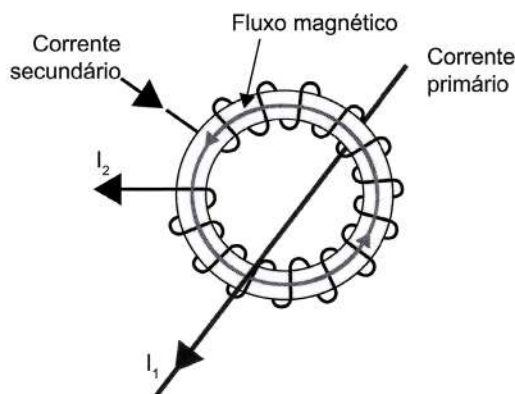


Figura 2.20 Sensor TC de núcleo de ferrite com condutor primário excitando o fluxo magnético, que por sua vez excita uma corrente no secundário.

A Figura 2.21 exemplifica a aplicação. Os resistores R_1 e R_2 são um divisor de tensão para limitar os valores máximos do secundário (sistemas microcontrolados costumam ter entrada analógica entre 0 e 5 V).

Dessa forma, pelo exemplo, uma tensão de 9 V RMS (+12,7 V de pico) passa por uma razão de proporção de 1/11, amostrando um pico máximo de 1,15 V. Já os resistores R_3 e R_4 servem para aplicar um nível DC deslocado do zero (nível de offset), entre os valores máximos (5 V) e mínimos (0 V) do sistema de controle (Arduino). O capacitor C_1 possibilita um caminho de baixa impedância para o terra para os sinais alternados.

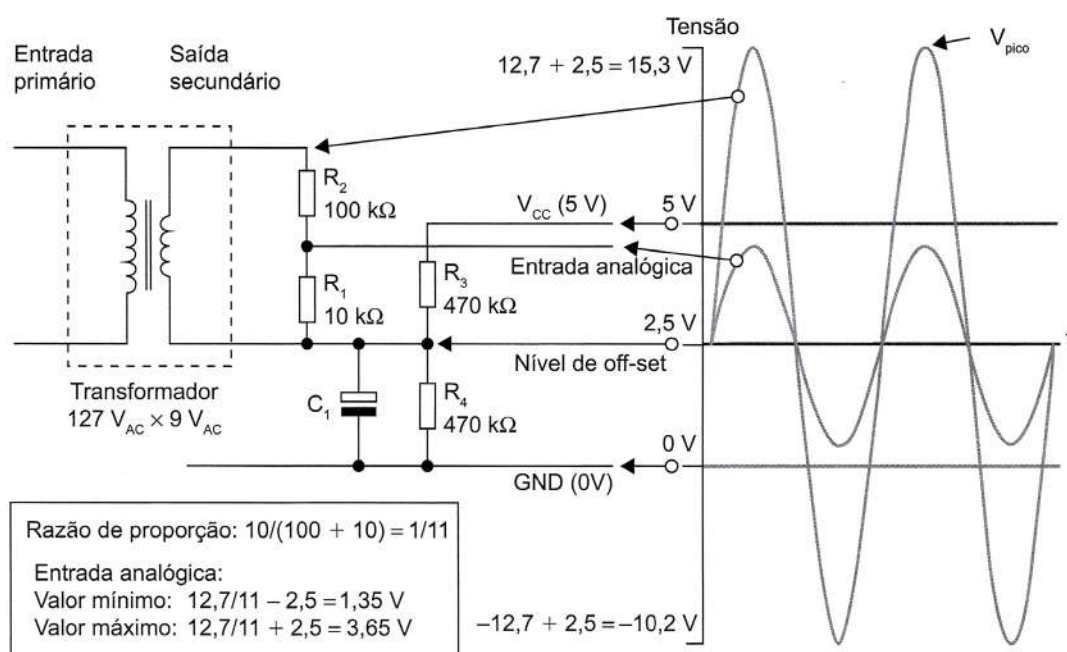


Figura 2.21 Exemplo de enquadramento de um sinal para um transformador de corrente.

EFEITO HALL

Para medição de corrente contínua ou alternada, podemos utilizar o princípio da indução de fluxo magnético em um toroide de ferrite, gerado por um condutor central com fluxo de corrente, conforme ilustrado na Figura 2.54, ou então contar com um enrolamento toroidal, que fornece um sinal de corrente proporcional ao fluxo gerado pela corrente que flui pelo condutor principal.

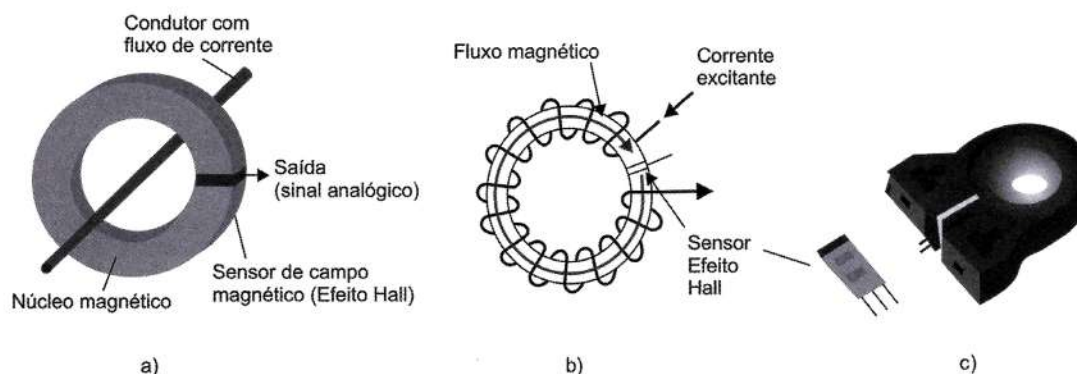


Figura 2.22 Sensor transformador de corrente de núcleo de ferrite: a) com uma passagem do condutor; b) para várias passagens do mesmo condutor; c) apresentação comercial do sensor.

Seu funcionamento baseia-se no dispositivo de efeito Hall (SS94A1 Honeywell) e, dependendo das características geométricas, faz leituras de dezenas de ampères, proporcionando uma saída nominal de dezenas de mV/A. Como exemplo, um certo sensor alimentado com $8 V_{DC}$ e polarizado para fornecer 4 V de saída sem corrente excitante tem sensibilidade de 32,7 mV/A, e para o pico de corrente de $\pm 72 A$ os valores de saída do sensor variam entre 1,64 V e 6,35 V, linearmente.

Para se aumentar o fluxo magnético no toroide, e, portanto, aumentar o valor de saída do sensor de efeito Hall, costuma-se enrolar o condutor com fluxo de corrente. Para o exemplo anterior, se a saída nominal para um condutor passante (uma volta) é de 32,78 mV/A, para 10 voltas a saída nominal será de 327 mV/A.

BOBINA DE ROGOWSKI

Diferentemente dos transformadores de corrente, que têm um núcleo ferromagnético, o método da bobina de Rogowski utiliza uma bobina toroidal enrolada em uma bobina de ar, sem núcleo ferromagnético (Figura 2.23). Uma vez colocado um condutor (primário) ao centro dessa bobina toroidal, com um fluxo de corrente, irá surgir um campo magnético que por sua vez induzirá uma corrente pelo enrolamento da bobina, gerando em seus terminais uma tensão proporcional à corrente do condutor primário.

Como a bobina não tem núcleo com elemento magnético, não aparecem sintomas de saturação magnética, como o aquecimento, eliminando efeitos como histerese; entretanto, fica mais suscetível a campos magnéticos externos.

Assim, da mesma forma que os TC, a bobina de Rogowski permite mensurar a corrente elétrica em um condutor sem interrompê-lo, portanto, de modo não invasivo.

A Tabela 2.8 apresenta uma comparação das principais características dos sensores de corrente descritos até aqui.

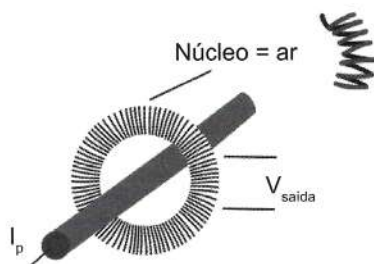


Figura 2.23 Bobina de Rogowski.

Tabela 2.4 Comparação entre os métodos de medição de corrente

	Custo	Largura de banda	Corrente Contínua	Sensibilidade	Saturação / histerese	Linearidade	Temperatura de operação	Integrabilidade	Tecnologia dos materiais
Shunt	Baixo	DC-10MHz	Sim	mV/A	Não	Muito boa	-55 a 125 °C	Excelente	Simples
TC	Médio	0,1-100MHz	Não	V/A	Sim	Fraca	-50 a 150 °C	Bom	Simples
Rogowski	Baixo	0,1-100MHz	Não	10mV/ μ A	Não	Muito boa	-20 a 100 °C	Excelente	Simples
Efeito Hall	Alto	<1MHz	Sim	10 gauss	Sim	Pobre	-40 a 125 °C	Fraco	Complicado

CIRCUITOS INTEGRADOS ESPECÍFICOS PARA MEDIÇÃO DE CORRENTE ELÉTRICA

Uma forma prática de medir corrente elétrica de baixa intensidade em circuitos eletrônicos é utilizando alguns circuitos integrados desenvolvidos para esse fim. O CI ACS712 (ilustrado na Figura 2.24) é um exemplo desse sensor, o qual é vendido em diferentes opções consoante a corrente máxima (5 A, 20 A e 30 A).

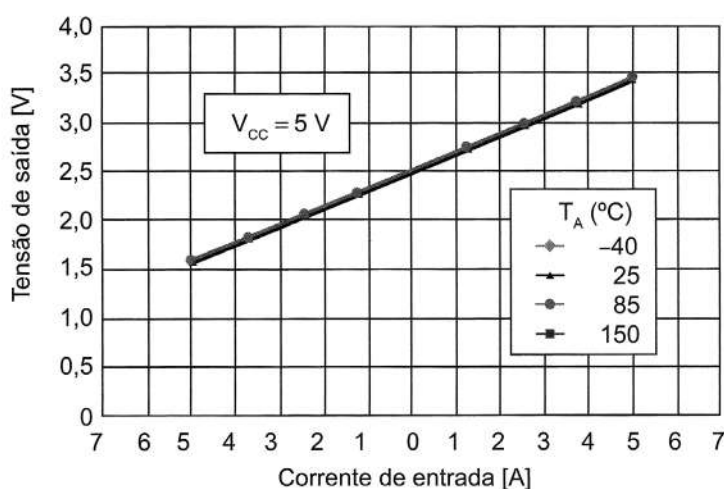
Para exemplificar, o Módulo ACS712-30 apresenta entrada entre -30 A a +30 A, e saída analógica entre 0 e 5 V. Possui uma resolução de 185 mV/A e a saída é dada pelo cálculo $0,5 \cdot V_{CC} + 185 \text{ mV/A}$, ou seja, quando alimentado com $V_{CC} = 5 \text{ V}$, temos:

$V_{\text{saída}} = 2,5 + 185 \text{ mV/A}$, o que evidencia um comportamento linear, como ilustrado na curva característica do Gráfico 2.4.



Figura 2.24 Módulo de corrente ACS712.



Gráfico 2.4 Curva característica ACS712**Tabela 2.5** Características do sensor de corrente ACS712

	Módulo 5 A	Módulo 20 A	Módulo 30 A
Tensão alimentação (V_{CC})	5 V _{DC}	5 V _{DC}	5 V _{DC}
Faixa de medição	-5A a +5A	-20A a +20A	-30A a +30A
Tensão a 0 A	$V_{CC}/2$ (2,5 V)	$V_{CC}/2$ (2,5 V)	$V_{CC}/2$ (2,5V)
Fator de escala	185 mV/A	100 mV/A	66 mV/A
Código	ACS712ELC-05A	ACS712ELC-10A	ACS712ELC-30A

SENSORES DE LUZ

Sensores de luz, ou sensores ópticos ou sensores optoeletrônicos são dispositivos capazes de converter energia de radiação incidente em liberação de elétrons e consequente geração de um sinal elétrico (tensão, corrente, variações de condutividade etc.) suscetível a aplicações de medição de intensidades luminosas (fotometria) ou aplicações de detecção de sinal luminoso (presença de alvos, contagem, movimento etc.).

A energia associada aos fótons da radiação incidente (E) é quantificada por: $E = h\nu = hc/\lambda$, em que $h = 6,63 \times 10^{-34}$ J é a constante de Planck, ν , a frequência da radiação incidente, c , a velocidade de propagação e λ , o comprimento de onda do sinal incidente.

A banda de frequências da radiação a que o sensor óptico é sensível depende da energia dos fótons e do material usado como detector.

Dependendo da tecnologia na qual se baseia o sensor, o mesmo pode apresentar variação de sensibilidade para um determinado comprimento de onda ($\lambda = 1/f$).

O comprimento de onda do sinal está diretamente ligado às cores do espectro, e consequentemente a sensibilidade dos sensores está ligada a esse parâmetro. Entretanto, os sensores mais vulgares operam em grandes faixas de frequência, sendo usuais na luz visível ou no infravermelho.

Os principais tipos de sensores de luz são: LDR, fotodiodos e fototransistores. O primeiro utiliza o conceito da dependência da resistência de suas ligas metálicas em função da incidência de luz. Já os outros dois são semicondutores que geram corrente elétrica em função da absorção de luz. Os fotodiodos não são muito sensíveis, exigindo bons circuitos de amplificação, mas, em compensação, são extremamente rápidos, podendo detectar pulsos de luz em taxas que chegam a dezenas ou mesmo centenas de MHz. Já o fototransistor é semelhante ao fotodiodo, mas amplifica as correntes que são geradas nesse processo.

Outros elementos sensores são as células fotovoltaicas, que convertem energia luminosa em elétrica, e os sensores CCD (*charge-coupled devices* - dispositivos de carga acoplada), que geram cargas elétricas proporcionais à intensidade de luz recebida por elas. Dessa forma, capturam cenas em função da luz refletida/absorvida pelos objetos.

A seguir são detalhados os principais sensores de luz.

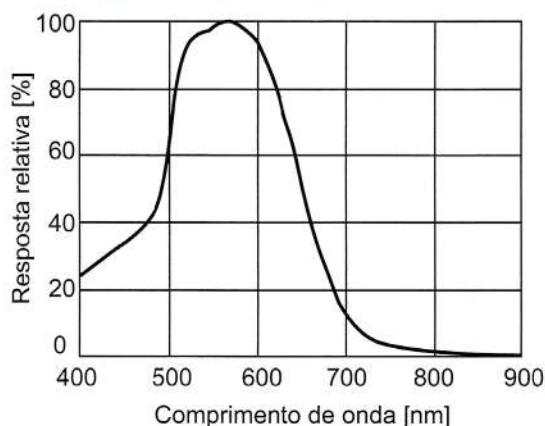


Figura 2.25 Espectro de luz.

LDR (RESISTOR DEPENDENTE DE LUZ)

O LDR (resistor dependente de luz) já foi anteriormente tratado na seção 2.1.1.4, quando foram abordados os sensores resistivos. Entretanto, neste tópico é importante chamarmos a atenção para sua resposta em frequência. Esse dispositivo apresenta maior alteração de sua resistividade para luzes entre o verde e o amarelo (entre 550 nm e 600 nm), como podemos verificar no Gráfico 2.5.

Gráfico 2.5 Resposta espectral de um LDR



Para a medição da resistência do sensor, normalmente utilizamos um divisor resistivo, e a amostragem deste é monitorada pela entrada analógica de um microcontrolador (Arduino), conforme circuito proposto na Figura 2.26.

A tensão sobre o LDR é

$$V_{saída} = \left(\frac{5}{10\text{ k} + R_{ldr}} \right) \cdot R_{ldr}$$

$$R_{ldr} = \frac{(10\text{ k} \cdot V_{saída})}{(5 - V_{saída})}$$

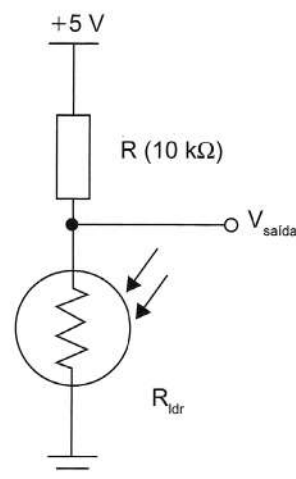


Figura 2.26 Divisor resistivo para leitura de um LDR.

Se considerarmos o monitoramento realizado pelo Arduino, o qual tem um conversor analógico-digital de 10 bits, ou seja, 1024 níveis, teremos portanto que a resolução obtida é de:

$$\text{Resolução: } \frac{5}{1024} = 0,0048828125$$

e a tensão $V_{saída}$ será aproximada por um valor inteiro $V_{A/D}$ (entre 0 e 1023), através da relação:

$$V_{A/D} = V_{saída} / 0,0048828125$$

Uma vez medido o nível de tensão sobre o LDR, pode-se correlacionar a iluminação em função desta tensão, conforme ilustrado na Tabela 2.16.

Tabela 2.6 Iluminação em função da tensão

Luz ambiente como...	Luz ambiente (lux)	Resistencia do LDR (Ω)	Resistência total LDR + R ($k\Omega$)	Corrente LDR + R (μA)	Tensão no LDR (V)
Corredor	0,1 lux	600	610	8,2	4,9
Luar	1 lux	70	80	62,5	1,4
Quarto escuro	10 lux	10	20	250,0	2,5
Dia encoberto, quarto brilhante	100 lux	1,5	11,5	434,8	0,7
Dia nublado	1000 lux	0,3	10,03	498,5	0,1

Uma forma de utilizar esses dados com o Arduino é correlacionar o valor obtido pelo conversor AD com informação de iluminação, como proposto pela Tabela 2.7.

Tabela 2.7 Informação de iluminação

Faixa de conversão	Informação
0-99	Dia claro
100-199	Dia nublado
200-499	Ambiente claro
500-899	Penumbra
900-1023	Noite/escurecimento

Com relação às diversas aplicações, podemos exemplificar:

Aplicações analógicas para um LDR:

- Câmera Controle de Exposição.
- Fotocopiadoras - densidade de toner.
- Equipamentos de teste colorimétrico.
- Densitômetro.
- Controle automático de ganho - fonte de luz modulada.

Aplicações digitais para um LDR:

- Controle automático de farol.
- Controle de luz noturna (fotocélula).
- Controle de chama em queimadores.
- Ausência / presença (de anteparo).
- Sensor de posição (alvo opaco).

SENSOR DE LUZ AMBIENTE

Sensor de luz ambiente - TEMT6000 é um sensor de luz ambiente com saída analógica no qual, com o aumento da intensidade luminosa, há um aumento na tensão analógica no pino de saída (SIG) do sensor ilustrado na Figura 2.27.

Os Gráficos 2.6a e b apresentam as curvas características desse sensor, sendo a: corrente de coletor em função da iluminância e b: a curva espectral da sensibilidade do sensor.

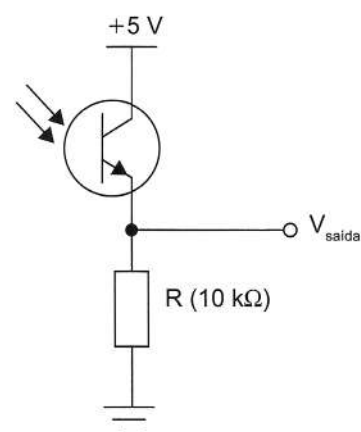
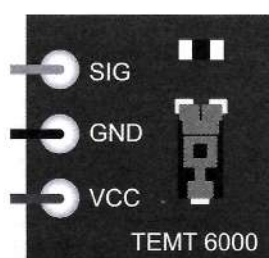
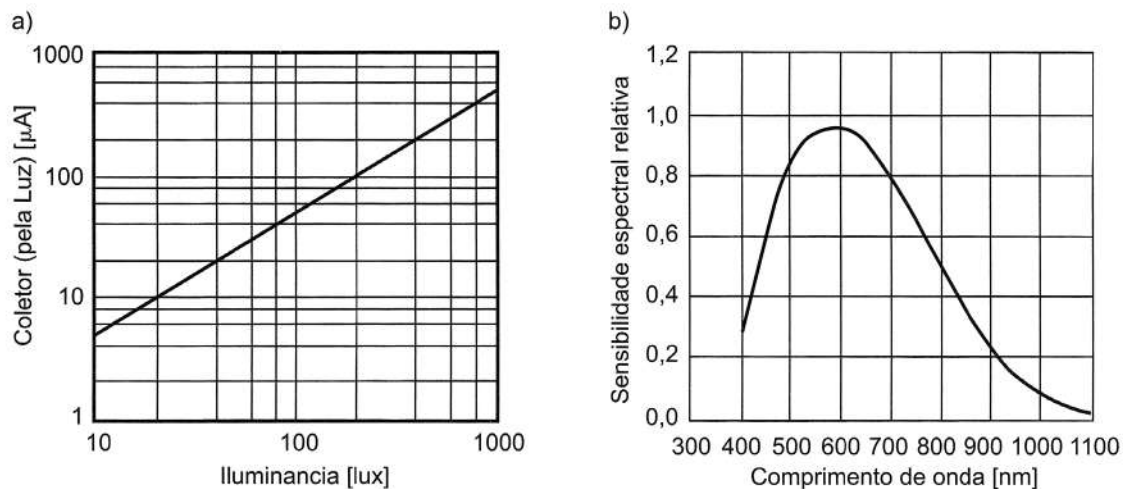


Figura 2.27 Sensor TEMT6000 de luz ambiente e sua conexão.



Gráfico 2.6 Curvas características do sensor TEMA6000

Fonte: Adaptado de Vishay, 2011.

A Tabela 2.8 exemplifica os diferentes níveis de iluminação para entendermos melhor os valores dos Gráficos 2.6a e b.

Tabela 2.8 Exemplos de iluminação

Iluminância	Exemplo
0,002 lux	Noite clara sem luar
0,2 lux	Nível mínimo de luz de emergência - norma
0,27-1,0 lux	Noite clara com lua cheia
34 lux	Crepúsculo sob um céu limpo
50 lux	Luz de uma sala de estar familiar
100 lux	Dia muito nublado
300-500 lux	Nascer ou pôr do sol ou luz de escritório
1000 lux	Dia nublado ou iluminação de estúdio
10.000-25.000 lux	Luz de um dia ensolarado (não sol direto)
32.000-130.000 lux	Luz solar direta

SENSOR DIGITAL DE LUZ TSL2561

Diferentemente dos tradicionais sensores de luz analógicos, o TSL2561 é um módulo com comunicação digital I2C com dois sensores: um infravermelho e outro de espectro completo (Figura 2.28).

Principais características:

- Modos de detecção selecionáveis.
- Resolução de 16 bits de saída.

- 400 kHz de comunicação serial I²C.
- Faixa de leitura entre 0,1 e 40.000 lux.
- Faixa de operação entre 40 °C e 85 °C.

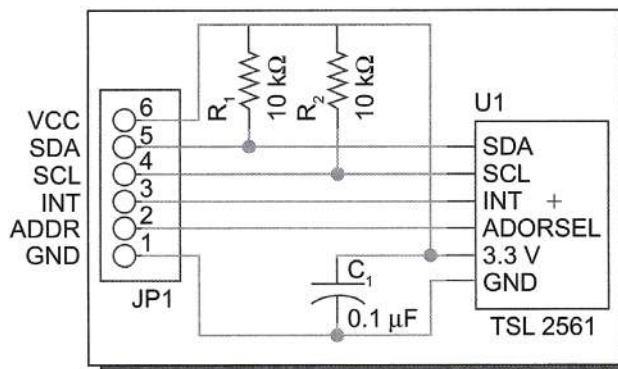


Figura 2.28 Sensor digital de luz TSL2561.

SENSOR DIGITAL DE LUZ VCNL4000

O sensor VCNL4000 é uma combinação entre um sensor de infravermelho de proximidade e um sensor de luz ambiente. Usando a luz IR, o sensor pode detectar objetos em uma região de aproximadamente 20 cm. A comunicação acontece pela interface serial I²C.

A Figura 2.29 ilustra uma interface comercialmente conhecida desse sensor.

Os Gráficos 2.7a, b e c foram obtidos para o sensor em questão, nomeadamente: a) espectro do sinal absorvido pelo sensor; b) correlação entre o sinal elétrico e a intensidade de luz (iluminância); e c) responsividade relativa do sensor.

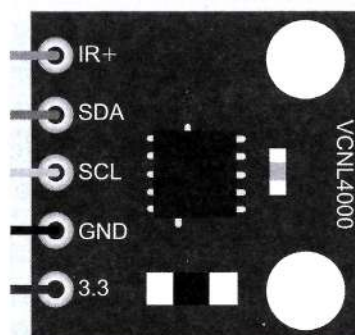
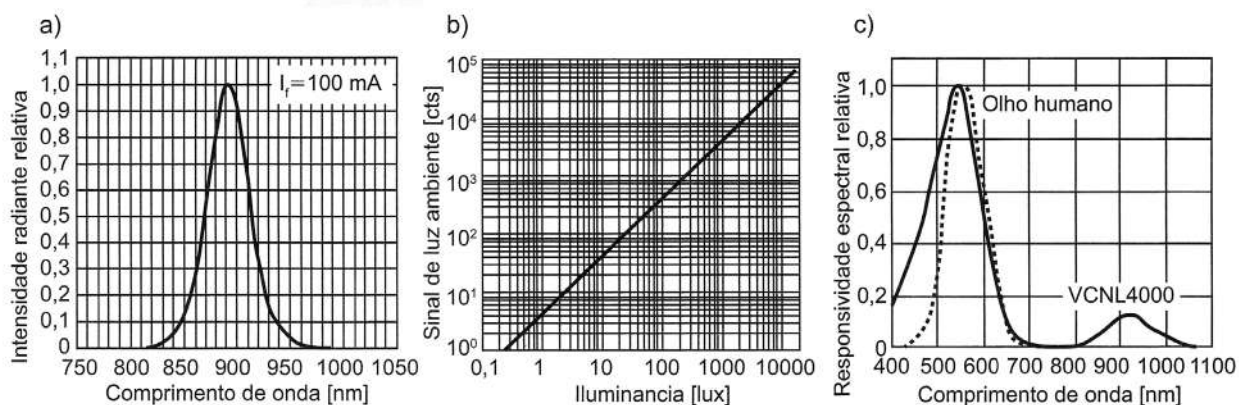


Figura 2.29 Sensor VCNL4000.

Gráfico 2.7 Curvas características do sensor VCNL4000



SENSOR DE INTENSIDADE LUMINOSA BH1750

O sensor de intensidade de luz BH1750 apresenta um conversor analógico-digital de 16 bits e entrega esse valor diretamente em sua saída digital. É alimentado com tensão entre 3 e 5 V_{DC}. Possui ampla faixa de medição e alta resolução (1 a 65.535 lux). A Figura 2.30 ilustra o sensor BH1750 e sua resposta espectral.

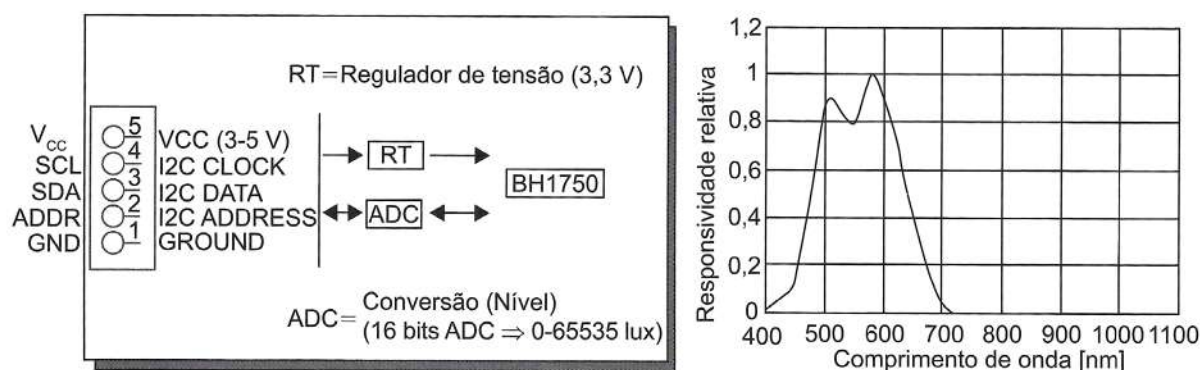


Figura 2.30 Sensor de luz BH1750 e seu espectro de sensibilidade.

SENSOR DE INTENSIDADE DE LUZ TSL230R

O sensor TSL230, ilustrado na Figura 2.31, é um sensor que converte luz (que estimula corrente elétrica) em frequência. A saída pode ser um trem de pulso ou uma onda quadrada (50% ciclo de trabalho) com frequência diretamente proporcional à intensidade da luz. A sensibilidade do dispositivo é selecionável em três faixas, proporcionando duas décadas de ajuste (ou seja, sensibilidade de 1x, 10x ou 100x).

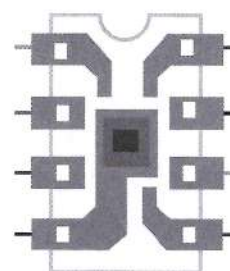


Figura 2.31
Sensor TSL230R.

A Figura 2.32 ilustra as curvas características: à esquerda, a frequência do sinal de saída em função da quantidade de iluminação em sua face, e à direita, o espectro de absorção, com o pico de responsividade próximo a 780 nm.

Relação dos terminais do sensor TSL230R

Terminal		Descrição
Nome N.º		
GND	4	Terra/referência
/OE	3	Habilitar para fo
OUT	6	Sinal periódico de saída (fo)
S0,S1	1,2	Entradas de sensibilidade
S2,S3	7,8	Entradas de escala fo
VDD	5	Alimentação

Relações de ajuste/configuração

S1	S0	Sensibilidade	S3	S2	Escala para f _o
L	L	Desligado	L	L	1
L	H	1x	L	H	2
H	L	10x	H	L	10
H	H	100x	H	H	100

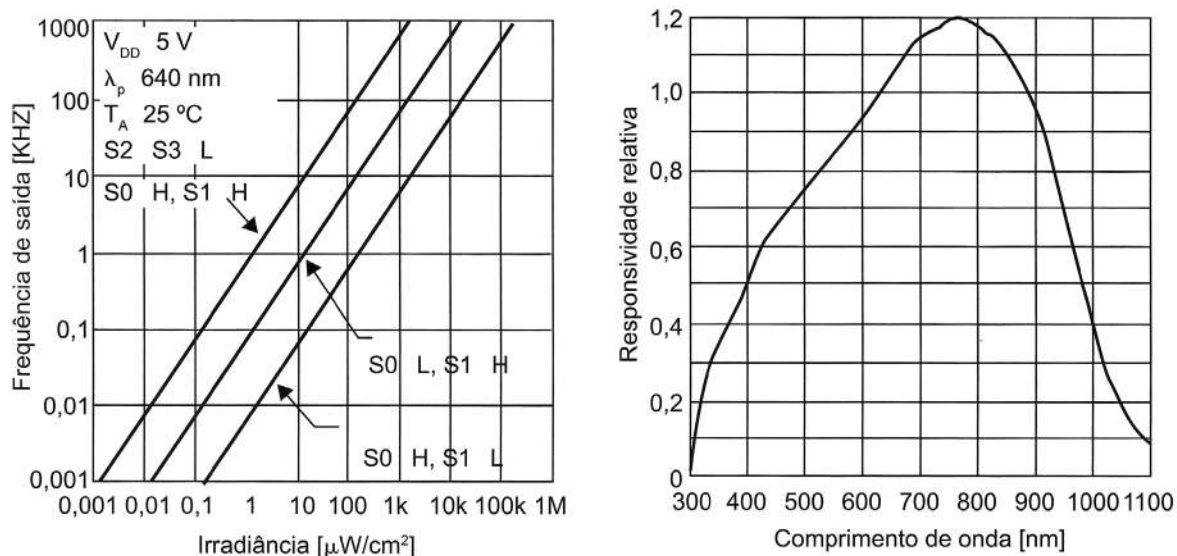


Figura 2.32 Curvas características do sensor TSL230R.

SENSOR DE CORES (TCS3200)

O sensor TCS3200 é um sensor de cores que possui um conjunto de 64 fotodiodos para detecção de cores: 16 com filtro para a frequência de cor vermelha (Red), 16 com filtro para a frequência de cor verde (Green), 16 com filtro para a frequência de cor azul (Blue), e ainda 16 fotodiodos sem filtro de cores (ver Figura 2.33).

O TCS230/3200 possui quatro entradas: S0, S1, S2 e S3. As duas primeiras servem para selecionar uma escala de frequência (2%, 20% ou 100%), ou simplesmente desligar o sensor. As duas últimas servem para selecionar o conjunto de fotodiodos cujo resultado você quer: como são dois bits, teremos exatamente quatro possibilidades, que são correspondentes aos quatro conjuntos de fotodiodos. Assim, o sensor dará um resultado com a intensidade de cada cor que ele consegue medir e a intensidade total, permitindo o desenvolvimento de uma aplicação que calcule o valor RGB para a cor detectada.

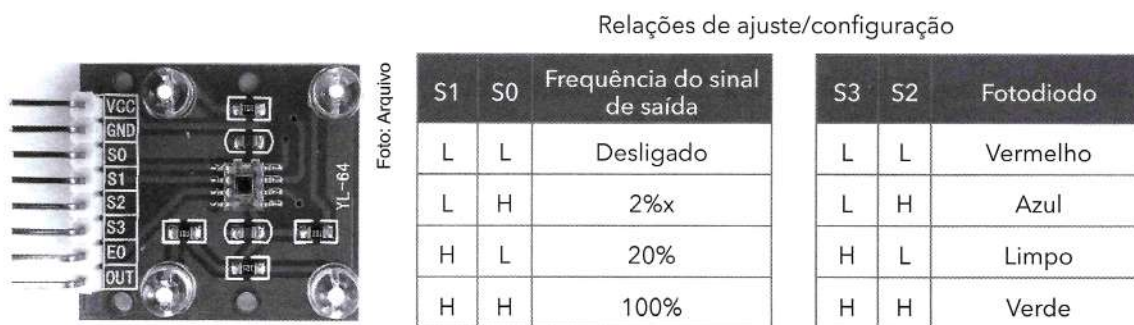


Figura 2.33 Sensor TSL3200.

Ao aproximar uma folha branca de papel desse sensor se vê como ele marca perfeitamente: [255,255,255] no Serial Monitor. Outras cores ficarão aproximadas, e o



preto é bastante difícil de detectar. Cores mais "claras" são mais fáceis por terem uma **saturação** maior e também maior **intensidade** de luz.

SENSORES INFRAVERMELHOS

O TIL32 é um led que emite luz infravermelha (IR), com comprimento de onda de 940 nm. Esse parâmetro é importante, pois o detector precisa trabalhar na mesma frequência. Sua aparência é idêntica à um led de luz visível, sendo encontrado com encapsulamento transparente ou azulado. Esse led possui uma queda de tensão entre 1,3 e 1,7 V, e deve-se ponderar o resistor para que a corrente máxima seja de 40 mA. A Figura 2.34 ilustra a polarização desse led com um resistor de 220 Ω fornecendo uma corrente de aproximadamente 15 mA.

O TIL78 e o SFH203 são exemplos de fototransistor (Gráficos 2.8a e b). Ele possui dois terminais, correspondendo ao coletor e ao emissor do transistor, e a base é ativada por luz.

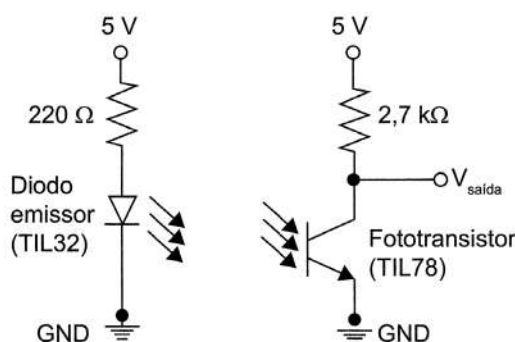
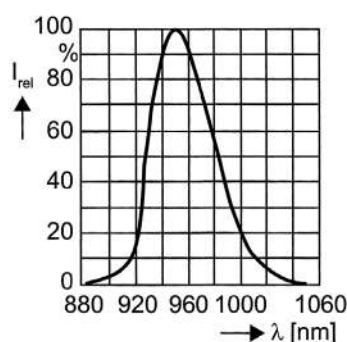


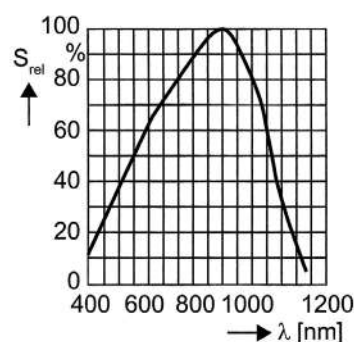
Figura 2.34 Polarização de led transmissor (TIL32) e fototransistor receptor IR (TIL78).

Gráfico 2.8 Diagrama espectral: a) do led transmissor; b) do transistor receptor

a)
Emissão espectral relativa $I_{rel} f(\lambda)$



b)
Sensibilidade espectral relativa $S_{rel} f(\lambda)$



Na presença de luz, o transistor conduz, e quanto mais luz, maior a condução do mesmo. Com uma fonte luminosa muito próxima dele (por exemplo no TIL32 a menos de 10 cm), ele apresenta uma saída de 0,2 V, ao passo que sem luz o transistor não conduz e a saída está em 5 V.

Um outro encapsulamento comum de um sensor IR é o que possui o transmissor e o receptor em um único invólucro, separados por um anteparo. Esse sensor é muito utilizado para identificação de proximidade de alvos, como ilustrado pela Figura 2.35.

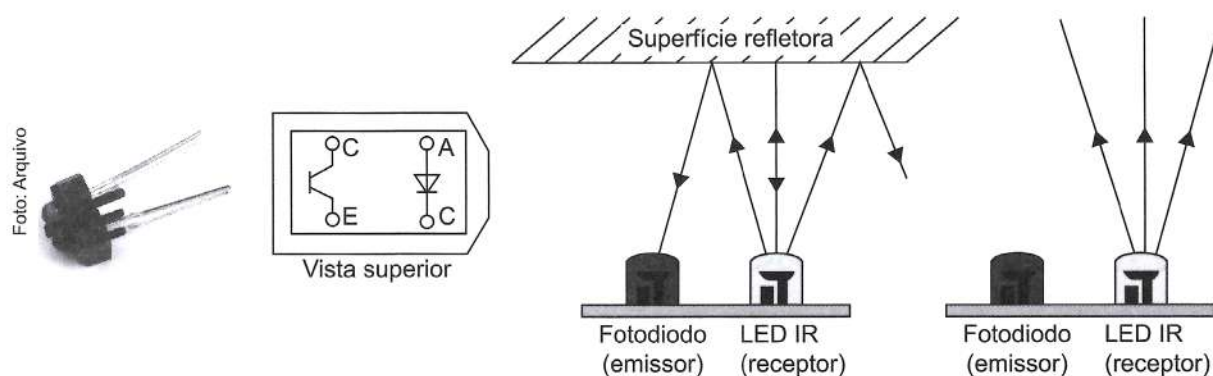


Figura 2.35 Sensor IR. Invólucro e funcionamento.

SENSOR PIR (INFRAVERMELHO PIROELÉTRICO) DE PRESENÇA

Um sensor muito utilizado é o sensor pirométrico infravermelho, baseado no sensor LHI778. Comercialmente, vem com os circuitos de condicionamento apresentando-se pronto para uso. Os shields prontos, DYP-ME003 e HC-SR501 só precisam ser alimentados com tensão $5 V_{DC}$ (tensão de alimentação entre 5 e 20 V), e ter os potenciômetros de tempo de atraso de ativação e sensibilidade (distância de ativação) ajustados. A Figura 2.36 ilustra o sensor em seu encapsulamento comercial.

Com relação à sensibilidade, o sensor consegue detectar o movimento de objetos quentes a uma distância de até 7 metros.

Quando um objeto é identificado, com um movimento por um tempo maior que o tempo de retardo, ajustado pelo potenciômetro de tempo (entre 2 s e 200 s), a saída (dados) apresenta nível lógico alto (3,3 V) e fica bloqueada por 2,5 s; caso contrário, apresenta nível lógico baixo (0 V).



Figura 2.36 Sensor de presença pirométrico IR.

Principais características do DYP-ME003:

- **Tensão de dados:** 3,3 V (Alto) – 0 V (Baixo).
- **Distância detectável:** 3-7 m (Ajustável).
- **Tempo de atraso (delay):** 5-200 s (Padrão: 5 s).
- **Tempo de bloqueio:** 2,5 s (Padrão).
- **Temperatura de trabalho:** –20 °C ~ +80 °C.

ACELERÔMETROS E GIROSCÓPIOS

Acelerômetros são dispositivos utilizados para medir aceleração e inclinação, enquanto giroscópios são usados para medir a velocidade angular e a orientação. A combinação desses dois sensores é conhecida por IMU (unidade de medida inercial), e proporciona um conhecimento completo de aceleração, velocidade, posição e orientação de um objeto.

Tipicamente, os acelerômetros são constituídos por uma massa de reação suspensa por uma estrutura estacionária. Sempre que o acelerômetro acelera, a inércia faz com que a massa resista, e descompensações são normalmente medidas por meios capacitivos, resistivos ou piezoelétricos.

Os acelerômetros capacitivos baseiam-se no princípio das placas paralelas, em que uma aceleração provoca variações de posição de uma placa móvel, resultando em alterações de capacitância.

Nos acelerômetros piezoelétricos, a massa é unida a um cristal piezoelétrico. Quando o corpo do acelerômetro é sujeito a vibração, a massa obedece às leis da inércia e o cristal piezoelétrico fica submetido a forças de tração e compressão, que por sua vez são proporcionais à aceleração (de acordo com a Lei de Newton, $F = m \cdot a$).

Se substituirmos os cristais piezoelétricos acima por um componente piezorresistivo, podemos ter em uma aceleração a força exercida pela massa variando a resistência, que é detectada por uma ponte de Wheatstone.

A última classe importante é a dos acelerômetros de microestruturas mecânicas (MEMS), que são microestruturas que também operam de acordo com a Lei de Newton.

Entretanto, existem muitos outros tipos de acelerômetros, como por exemplos os: baseados em efeito Hall; em transferência de calor; em strain gauge; em ressonância; ópticos, acústicos etc.

A saída de um acelerômetro é um valor escalar correspondente à magnitude de um vetor de aceleração. São normalmente referenciadas com a aceleração da gravidade g ($g = 9,8 \text{ m/s}^2$).

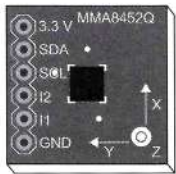
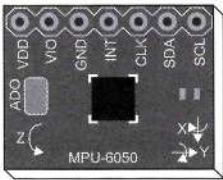
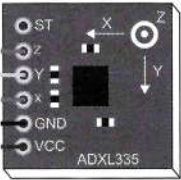
Podem possuir saídas digitais ou analógicas. Saídas digitais normalmente utilizam os protocolos seriais I²C ou SPI. Já as analógicas abrem mão do uso dos conversores analógico-digitais dos microcontroladores para analisarem o dado disponível em seus terminais.

Giroscópios medem a velocidade angular (em 1, 2 ou 3 eixos) e são utilizados para ajudar na estabilização ou informar alterações na direção e orientação. Ao contrário dos acelerômetros, giroscópios não têm uma referência fixa, e só são aptos a medir mudanças de estado.

Basicamente, acelerômetros podem operar em 1, 2 ou 3 eixos, assim como os giroscópios (1, 2 ou 3 eixos).

A Tabela 2.9 ilustra alguns acelerômetros.

Tabela 2.9 Exemplos de acelerômetros disponíveis comercialmente

Identificação	MMA8452	MMA7361	MPU-6050	ADXL3xx
Imagem				
Funções	Acelerômetro Digital	Acelerômetro Analógico	Acelerômetro Giroscópio Digital	Acelerômetro Analógico
Tensão de Alimentação	1,95 a 3,6 V	3,3 V ou 5 V	3 a 5 V	5 V
Resolução	12 e 8 bits	-	16	-
Escala Aceleração	$\pm 2 \text{ g} / \pm 4 \text{ g} / \pm 8 \text{ g}$	1,5 g ou 6 g	$\pm 2 \pm 4 \pm 8 \pm 16 \text{ g}$	1,5 g ou 6 g
Escala giroscópio	-	-	+ 250 500 1000 2000/s	-
Eixos	3	3	3+3	3

GY-80 (Figura 2.37) é uma IMU (*Inertial Measurement Unit*) que conta com 10 graus de liberdade, sendo 3 eixos do Giroscópio L3G4200D, 3 eixos do Acelerômetro ADXL345, 3 eixos do Magnetômetro HMC5883L e mais o sensor de pressão e temperatura BMP085.

O Magnetômetro HMC5883L mede o campo magnético e funciona como uma bússola digital, podendo ser usado por exemplo com o Arduino para indicar o norte geográfico da Terra.

O Giroscópio L3G4200D tem capacidade para 3 eixos, bem como 3 níveis de sensibilidade. Os dados das velocidades angulares podem ser obtidos através da comunicação I²C.

O Acelerômetro de 3 eixos ADXL345 possui alta resolução com baixo consumo de energia. Os dados também são obtidos por comunicação I²C.



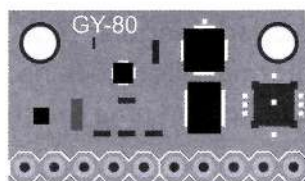


Figura 2.37 Sensor GY-80. Unidade de medida inercial com giroscópio, acelerômetro, magnetômetro e barômetro.

Especificações:

Protocolo de comunicação: I2C.

Tensão de operação: 3,3-5 V.

- | | |
|--|---------------------------------|
| ■ Acelerômetro: ADXL345 | ■ Magnetômetro: HMC5883L |
| Endereço I2C: 0x53 | Endereço I ² C: 0x1E |
| Faixa do Acelerômetro: ± 2 , ± 4 ,
± 8 , ± 16 g | ■ Barômetro: BMP085 |
| ■ Giroscópio: L3G4200D | Endereço I ² C: 0x77 |
| Endereço I ² C: 0x69 | |
| Faixa do Giroscópio: ± 250 , 500,
2000°/s | |

SENSORES DE SOM (MICROFONE)

Um microfone de eletreto é um tipo de microfone condensador de alta sensibilidade e resposta em frequência larga e aproximadamente constante na faixa audível pelo ser humano que elimina a necessidade de uma fonte de tensão para se polarizar. Baseia-se num capacitor de placas paralelas, energizado por uma fonte de corrente contínua. Sob pressão sonora, o diafragma movimenta uma das placas em relação à outra, alterando a capacitância e a corrente no circuito.

As características principais são:

- **Resposta em frequência:** 30 Hz a 8 kHz.
- **Sensibilidade:** -42 dB (V/Pa) a 1 kHz.
- **Impedância:** 1000 Ω .
- **Alimentação:** 2 a 8 V.

A Figura 2.38 apresenta alguns circuitos com o estágio de amplificação para utilização de microfones de eletreto.

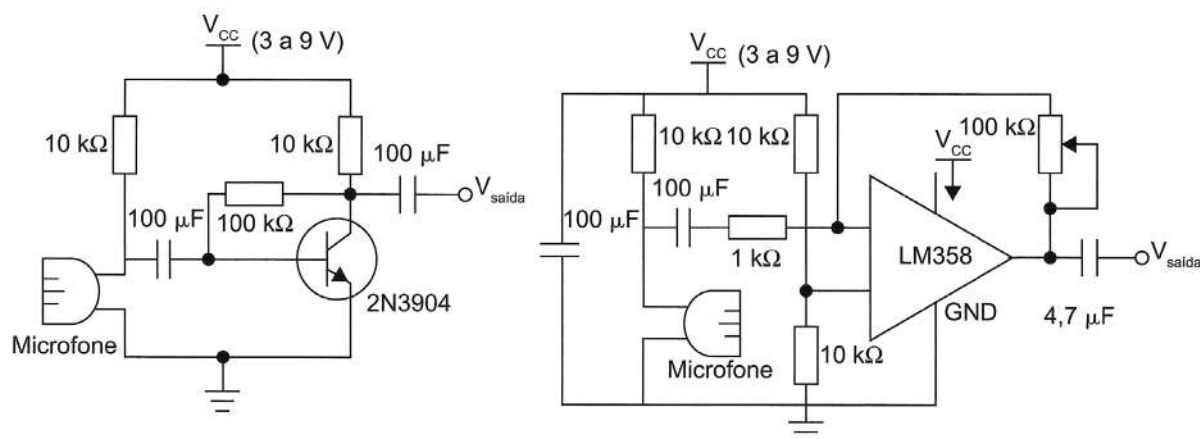


Figura 2.38 Exemplos de circuitos para microfones de eletreto.

Alguns módulos comerciais apresentam os circuitos de amplificação e comparação já implantados. O módulo KY-038 (Figura 2.39) é um exemplo desses módulos, o qual é alimentado entre 3 e 24 V; possui duas saídas, uma analógica (A0) e outra digital (D0). Para a saída digital, possui um comparador com o CI LM393, com regulagem de limiar pelo resistor variável.

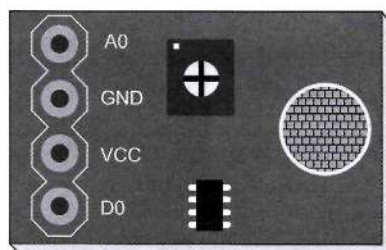


Figura 2.39 Módulo sensor de som.

CARACTERIZAÇÃO DE SENSORES

Um sensor é caracterizado em função das diversas características que ele apresenta. Conhecer a resposta de um sensor é importantíssimo para que as ações tomadas em função dessas respostas as considerem.

A seguir são descritas algumas das principais características que devemos conhecer para um sensor.

REGIÃO DE OPERAÇÃO (RANGE)

Também conhecida como zona detectável ou região de atuação, representa a faixa de valores da grandeza que se deseja medir e para a qual o dispositivo consegue efetivamente



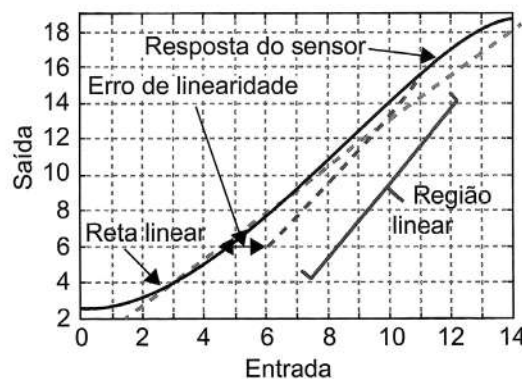
operar. Normalmente é relacionada com a região linear do transdutor, porém deve-se considerar outros limitantes, como integridade física do material, detalhes construtivos etc.

Há que se considerar que alguns sensores possuem uma zona cega, que é a região dentro da distância sensora na qual o sensor, por questões tecnológicas ou de montagem, não consegue detectar o objeto. Portanto, nesse caso, a região de sensibilidade é a região da zona detectável em que o dispositivo é efetivamente sensibilizado.

LINEARIDADE

A linearidade de um sensor refere-se à aproximação da curva de resposta de uma dada entrada a uma reta de aproximação. Normalmente, os sensores são lineares em apenas certas faixas restritas de operação, como exemplificado no Gráfico 2.9.

Gráfico 2.9 Exemplo de resposta de sensor não linear e de faixa linear de operação



Quando se deseja utilizar sensores não lineares, deve-se ponderar o uso de técnicas de linearização para facilitar o modo de controle sobre o sensor.

Um bom exemplo refere-se aos sensores de temperatura do tipo NTC (exponenciais), que, com o auxílio de amplificadores logarítmicos, têm sua resposta linearizada.

Entretanto, é interessante analisar o comportamento do sensor e o vir operar em uma faixa de linearidade para poder simplificar os circuitos eletrônicos de compensação de não linearidade e reduzir as fontes de erros.

RESOLUÇÃO

A resolução define o menor incremento da variável medida que produz uma variação na saída do sensor.

VELOCIDADE DE RESPOSTA

Define o tempo de reação de um sensor a variações na entrada.

HISTERESE

Refere-se ao fato de uma resposta a um estímulo crescente ser diferente à resposta ao mesmo estímulo decrescente, conforme ilustrado no Gráfico 2.10.

Para sensores comerciais, essa excursão diferente refletirá em diferença entre a distância na qual o sensor é ativado quando o objeto se aproxima dele e a distância na qual o sensor é desativado quando o objeto se afasta dele (ver Figura 2.40). Normalmente é dada na forma percentual.

Gráfico 2.10 Característica de histerese de um sensor

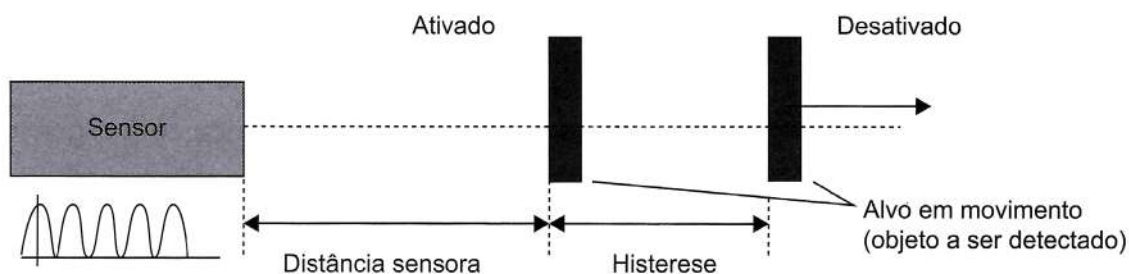
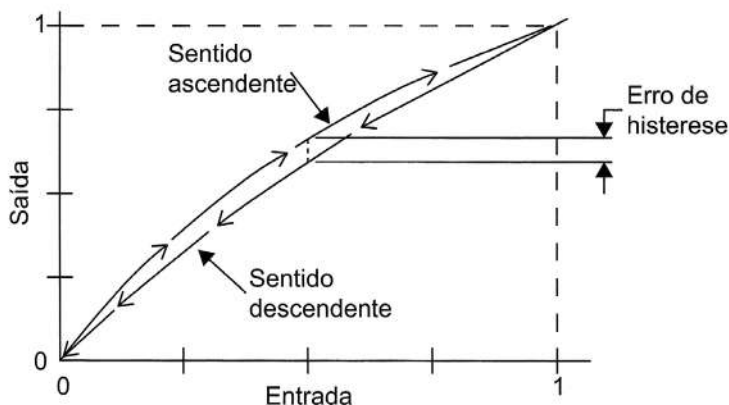


Figura 2.40 Característica de histerese de um sensor comercial.

REPETIBILIDADE

A repetibilidade refere-se às diferenças de leituras obtidas na saída para o mesmo valor de grandeza de entrada, quando esse sinal varia na mesma direção. Para sensores comerciais, o parâmetro de repetibilidade indica a pequena variação na distância sensora quando se procedem a duas ou mais tentativas de detecção.



CAPÍTULO 03

Muitas aplicações necessitam de medições em ambientes ou estruturas, como temperatura e vibração, realizadas a partir de sensores. Esses sensores, por sua vez, necessitam de condicionamento de sinal para que o dispositivo de aquisição de dados e controle efetue a medição de forma eficaz e exata. Assim, um circuito de condicionamento deve atender tanto às características do dispositivo sensor quanto às necessidades do controlador, que precisa ter uma grandeza elétrica em determinadas condições de amplitude e frequência para que possam desempenhar adequadamente suas funções.

Considerando que nosso dispositivo de controle focado aqui é um sistema microcontrolado baseado na plataforma Arduino, as características do sinal entregue a ele devem considerar as características de leitura de portas digitais e analógicas do microcontrolador ATMEGA 328. Ou seja, para sinais digitais, considerando $V_{CC} = 5\text{ V}$, a amplitude de um nível lógico baixo deve estar entre $-0,5\text{ V}$ e $1,5\text{ V}$ ($\text{GND}-0,5\text{ V}$ e $0,3 \cdot V_{CC}$); e para um nível lógico alto entre $3,5\text{ V}$ e $5,5\text{ V}$ ($0,7 \cdot V_{CC}$ e $V_{CC} + 0,5$).

Já para sinais analógicos, é importante conhecer as características do conversor analógico digital disponível no microcontrolador. No Arduino, tem-se presente um conversor analógico digital de aproximações sucessivas de 10 bits ($2^{10} = 1024$ níveis). Ou seja, a resolução padrão para $V_{CC} = 5\text{ V}$ é de: $(5-0)/1023 = 4,88\text{ mV}$. Em muitas situações, uma sensibilidade de $4,88\text{ mV}$ é muito baixa para se analisar o sinal direto do sensor, sendo necessário um circuito de condicionamento de sinais para que a sensibilidade seja adequada às restrições de operação do conversor analógico digital (ADC) utilizado. Outra questão com relação a esse conversor refere-se à sua velocidade de resposta, uma vez que a frequência de operação do conversor está entre 50 e 200 kHz, necessitando da configuração de uma pré-escala. O conversor inserido na família ATMEGA (Arduino), segundo o fabricante, demora entre 13 e até $260\text{ }\mu\text{s}$, dependendo das suas configurações de resolução desejadas. Entretanto, utilizando um cristal de 16 MHz (padrão nas placas Arduino), e considerando que a conversão demora aproximadamente 13 ciclos para terminar, teremos uma quantidade máxima de $16\text{M}/128/13 = 9600$ conversões por segundo, o que restringe a gama de frequência dos sinais analógicos a serem analisados.

Considerando esta breve introdução, vamos abordar os principais tipos de condicionamento de sinais necessários para os sensores apresentados no capítulo anterior.

Entre tantas técnicas de condicionamento de sinais, as principais são:

- Amplificação.
- Atenuação.
- Isolação.
- Filtragem.
- Excitação.
- Linearização.
- Compensação de junção fria.
- Configurações potenciométricas.
- Retificadores.
- Correção de nível DC (offset).



- Detectores de pico.
- Conversão de corrente em tensão.
- Conversão de tensão em corrente.
- Técnicas de condicionamento para sensores capacitivos.

AMPLIFICAÇÃO

Os amplificadores são circuitos que aumentam o nível de tensão de um sinal. São utilizados, nesse caso, para adequar o nível máximo de tensão de um sensor para que ele esteja o mais próximo da faixa de operação do conversor analógico digital, aumentando assim a sensibilidade de medição.

Um sistema de amplificação benfeito e posicionado mais próximo do sensor possibilita uma melhoria na relação sinal-ruído, melhorando a eficiência do processo.

Amplificadores Operacionais (AmpOps) são circuitos integrados que nos possibilitam realizar operações matemáticas com grande facilidade e poucos componentes externos. Diferentes configurações podem ser utilizadas em função do sinal proveniente do sensor e da aplicação em si.

AMPLIFICADOR NÃO INVERSOR

Nesse caso, a saída do amplificador apresenta ganho constante e saída de mesmo sinal, determinada por:

$$V_{saída} = \left(\frac{R_1 + R_2}{R_1} \right) \cdot V_{entrada} \quad (3.1)$$

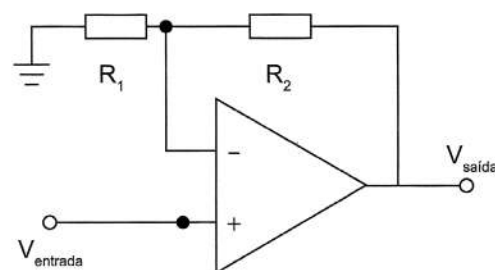


Figura 3.1 Amplificador não inversor.

AMPLIFICADOR INVERSOR

O amplificador inversor é a configuração mais utilizada com ganho constante, apresentando-se mais estável que a versão não inversora. Nesse caso, a saída do amplificador é determinada por:

$$V_{saída} = - \left(\frac{R_2}{R_1} \right) \cdot V_{in} \quad (3.2)$$

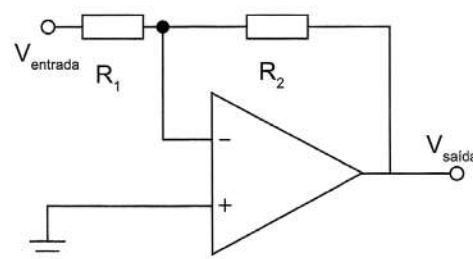


Figura 3.2 Amplificador inversor.



AMPLIFICADOR SOMADOR INVERSOR

Essa configuração fornece uma forma de somar algebricamente várias tensões (neste exemplo estão ilustradas 4 entradas de tensão).

Dessa forma, a tensão de saída pode ser determinada por:

$$V_{\text{saída}} = -R_f \left(\frac{V_1}{R_1} + \frac{V_2}{R_2} + \frac{V_3}{R_3} + \frac{V_4}{R_4} \right) \quad (3.3)$$

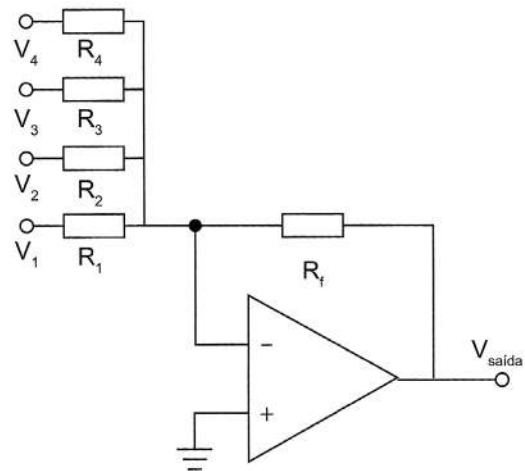


Figura 3.3 Amplificador somador inversor.

AMPLIFICADOR DIFERENCIAL (SUBTRATOR)

Nesse caso, a tensão de saída ($V_{\text{saída}}$) é igual à diferença entre os dois sinais de entrada (V_1 e V_2), multiplicados por um ganho.

Dessa forma, considerando

$$R_1 = R_2 \text{ e } R_g = R_f$$

a saída é dada por:

$$V_{\text{saída}} = \frac{R_f}{R_1} (V_2 - V_1) \quad (3.4)$$

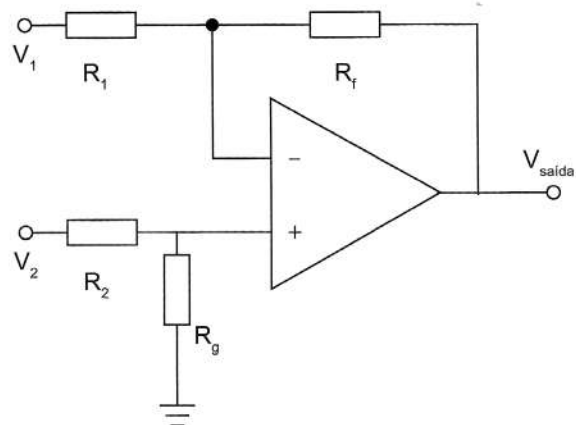


Figura 3.4 Amplificador subtrator.

O amplificador integrador e o diferenciador são circuitos que simulam os operadores matemáticos de integral e derivada, respectivamente. Além disso, são usados para modificar formas de onda, gerando pulsos, ondas quadradas, ondas triangulares etc.

AMPLIFICADOR INTEGRADOR

Nesse caso, a tensão de saída é proporcional à integral da tensão de entrada.

$$V_{\text{saída}} = -\frac{1}{R_1 C_1} \int V_{\text{entrada}} dt \quad (3.5)$$

Por exemplo, se a entrada for uma tensão constante, a saída será uma rampa. Se for uma tensão positiva, a rampa será descendente (inclinação negativa), e se for uma tensão negativa, a rampa será ascendente (inclinação positiva).

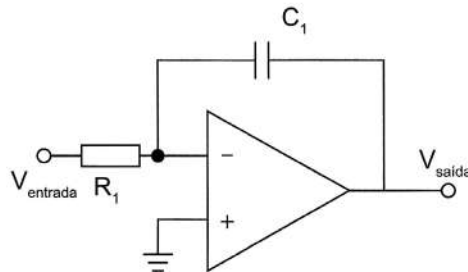


Figura 3.5 Amplificador somador inversor.

Na prática, o circuito da Figura 3.5 apresenta problemas de saturação, uma vez que ele não possui realimentação em Corrente Contínua (CC) (capacitor é circuito aberto em CC). Dessa forma, em CC o ganho do amplificador é muito alto, fazendo o AmpOp saturar mesmo com tensões da ordem de mV. A solução é reduzir o ganho em CC colocando em paralelo com o capacitor C_1 um resistor (R_p). Como resultado, o AmpOp se comportará como integrador para frequências acima da frequência de corte f_c , determinada por $f_c = \frac{1}{(2\pi \cdot R_p \cdot C_1)}$.

Abaixo de f_c o circuito se comporta como amplificador inversor de ganho igual a $-R_p/R_1$. A Figura 3.6 ilustra o comportamento do ganho em função da frequência e das tensões de entrada e saída em função do tempo, para frequências maiores e menores que a frequência de corte.

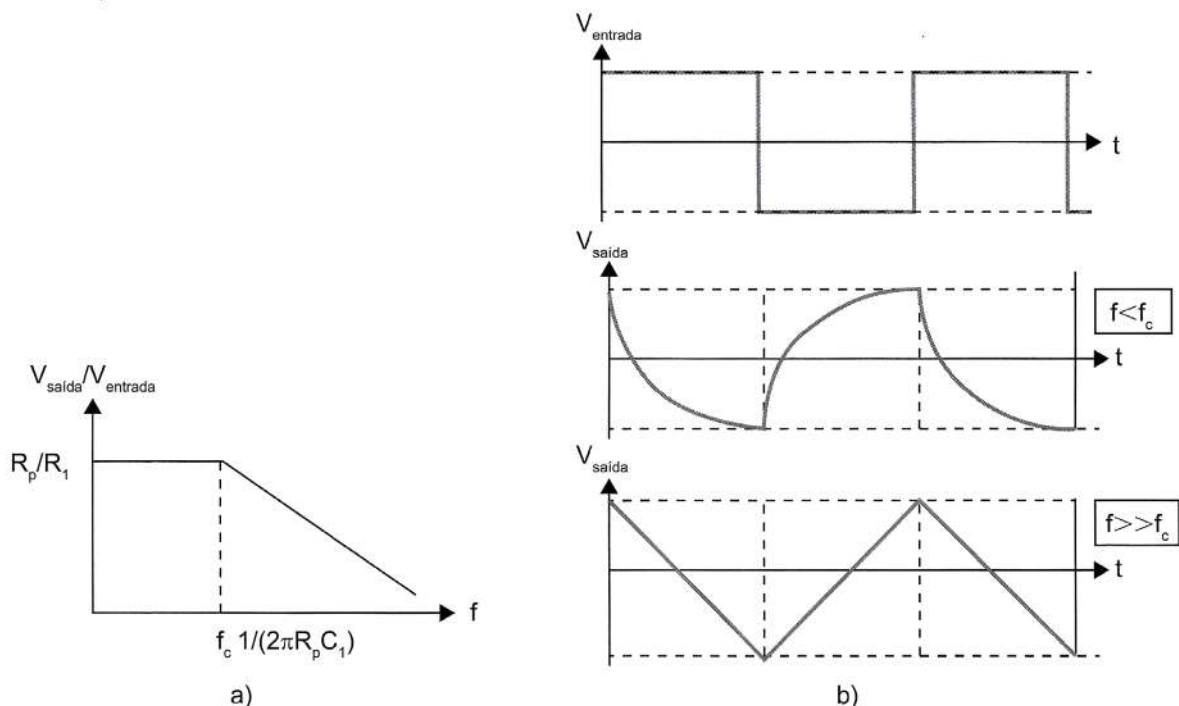


Figura 3.6 a) Resposta de ganho em função da frequência de um amplificador integrador; b) resposta temporal da entrada e saída, em função da frequência de operação $f < f_c$ e $f > f_c$.



AMPLIFICADOR DIFERENCIADOR

A derivada é um operador dual da integral, e em um circuito eletrônico os componentes trocam de posição. A tensão de saída é proporcional à derivada da tensão de entrada. Por exemplo, se a entrada for uma tensão constante a saída será nula, pois a derivada de uma constante é zero.

$$V_{\text{saída}} = -R_1 C_1 \frac{dV_{\text{entrada}}}{dt} \quad (3.6)$$

Na prática, o circuito da Figura 3.7 é sensível a ruído, tendendo a saturar. Para diminuir esse inconveniente, a solução é limitar o ganho em altas frequências colocando em série com C_1 uma resistência R_S

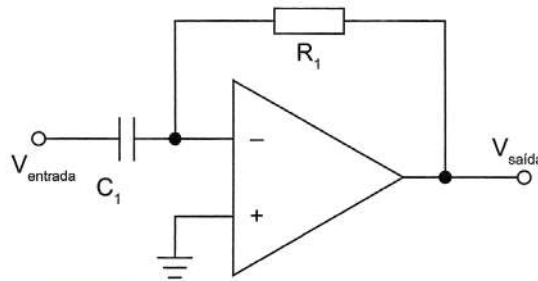


Figura 3.7 Amplificador diferenciador.

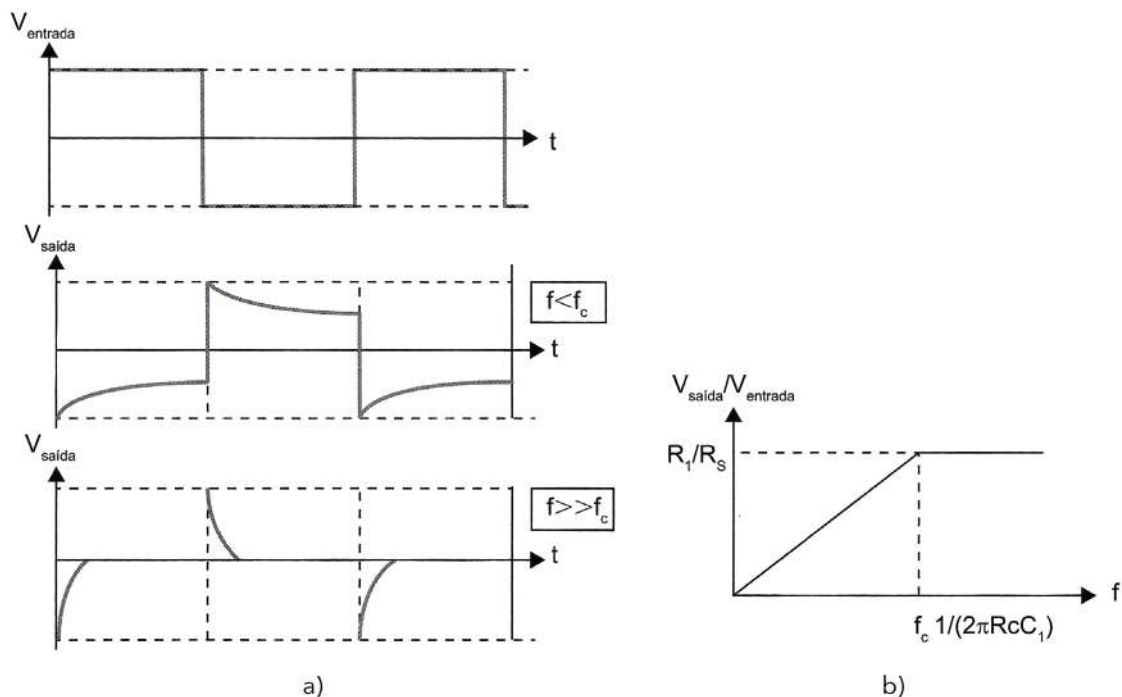


Figura 3.8 a) Resposta temporal do amplificador diferenciador;
b) resposta em frequência do amplificador diferenciador.

O circuito só se comportará como diferenciador se $f \ll f_c$, pois nessas condições a reatância de C será muito maior do que R_S e na prática é como se não existisse R_S .

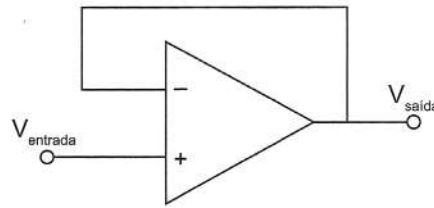


Figura 3.10 Amplificador unitário – seguidor de tensão.

ATENUAÇÃO

A atenuação é o oposto da amplificação. É necessária quando as tensões a serem digitalizadas estão acima da tensão do conversor. Essa forma de condicionamento diminui a amplitude do sinal de entrada, de forma que o sinal condicionado fique dentro da faixa do ADC.

Muitas vezes, apenas divisores de tensão são utilizados, porém podemos utilizar a configuração dos amplificadores inversor e não inversor, com $R_2 > R_1$, e dessa forma obteremos uma relação entre os resistores menor que 1, resultando em atenuação do sinal.

ISOLAÇÃO

Os dispositivos com isolação passam o sinal da fonte para o dispositivo de medição sem uma conexão física, sendo usadas técnicas como, por exemplo, por meio de transformadores e acopladores capacitivos ou ópticos. Além de evitar loops de terra, a isolação bloqueia surtos de alta tensão e rejeita tensão elevada em geral, protegendo assim os operadores e o caro equipamento de medição.

FILTROS

Os filtros rejeitam ruídos indesejados dentro de uma determinada faixa de frequência. Normalmente, filtros passa-baixa são usados para bloquear ruídos de alta frequência em medições elétricas, como uma tensão com frequência de 60 Hz.

Por outro lado, filtros passa-alta são necessários para evitar ruídos do ambiente e ruídos oriundos da rede elétrica (60 Hz).

A combinação de um filtro passa-alta com um filtro passa-baixa resulta em um filtro passa-faixa. Por outro lado, podemos projetar um filtro para eliminar exclusivamente uma faixa de frequência, nesse caso, um filtro rejeita-faixa.

Sinais AC, como vibração, geralmente requerem um tipo diferente de filtro, conhecido como filtro anti-aliasing. O filtro anti-aliasing é um filtro passa-baixa que requer

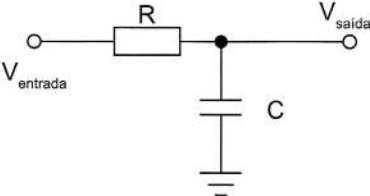
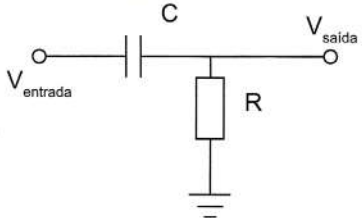
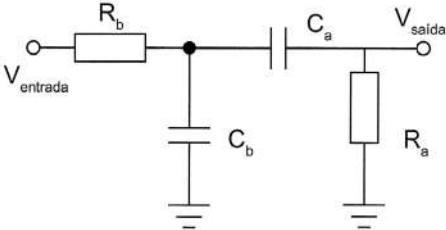


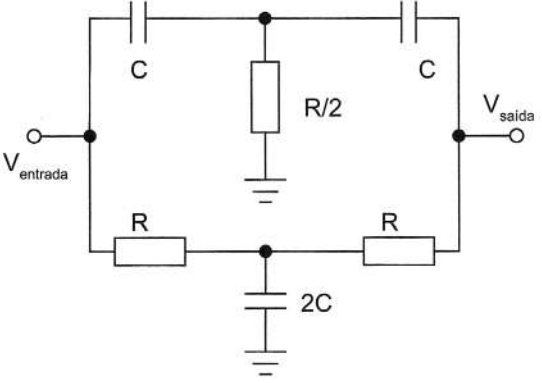
uma taxa de corte muito alta, e geralmente remove completamente todas as frequências do sinal que são maiores que a largura de banda de entrada do equipamento. Se esses sinais não forem removidos, eles irão aparecer erroneamente com os sinais da largura de banda de entrada do equipamento.

Os filtros podem ser passivos (utilizando componentes discretos: resistores, capacitores e indutores) ou ativos (baseados normalmente em amplificadores operacionais).

A seguir, serão apresentados alguns exemplos de circuitos de filtragem.

FILTROS PASSIVOS

Tipo de filtro	Cálculo da frequência de corte
Filtro passa-baixa  Figura 3.11 Filtro passivo passa-baixa.	Frequência de corte do filtro passa-baixa (f_c): $f_c = \frac{1}{2\pi RC} \quad (3.9)$
Filtro passa-alta  Figura 3.12 Filtro passivo passa-alta.	Frequência de corte do filtro passa-alta (f_c): $f_c = \frac{1}{2\pi RC} \quad (3.10)$
Filtro passa-faixa  Figura 3.13 Filtro passivo passa-faixa.	Frequências de corte do filtro passa-faixa, sendo $f_{c, alta}$ a frequência alta e $f_{c, baixa}$ a frequência baixa: $f_{c, alta} = \frac{1}{2\pi R_a C_a} \quad (3.11)$ $f_{c, baixa} = \frac{1}{2\pi R_b C_b} \quad (3.12)$

Tipo de filtro	Cálculo da frequência de corte
Filtro rejeita-faixa  Figura 3.14 Filtro passivo rejeita-faixa.	Frequência de corte do filtro rejeita-faixa (f_c): $f_c = \frac{1}{2\pi RC} \quad (3.13)$

FILTROS ATIVOS

TOPOLOGIA Sallen Key – PASSA-BAIXAS 2.ª ORDEM:

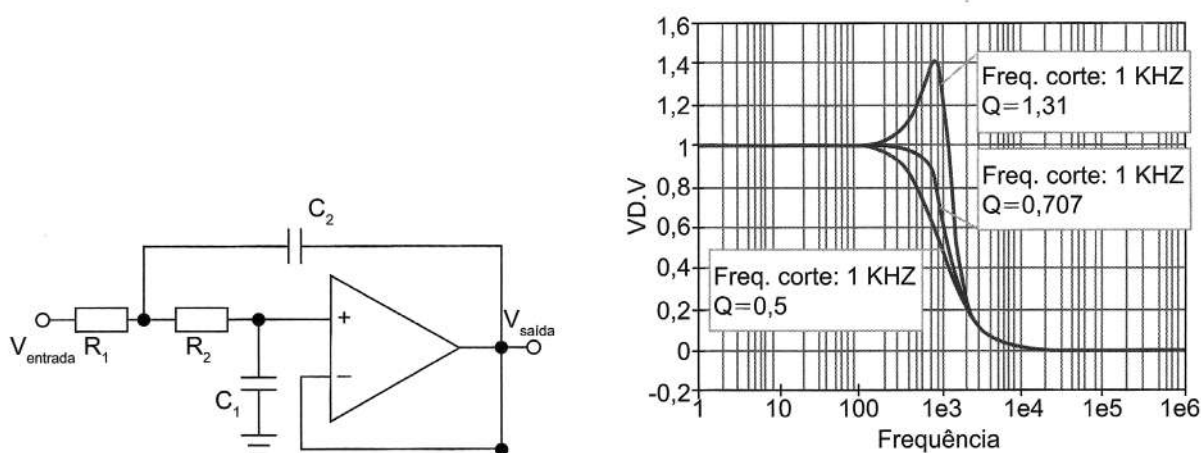


Figura 3.15 Filtro ativo passa-baixa e sua resposta em frequência.

Para essa topologia, podemos calcular a frequência de corte f_c e o fator de qualidade do filtro Q :

$$f_c = \frac{1}{2\pi\sqrt{R_1 R_2 C_1 C_2}} \quad (3.14)$$

e

$$Q = \frac{1}{2\pi f_c C_1 (R_1 + R_2)} \quad (3.15)$$

Nota: Para projetar, fazer $R_1 = R_2$ e calcular C_1 e C_2 .



TOPOLOGIA SALLEN KEY – PASSA-ALTAS 2.^a ORDEM

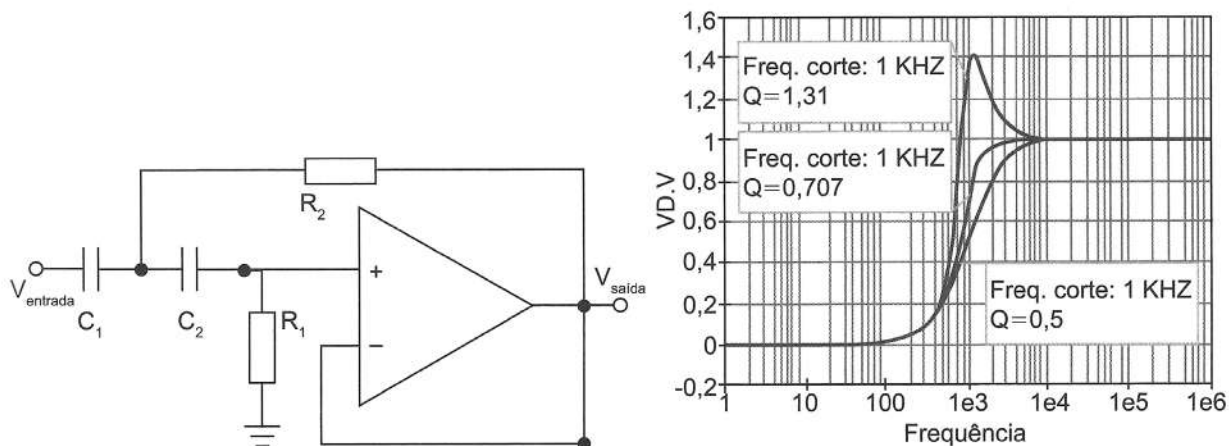


Figura 3.16 Filtro ativo passa-faixa e sua resposta em frequência.

Para essa topologia, podemos calcular a frequência de corte f_c e o fator de qualidade do filtro Q :

$$f_c = \frac{1}{2\pi\sqrt{R_1 C_1 R_2 C_2}} \quad (3.16)$$

e

$$Q = \frac{1}{2\pi f_c R_2 (C_1 + C_2)} \quad (3.17)$$

Nota: Para projetar, fazer $R_1=R_2$ e calcular C_1 e C_2 .

TOPOLOGIA SALLEN KEY – PASSA-FAIXAS 2.^a ORDEM

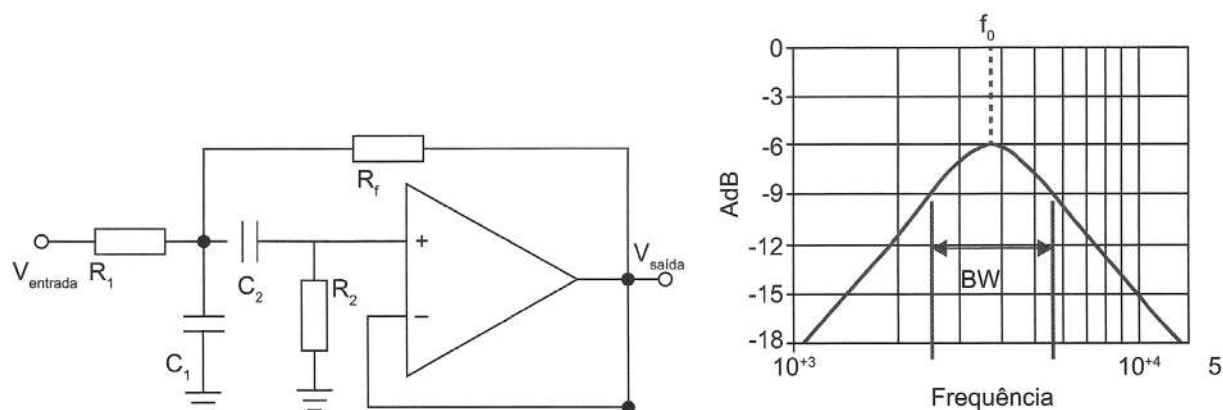


Figura 3.17 Filtro ativo passa-alta e sua resposta em frequência.

Para essa topologia, podemos calcular a frequência central f_o e o fator de qualidade do filtro Q :

$$f_o = \frac{1}{2\pi \sqrt{\frac{R_f + R_1}{R_1 C_1 R_2 C_2 R_f}}} \quad (3.18)$$

e

$$Q = \frac{2\pi f_o}{\left(\frac{1}{R_1 C_1} + \frac{1}{R_2 C_1} + \frac{1}{R_2 C_2} \right)} \quad (3.19)$$

Relação entre fator de qualidade (Q) e largura de banda (BW):

$$Q = \frac{f_o}{BW} \quad (3.20)$$

Outra topologia de passa-faixa, baseada na união de um passa-baixa e um passa-alta, conforme ilustrado na Figura 3.18.

Nesse caso, as equações são idênticas aos passa-baixa (3.14) e passa-alta isolados (3.16), relembrando:

Frequência de corte $f_{c, \text{baixa}}$ é calculada por Enquanto a frequência de corte $f_{c, \text{alta}}$

$$f_{c, \text{baixa}} = \frac{1}{2\pi \sqrt{R_1 C_1 R_2 C_2}}$$

$$f_{c, \text{alta}} = \frac{1}{2\pi \sqrt{R_3 C_3 R_4 C_4}}$$

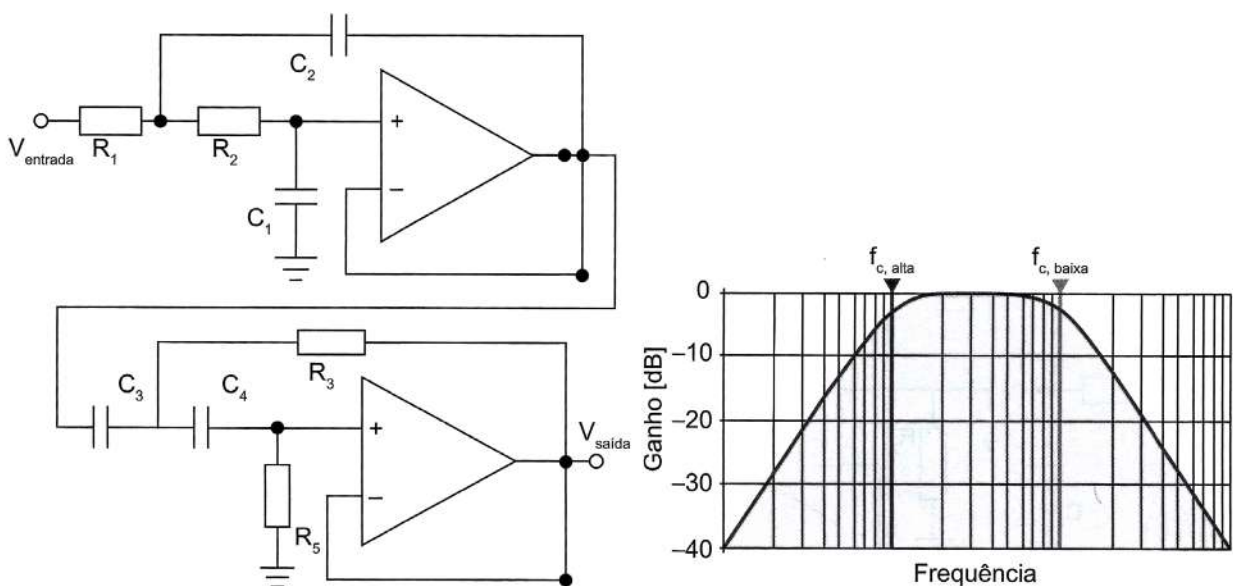
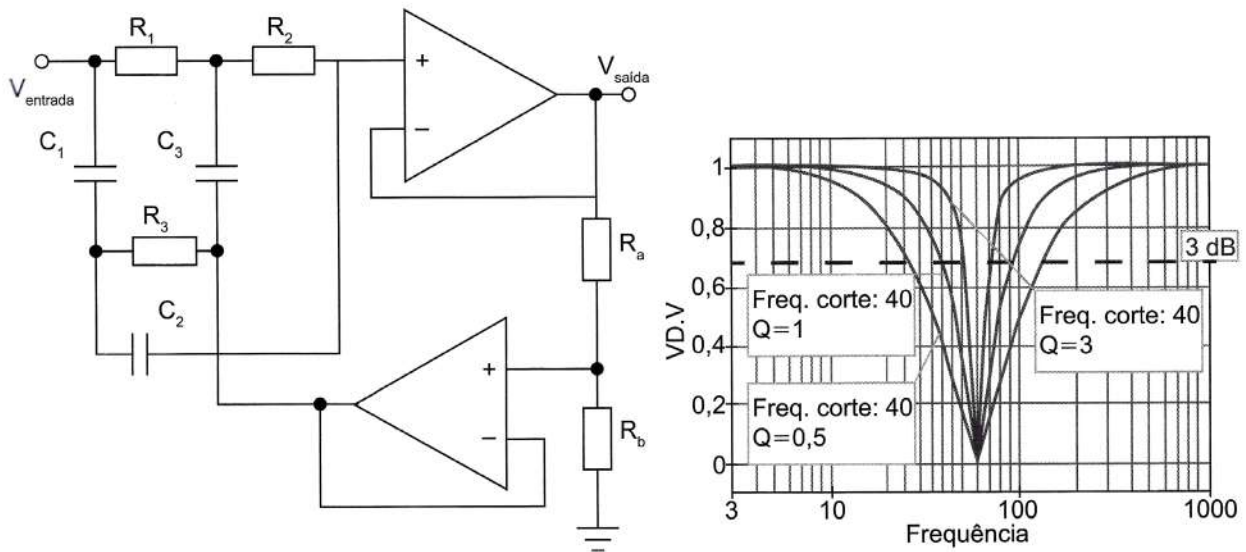


Figura 3.18 Filtro ativo passa-faixa, baseado na união de um passa-baixa e outro passa-alta, e sua resposta em frequência.

TOPOLOGIA DUPLO T – REJEITA-FAIXAS 2.^a ORDEM**Figura 3.19** Filtro ativo rejeita-faixa e sua resposta em frequência.

Considerando $R_1 = R_2 = 2R_3 = R$ e $V_1 = C_2 = C_3/2 = C$, podemos calcular a frequência central de f e o fator de qualidade Q por:

$$f = \frac{1}{2\pi RC} \quad (3.21)$$

$$Q = \frac{(R_a + R_b)}{4R_a} \quad (3.22)$$

Vale a pena ressaltar que os projetos de filtros são um ramo específico dentro da eletrônica, e que aqui nossa intenção foi apresentá-los e exemplificá-los, e não entrar a fundo em projeto de filtros.

EXCITAÇÃO

A excitação é necessária para muitos tipos de transdutores. Por exemplo, strain gages, acelerômetros, termistores e medidores de temperatura por resistência (RTDs) necessitam da excitação de tensão ou corrente externa. Medições de RTD e termistor são normalmente feitas com uma fonte de corrente que converte a variação na resistência em uma tensão mensurável. Acelerômetros normalmente possuem um amplificador integrado, o qual requer uma corrente de excitação fornecida pelo dispositivo de medição. Strain gauges, que são dispositivos de resistência muito baixa, são usados tipicamente em uma configuração de ponte de Wheatstone com uma fonte de tensão de excitação, como ilustrado na Figura 3.20.

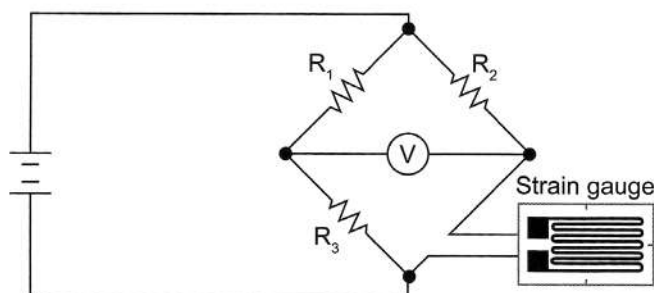


Figura 3.20 Exemplo de circuito de excitação de sensor (strain gauge).

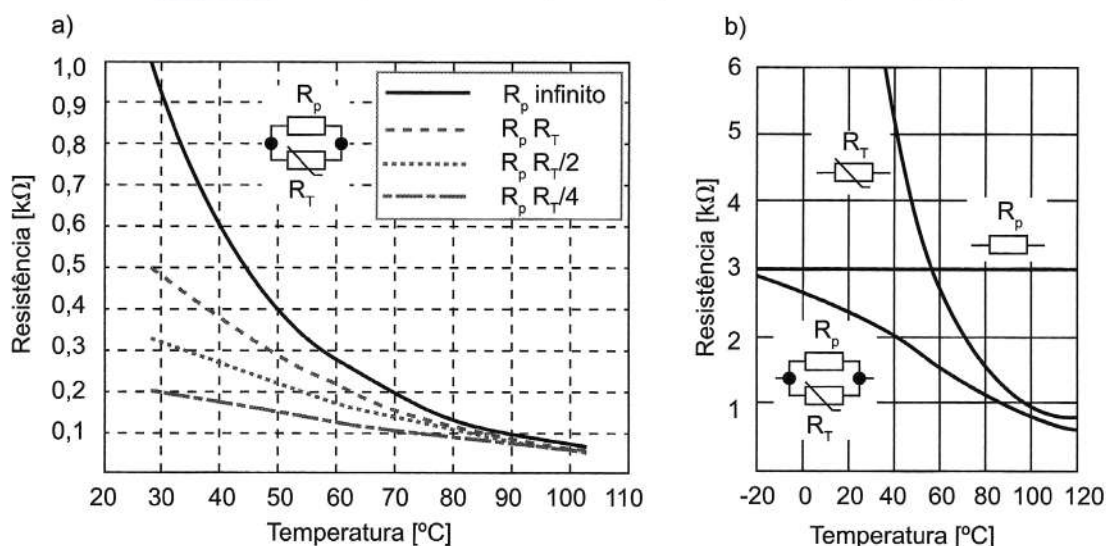
LINEARIZAÇÃO

A linearização é necessária quando os sensores produzem sinais de tensão que não estão linearmente relacionados com a medição física. A linearização é o processo de interpretar o sinal medido pelo sensor, podendo ser feita tanto por condicionamento de sinal quanto por software. Um sinal com uma resposta linear simplifica o processo de tratamento do sinal, além de reduzir o tempo necessário para tal.

O termopar e o termistor são exemplos clássicos de sensores que requerem linearização.

Os Gráficos 3.1a e b ilustram um exemplo simples de linearização de um termistor (R_T), com um resistor de compensação em paralelo (R_p). No lado esquerdo, são apresentadas 4 curvas, com 4 valores diferentes de resistência de compensação. Percebemos que, reduzindo o valor do resistor R_p , é possível reduzir o comportamento exponencial do sensor, aproximando sua resposta de uma reta. No lado direito da figura são apresentados o comportamento de R_T e R_p separadamente e depois o de ambos em paralelo, induzindo à linearização da resposta.

Gráfico 3.1 Exemplos de linearização de resposta de sensor (termistor)



COMPENSAÇÃO DE JUNÇÃO FRIA

A compensação de junção fria (comumente conhecida pela sigla em inglês: CJC) é uma tecnologia necessária para medições precisas com termopar. Os termopares medem a temperatura por meio da diferença de tensão entre dois metais com características diferentes. Com base nesse conceito, outra tensão é gerada na conexão entre o termopar e o terminal do seu dispositivo de aquisição de dados. A CJC melhora a exatidão da sua medição fornecendo a temperatura nessa junção e aplicando a correção apropriada. A Figura 3.21 ilustra duas técnicas de compensação.

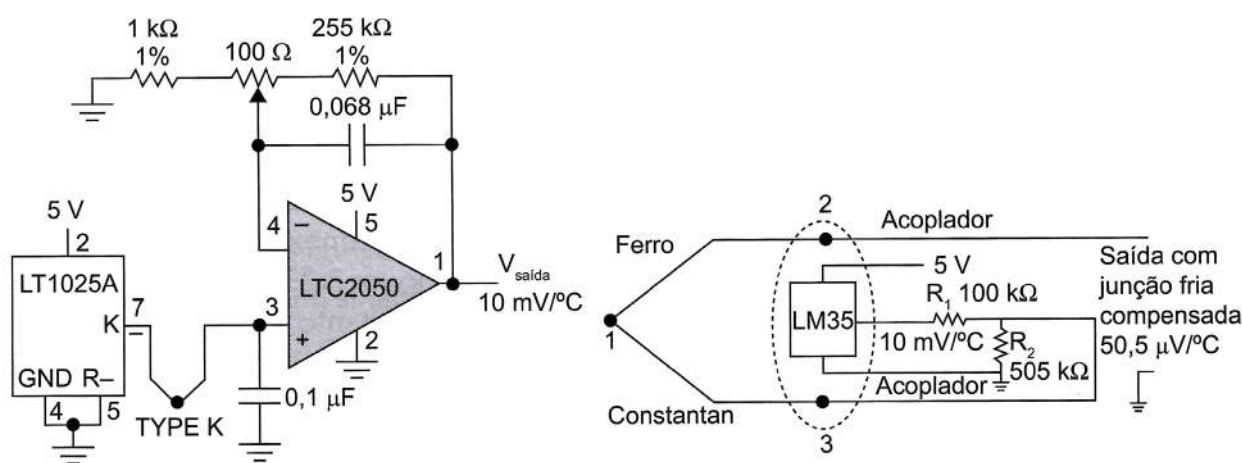


Figura 3.21 Exemplos de circuitos de compensação de junção fria.

CONFIGURAÇÕES RESISTIVAS E POTENCIOMÉTRICAS

Alguns sensores operam essencialmente como divisores resistivos, ou, quando associados a uma resistência em série, possibilitam a medição de alguma grandeza (como por exemplo temperatura ou tensão mecânica) através da sua dependência resistiva com a grandeza de medida em questão. Nesse caso, a utilização de estágios de casamento de impedâncias (seguidor de tensão) e filtragens é importante para que o circuito de medida não interfira no sensor e para que ruídos de alta frequência sejam eliminados.

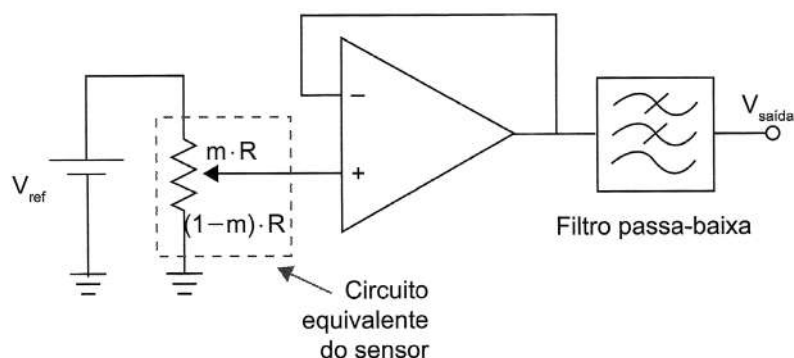


Figura 3.22 Exemplo de seguidor de tensão para isolamento do sensor.

Neste exemplo, a tensão de saída é exatamente igual à tensão de entrada:

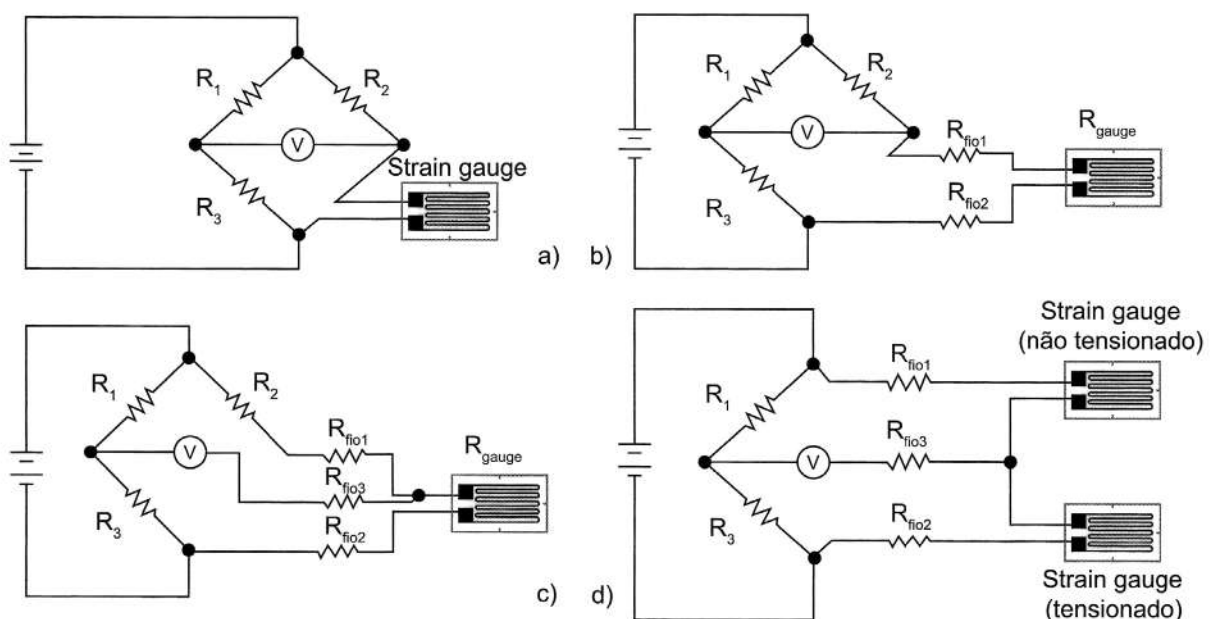
$$V_{\text{saída}} = m \cdot V_{\text{ref}} \quad (3.23)$$

com $0 < m < 1$

A configuração da ponte é necessária para definir os demais elementos dos sensores de quarto e meia ponte, compreendendo então uma ponte de Wheatstone de quatro resistores. Condicionadores de sinal de strain gauge normalmente fornecem redes de elementos para definição de meia ponte, consistindo em resistores de referência de alta precisão. Esses resistores fornecem uma referência fixa para detecção de pequenas variações de tensão no(s) resistor(es) ativo(s).

A seguir, na Figura 3.23, são apresentados diferentes métodos de concepção de pontes resistivas para medição de extensômetros. A Figura 3.23a ilustra o método mais simples de medição de um strain gauge, normalmente utilizada com a ponte próxima ao sensor. Analogamente, a Figura 3.23b apresenta o sensor em uma configuração em que a ponte está distante do sensor. Nesse caso a resistência das conexões deve ser considerada no balanceamento da ponte. Para resolver isso, a Figura 3.23c ilustra a compensação das conexões do sensor distante, utilizando 3 fios. Entretanto, esse método pode apresentar interferência cruzada entre grandezas (como temperatura e torção mecânica). A Figura 3.23d apresenta uma solução, colocando um segundo sensor sob a ação de apenas uma grandeza, eliminando assim a variação cruzada.

Já a Figura 3.23e ilustra 2 sensores, colocados para operarem em sentidos opostos à grandeza. Por exemplo, no caso de flexão de uma barra, um sensor posto na parte superior da barra sofreria extensão, enquanto outro sensor, posto na parte inferior da barra, sofreria compressão. Por fim, a Figura 3.23f ilustra 4 sensores idênticos, colocados para operarem juntamente. Essa configuração facilita o balanceamento inicial da rede e maximiza a interpretação das ações.



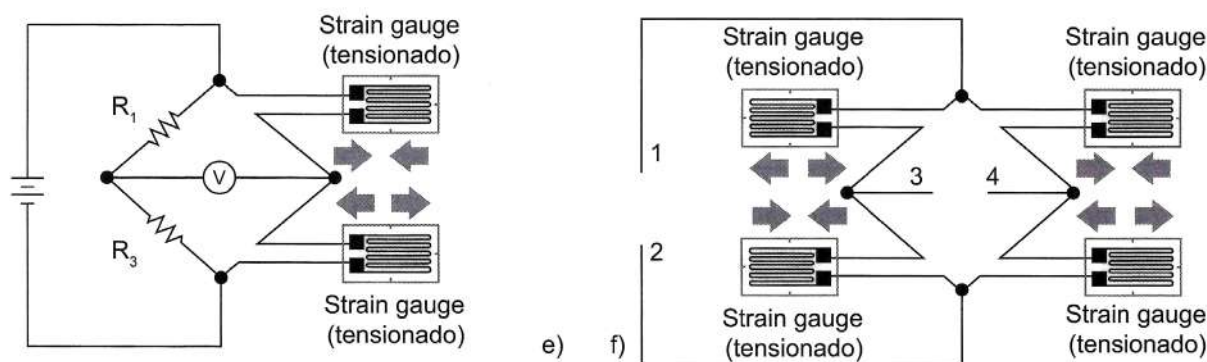


Figura 3.23 Diferentes configurações para medição de sensores strain gauge em pontes de Wheatstone.

RETIFICADORES

Outros circuitos importantes são os retificadores, que eliminam os níveis negativos e ainda podem converter o sinal negativo em positivo (invertendo a fase do sinal). Deseja-se retificar um sinal quando é preciso analisar nível lógico, como por exemplo o valor de tensão RMS obtido de um sinal induzido por um sensor indutivo.

Temos dois casos: o retificador de meia onda, quando se elimina a parte negativa do sinal, e o retificador de onda completa, quando se inverte a parte negativa do sinal, invertendo-se a fase do mesmo. A Figura 3.24 ilustra dois exemplos de circuitos para essas aplicações.

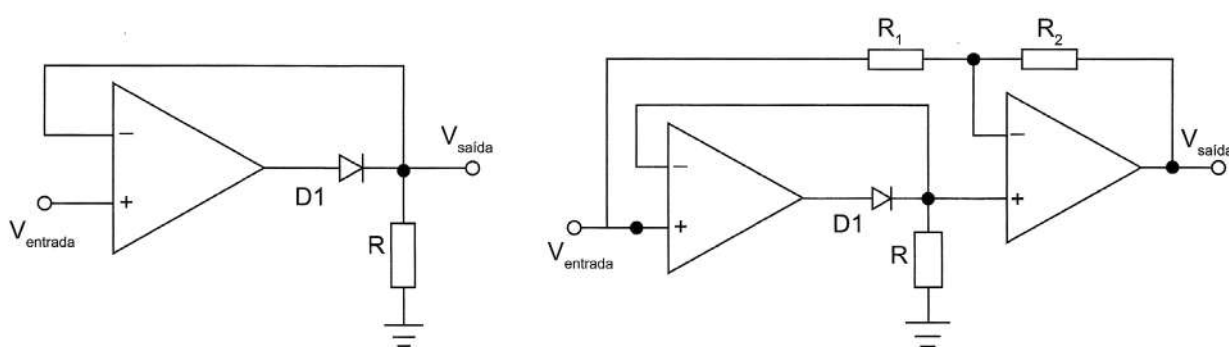


Figura 3.24 Exemplos de retificadores (à esquerda) de meia onda e (à direita) de onda completa.

CORREÇÃO DE NÍVEL DC (OFFSET)

O nível de tensão contínua (DC), também conhecido como offset, de um sinal muitas vezes precisa ser corrigido após a amplificação de um sinal de amplitudes muito baixas. Outro exemplo é quando um sinal tem amplitudes positivas e negativas e precisamos de uma excursão exclusivamente positiva para que o sinal possa ser enquadrado aos níveis de um conversor analógico digital (por exemplo do Arduino).

Dependendo do amplificador operacional utilizado, ele pode ter entradas específicas para correção (ex.: LM741), ou esta deve ser feita através de um circuito externo. A Figura 3.25 ilustra a questão de inserção de um nível DC, de um sinal com amplitudes positivas e negativas.

O amplificador operacional 741 dispõe de duas entradas para a correção da tensão de DC (offset) de entrada. O circuito típico para essa correção é apresentado na Figura 3.26 e é composto basicamente por um resistor variável conectado nos pinos de offset nulo (pinos 1 e 5).

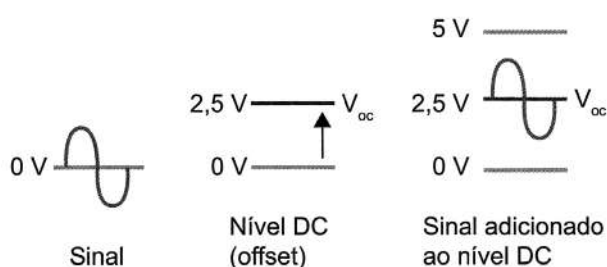


Figura 3.25 Inserção de nível offset em sinal.

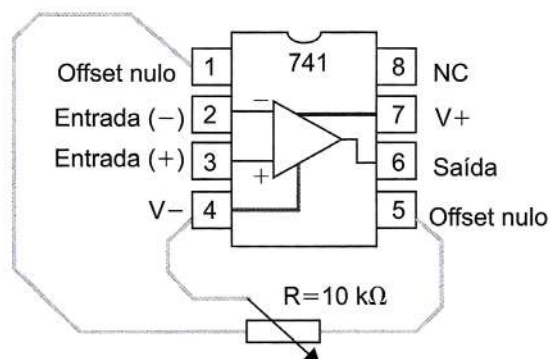


Figura 3.26 Controle de nível offset no LM741.

Os amplificadores operacionais que não possuem controle de offset interno podem utilizar circuitos como os ilustrados na Figura 3.27.

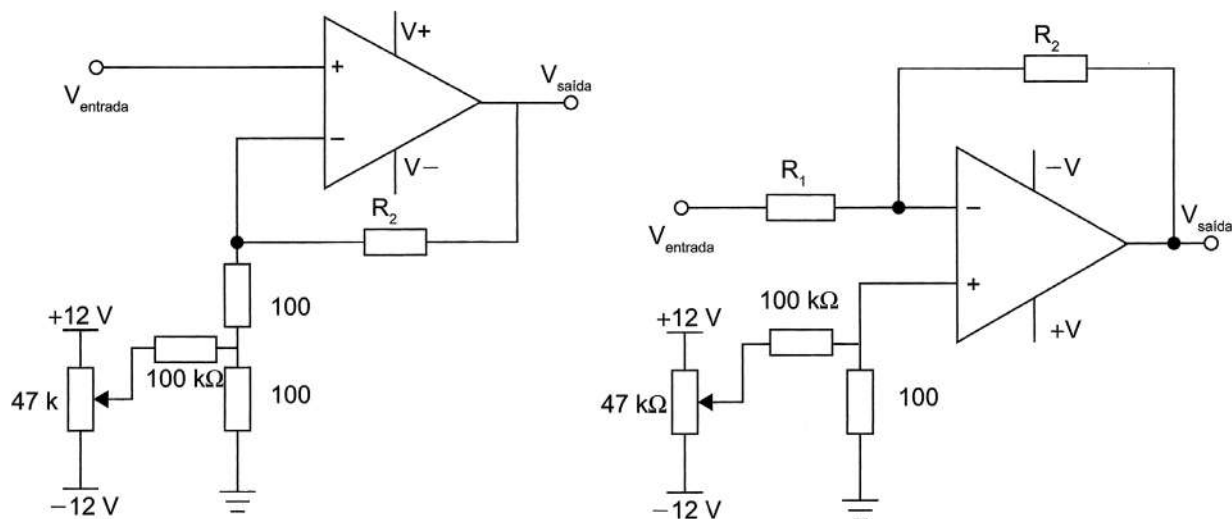


Figura 3.27 Exemplos de circuitos de configuração de offset em amplificadores operacionais.

DETECTOR DE PICO

Um circuito muito utilizado para avaliar nível de tensão é o detector de pico. O mais simples é composto por um diodo, para garantir apenas a amplitude positiva da



onda, um capacitor, responsável pelo acúmulo de energia (tensão), e um resistor para descarregar essa energia vagarosamente.

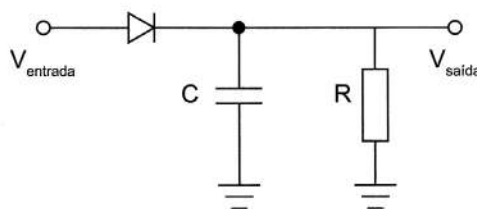


Figura 3.28 Detector de pico de sinal.

Dessa forma, escolhem-se os valores de R e C , de forma que a constante de tempo $\tau = RC$ deve ser no mínimo um décimo da frequência do sinal.

A Figura 3.28 ilustra o circuito básico de detecção de pico.

Como vantagens, esse circuito apresenta-se como de baixo custo, devido inclusive à quantidade de componentes utilizados; entretanto, tem como principal desvantagem um nível de amplitude de saída abaixo da queda de tensão do diodo ($-0,7\text{ V}$, se de silício).

Para as aplicações em que a queda de tensão do diodo não possa ser desconsiderada, alguns circuitos ativos podem realizar a detecção de pico. A Figura 3.29 ilustra um circuito para esse fim, com dois amplificadores operacionais.

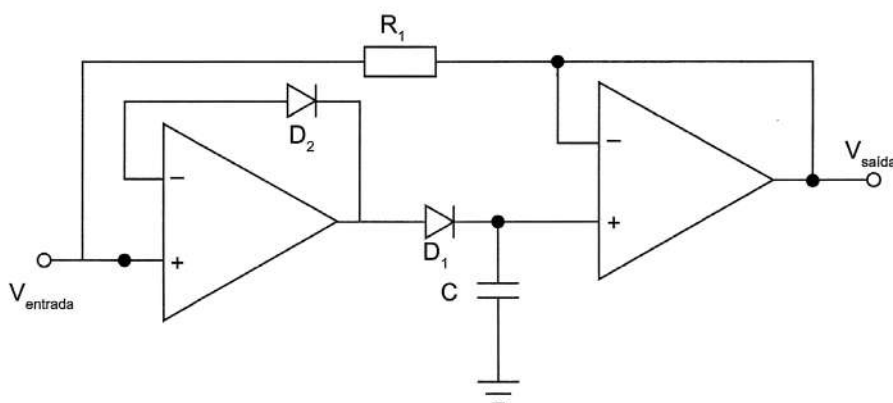


Figura 3.29 Exemplo de detector de pico ativo.

DETECTOR DE CRUZAMENTO DE NÍVEL

Circuitos detectores de cruzamento de nível (seja nível $DC = 0\text{ V}$ - conhecidos como cruzamento pelo zero - *Zero Crossing Detector* - ZCD, seja em um nível diferente de 0 V) são utilizados para determinação de frequência ou relação de fases, além de operações com limiares (quando $DC \neq 0$). Basicamente, utilizamos amplificadores operacionais em sua configuração de comparador, e utilizamos o ajuste de limiar do valor de comparação para tomada de decisão.

NÍVEL ZERO (CRUZAMENTO AO ZERO)

Neste exemplo, ilustrado pelo circuito da Figura 3.30a, fixando a entrada invertida em 0 V (GND) e variando a tensão de entrada V_{entrada} , ter-se-á uma comparação entre o valor de entrada em relação ao nível zero. Assim, sempre que $V_{\text{entrada}} > V_{\text{ref}}$ (0 V) a saída do comparador será o valor da tensão de saturação positiva, ao passo que quando $V_{\text{entrada}} < V_{\text{ref}}$ (0V) a saída do comparador será o valor da tensão de saturação negativa. Nesse caso, a tensão de saturação equivale à tensão de alimentação do amplificador operacional, por exemplo: +12 V e -12 V, ou +5 V e -5 V. Esse é o comportamento esperado pelo circuito ilustrado na Figura 3.31, onde são exemplificados comportamentos de saída em função de uma onda triangular de entrada (Figura 3.30b) e a relação entre $V_{\text{saída}}$ e V_{entrada} (Figura 3.30c).

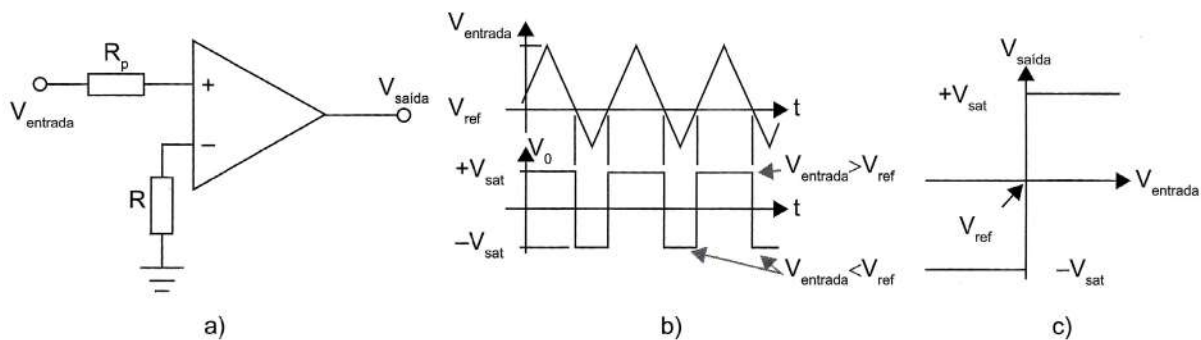


Figura 3.30 Exemplo de um comparador de nível zero (detector de cruzamento ao zero).

NÍVEL DIFERENTE DE ZERO

Se a tensão de referência for variada entre os valores das tensões de alimentação positiva e negativa, teremos um comparador com nível diferente de zero. A Figura 3.31b ilustra o comportamento do comparador com uma tensão positiva, e a Figura 3.31c o comportamento para uma tensão V_{ref1} positiva e outra V_{ref2} negativa.

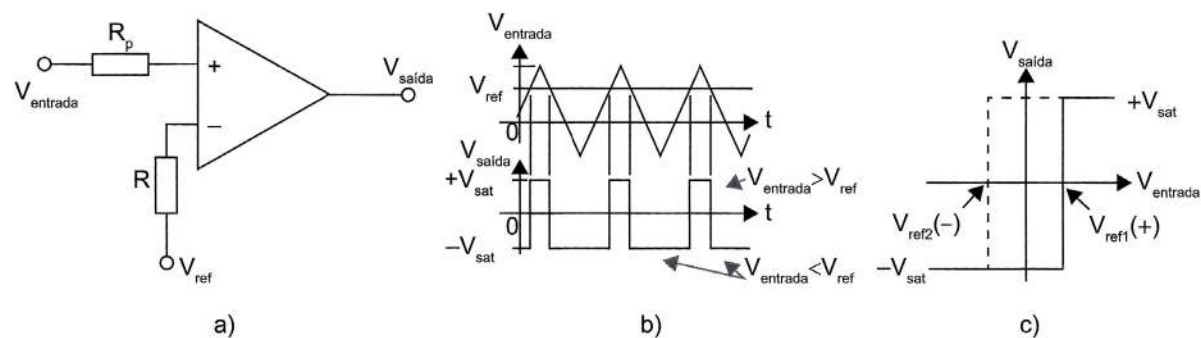


Figura 3.31 Exemplo de um comparador de nível de referência diferente de zero:
a) circuito; b) tensões no tempo; c) sinal de saída pela entrada.



DETECÇÃO ENTRE DOIS NÍVEIS – COMPARADOR DE JANELA

O circuito da Figura 3.32 considera não apenas a identificação de um nível, mas de dois, indicando uma janela de operação dentro da faixa de operação do amplificador operacional. Nesse caso, considerando a entrada de referência 1 (V_{ref1}) maior que a entrada de referência 2 (V_{ref2}), temos três situações:

- Entrada de sinal $V_{entrada} > V_{ref1}$: $\Rightarrow$ a saída do AmpOp1 é $+V_{CC}$ e portanto D_1 conduz. Por outro lado, a saída do AmpOp2 é $-V_{CC}$ e portanto D_1 estará polarizado reversamente. Portanto, $V_{saída} = +V_{CC}$.
- $V_{ref1} < V_{entrada} < V_{ref2}$ $\Rightarrow$ a saída dos dois AmpOps será $-V_{CC}$ e portanto os dois diodos estarão cortados. Portanto, a saída $V_{saída} = 0$.
- $V_{entrada} < V_{ref2}$ $\Rightarrow$ a saída do AmpOp2 é $+V_{CC}$, logo D_2 conduz. A saída do AmpOp1 é $-V_{CC}$ e D_1 estará reversamente polarizado. Nessas condições, a saída $V_{saída} = +V_{CC}$.

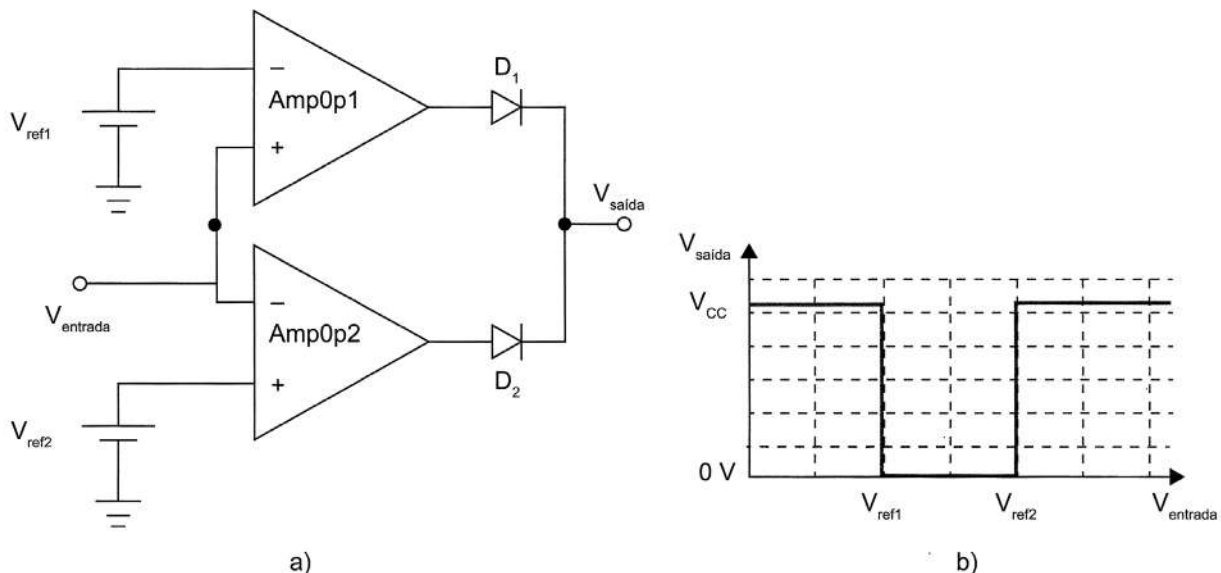
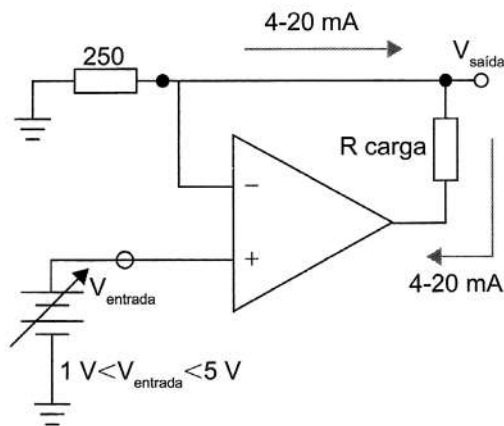


Figura 3.32 Exemplo de um comparador de janela (entre dois níveis):
a) circuito; b) diagrama de tensão de saída em função da tensão de entrada.

CONVERSÃO DE TENSÃO EM CORRENTE

Muitos dispositivos, como CLPs, têm suas entradas no modo corrente, e nesse caso os dispositivos sensores que têm sua saída em tensão precisam ter essa grandeza convertida em corrente. Outra possibilidade comum é termos dispositivos de campo (sensores) e controladores afastados, e nesse caso uma possibilidade de comunicação entre os dispositivos é utilizar o padrão analógico de corrente entre 4 e 20 mA. Há relatos de que esse tipo de comunicação alcança distâncias superiores a 1000 metros.

A Figura 3.33 ilustra um circuito com esse princípio, em que a variação de um sinal de tensão controlado (por exemplo, sensor ou transmissor) tem sua saída de nível de corrente determinada apenas pelo resistor de 250Ω , independentemente da carga do circuito.



$$I_{\min} = \frac{V_{\min}}{250} = \frac{1}{250} = 4 \text{ mA}$$

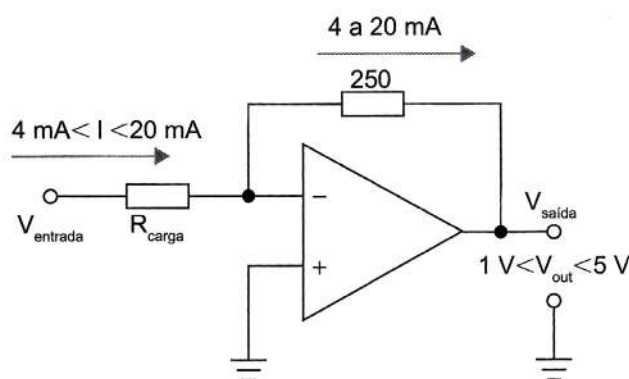
e

$$I_{\max} = \frac{V_{\max}}{250} = \frac{5}{250} = 20 \text{ mA}$$

Figura 3.33 Conversor tensão-corrente (de 1 a 5 V para 4 a 20 mA).

CONVERSÃO DE CORRENTE EM TENSÃO

De modo análogo, como no exemplo anterior da comunicação analógica de 4 a 20 mA, um circuito receptor que receba esse sinal em corrente e ofereça um sinal de tensão em sua saída é ilustrado na Figura 3.34.



$$V_{\min} = 250 \cdot I_{\min} = 250 \cdot 4 \text{ mA} = 1 \text{ V}$$

e

$$V_{\max} = 250 \cdot I_{\max} = 250 \cdot 20 \text{ mA} = 5 \text{ V}$$

Figura 3.34 Conversor corrente-tensão (de 4 a 20 mA para 1 a 5 V).

CONDICIONAMENTO PARA SENSORES CAPACITIVOS

Até aqui, focamos no condicionamento de sinais elétricos de tensão ou corrente. Para a análise de sensores capacitivos, utilizamos circuitos de condicionamento de sinais que normalmente são traduzidos em uma outra grandeza, como controle de tempo (de carga ou descarga do capacitor), de temporização (para identificação de diferença de fase) ou ainda de frequência, na concepção de osciladores de frequência dependente do sensor capacitivo. Dessa forma, citamos algumas das técnicas de detecção de sensores capacitivos.



FREQUÊNCIA DE OSCILADOR

Um oscilador astável utilizando um CI 555 produz um sinal periódico de onda quadrada de saída, com frequência e a duração do nível lógico alto dependentes de dois resistores (R_1 e R_2) e um capacitor (C), conforme ilustrado na Figura 3.35. Basicamente, o circuito trabalha com a carga e descarga do capacitor C e controla os disparos em função do limiar de operação, determinado pelos resistores (R_1 e R_2).

O tempo que o sinal de saída apresenta nível lógico alto é determinado pela Equação 3.24, enquanto o tempo em nível lógico baixo é determinado pela Equação 3.25. A soma desses dois tempos determina o período da onda gerada, dada pela Equação 3.26. Para determinar a frequência, basta inverter o período ($f = 1/T$), o que pode ser calculado pela Equação 3.27.

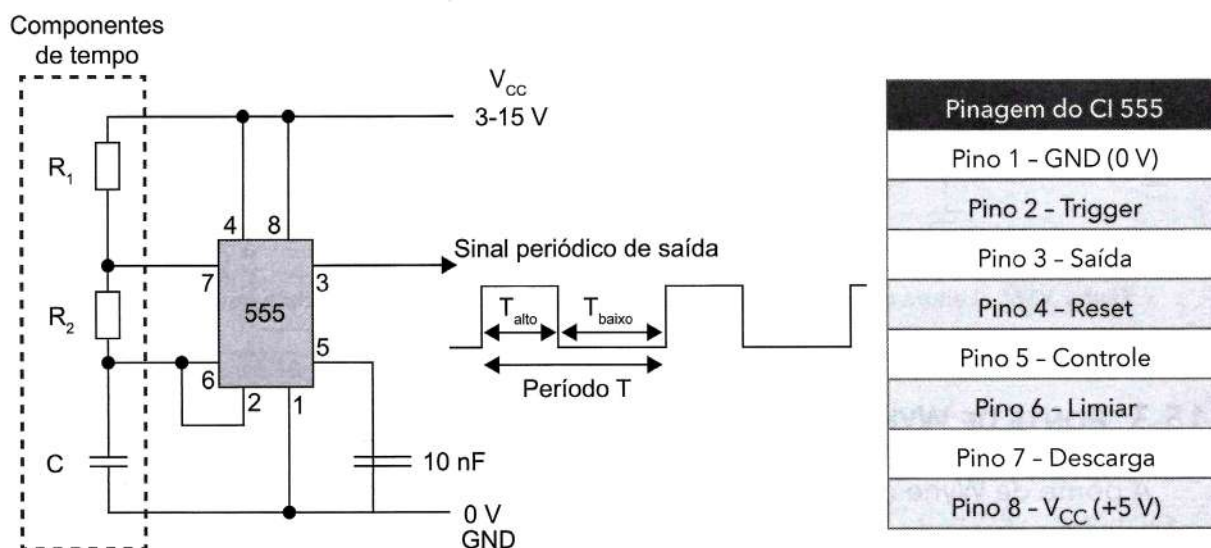


Figura 3.35 Oscilador astável com CI 555.

Equações:

$$T_{\text{alto}} = 0,69 \cdot (R_1 + R_2) \cdot C \quad (3.24)$$

$$T_{\text{baixo}} = 0,69 \cdot R_2 \cdot C \quad (3.25)$$

$$T = 0,69 \cdot (R_1 + 2 \cdot R_2) \cdot C \quad (3.26)$$

$$f = \frac{1,44}{((R_1 + 2 \cdot R_2) \cdot C)} \quad (3.27)$$

! DIFERENÇA DE FASES

Outra técnica baseia-se em um capacitor de referência (C_{ref}), o qual é colocado em paralelo com o capacitor sensor a ser medido (C_m), formando uma ponte, ambos excitados por uma fonte senoidal. Quando a ponte está equilibrada (ou seja, C_{ref} igual a C_m), a diferença de fase entre V_{sensor} e $V_{referência}$ é nula. Uma variação em C_m causa uma diferença de fase entre os sinais em cada lado da ponte. Se utilizarmos comparadores para um mesmo limiar, é possível coletar o tempo de transição dos dois capacitores e posteriormente o tempo entre os disparos, o qual está relacionado com a diferença sobre os capacitores. A Figura 3.36 ilustra essa técnica.

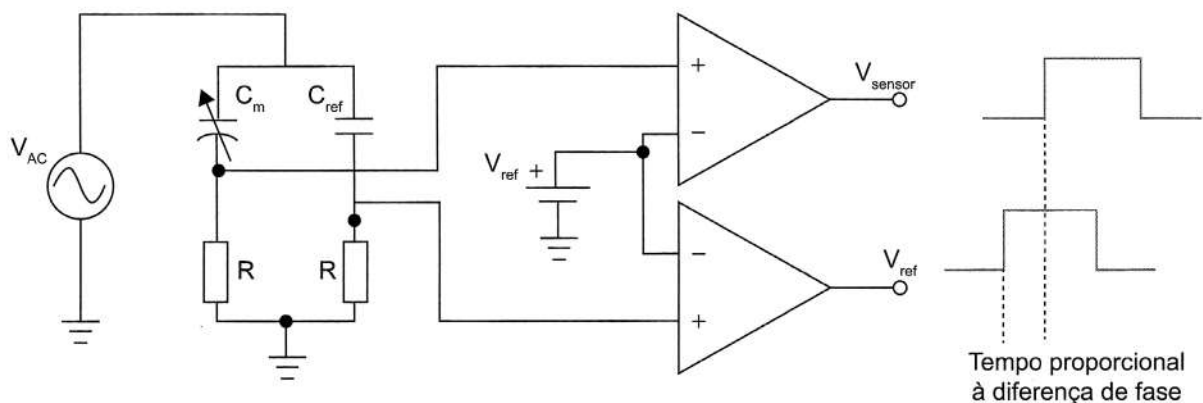
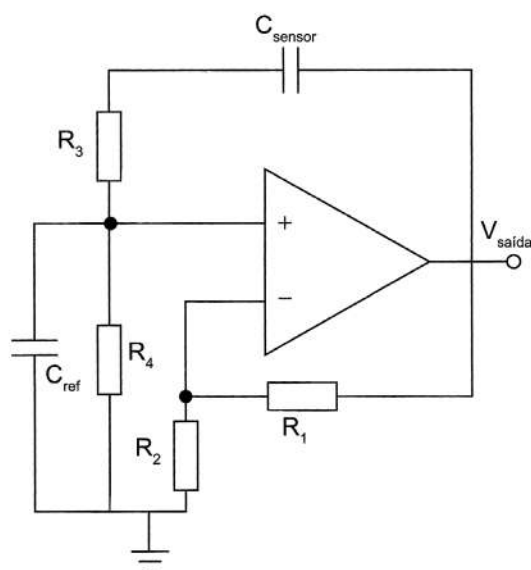


Figura 3.36 Leitura de diferença de fase sobre um capacitor de referência e outro sensor.

PONTE DE WYNE

A ponte de Wyne é outro circuito que relaciona a frequência de um oscilador em função da frequência de operação do mesmo, determinada por uma capacitância variável (sensor). Um circuito típico dessa ponte é ilustrado na Figura 3.37.



O valor da frequência (f), nessa ponte, é determinado por:

$$f = \frac{1}{2\pi\sqrt{R_3 \cdot R_4 \cdot C_{ref} \cdot C_{sensor}}} \quad (3.28)$$

Figura 3.37 Ponte de Wyne para medição de variação capacitiva.



CONDICIONAMENTO PARA SENSORES INDUTIVOS

A grande parte dos sensores indutivos baseia-se na alimentação de uma bobina com um sinal senoidal e na análise do sinal induzido em outra(s) bobina(s). Assim sendo, o mais simples circuito de condicionamento refere-se a retificar e filtrar o sinal de saída induzido no secundário do sensor indutivo, amplificá-lo e ajustar seu nível de offset. Dessa forma, pode-se enquadrar o sinal de saída dentro dos requisitos do controlador (Arduino, com entradas entre 0 e 5 V). O circuito da Figura 3.38 ilustra essa técnica. Outra possibilidade seria substituir o retificador por detector de pico.

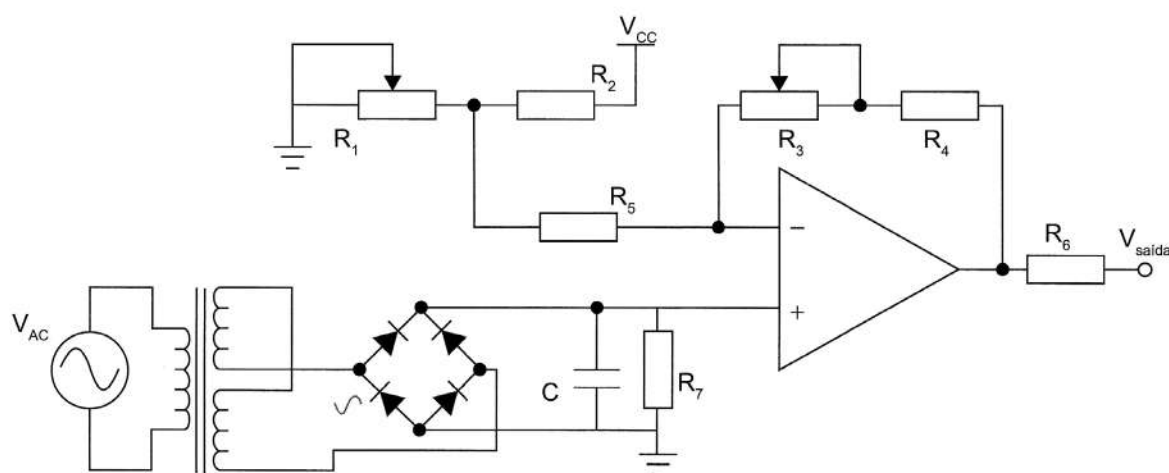


Figura 3.38 Exemplo de circuito de condicionamento para um sensor indutivo.

CONVERSÃO ANALÓGICO-DIGITAL (ADC)

A conversão analógico-digital (ADC) pode ser realizada diretamente pela maioria dos microcontroladores, por possuírem conversores internos. Esse é o caso do Arduino, o qual possui um único conversor e vários canais analógicos (6 no caso do Arduino UNO e 16 no Arduino MEGA), com resolução de 10 bits.

O Arduino opera normalmente a 16 MHz, e o conversor AD interno opera com uma pré-escala de 128 (configurável), trabalhando este com um clock interno de 125 kHz. Nessa escala, utiliza até 13 ciclos de clock para completar uma conversão analógico-digital, resultando em uma frequência de conversão de aproximadamente 9,6 kHz.

Quando a resolução não é suficiente, ou, ainda, o tempo de conversão for muito alto, precisamos utilizar circuitos dedicados de conversão analógico-digital.

Como exemplo, podemos citar os conversores ADS1115 e ADS1015, que são conversores de 12 e 16 bits, respectivamente, que operam através de comunicação serial (I²C). Nesse caso, basta alimentar o conversor e utilizar 2 fios de comunicação sem um circuito de condicionamento especial. No quadro a seguir, são ilustradas as principais características desses dois conversores.

ADS1115	ADS1015
■ Resolução: 16 bits. ■ Taxa de amostragem entre 8 e 860 amostras por segundo. ■ Tensão de operação: 2 V a 5,5 V. ■ Oscilador interno. ■ Interface serial I²C. ■ 4 canais.	■ Resolução: 12 bits. ■ Taxa de amostragem entre 128 e 3300 amostras por segundo. ■ Tensão de operação: 2 V a 5,5 V. ■ Oscilador interno. ■ Interface serial I²C. ■ 4 canais.

Vale lembrar que todo sinal convertido de analógico para digital carrega consigo uns erros de discordância com o sinal original, erros esses que se devem principalmente a limitação de resolução e atrasos de conversão.

CONVERSÃO DIGITAL-ANALÓGICO (DAC)

Diferentemente dos conversores ADC, todas as saídas do Arduino são digitais. Assim, quando se deseja ter um sinal analógico de saída, algumas técnicas podem ser utilizadas.

TÉCNICA DOS RESISTORES PONDERADOS

Utiliza uma rede de resistores proporcionais entre si, os quais contribuem com a corrente de saída do conversor, em função do nível lógico em suas entradas (fornecido pelo sistema controlador, nesse caso), como ilustrado por um conversor de 4 bits na Figura 3.39.

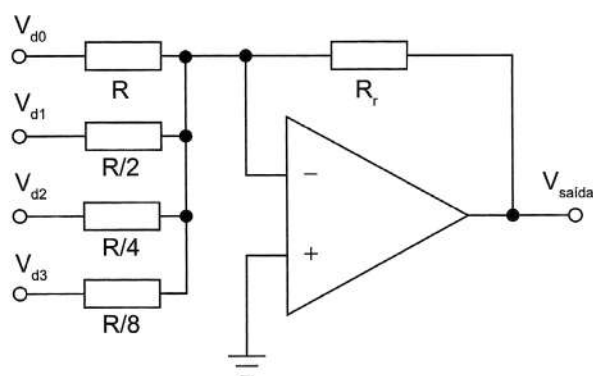


Figura 3.39 Exemplo de circuito de conversor digital-analógico com resistores ponderados.

Dessa forma, a tensão de saída é dada por:

$$V_{saída} = R_r \cdot I_{Rr} \quad (3.29)$$

e, considerando $V_{d0} = V_{CC} \cdot d0$, onde $d0$ representa a ausência de nível lógico (= 0) ou a existência dele (= 1), e assim analogamente para V_{d1} , V_{d2} e V_{d3} , a corrente elétrica pode ser determinada por:

$$I_{Rr} = V_{CC} \cdot \left(\frac{d0}{R} + \frac{d1}{\frac{R}{2}} + \frac{d2}{\frac{R}{4}} + \frac{d3}{\frac{R}{8}} \right) \quad (3.30)$$

resultando na tensão de saída do conversor, calculada por:

$$V_{saída} = \frac{R_r}{R} \cdot V_{CC} \cdot (8 \cdot d3 + 4 \cdot d2 + 2 \cdot d1 + 1 \cdot d0) \quad (3.31)$$



Essa técnica apresenta como principal desvantagem a necessidade de um conjunto de resistores proporcionais entre si, o que dificulta sua realização com o aumento da resolução do conversor e com a tolerância dos componentes.

TÉCNICA DA REDE R-2R

Essa técnica apresenta como principal vantagem a necessidade de apenas dois valores de resistores, ainda que proporcionais entre si, como ilustrado por um conversor de 4bits na Figura 3.40.

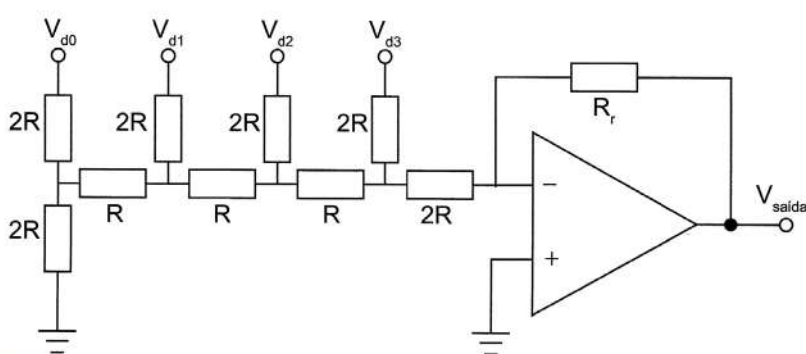


Figura 3.40 Exemplo de circuito de um conversor digital-analógico com rede R-2R.

Nesse caso, a tensão de saída, em função do nível lógico das entradas digitais, é determinada por:

$$V_{saída} = -\frac{R_f}{6R} \frac{V_{cc}}{8} \cdot (8 \cdot d_3 + 4 \cdot d_2 + 2 \cdot d_1 + d_0) \quad (3.32)$$

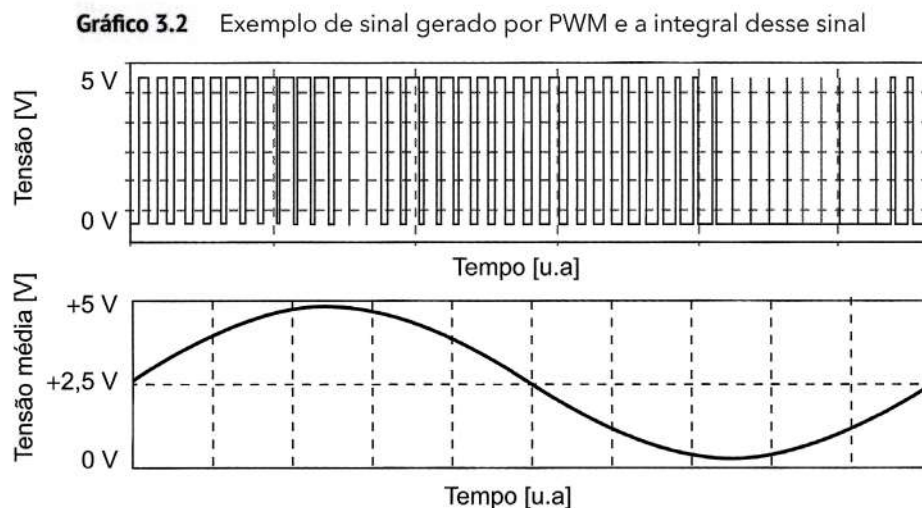
onde, igualmente ao conversor anterior, $V_{d0} = V_{CC} \cdot d_0$, onde d_0 representa a ausência de nível lógico (= 0) ou a existência dele (= 1), e assim analogamente para V_{d1} , V_{d2} e V_{d3} .

TÉCNICA DA SAÍDA POR MODULAÇÃO DE PULSOS (PWM)

A técnica mais comumente utilizada em sistemas microcontrolados é a utilização da modulação por largura de pulsos (PWM). Essa técnica utiliza a variação temporal do nível lógico 1, em relação ao nível lógico 0, de um sinal de uma onda quadrada. O sinal contínuo é obtido integrando-se o sinal de saída (colocando um capacitor em paralelo com a carga).

Assim, vamos imaginar que precisamos de uma onda senoidal analógica de saída de um microcontrolador. Esse sinal pode ser obtido variando-se a largura do pulso de uma onda quadrada, de modo que, quando precisarmos do maior nível de tensão de saída, o pulso terá 100% em nível lógico alto; quando precisarmos de um sinal de nível

lógico baixo "0", o pulso terá 0% em nível lógico alto "1" (e 100% em nível baixo "0"); e, por consequência, os níveis intermediários terão valores intermediários ponderados entre 0 e 1. O Gráfico 3.2 ilustra um sinal gerado por PWM e a média (integral) do sinal de saída.



USO DE UM CONVERSOR EXTERNO DEDICADO

Além dos circuitos apresentados anteriormente, podemos utilizar conversores dedicados (externos), que podem prover maior precisão do sinal de saída, com curtos tempos de estabilização. O DAC0808 é um exemplo desses circuitos integrados dedicados, que apresenta como principais características:

- compatibilidade de sinais TTL ou CMOS de entrada;
- 8 bits de digitais;
- tempo de estabilização de aproximadamente 150 ns; e
- alimentação entre $\pm 4,5\text{ V}$ a $\pm 18\text{ V}$.

A Figura 3.41 ilustra uma proposta de conexão desse conversor, onde as tensões de referência são +10 V e 0 V e a tensão de saída é determinada por:

$$V_{\text{saída}} = \frac{V_{\text{ref}} R_f}{256 R_l} \left(A1 \cdot 128 + A2 \cdot 64 + A3 \cdot 32 + A4 \cdot 16 + A5 \cdot 8 + A6 \cdot 4 + A7 \cdot 2 + A8 \cdot 1 \right) \quad (3.33)$$

Por último, outro conversor digital-analógico a ser ilustrado é o AD5662, o qual apresenta 16 bits de resolução digital e utiliza apenas 3 pinos para controle e operação, além da alimentação (5V-GND). A Figura 3.42 ilustra a pinagem para utilização desse conversor.



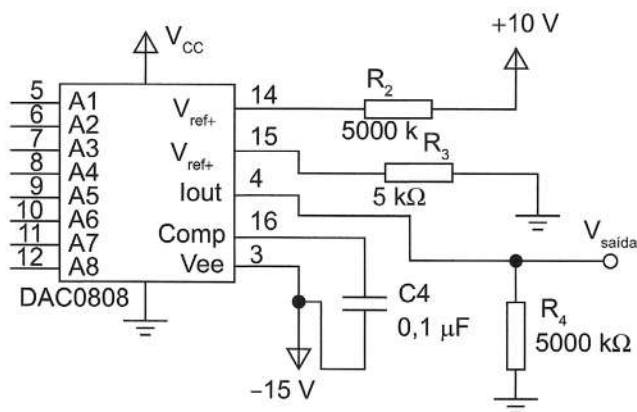


Figura 3.41 Circuito de interface do conversor DAC0808.

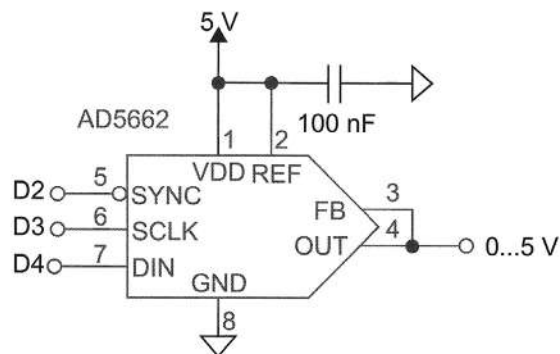


Figura 3.42 Conversor digital-analógico AD5662.

Para finalizar, a Tabela 3.1 fornece um resumo de algumas das técnicas de condicionamento de sinal abordadas, comuns para diferentes tipos de sensores e medições.

Tabela 3.1 Análise comparativa da aplicação das diferentes técnicas de condicionamento de sinais

	Amplificação	Atenuação	Isolação	Filtragem	Excitação	Linearização	CJC	Uso de pontes	Detector de pico	Detector de zero e nível DC	Retificadores	Ponte de Wyne	Diferença de fases	Oscilador	Conversão V em I	Conversão I em V
Termopar	■		■	■		■	■									
Termistor	■		■	■	■	■										
RTD	■		■	■	■	■										
Strain gauge	■		■	■	■	■		■							■	■
Carga, pressão e torque (mV/V)	■		■	■	■	■		■								
Carga, pressão e torque	■		■	■	■	■										
Acelerômetro	■		■	■	■	■										
Microfone	■			■	■	■			■		■					
Proxímetro	■			■	■	■			■	■	■					
LVDT/RVDT	■		■	■	■	■			■	■	■			■		■
Alta-tensão		■	■						■		■					
Sensores capacitivos									■	■	■	■	■	■	■	

**Revisão**

Este capítulo buscou apresentar os diferentes circuitos de condicionamento de sinais, muitas vezes necessários para intercomunicar um sensor com seu controlador (em nosso caso o Arduino).

Basicamente, é preciso respeitar o nível lógico de entrada do dispositivo (0 e 5 V das entradas digitais e 0 a 5 V ou a uma tensão de referência intermediária, para as entradas analógicas). Mas também é necessário respeitar o nível de corrente a ser drenada por cada pino e pelo microcontrolador inteiro, além de ter cuidado com a frequência de um sinal em função da limitação do dispositivo.

**Atividades de reforço**

- 1) Com base nisso, relacione alguns dos circuitos de condicionamento necessários a serem acoplados em alguns dos sensores apresentados no capítulo anterior, que complementem a Tabela 3.1.
- 2) O essencial deste capítulo, uma vez identificado qual circuito de condicionamento se utiliza com qual sensor, é “testar” os valores dos componentes do circuito para obter a parametrização necessária para o sinal. Dessa forma, analise as seguintes hipóteses e apresente os circuitos de condicionamento mais indicados para cada caso:
 - a) um sensor apresenta um sinal de saída de tensão, com variações lentas e amplitudes entre 0,5 mV e 0,580 mV;
 - b) um sinal de saída em corrente, com variações entre 4 e 20 mA;
 - c) um sinal do tipo exponencial de x;
 - d) um sinal com variação de -3 a -2 V;
 - e) um sinal com variação de 2 a 24 V.



Arduino é uma plataforma open-source, ou software livre. Software livre é o software que é distribuído juntamente com o seu código-fonte, e é liberado sob os termos que garantem aos usuários a liberdade de estudar, adaptar/modificar e distribuir o software.

Essa plataforma é baseada em microcontroladores, que são unidades de processamento com diversos periféricos internos ao mesmo circuito integrado, que possibilitam que esse dispositivo possa interagir com o ambiente e ter controle sobre atuadores no mesmo, por possuir internamente, além de uma unidade central de processamento, periféricos como memórias (tanto de dados quanto de programa), comunicação serial, conversores analógico-digitais, dispositivos PWM, comparadores, entre outros. Diferentemente de um microprocessador, um microcontrolador dispõe de praticamente tudo que precisa para operar sozinho, bastando interligar as interfaces com o meio e programá-lo com as funções desejadas.

Baseado nas linhas de microcontroladores da ATMEL, o Arduino nada mais é que uma plataforma de padronização de interfaces de hardware que permite uma programação de fácil acesso e aprendizagem, o que tem resultado em aumento contínuo dos interessados e colaboradores nesse tipo de ferramenta.

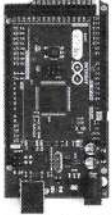
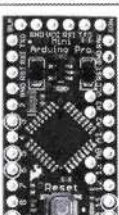
A Tabela 4.1 ilustra os diferentes hardwares e versões de Arduino disponíveis no mercado. É importante notar que temos diferentes processadores, cada um com suas peculiaridades de pinagem, memória, processamento e periféricos disponíveis.

Além disso, é uma plataforma que permite o desenvolvimento de hardware livre, o que permite que diversos fornecedores atendam a suas características padronizadas e desenvolvam placas periféricas chamadas de shields.

A utilização dessas placas periféricas permite que sejam eliminados problemas de conexões inerentes a maus contatos ou erros de conexão. Isso certamente acelera o desenvolvimento de qualquer projeto, reduzindo o tempo inicial, normalmente gasto com acerto de hardware, e concentra o desenvolvimento na aplicação em si.

As principais shields terão suas características abordadas mais à frente.

Tabela 4.1 Comparativo entre as diferentes versões da plataforma Arduino

	Arduino Uno	Arduino Mega2560	Arduino Leonardo	Arduino Due	Arduino ADK	Arduino Nano	Arduino Mini
							
Microcontrolador	ATmega328	ATmega2560	ATmega32u4	AT91SAM3X8E	ATmega2560	ATmega168 (versão 2x) ou ATmega328 (versão 3x)	ATmega168
Portas digitais	14	54	20	54	54	14	14
Portas PWM	6	15	7	12	15	6	6
Portas analógicas	6	16	12	12	16	8	8
Memória	32 K (0,5 K usado pelo bootloader)	256 K (8 K usados pelo bootloader)	32 K (4 K usados pelo bootloader)	512 K disponível para aplicações	256 K (8 K usados pelo bootloader)	16 K (ATmega168 ou 32 K (ATmega328), 2 K usados pelo bootloader)	16 K (2 K usados pelo bootloader)
Clock	16 Mhz	16 Mhz	16 Mhz	84 Mhz	16 Mhz	16 Mhz	8 Mhz (modelo 3.3v) ou 16 Mhz (modelo 5v)
Conexão	USB	USB	Micro USB	Micro USB	USB	USB Mini-B	Serial/Módulo USB externo
Conector para alimentação externa	Sim	Sim	Sim	Sim	Sim	Não	Não
Tensão de operação	5 V	5 V	5 V	3,3 V	5 V	5 V	3,3 V ou 5 V, dependendo do modelo
Corrente máxima portas E/S	40 mA	40 mA	40 mA	130 mA	40 mA	40 mA	40 mA
Alimentação	7 - 12 V _{DC}	7 - 12 V _{DC}	7 - 12 V _{DC}	7 - 12 V _{DC}	7 - 12 V _{DC}	7 - 12 V _{DC}	3,35 - 12 V (modelo 3.3), ou 5 - 12 V (modelo 5v)



ARDUINO UNO

O UNO é a versão mais disseminada da família Arduino, por contar com um microcontrolador de entrada, bons recursos e um número de interfaces suficiente para muitos projetos simples. Conta com algumas versões que se diferenciam principalmente pelo modo de fixação do microcontrolador, além de melhorias no circuito de reset (e posição do botão de reset na placa), como ilustrado na Figura 4.1.

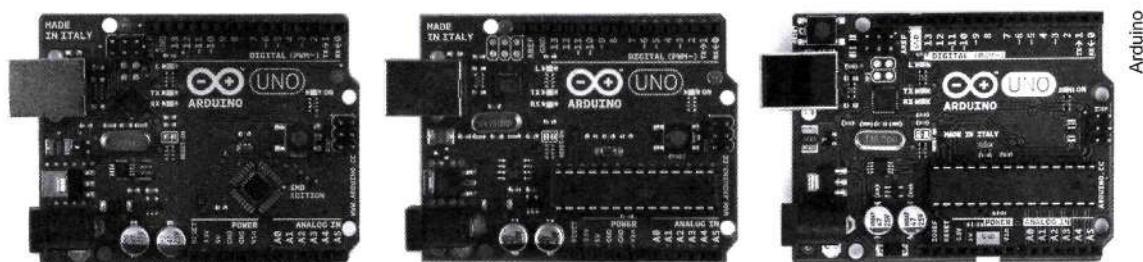


Figura 4.1 Arduino UNO.

O UNO é uma placa baseada no microcontrolador ATMEGA 328, a qual possui 14 pinos de entrada/saída digital (dos quais 6 podem ser usados como saídas PWM), 6 entradas analógicas, um cristal oscilador (clock) de 16 MHz. Internamente, ele possui um oscilador de 32 kHz que pode ser usado, por exemplo, em situações de baixo consumo.

Possui uma conexão USB, a qual é utilizada como porta serial, ou também pode ser utilizada para alimentar o circuito eletrônico. A alimentação, quando não realizada pela USB, é feita por fonte externa, preferencialmente entre 7 e 12 V contínua.

Ainda sobre a alimentação, o ATMEGA opera a 5 V, obtida por um regulador de tensão. Outra tensão disponível é 3,3 V, obtida por outro regulador, e, nesse caso, limitado a uma corrente máxima de 50 mA. Há ainda um pino Vin, que se refere à tensão de alimentação da placa, quando esta for realizada por uma fonte externa.

A Figura 4.2 ilustra as principais interfaces da placa Arduino UNO, além de apontar o microcontrolador.

Atualmente está em sua versão R3, na qual foram adicionados pinos de comunicação serial (SDA e SCL) próximos à entrada de referência (AREF) e melhorias no circuito de reset.

Tensões presentes na placa:

- **3,3 V:** fornece tensão de 3,3 V para alimentação de shield e módulos externos. Corrente máxima de 50 mA.
- **5 V:** fornece tensão de 5 V para alimentação de shields e circuitos externos.
- **GND:** pinos de referência, terra.
- **VIN:** pino para alimentar a placa através de shield ou bateria externa. Quando a placa é alimentada através do conector Jack, a tensão da fonte estará nesse pino.

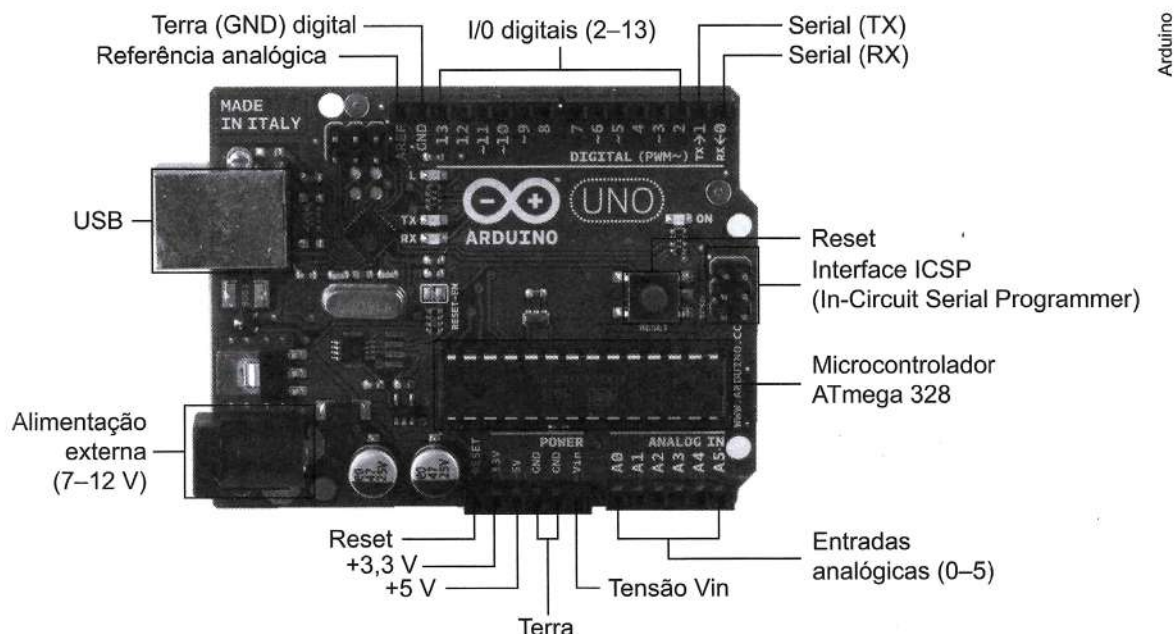


Figura 4.2 Interfaces de comunicação e alimentação do Arduino UNO.

INTERFACE USB

O Arduino utiliza um microcontrolador específico (ATMEL ATMEGA16U2) para controle da comunicação USB com o exterior. Esse microcontrolador é o responsável pela forma transparente como funciona a placa Arduino UNO, possibilitando o upload do código binário gerado após a compilação do programa feito pelo usuário.

Para sinalizar a comunicação serial pela USB, a placa conta com dois leds (TX, RX), que indicam o envio e a recepção de dados da placa para o computador. Esse microcontrolador possui um cristal externo de 16 MHz. A comunicação desse microcontrolador com o ATMEL ATMEGA328 (microcontrolador principal do Arduino) é realizada pelo canal serial de ambos. Outro ponto interessante que facilita o uso da placa Arduino é a conexão do pino 13 do ATMEGA16U2 ao circuito de RESET do ATMEGA328, possibilitando a entrada no modo bootloader automaticamente quando é pressionado o botão Upload na IDE. Essa característica não acontecia nas primeiras placas Arduino, onde era necessário pressionar o botão de RESET antes de fazer o upload na IDE.

O MICROCONTROLADOR

O componente principal da placa Arduino UNO é o microcontrolador ATMEL ATMEGA328, que é um microcontrolador de 8 bits da família AVR com arquitetura RISC avançada, que pode ser encontrado em encapsulamento DIP28 ou SMD. O ATMEGA328



possui 32 KB de memória de programa do tipo Flash (mas 512 bytes são utilizados para o bootloader), 2 KB de dados do tipo RAM e 1 KB de memória de dados do tipo EEPROM.

Na plataforma UNO, esse microcontrolador utiliza um cristal externo de 16 MHz (conectado nos pinos 9 e 10 do microcontrolador na versão DIP e 7 e 8 nas versões SMD). Entretanto, em um projeto dedicado ou até mesmo em caso de substituição do cristal padrão da placa Arduino, ele pode operar com uma frequência máxima de 20 MHz.

A versão DIP possui 28 pinos, dos quais 23 podem ser utilizados como I/O. Desses 23, 14 são utilizados para I/O digitais, 6 são entradas analógicas, 2 estão utilizados para o oscilador e o último, para o reset. A Figura 4.3 ilustra a pinagem do ATMEGA328, bem como ilustra as funções de cada pino.

As 14 I/O digitais operam com nível lógico alto de 5 V e baixo de 0 V, e suportam uma corrente máxima de 40 mA. Entretanto, a soma das correntes de todo os pinos do microcontrolador não pode ultrapassar 200 mA. Cada pino possui um resistor de pull-up interno que pode ser habilitado por software. Alguns desses pinos possuem outras funções, como PWM, comunicação serial e interrupção externa.

Os pinos 0 e 1 do Arduino podem ser utilizados para comunicação serial. Esses pinos também são responsáveis pela comunicação serial via USB com o PC.

Já os pinos 2 e 3 possuem prioridade de evento externo, chamado de interrupção externa, quando configurados para tal, através da função *attachInterrupt()*, enquanto os pinos 3, 5, 6, 9, 10 e 11 podem ser usados como saídas PWM de 8 bits através da função *analogWrite()*.

Os pinos A0 a A5 são 6 canais analógicos, que operam com um conversor analógico-digital (ADC) de 10 bits. Internamente, por definição, o conversor opera com referências de 0 a 5 V, proporcionando uma resolução de 4,882 mV (5 V/1024). Essa resolução pode ser alterada se utilizarmos um valor de referência positiva maior que 0 V e menor que 5 V, utilizando o pino AREF. Nesse caso, se utilizarmos uma tensão de referência de 1,5 V, a resolução máxima do ADC será de 1,466 mV (1,5 V/1024).

A Figura 4.3 ilustra a relação entre os pinos do microcontrolador ATMEL ATMEGA328 e a pinagem do Arduino UNO.

Além do conversor analógico-digital, ele possui outros periféricos, dentre os quais destacam-se: três periféricos de comunicação serial, sendo um USART que funciona a até 250 kbps, um SPI, que vai a até 5 MHz, e um I2C que pode operar até 400 kHz; um comparador analógico interno ao CI e diversos temporizadores, além de 6 PWMs já comentadas.

Uma das grandes vantagens de plataformas como essa é o modo de programação, o qual é realizado pelo próprio microcontrolador auxiliar que faz a interface com o computador através da serial USB. Entretanto, a programação direta também pode ser realizada através do conector ICSP (*in-circuit serial programming*), mas nesse caso é preciso ter um programador externo específico para microcontroladores da família ATMEL ATMEGA.

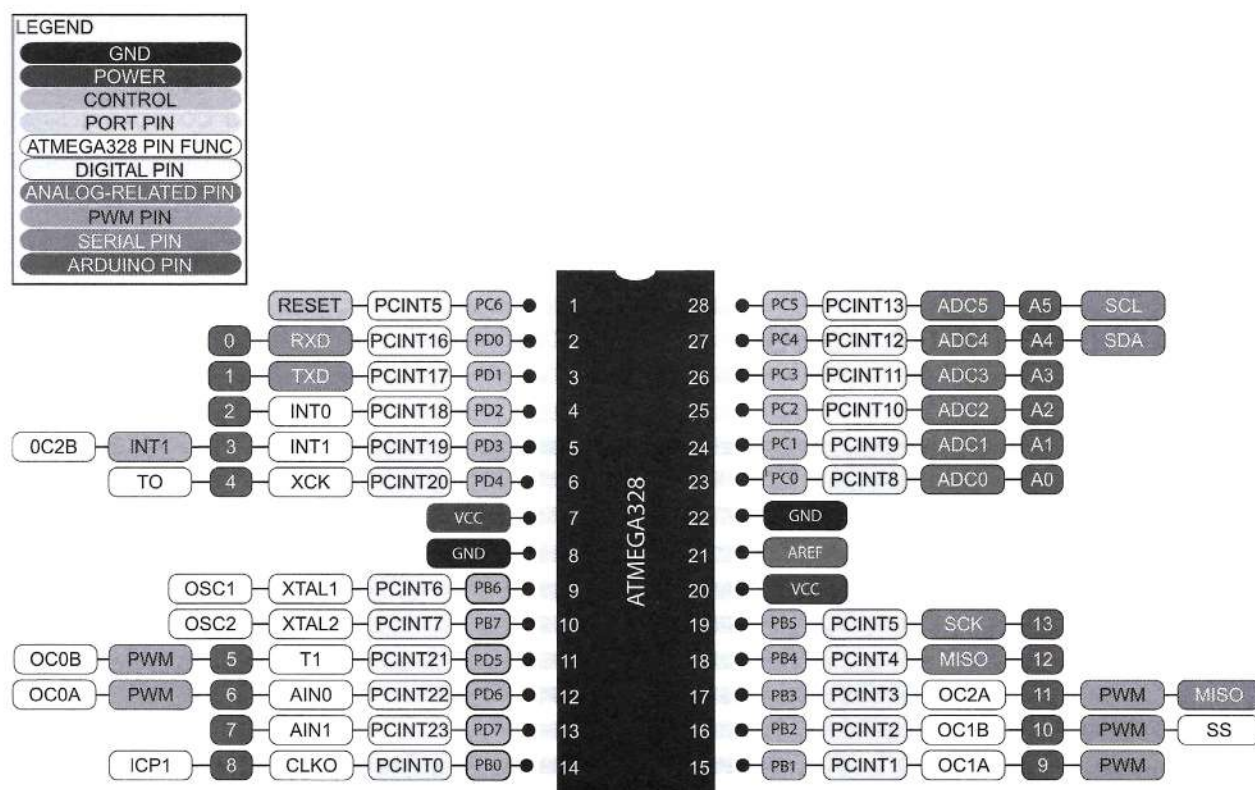


Figura 4.3 Esboço de pinagem e funções do ATMEGA328.

As principais características do Arduino UNO estão resumidas na Tabela 4.2.

Tabela 4.2 Principais características da plataforma Arduino UNO

Microcontrolador	ATMEGA328
Tensão de funcionamento	5 V
Tensão de entrada fonte externa (recomendado)	7-12 V
Tensão de entrada (limites)	6-20 V
Digital pinos de I/O	14 (dos quais 6 fornecem uma saída de PWM)
Entrada Analógica	6
Corrente DC para pinos I/O	40 mA
Corrente DC para Pino 3,3 V	50 mA
Memória Flash	32 KB (ATMEGA328), dos quais 0,5 KB utilizado pelo bootloader
SRAM	2 KB (ATMEGA328)
EEPROM	1 KB (ATMEGA328)
Velocidade de clock	16 MHz

Fonte: Adaptado de Arduino, 2015.



Para aplicações em que o número de pinos de I/O disponível no UNO é restrito, ou o número de temporizadores ou a quantidade de interrupções disponíveis são limitados, ou ainda a memória interna aponte uma restrição com relação à complexidade do projeto, uma alternativa é optar por uma plataforma estendida. Um exemplo é a plataforma Arduino MEGA.

ARDUINO MEGA 2560

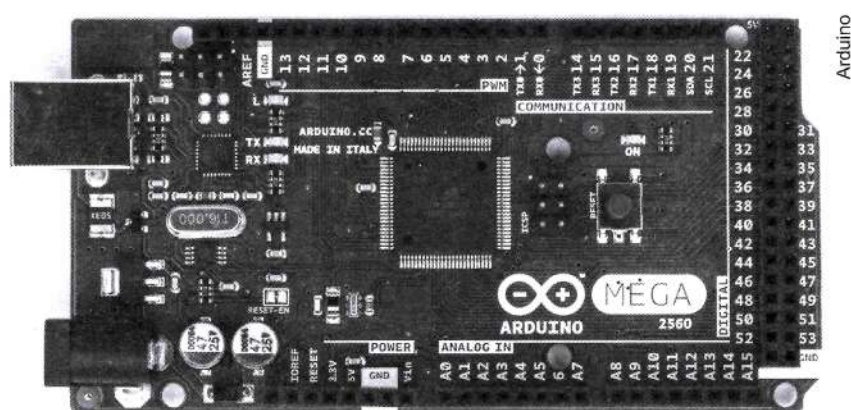


Figura 4.4 Arduino MEGA 2560.

A placa Arduino MEGA 2560 baseia-se no microcontrolador ATMEGA2560, que é um microcontrolador de baixo custo de 8 bits baseado em arquitetura RISC de 100 pinos, sendo 86 pinos de propósito geral, dos quais 16 permitem ser utilizados como saídas PWM, possui 4 blocos de comunicação serial (USART), além de dispor de 16 canais analógico-digitais. Desses, estão organizados no Arduino MEGA 2560 54 pinos de entradas e saídas (I/O) digitais, 12 dos quais podem ser utilizados como saídas PWM, 16 entradas analógicas e ainda 4 portas de comunicação serial.

A alimentação do Arduino MEGA ocorre de forma semelhante à do UNO, uma vez que ambos operam a uma tensão de 5 V, sendo realizada pela conexão externa específica (de 7 a 12 V) ou através da porta serial USB.

Além de um maior número de periféricos e I/O disponíveis, essa versão conta com 256 KBytes de memória de programa do tipo Flash (dos quais 8 KB são utilizados para o bootloader) e 4 KBytes de memória de dados EEPROM e 8 KBytes de memória de dados RAM.

TENSÕES E ALIMENTAÇÃO

- **IOREF:** fornece uma tensão de referência para que shields possam selecionar o tipo de interface apropriada. Dessa forma, shields que funcionam com a placas Arduino que são alimentadas com 3,3 V podem se adaptar para serem utilizadas em 5 V e vice-versa.

- **RESET:** pino conectado a pino de RESET do microcontrolador. Pode ser utilizado para um reset externo da placa Arduino.
- **3,3 V:** fornece tensão de 3,3 V para alimentação de shield e módulos externos. Corrente máxima de 50 mA.
- **5 V:** fornece tensão de 5 V para alimentação de shields e circuitos externos.
- **GND:** pinos de referência, ground, terra.
- **VIN:** pino para alimentar a placa através de shield ou bateria externa. Quando a placa é alimentada através do conector Jack, a tensão da fonte estará nesse pino.

De forma análoga ao UNO, no MEGA a interface USB é controlada por um microcontrolador auxiliar (ATMEGA16U2), que está conectado ao microcontrolador principal (ATMEGA2560) através do canal serial. Também utiliza um cristal oscilador de 16 MHz, o que proporciona 16 MIPS (a execução de 16 milhões de instruções por segundo)

Dentre outros periféricos adicionais, destacam-se: possui multiplicador por hardware, 4 canais de comunicação serial, 16 entradas analógicas e 15 saídas PWM. Possui ainda comunicação SPI, I²C e 6 pinos de interrupções externas.

Os 54 pinos de entradas e saídas digitais operam igualmente ao UNO, com tensões de 0 V para nível baixo e de 5 V para nível lógico alto, podendo fornecer ou drenar até 40 mA por pino. Alguns desses pinos possuem outras funções adicionais, a destacar:

- **Comunicação Serial:** Serial 0: 0 (RX) e 1 (TX); Serial 1: 19 (RX) e 18 (TX); Serial 2: 17 (RX) e 16 (TX); Serial 3: 15 (RX) e 14 (TX). Os pinos 0 e 1 estão conectados aos pinos do ATMEGA16U2 responsáveis pela comunicação USB.

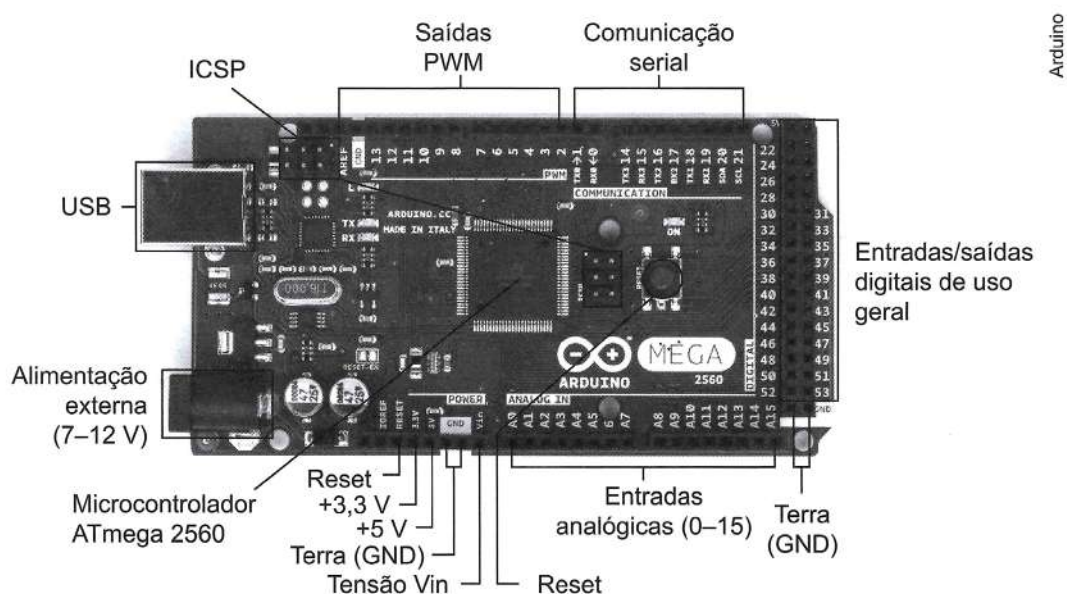


Figura 4.5 Interfaces de comunicação e alimentação do Arduino MEGA2560.



- **Interrupções externas:** 2 (interrupt 0), 3 (interrupt 1), 18 (interrupt 5), 19 (interrupt 4), 20 (interrupt 3), e 21 (interrupt 2). Esses pinos podem ser configurados para disparo da interrupção tanto na borda de subida quanto na de descida, ou em níveis lógicos alto ou baixo, conforme a necessidade do projeto.
- **PWM:** os pinos 2 a 13 e 44 a 46 podem ser utilizados como saídas PWM. O sinal PWM possui 8 bits de resolução.
- **Comunicação SPI:** pinos: 50 (MISO), 51 (MOSI), 52 (SCK), 53 (SS). Esses pinos estão ligados ao conector ICSP.
- **Comunicação I2C:** pinos 20 (SDA) e 21 (SCL).

A Arduino MEGA2560 possui 16 canais analógicos (pinos A0 a A15), suportados por um conversor analógico digital de 10 bits, ou seja, um valor entre 0 e 5 V é convertido entre 0 e 1023. Quando utilizado o pino de referência analógica AREF (entre 0 e 5 V), a resolução pode ser alterada, semelhantemente ao proposto na versão UNO.

A placa Arduino MEGA 2560 é uma ótima opção para expandir seus projetos, quando há a necessidade de mais pinos ou quantidade de memória de programa. Possui desempenho semelhante ao da placa Arduino UNO, porém possibilitando maior quantidade de recursos, como mais entradas analógicas e saídas PWM. Com essas características, pode atender facilmente projetos de automação residencial, comercial e até de pequenas empresas, como também atender projetos de robótica e vários projetos eletrônicos.

As principais características do Arduino MEGA2560 estão resumidas na Tabela 4.3.

A Figura 4.6 ilustra mais detalhadamente todas as interfaces e pinagem da placa Arduino MEGA2560.

Tabela 4.3 Principais características da plataforma Arduino MEGA2560

Microcontrolador	ATMEGA2560
Tensão de funcionamento	5 V
Tensão de entrada (recomendado)	7-12 V
Tensão de entrada (limites)	6-20 V
Digital pinos de I/O	54 (dos quais 15 podem operar com saída PWM)
Entrada analógica pinos	16
DC Current per I/O Pin	40 mA
Corrente DC para Pin 3.3V	50 mA
Memória Flash	256 KB dos quais 8 KB usados pelo bootloader
SRAM	8 KB
EEPROM	4 KB
Velocidade de clock	16 MHz

Fonte: Adaptado de Arduino, 2015.

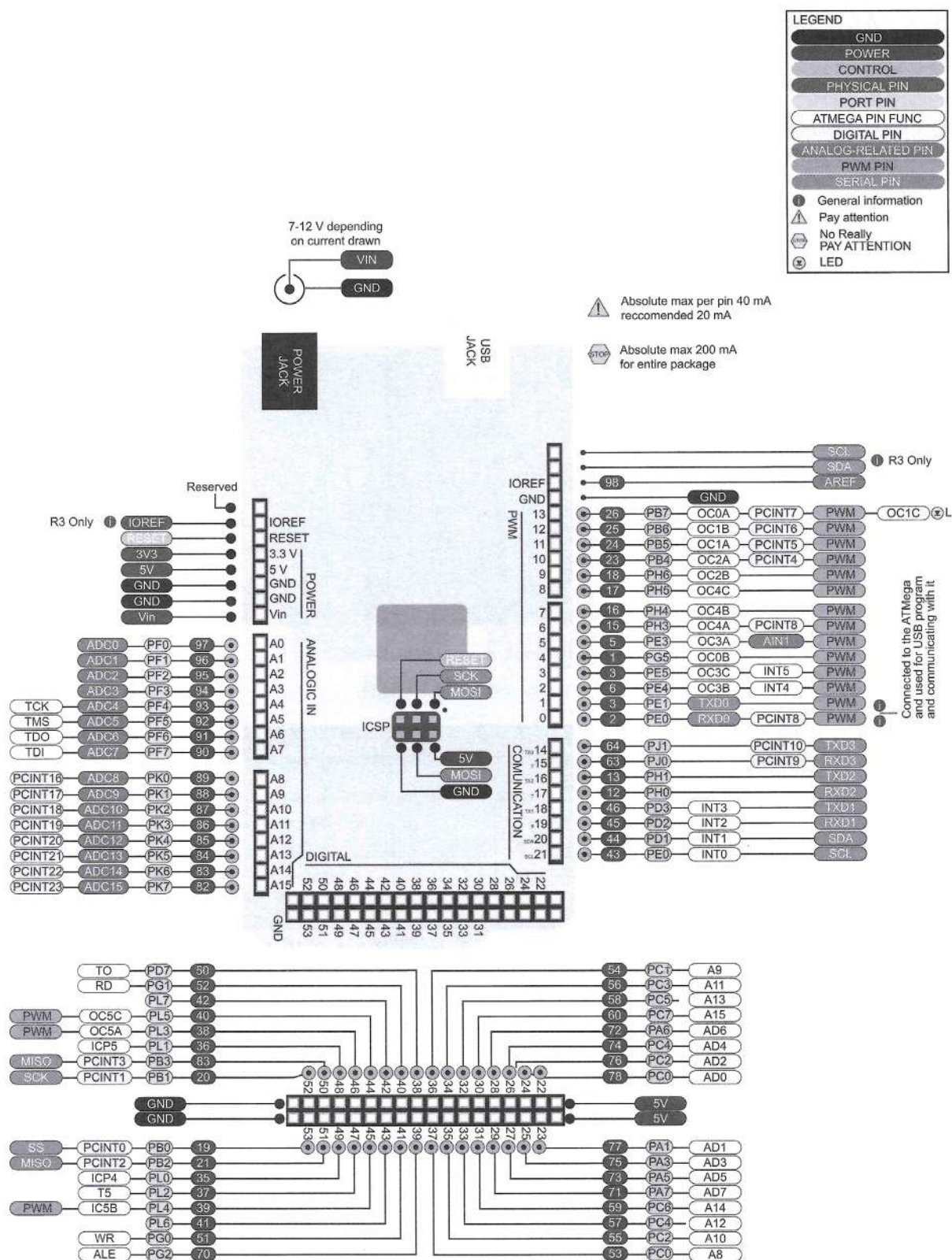


Figura 4.6 Detalhamento da pinagem do Arduino MEGA2560.



ARDUINO LEONARDO

O Arduino LEONARDO difere de outras placas Arduino por ter um único microcontrolador para gerenciar a aplicação desenvolvida (sketch) e controlar a comunicação com o computador pela USB.

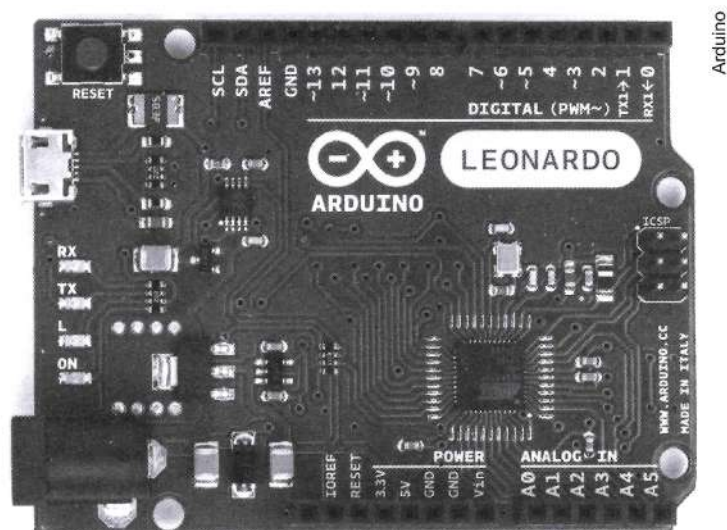


Figura 4.7 Arduino LEONARDO.

Microcontrolador	ATMEGA32u4
Tensão de operação	5 V
Tensão de entrada (recomendado)	7-12 V
Limites de tensão de entrada	6-20 V
Pinos de I/O digital	20
Canais PWM (Número de bits PWM)	7 : (pinos 3, 5, 6, 9, 10, 11, e 13) (PWM de 8 bits de variação)
Canais analógicos	12 : (A0-A5 e A6-A11 - nos pinos digitais 4, 6, 8, 9, 10, e 12), todos com 10 bits de resolução, entre 0 e 5V ou 0 e AREF (referência)
Corrente máxima por I/O digital	40 mA
Corrente máxima por pino 3,3 V	50 mA
Memória de programa Flash	32 KB dos quais 4 KB são usados pelo bootloader
Interrupções externas	5
Memória SRAM	2,5 KB (ATMEGA32u4)
Memória EEPROM	1 KB (ATMEGA32u4)
Frequência de clock	16 MHz

ARDUINO MICRO

O Arduino MICRO é similar ao LEONARDO, pois utiliza somente o microcontrolador ATMEGA32u4 também para a comunicação USB, eliminando a necessidade de um segundo microcontrolador.

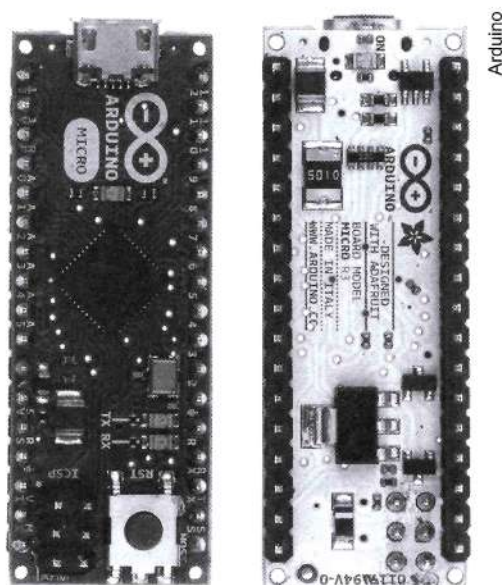


Figura 4.8 Arduino MICRO.

Microcontrolador	ATMEGA32u4
Tensão de operação	5 V
Tensão de entrada recomendada	7-12 V
Limites de tensão de entrada	6-20 V
Pinos de I/O digital	20
Canais PWM	7
Canais de entrada analógica	12
Corrente máxima por pino de I/O	40 mA
Corrente máxima por pino de 3,3 V	50 mA
Memória de programa - Flash	32 KB (4 KB para bootloader)
Memória SRAM	2,5 KB (ATMEGA32u4)
Memória EEPROM	1 KB (ATMEGA32u4)
Frequência de clock	16 MHz



ARDUINO DUEMILANOVE

O Arduino DUEMILANOVE (2009) baseia-se no microcontrolador ATMEGA168 ou no ATMEGA328. Possui 14 pinos de I/O digitais, dos quais 6 podem ser utilizados como saídas PWM e 6 como entradas analógicas e conexão USB. Opera com um oscilador de 16 MHz.

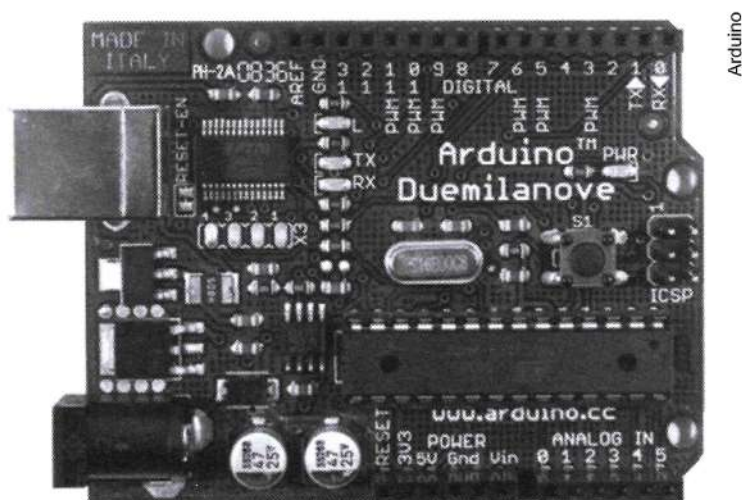


Figura 4.9 Arduino DUEMILANOVE.

Microcontrolador	ATMEGA168
Tensão de operação	5 V
Tensão de entrada (recomendado)	7-12 V
Limites de tensão de entrada	6-20 V
Pinos de I/O digital	14 (6 podem ser saídas PWM)
Pinos de entrada analógica	6
Corrente máxima por pino de I/O	40 mA
Corrente máxima por pino de 3,3 V	50 mA
Memória de programa - Flash	16 KB (ATMEGA168) or 32 KB (ATMEGA328) dos quais 2 KB são usados pelo bootloader
SRAM	1 KB (ATMEGA168) ou 2 KB (ATMEGA328)
EEPROM	512 bytes (ATMEGA168) ou 1 KB (ATMEGA328)
Frequência de clock	16 MHz

ARDUINO NANO

O Arduino NANO é uma pequena e completa plataforma Arduino, baseada no ATMEGA328. Ele tem praticamente as mesmas funcionalidades do Arduino DUEMILANOVE.

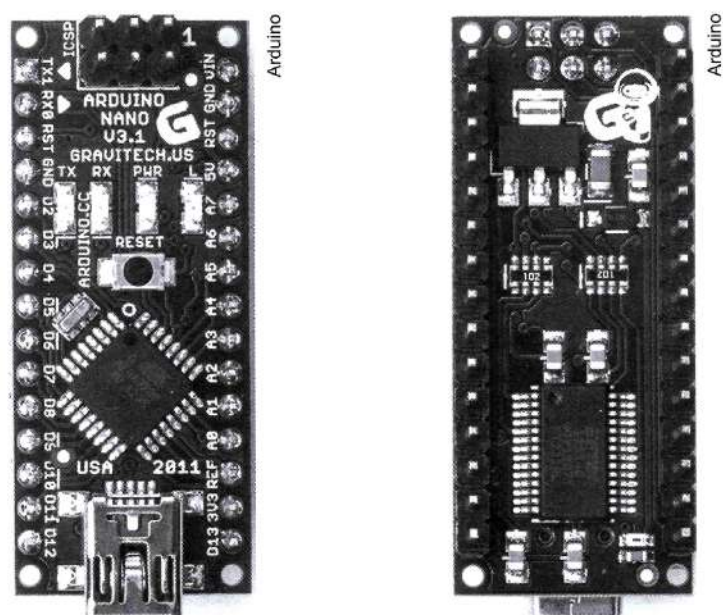


Figura 4.10 Arduino NANO.

Microcontrolador	Atmel ATMEGA168 ou ATMEGA328
Tensão de operação	5 V
Tensão de entrada (recomendado)	7-12 V
Limites de tensão de entrada	6-20 V
Pinos de I/O digital	14 (dos quais 6 dispõem de saídas PWM)
Pinos de entrada analógica	8
Nível de corrente por Pino I/O	40 mA
Memória de programa - Flash	16 KB (ATMEGA168) ou 32 KB (ATMEGA328) (2 KB para bootloader)
Memória SRAM	1 KB (ATMEGA168) ou 2 KB (ATMEGA328)
Memória EEPROM	512 bytes (ATmega168) ou 1 KB (ATMEGA328)
Frequência de clock	16 MHz



ARDUINO ETHERNET

O Arduino ETHERNET baseia-se no microcontrolador ATMEGA328. Possui 14 I/O digitais, 6 pinos analógicos, conexão RJ45 e USB, e opera com um oscilador de 16 MHz.

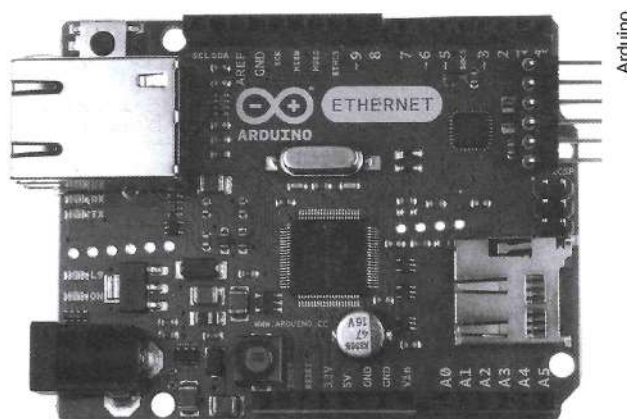


Figura 4.11 Arduino ETHERNET.

Microcontrolador	ATMEGA328
Tensão de operação	5 V
Tensão de entrada (recomendada)	7-12 V
Limites de tensão de entrada	6-20 V
Limites de tensão de entrada PoE	36-57 V
Pinos de I/O digital	14 (dos quais 4 oferecem saída PWM)
Pinos reservados do Arduino	10 a 13: usados para SPI 4: usado para o cartão de memória SD 2: interrupção usado por W5100
Pinos de entrada Analógica	6
Corrente máxima por pino I/O	40 mA
Corrente máxima por pino de 3,3 V	50 mA
Memória de programa - Flash	32 KB (0,5 KB usado pelo bootloader)
Memória SRAM	2 KB (ATMEGA328)
Memória EEPROM	1 KB (ATMEGA328)
Frequência de clock	16 MHz
Controlador embarcado de TCP/IP (Ethernet): W5100	
Suporte a cartão Micro SD, com tradutores de tensão ativos	

ARDUINO ZERO

Essa é a primeira placa Arduino com processador Cortex M+, de 32 bits, com as padronizações geométricas da família Arduino UNO.

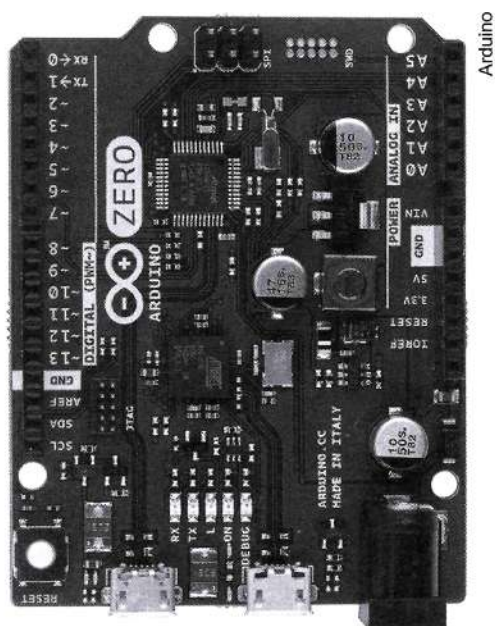


Figura 4.12 Arduino ZERO.

Microcontrolador	ATSAMD21G18, 48pins LQFP
Núcleo do microcontrolador	32-bit ARM Cortex® M0+
Tensão de operação	3,3 V
Pinos de I/O digitais	14, com 12 PWM e UART
Pinos de entrada analógica	6 canais (ADC de 12 bits)
Pinos de saída analógica	1 (DAC de 10 bits com velocidade de até 350 ksps)
Corrente máxima por pino I/O	7 mA
Memória de programa - Flash	256 KB
Memória SRAM	32 KB
Memória EEPROM	Acima de 16 KB por emulação
Frequência de clock	48 MHz
Consumo ativo	até 70 μ A/MHz
Consumo em baixo consumo	menos de 2 μ A



ARDUINO DUE

O Arduino DUE é baseado no microcontrolador Atmel SAM3X8E-Cortex-M3. É o primeiro Arduino baseado em um núcleo de 32 bits. Possui 54 I/O digitais (dos quais 12 podem ser utilizadas como saídas PWM), 12 entradas analógicas, 4 portas seriais em hardware (USART) e opera com um clock de 84 MHz, além de possuir 2 conversores digital-analógicos (DAC).

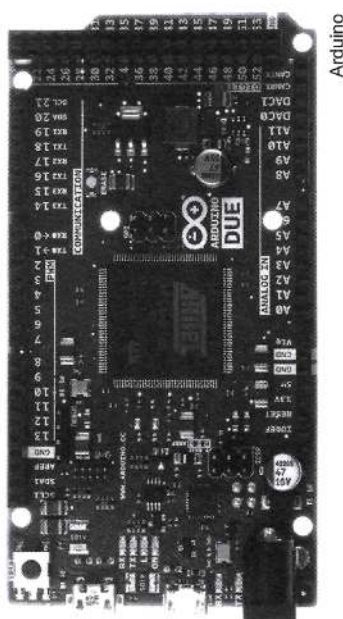


Figura 4.13 Arduino DUE.

Microcontrolador	AT91SAM3X8E
Tensão de operação	3,3 V
Tensão de entrada recomendada	7-12 V
Limites de tensão de entrada	6-16 V
Pinos de I/O digital	54 (dos quais 12 oferecem saída PWM)
Pinos de entrada analógica	12
Pinos de saída analógica	2 (DAC)
Corrente máxima de saída (placa)	130 mA
Corrente máxima pinos de 3,3 V	800 mA
Corrente máxima I/O de 5 V	800 mA
Memória de programa - Flash	512 KB disponíveis para usuário
Memória SRAM	96 KB (dois bancos: 64 KB e 32 KB)
Frequência de Clock	84 MHz

ARDUINO ADK

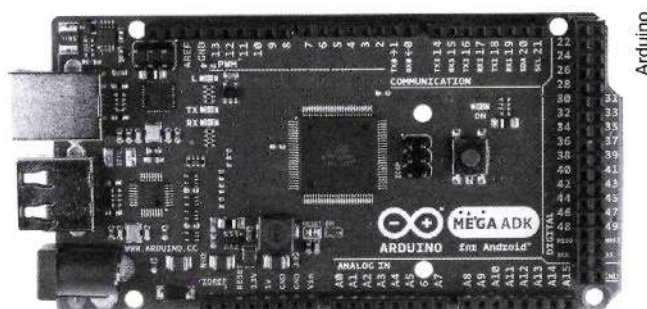


Figura 4.14 Arduino ADK.

O Arduino MEGA ADK é um microcontrolador baseado no ATMEGA2560. Possui uma interface USB host baseado no CI MAX3421, para conectá-lo com dispositivos Android.

Possui 54 pinos de I/O digitais, dos quais 15 podem ser usados como saída PWM, 16 entradas analógicas, 4 portas seriais (USARTs), e opera com um clock de 16 MHz.

ARDUINO YUN

Baseia-se nos microcontroladores ATMEGA32u4 e Atheros AR9331. O processador Atheros suporta uma distribuição Linux baseada no OpenWrt chamada OpenWrt-Yun. A placa conta com suporte a Ethernet e Wi-fi, uma porta USB-A, slot de cartão micro-SD. Possui 20 pinos de I/O digitais, dos quais 7 podem ser usados como saída PWM e 12 como entradas analógicas, e opera com um cristal de frequência de 16 MHz.

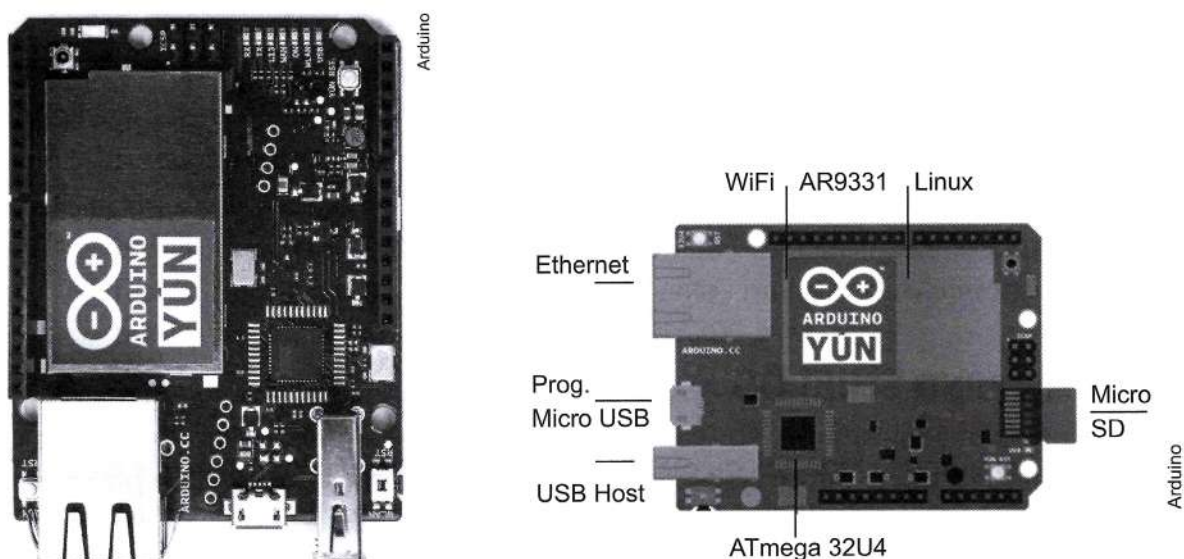


Figura 4.15 Arduino YUN.

Microcontrolador	ATMEGA32u4
Tensão de operação	5 V
Tensão de entrada	5 V
Pinos de I/O digital	20
Canais PWM	7
Canais de entrada analógica	12
DC Current per I/O Pin	40 mA
DC Current for 3.3 V Pin	50 mA
Memória de programa - Flash	32 KB (4 KB usados pelo bootloader)
Memória SRAM	2,5 KB
Memória EEPROM	1 KB
Frequência de clock	16 MHz
Linux microprocessador	
Processador	Atheros AR9331
Arquitetura	MIPS @400MHz
Tensão de operação	3,3 V
Protocolo de Ethernet	IEEE 802.3 10/100 Mbit/s
Protocolo de Wi-Fi	IEEE 802.11b/g/n
USB Tipo A	2.0 Host
Leitor de cartão	Micro-SD
Memória RAM	64 MB DDR2
Memória Flash	16 MB

Existem outras placas que estão sendo constantemente desenvolvidas pelo próprio fabricante Arduino. Para buscar mais informações, acesse o endereço eletrônico (<<http://arduino.cc/>>). Também existem outras placas em constante desenvolvimento e que respeitam a compatibilidade com a plataforma Arduino. Informações adicionais a respeito de compatibilidade ou recursos adicionais que uma nova placa pode oferecer podem ser encontradas na internet, nos datasheets dos fabricantes.

ARDUINO SHIELDS

Um dos grandes legados da plataforma Arduino é a padronização de características geométricas, posição de pinos de entradas e saídas e tensões e a possibilidade de conexões rápidas através de conectores de barras de pinos que permitem a conexão de periféricos sobre a plataforma Arduino utilizada.



Esses periféricos são chamados de **shields**, que são placas de circuito impresso que utiliza a padronização geométrica e de pinos para alimentar e se comunicar com os periféricos adicionados, aumentando as funcionalidades disponíveis.

Justamente por haver padronização, uma vez acoplada uma shield sobre uma plataforma Arduino, há a possibilidade de uma outra shield ser acoplada à anterior, reduzindo a necessidade do uso de condutores de interligação, acelerando o processo de desenvolvimento de projeto em sua fase de concepção. A Figura 4.16 ilustra justamente esse acoplamento de placas de expansão (shields) em uma plataforma Arduino e o acoplamento de várias placas na mesma estrutura.

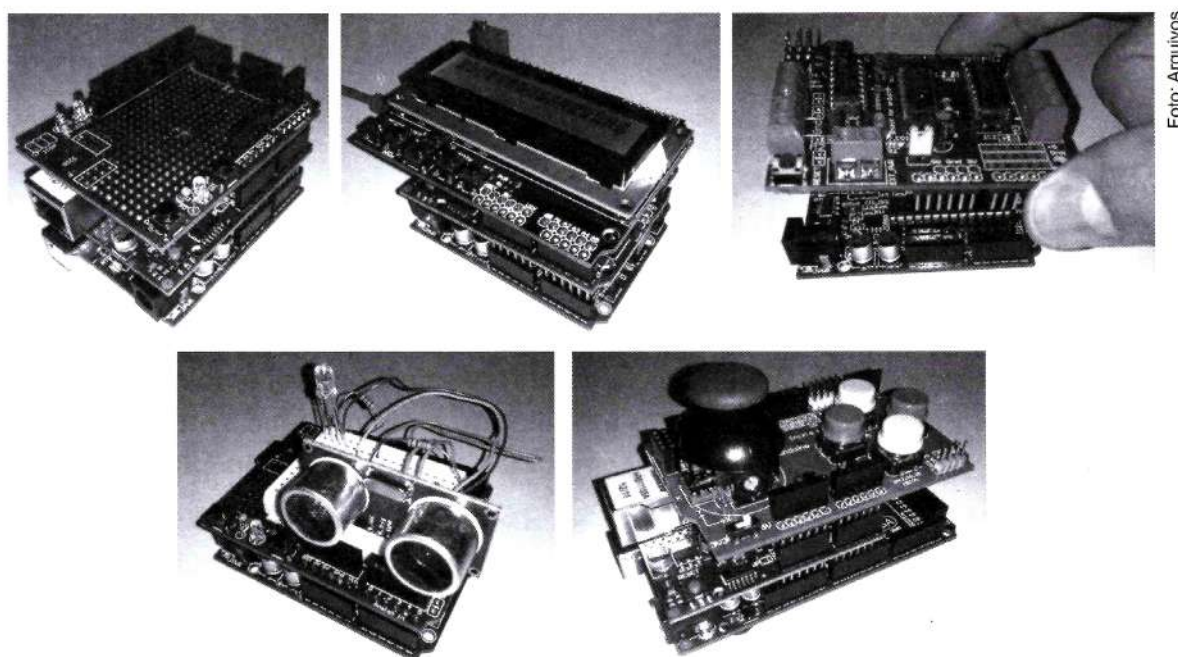


Figura 4.16 Ilustração do acoplamento de placas de expansão (shields) no Arduino.

Claro que, para o bom acoplamento de diversas shields devem ser observados diversos critérios, como:

- compatibilidade de pinos entre periféricos, evitando conflito entre eles;
- monitoramento da corrente total de drenagem do Arduino (o qual tem um fusível para proteção de picos de correntes maiores que 500 mA, embora a média de consumo não deva passar dos 200 mA).

Existe uma variedade enorme de shields compatíveis com a plataforma Arduino. A seguir iremos apresentar as principais características de algumas das principais shields comerciais que ajudarão na maioria dos projetos iniciais e que serão utilizadas no próximo capítulo, quando serão vistos exemplos de aplicação/projetos com Arduino. Com isso, nosso objetivo é informar ao leitor quais são os periféricos mais comumente aplicáveis em projetos e automação e instrumentação, e quais são as características de cada

módulo “pronto”. Dessa forma, o leitor poderá avaliar mais adequadamente seu projeto em termos de solução, acelerando a fase de desenvolvimento do projeto. De forma alguma temos a pretensão de apresentar todas as shields, até mesmo porque estão em constante desenvolvimento.

Uma vez desenvolvido um projeto com o Arduino e com as shields de extensão, cabe ao projetista projetar a sua placa de circuito impresso utilizando o microcontrolador do Arduino que usou. Entretanto, haverá muitas situações em que o projeto final pode ficar na própria solução do Arduino.

Para esta apresentação, faremos uma separação básica em função da aplicação, sendo shields de comunicação, controle de motores (potência), expansão de portas e controle de sensores e, por fim, shields de interface homem-máquina.

SHIELDS DE COMUNICAÇÃO

As principais shields de comunicação são: comunicação ethernet (com dois modelos facilmente encontrados comercialmente, e inclusive uma plataforma Arduino já com o controle de Ethernet implantado na mesma placa), comunicação serial RS232 e RS485, e comunicação sem fio Bluetooth. Entretanto, existem outras como a de rede wi-fi, a de suporte aos módulos ZigBee, alguns módulos de radiofrequência etc.

SHIELD RS232/485

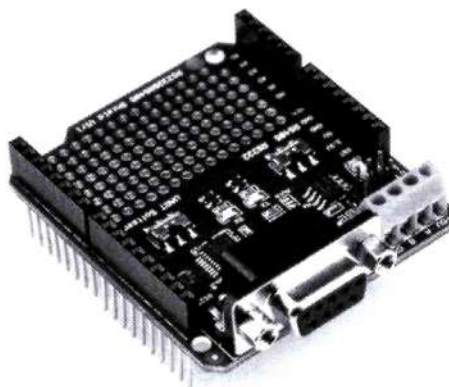


Figura 4.17 Shield RS232/485.

Placa de expansão que permite que o microcontrolador (que opera com níveis de tensão TTL = 0 V e 5 V) se comunique utilizando os protocolos, RS232 ou RS485, configurável através de chaves de seleção. A comunicação serial com o Arduino pode ser feita tanto por software quanto por hardware, também configurável por chaves de seleção. Uma das apresentações está ilustrada na Figura 4.17.

SHIELD ETHERNET W5100

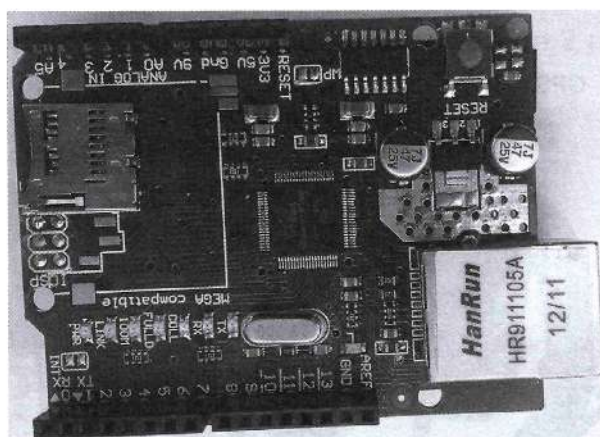


Foto: Arquivos

Figura 4.18 Shield Ethernet W5100.

Existem duas shields de Ethernet comuns. Uma delas é a apresentada na Figura 4.18. Essa shield Ethernet baseia-se no controlador W5100 com buffer interno de 16 KB. Opera com tensão TTL (5 V) e apresenta velocidade de comunicação de 10 ou 100 Mbps. Comunica-se com o Arduino através da porta serial SPI.

Possui alguns leds de informação do seu funcionamento. São eles:

- **PWR:** indica que a placa está alimentada.
- **LINK:** indica que há conexão de rede, e pisca quando há transmissão ou recepção de dados.
- **FULLD:** indica que a conexão de rede é full duplex.
- **100 M:** indica que a velocidade de conexão de rede é de 100 Mb/s quando acesa e de 10 Mb/s quando apagada.
- **RX:** indica a recepção de dados.
- **TX:** indica a transmissão de dados.
- **COLL:** indica que houve colisão na rede.

SHIELD BLUETOOTH

Os módulos de comunicação Bluetooth podem ser apresentados como módulos (com pinagem de encaixe) ou como shields. Nesse caso, apresentamos o que é mais comumente utilizado, ilustrado na Figura 4.19.

São apresentados em 6 pinos, mas utilizamos apenas 4 (Alimentação $V_{CC} = 5\text{ V}$ e $GND = 0\text{ V}$) e pinos de recepção RXD e transmissão TXD.

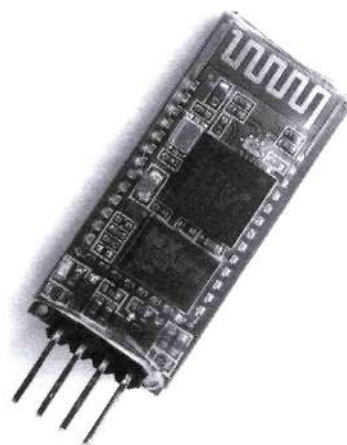


Figura 4.19 Módulo Bluetooth.



Apenas para ilustrar, também são exemplificadas outras shields de comunicação para projetos mais elaborados, como as shields de GPS e de GPRS, para Arduino.

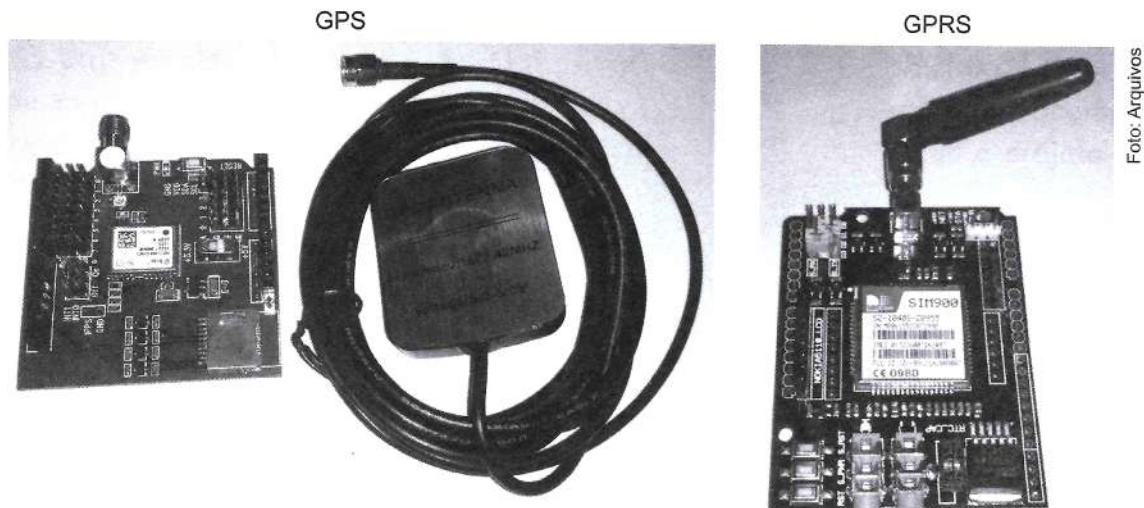


Figura 4.20 Módulos shields GPS e GPRS.

SHIELDS DE CONTROLE DE MOTOR E POTÊNCIA

Para controle de motores, existem diversas placas de extensão tanto para controle de pequenos servomotores quanto para controle de motores de passo.

SHIELD DE MOTOR PONTE H - L293D

O shield de motor apresentado na Figura 4.21 utiliza 2 CI L293D e 1 74HC595, o que permite o controle de 4 motores DC, 2 servos ou 2 motores de passo, e suporta motores com tensões de 4,5 a 25 V_{CC}.

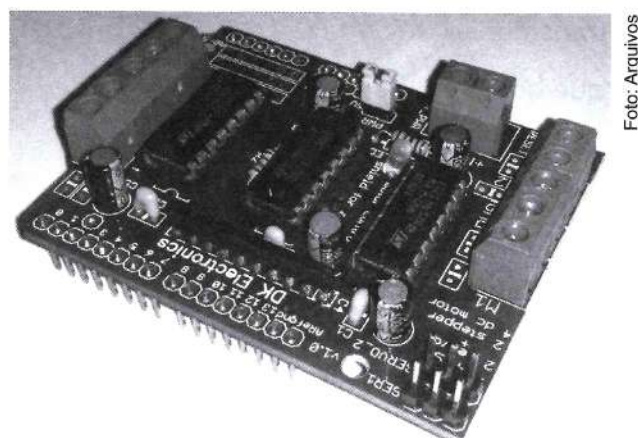


Figura 4.21 Shield Motor- Ponte H - L293D.

MOTOR SHIELD – PONTE H- L298N

Com este driver de motor baseado na Ponte H L298M (Figura 4.22), é possível controlar independentemente a velocidade e a rotação de 2 motores DC ou 1 motor de passo.

Tem como principais características:

- **Tensão de operação:** 4~35 V.
- **Corrente de operação máxima:** 2 A por canal.
- **Tensão lógica:** 5 V.
- **Corrente lógica:** 0~36 mA.

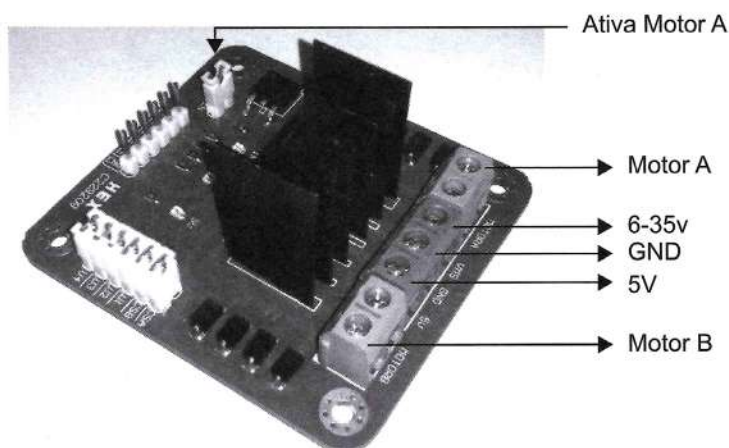


Figura 4.22 Shield Motor - Ponte H - L298N.

SHIELD DE RELÉS

Esta shield apresentada na Figura 4.23 possibilita o controle de 4 relés, porém são várias as possibilidades no comércio, desde placas de expansão com apenas 1 relé, quanto placas com controle de até 8 relés, que se encontram facilmente.

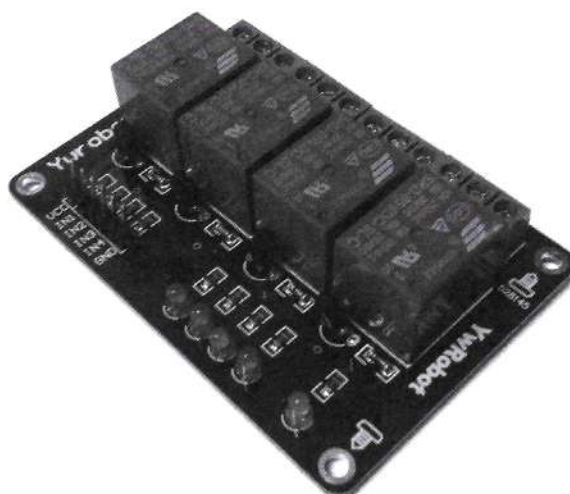


Figura 4.23 Shield de relés.

SHIELDS DE EXPANSÃO DE PORTAS E CONTROLE DE SENSORES

As principais placas de extensão referem-se às placas de prototipagem, placas com trilhas universais, placas de extensão a conexão de sensores, entre outras.

PROTO SHIELD

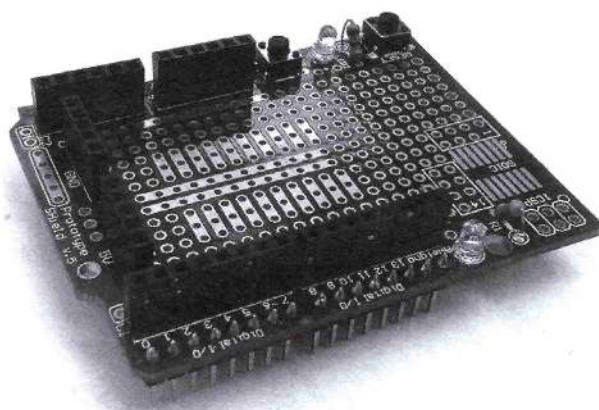


Figura 4.24 Proto Shield.

A Proto Shield é uma placa de expansão que permite o desenvolvimento em placa universal ou com o uso de uma pequena matriz de contatos (ver Figura 4.24).

Conta com 2 botões e 2 leds que podem ser facilmente adequados para uma aplicação rápida.

SENSOR SHIELD V4.0 PARA ARDUINO

Esta shield é uma maneira fácil de construir projetos com Arduino, pois utiliza interface de conectores padrão, facilitando a interação com diversos componentes como servos, motores e sensores (ver Figura 4.25).

Possui 6 portas analógicas (conector lateral); 6 portas analógicas (conector frontal); 1 porta de comunicação I2C e UART; e 16 portas digitais entrada-saída (conector frontal).

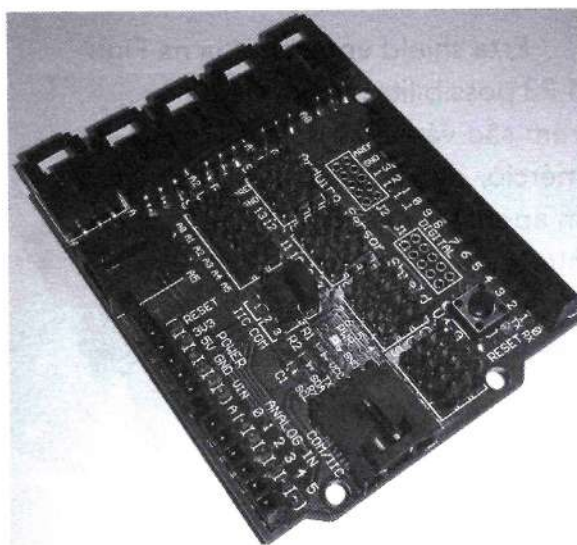


Figura 4.25 Sensor Shield V4.0.

SHIELDS DE INTERFACE HOMEM-MÁQUINA (IHM)

As principais shields de IHM referem-se, em um primeiro momento, à sinalização. Nesse caso, shields de displays de 7 segmentos.

LCD 16x2 SHIELD COM TECLADO

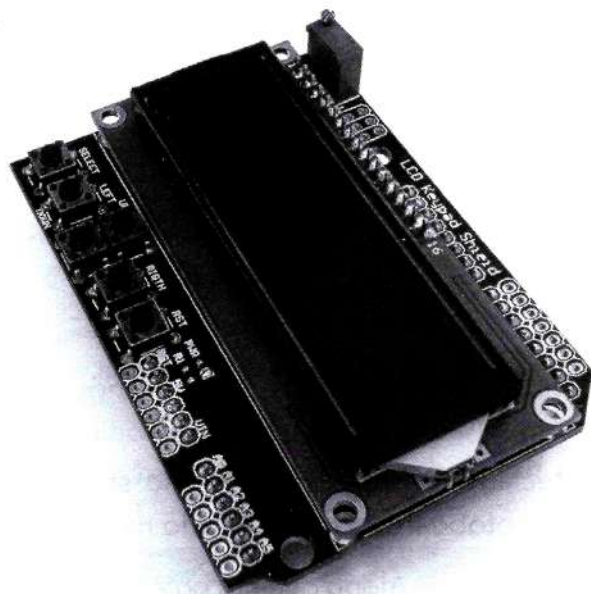


Figura 4.26 Shield LCD 16x2 com teclado.

A placa (shield LCD com botões) possui um teclado de navegação com 5 teclas, no entanto utiliza somente uma entrada analógica. Dessa forma, conforme pode ser visto na Figura 4.26, esta shield utiliza 5 níveis de tensão analógica baseados em divisores de tensão para monitorar cada botão.

SHIELD DE JOYSTICK

Existem diferentes joysticks ofertados no mercado. A shield apresentada na Figura 4.27 tem os botões momentâneos que são conectados aos pinos digitais do Arduino de 2 a 6 e quando pressionados rebaixam o pino (utilizando os interruptores do Arduino). Um movimento vertical do joystick produzirá uma tensão analógica proporcional no pino 0; do mesmo modo, um movimento horizontal pode ser monitorado no pino 1.

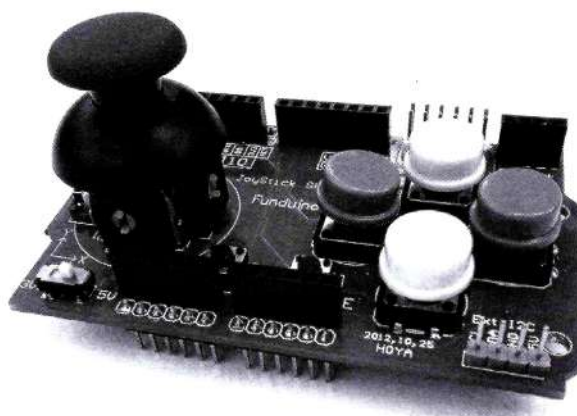


Figura 4.27 Shield de joystick.

SHIELD DE TECLADO MATRICIAL

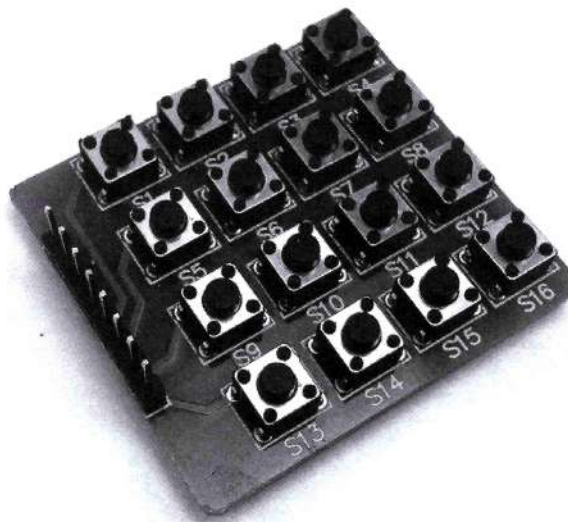


Figura 4.28 Shield de teclado matricial.

Existem diferentes teclados, apresentados como shields ou como dispositivos para serem conectados via protoboard, como ilustrado na Figura 4.28.

A maioria dos teclados opera de modo que um botão apertado curto-circuita uma coluna com uma linha. Assim, normalmente operam induzindo um nível lógico em uma linha e monitorando os níveis lógicos das colunas através de varredura, buscando um nível lógico refletido por ela.

Novamente, reforçamos a ideia de que o objetivo deste subcapítulo foi ilustrar que existem diversas placas de extensão que permitem que o tempo de projeto seja reduzido em função da simplificação que a padronização proposta pela plataforma Arduino permite. Assim, o desenvolvimento de novas shields tem uma dinâmica que justifica que o projetista esteja constantemente pesquisando novas ferramentas e dispositivos que acelerem o andamento dos seus projetos.

UTILIZANDO O ARDUINO PELA PRIMEIRA VEZ

O processo de utilização do Arduino é padronizado, entretanto todas as aplicações apresentadas neste livro utilizarão o Arduino UNO e o MEGA2560.

Já foi descrito, quando falamos das diferentes versões da plataforma Arduino, que essas placas contêm um microcontrolador auxiliar que faz a comunicação via porta serial USB com um computador, e que os microcontroladores já possuem o bootloader que permite a programação fácil da aplicação em si.

Dessa forma, os requisitos necessários para um primeiro contato com o Arduino são: um computador (com a ferramenta de programação instalada), uma placa Arduino e um cabo USB. Esses são suficientes para iniciar o desenvolvimento de um projeto (Figura 4.29).

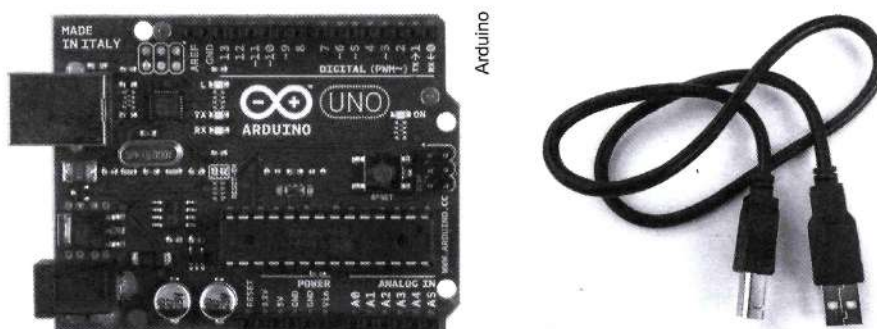


Figura 4.29 Arduino UNO e cabo USB.

A ferramenta de programação a ser instalada é gratuita e está disponível no endereço eletrônico do Arduino (<<http://arduino.cc/en/Main/Software>>). A Figura 4.30 ilustra a página onde se pode realizar o download da ferramenta.

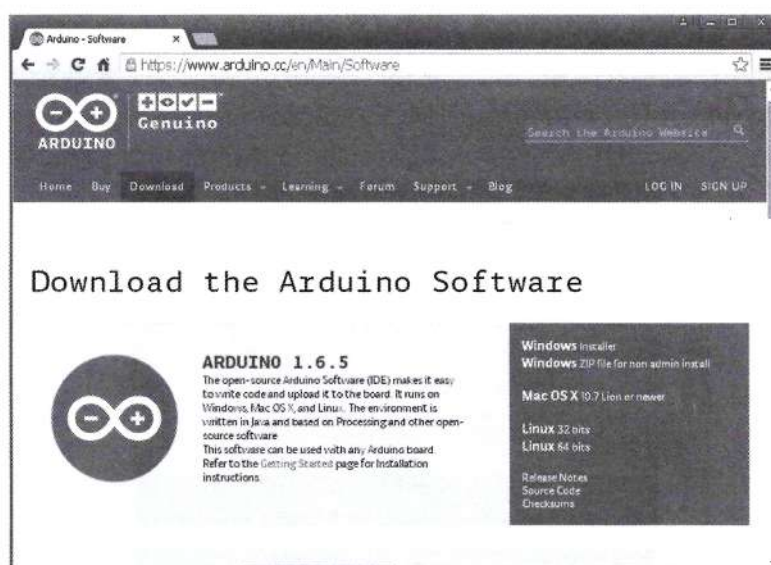


Figura 4.30 Endereço eletrônico para fazer download da ferramenta.

Os próximos passos serão descritos para a versão de execução na plataforma Windows. Existem duas versões: a de instalação e aquela cujo conteúdo se pode descompactar e executar. Para essa segunda opção, assim que o download terminar, descompacte o arquivo em uma pasta específica para o software. É importante preservar a estrutura de pastas nesse processo.



A Figura 4.31 ilustra o conjunto de arquivos descompactados do arquivo executável.



Figura 4.31 Conjunto de arquivos descompactados do software.

Conecte a placa Arduino ao seu computador utilizando o cabo USB. Na primeira vez que isso for feito, os drives de comunicação serão automaticamente instalados.

Uma vez (automaticamente) estabelecida a comunicação, o led verde (indicativo de energia – indicado por PWR) deve acender.

Navegue até a pasta onde você salvou o software Arduino e dê um duplo clique no arquivo ARDUINO.EXE para abrir a plataforma IDE de programação do Arduino (ilustrada na Figura 4.32).



Figura 4.32 Plataforma IDE de programação Arduino.

O próximo passo é indicar para a ferramenta qual é a plataforma Arduino que se estará programando. Para tal, na barra de ferramentas da plataforma IDE, selecione a aba Tools.

Dentro dessa aba, selecione Board para abrir um menu no qual se deverá escolher a placa que corresponde ao seu Arduino. Essa etapa está ilustrada na Figura 4.33.

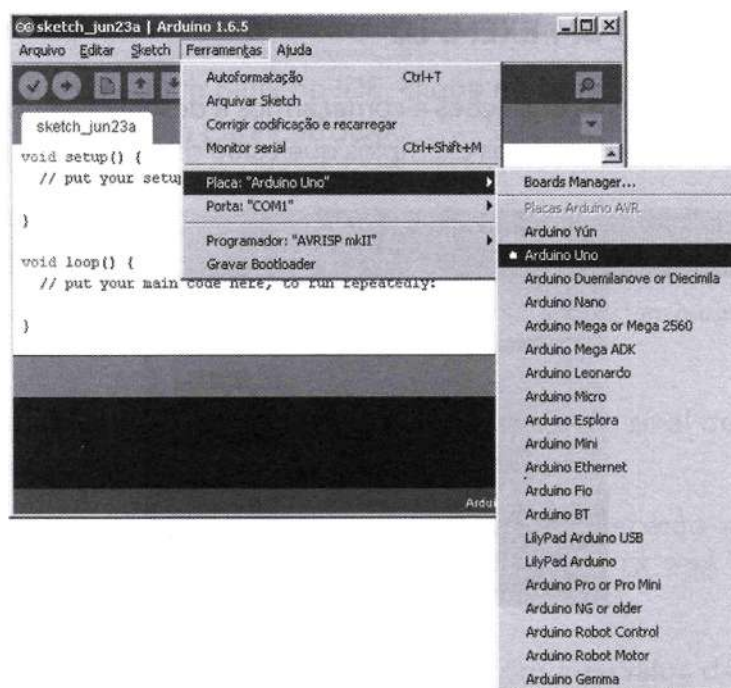


Figura 4.33 Selecionando o modelo da placa Arduino na plataforma IDE de programação.

O último passo é formalizar qual é o canal de comunicação serial utilizado pela plataforma para conversar com a placa Arduino. Nesse caso, acesse novamente a aba de ferramentas Tools e escolha a opção Serial Port. Se aparecer mais de uma opção, confirme qual a porta de comunicação COM que está sendo utilizada pelo Arduino, conforme ilustrado na Figura 4.34.

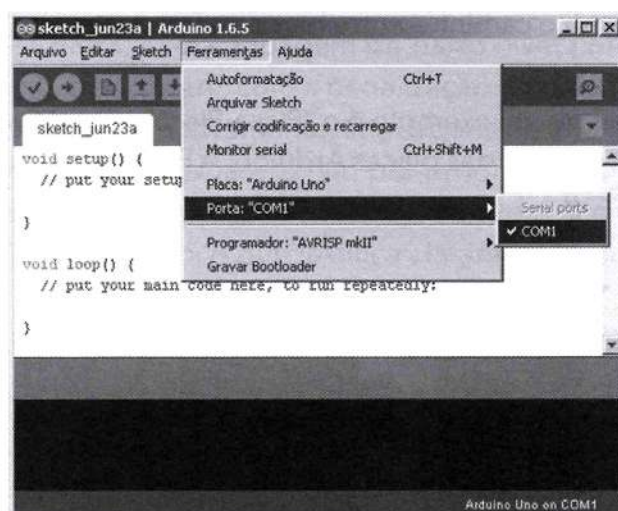


Figura 4.34 Escolha da porta de comunicação COM para a plataforma IDE com o Arduino.

Pronto. Se tudo correu bem, você já está apto a começar a escrever seus primeiros projetos com Arduino.



EXECUTANDO UM EXEMPLO

Antes de abordarmos as instruções e começarmos a desenvolver nossos próprios projetos, é importante analisar os projetos exemplos que estão disponíveis na plataforma IDE.



Figura 4.35 Exemplos dentro da IDE Arduino.

Abra a plataforma IDE, na aba File e depois Examples. Você verá que existem dezenas de exemplos, inicialmente separados em grupos de 1 a 10.

A Figura 4.35 ilustra o caminho entre barra de ferramentas e abas para chegar nos exemplos.

O primeiro e mais visível exemplo é o “Pisca-Pisca” que utiliza o led (conectado ao pino 13) conectado na maioria das placas Arduino. A Figura 4.36 ilustra o led pertencente às placas UNO e MEGA.

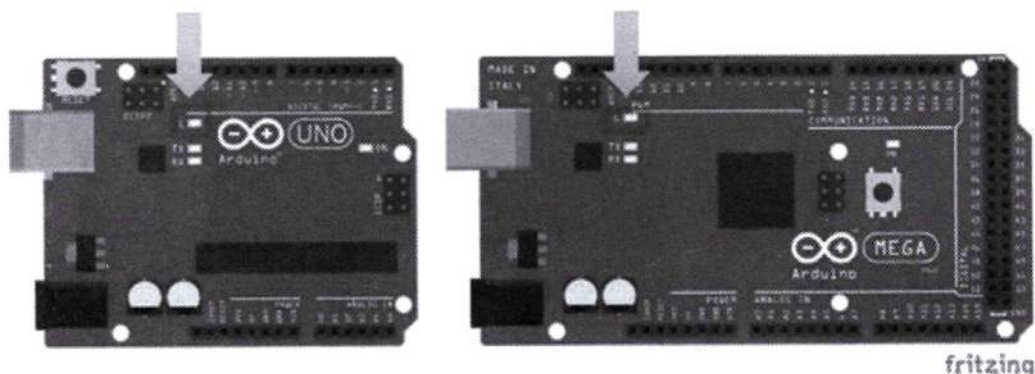


Figura 4.36 Leds conectados nos pinos 13 das placas UNO e MEGA.

Abra a aba File>Examples>01.Basics>Blink.

Uma vez aberto o sketch Blink no IDE, clique no botão upload (Figura 4.37) para fazer a transferência do sketch para o Arduino.



Figura 4.37 Comando upload na barra de ferramentas, para carregar um programa no Arduino.

No Arduino as luzes RX e TX começarão a piscar como sinal de que os dados estão sendo transmitidos do seu computador para a placa.

Se não tiver nenhum problema no seu sketch aparecerão as palavras "Done uploading" na barra de status do seu IDE, as luzes RX e TX irão parar de piscar e seu upload estará completo.

Se o led laranja na placa começou a piscar com intervalos de 1 segundo, você obteve êxito na execução do seu primeiro exemplo.

LINGUAGEM C PARA ARDUINO

Os microcontroladores ATMEGA, que dão vida à plataforma Arduino, têm como linguagem base o Assembly. É o Assembly que é conhecido como linguagem de máquina e convertido em arquivos de formato hexadecimal que são transferidos para o microcontrolador.

Um tanto quanto difícil, essa linguagem de baixo nível permite que o programador controle plenamente o hardware, com a penalização de dificultar em muito o desenvolvimento de um projeto. Alternativamente, linguagens de alto nível, como linguagem C, propiciam que diversas funções já estejam pré-programadas na linguagem de máquina, de modo transparente ao programador, facilitando e acelerando o processo de desenvolvimento da programação, o que justifica claramente a sua utilização.

Facilitar o processo de desenvolvimento e acelerar a criação das lógicas de programação com uma linguagem de alto nível trazem como problema um código não otimizado, além de alguns problemas em nível mais específico. Quando o projeto não é de alta complexidade, nem o tamanho do código gerado pelo compilador é um limitador para o microcontrolador escolhido, a linguagem de alto nível é, acertadamente, a melhor opção de desenvolvimento.

O software IDE Arduíno é, em primeira instância, uma ferramenta de edição de linguagem de alto nível (C); mas, em uma segunda instância, é quem compila o código nessa linguagem e o converte para Assembly, para que ele possa converter em código binário e embarcar o código no Arduino.



O arquivo em que programamos nosso código em linguagem C é conhecido como **sketch**. Basicamente, a estrutura de um sketch é formada por três blocos (conforme ilustrado pela Figura 4.38):

- o bloco de declaração das variáveis, e depois por dois blocos principais;
- o bloco da função `setup()`, com declarações, inicializações e funções internas, as quais serão executadas uma única vez; e
- o bloco da função `loop()`, com funções que se repetirão.

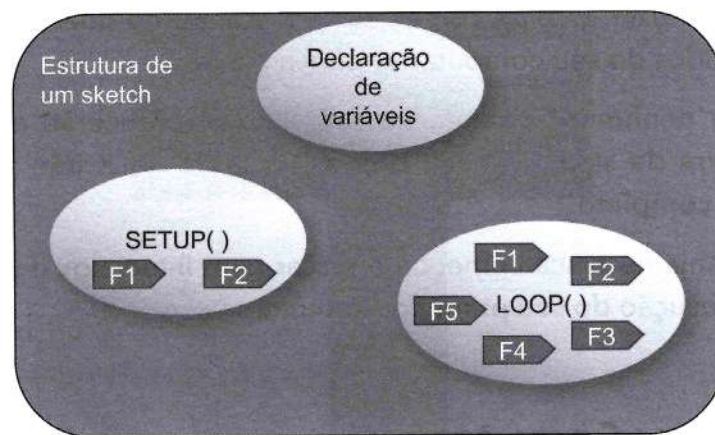


Figura 4.38 Estrutura de um sketch.

Para construir cada um desses blocos, é necessário conhecer alguns símbolos que permitirão escrever corretamente o código:

- Todos os procedimentos de uma função devem estar limitados por chaves "{ }".
- Cada procedimento deve ser finalizado por um ponto e vírgula ";" .
- Usa-se "//" antes de um comentário em uma linha.
- Usa-se "/*.....*/" para comentar um conjunto de linhas.

Vamos analisar a estrutura do código de exemplo ilustrado na Figura 4.39.

Nesse código, pode-se notar 3 linhas de comentário inicial, entre /* e */, e também 5 comentários após código em continuação de linhas, iniciados por //. Temos a declaração de uma variável `ledPin`, do tipo inteira, e com valor inicial 13. No bloco `SETUP`, temos uma configuração inicial (`pinMode(ledPin, OUTPUT)`), que indica que a variável pino 13 (`ledPin = 13`) é configurada como saída (`OUTPUT`). No bloco `LOOP`, tem-se um procedimento de atribuição de valor lógico (`HIGH` ou `LOW`) para o pino indicado (`ledPin = 13`); e temos também o procedimento de temporização (`delay`) de um tempo de 1000 ms. Note que todas as instruções, desde atribuição de variáveis até as instruções finais, todas, são finalizadas por ponto e vírgula.

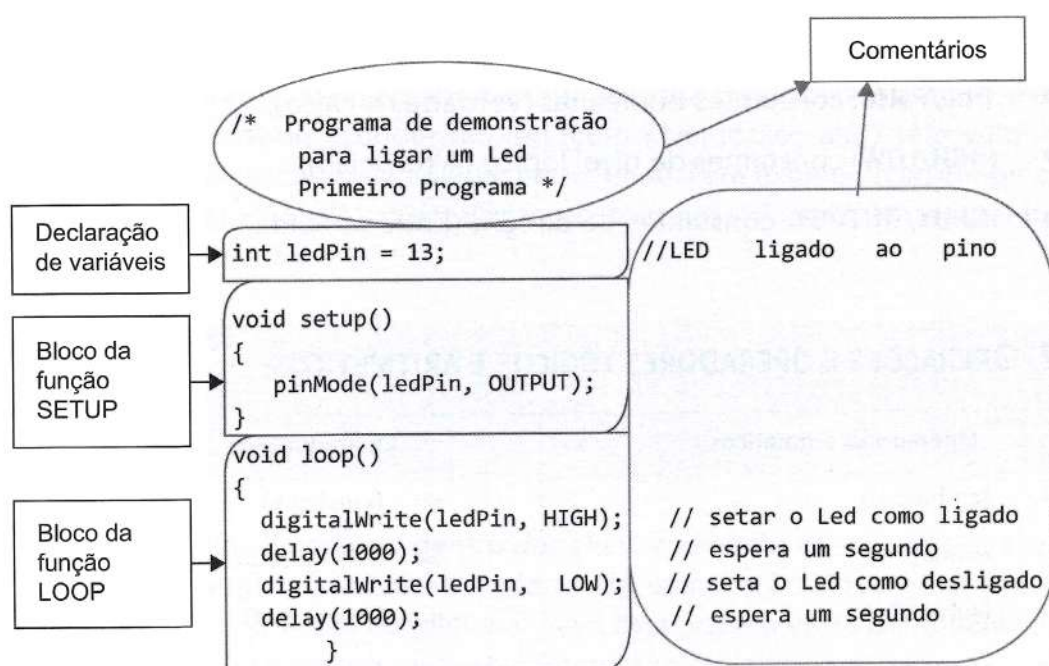


Figura 4.39 Conhecendo o primeiro sketch e identificando sua estrutura.

Para falar mais da sintaxe e das funções, vamos separar o conhecimento da programação em tópicos: tipos de dados; operações e operadores lógicos e aritméticos.

TIPOS DE DADOS

Os tipos mais comuns de dados (variáveis) que utilizamos no Arduino são:

- **boolean:** valor verdadeiro (true) ou falso (false);
- **char:** um caractere;
- **byte:** um byte, ou sequência de 8 bits;
- **int:** número inteiro de 16 bits com sinal (–32768 a 32767);
- **unsigned int:** número inteiro de 16 bits sem sinal (0 a 65535);
- **long:** número inteiro de 32 bits com sinal (–2147483648 a 2147483647);
- **unsigned long:** número inteiro de 32 bits sem sinal (0 a 4294967295);
- **float:** número real de precisão simples (ponto flutuante);
- **double:** número real de precisão dupla (ponto flutuante);
- **string:** sequência de caracteres;
- **void:** tipo vazio (não tem tipo).



Já os tipos padronizados de constantes são:

- **True/False:** constantes booleanas (Verdadeiro/Falso);
- **HIGH/LOW:** constantes de nível lógico (Alto/Baixo);
- **INPUT/OUTPUT:** constantes de direção (Entrada/Saída).

OPERAÇÕES E OPERADORES LÓGICOS E ARITMÉTICOS

Operadores aritméticos = (atribuição) + (soma) - (subtração) * (multiplicação) / (divisão) % (módulo)	Operadores de comparação == (igual que) != (não igual que) < (menor que) > (maior que) <= (menor ou igual que) >= (maior ou igual que)
Operadores compostos ++ (incremento) -- (decremento) += (soma composta) -= (resta composta) *= (multiplicação composta) /= (divisão composta)	Operadores booleanos && (and) (or) ! (not)

ESTRUTURAS DE CONTROLE OU CONDICIONAIS

O quadro a seguir apresenta as principais estruturas de controle da programação C.

if()	do()... while
if()...else	break
for()	continue
switch()...case	return
while()	goto

CONDICIONAIS IF E IF...ELSE

Instruções condicionais são um meio de tomada de decisões em um sketch. Por exemplo, seu sketch pode acender um led (com nível lógico alto) se o valor de uma variável de temperatura cai abaixo de um certo limite. Para exemplificar, imagine que só executaremos a instrução `digitalWrite()` caso o conteúdo da variável temperatura seja menor do que o valor 15.

```
if (temperature < 15)
{
  digitalWrite(ledPort, HIGH);
}
```

A linha ou linhas de código dentro das chaves só serão executadas se a condição for verdadeira. Qualquer um dos operadores de comparação pode estar presente no teste condicional. A expressão condicional deve estar escrita entre parênteses e poderá ter duas situações lógicas como resposta: verdadeiro ou falso.

O teste condicional pode conter duas ou mais expressões, que podem estar relacionadas através dois operadores booleanos (`&&` e `||`). Um exemplo de teste condicional pode ser ilustrado imaginando que desejamos acender um led se a variável temperatura for menor que 15 ou maior que 20.

```
if ((temperature < 15) || (temperature > 20))
{
  digitalWrite(ledPort, HIGH);
}
```

E ainda podemos escrever um conjunto de instruções também para o caso de resultado falso do teste condicional, através da instrução `ELSE`.

```
if ((temperature < 15) || (temperature > 20))
{
  digitalWrite(ledPort, HIGH);
}
else
{
  digitalWrite(ledPort, LOW);
}
```

LAÇO FOR

O laço `for` executa uma contagem entre um valor inicial até um valor final, segundo a regra aritmética determinada, e essas três informações devem estar entre parênteses e separadas por ponto e vírgula.



Por exemplo: na instrução `for (int i= 0; i <= 255; i++) {}`, é declarado um contador `i`, que se inicia em 0, tem um valor máximo de 255 e uma dinâmica de crescimento incremental, de um em um. Para exemplificar a aplicação, temos um código a seguir que ilustra a variação de luminosidade de um led, através da variação de um contador, nesse caso 0, que representaria o mínimo (led desligado) e 255 o valor máximo (máxima luminosidade do led).

```
// Exemplo: Dimmer de um LED usando um pino PWM como saída

int pinoPWM = 10; // LED ligado em série com um resistor de 11 kΩ ao pino 10

void setup()
{ } // não é necessário setup

void loop()
{
    for (int i= 0; i <= 255; i++) { // a variável i é incrementada até 255,
                                    // depois o loop é terminado
        analogWrite(pinoPWM, i); // o valor de i é escrito no pinoPWM
        delay(10); // temporização de 10 ms entre mudanças de valor
    }
}
```

LAÇO DO...WHILE

Esse laço executa as instruções dentro das chaves, de forma cíclica, enquanto uma condição while for verdadeira.

Para ilustrar a instrução, o código a seguir faz uma leitura da porta analógica AN0, com intervalos de 50 ms, enquanto o valor analógico for menor que uma constante (100).

```
do {
    delay(50); // espera os sensores se estabilizarem
    x = analogRead(0); // lê um sensor analógico na porta 0
} while (x < 100); // fica dentro do do..while enquanto < 100
```

CONDICIONAL SWITCH / CASE

Essa condição avalia o valor de entrada e faz comparações sequenciais de igualdade. Quando a condição for verdadeira, executa as instruções a seguir até encontrar a instrução `break` e sair do condicional. O exemplo que se segue primeiramente mapeia

os valores analógicos de um sensor (valor mínimo e valor máximo) em uma escala de valores inteiros de 3 níveis (0 a 2). Depois efetivamente utiliza o laço condicional comparando a variável de leitura com os valores 0, 1 e 2.

```
const int sensorMin = 50;    // valor mínimo do sensor (experimental)
const int sensorMax = 200;   // valor máximo do sensor (experimental)

void setup() {
  Serial.begin(9600);        // inicialização da comunicação serial
}
void loop() {
  int sensor = analogRead(A0);    // leitura analógica
  int ajuste = map(sensor, sensorMin, sensorMax, 0, 2);
  //mapeia os valores de entrada, de SensorMin a SensorMax, para 0 a 2.

  switch (ajuste) {
    case 0:    Serial.println("escuro");           //0 representa escuro
               break;
    case 1:    Serial.println("intermediário");     //1, intermediário
               break;
    case 2:    Serial.println("claro");             //2 representa claro.
               break;
  }
  delay(10);    // tempo entre leituras analógicas;
}
```

RETURN

Instrução utilizada para finalizar uma função e retornar valor, caso desejado.

```
int checkSensor(){
  if (analogRead(0) > 400) {
    return 1;
  }
  else{
    return 0;
  }
}
```



FUNÇÕES

Como já pudemos reparar nos exemplos anteriores, temos diversas funções que interagem como o hardware do microcontrolador, além das funções tradicionais de linguagem C.

Constantemente, novas bibliotecas são desenvolvidas para que novos dispositivos possam ser facilmente utilizados.

O quadro a seguir concentra as principais funções em linguagem C para Arduino.

Funções de I/O digitais ▪ <code>pinMode(pin, mode)</code> ▪ <code>digitalWrite(pin, value)</code> ▪ <code>int digitalRead(pin)</code>	Funções I/O avançadas ▪ <code>shiftOut(dataPin, clockPin, bitOrder, value)</code> ▪ <code>unsigned long pulseIn(pin, value)</code>	Funções matemáticas ▪ <code>min(x, y)</code> ▪ <code>max(x, y)</code> ▪ <code>abs(x)</code> ▪ <code>constrain(x, a, b)</code> ▪ <code>map(value, fromLow, fromHigh, toLow, toHigh)</code> ▪ <code>pow(base, exponente)</code> ▪ <code>sq(x)</code> ▪ <code>sqrt(x)</code> ▪ <code>sin(rad)</code> ▪ <code>cos(rad)</code> ▪ <code>tan(rad)</code>
Funções I/O analógicas ▪ <code>int analogRead(pin)</code> ▪ <code>analogWrite(pin, value)</code>	Funções de tempo ▪ <code>unsigned long millis()</code> ▪ <code>delay(ms)</code> ▪ <code>delayMicroseconds(μs)</code>	
Comunicação serial ▪ <code>Serial.begin(baudios)</code> ▪ <code>int Serial.available()</code> ▪ <code>int Serial.read()</code> ▪ <code>Serial.flush()</code> ▪ <code>Serial.print(datos)</code> ▪ <code>Serial.println(datos)</code>	Funções de números aleatórios: ▪ <code>randomSeed(semilla)</code> ▪ <code>long random(max)</code> ▪ <code>long random(min, max)</code>	

A seguir serão descritas as principais funções.

FUNÇÕES DE ENTRADA E SAÍDA DIGITAL

PINMODE()

Configura o pino especificado para que se comporte ou como uma entrada ou como uma saída. Deve-se informar o número do pino que se deseja configurar e em seguida se o pino será determinado como uma entrada (INPUT) ou uma saída (OUTPUT).

Sintaxe:
`pinMode(pino, modo)`

Exemplo:
`pinMode(13, OUTPUT);`
`pinMode(2, INPUT);`

DIGITALWRITE()

Escreve um valor HIGH ou LOW em um pino digital. Se o pino foi configurado como uma saída, sua tensão será determinada ao valor correspondente: 5 V para HIGH e 0 V para LOW. Se o pino está configurado como uma entrada, HIGH liga o resistor interno de pull-up (de 20 k Ω) e LOW desliga o resistor de pull-up.

Sintaxe: <code>digitalWrite(pino, valor);</code>	Exemplo: <code>digitalWrite(13, LOW);</code> <code>digitalWrite(12, HIGH);</code>
--	--

DIGITALREAD()

Lê o valor de um pino digital especificado e retorna um valor HIGH ou LOW.

Sintaxe: <code>int digitalRead(pino);</code>	Exemplo: <code>Valor = digitalRead(2);</code>
--	---

FUNÇÕES DE ENTRADA E SAÍDA ANALÓGICA**ANALOGREAD()**

Essa função lê o valor de um pino analógico especificado. O Arduino contém um conversor analógico-digital de 10 bits (com 6 canais, no UNO). Com isso ele pode mapear voltagens de entrada entre 0 e 5 V para valores inteiros entre 0 e 1023. Isso permite uma resolução entre leituras de 5 V / 1024 unidades ou 0,00488 V (4,88 mV) por unidade.

Sintaxe: <code>int analogRead(pino);</code>	Exemplo: <code>Valor = analogRead(3);</code>
---	--

ANALOGWRITE()

A escrita analógica utiliza as portas de saída PWM (modulação por largura de pulso) para prover um sinal pseudoanalógico. Ao executar a função `analogWrite( )`, é gerada no pino em questão, uma onda quadrada estável, onde configuramos o tempo de duração da parte em nível lógico alto, com valores entre 0 e 255, equivalendo, respectivamente, a 0% e 100% do período da onda. Em kits Arduino com o chip ATMEGA168, essa função está disponível nos pinos 3, 5, 6, 9, 10 e 11.

Para usar essa função, deve-se informar o pino ao qual deseja-se escrever e em seguida informar um valor entre 0 (pino sempre desligado) e 255 (pino sempre ligado).

Sintaxe: <code>analogWrite(pino, valor);</code>	Exemplo: <code>analogWrite(10, 128);</code>
---	---



■ Tempo

millis()

Retorna o número de milissegundos desde que o kit Arduino começou a executar o programa. Esse número extrapolará (voltará a zero) depois de aproximadamente 50 dias.

Sintaxe: <i>unsigned long millis()</i>	Exemplo: <i>unsigned long tempo;</i> ... <i>tempo = millis()</i>
--	--

delay()

Gera um atraso na continuação da execução do programa pelo tempo especificado (ms).

Sintaxe: <i>delay(tempo);</i>	Exemplo: <i>Delay(1000); //gera um tempo de 1 segundo</i>
---	---

■ Comunicação serial

Serial.begin()

Ajusta a taxa de transferência em bits por segundo para uma transmissão de dados pelo padrão serial. Para comunicação com um computador, use uma destas taxas: 300, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 57600, 115200. Pode-se, entretanto, especificar outras velocidades, por exemplo, para comunicação através dos pinos 0 e 1 com um componente que requer uma taxa específica. Ainda é possível configurar a quantidade de bits, paridade e bit de parada, sendo que o valor padrão de configuração é SERIAL_8N1.

Sintaxe: <i>Serial.begin(taxa);</i> <i>Serial.begin(taxa, config);</i>	Exemplo: <i>Serial.begin(9600);</i>
---	---

Serial.read()

Lê o byte mais recente apontado no buffer de entrada serial.

Sintaxe: <i>Serial.read();</i>	Exemplo: <i>Dado = Serial.read();</i>
--	---

Serial.available()

Retorna a quantidade de bits disponíveis no buffer para leitura (valor máximo 64 bytes).

Sintaxe:
Serial.available();

Exemplo:
NDadosDisponíveis = Serial.available();

Serial.write()

Retorna a quantidade de bits disponíveis no buffer para leitura (valor máximo 64 bytes).

Sintaxe:
Serial.available();

Exemplo:
NDadosDisponíveis = Serial.available();

Serial.print() e Serial.println()

Escreve na serial um texto em formato ASCII. No segundo caso, acrescenta uma quebra de linha ao dado escrito.

Sintaxe:
Serial.print();
Serial.println();

Exemplo:
Serial.print (2015); // Envia "2015"
Serial.print (2.015); // Envia "2.01"
Serial.print ('A'); // Envia "A".
Serial.print ("Arduino"); // Envia "Arduino"

O código a seguir exemplifica algumas instruções de comunicação serial:

```
void setup() {
  Serial.begin(9600); //inicializa a comunicação serial a uma
                      //taxa de 9600 bits por segundo
}
void loop() {
  while (Serial.available()) { // Analisa a existência de dados
                              //...recebidos no buffer
    char c = Serial.read();    // Lê byte do buffer serial;
    Serial.print(c);           // Escreve na serial, enviando o dado
                              // ...anteriormente recebido;
    delay(10);                 // gera um atraso de 10 ms
  }
}
```



INTERRUPÇÕES EXTERNAS

O Arduino UNO possui 2 interrupções, enquanto o MEGA2560 possui 6. São identificadas pelo número da interrupção (exemplo: interrupção 0 – ligada ao pino 2 do Arduino UNO). O quadro a seguir indica a relação entre a interrupção e o pino do circuito integrado.

Placa	int.0	int.1	int.2	int.3	int.4	int.5
UNO	2	3				
MEGA2560	2	3	21	20	19	18

As instruções específicas para configurar e habilitar as interrupções estão listadas a seguir. A configuração deve indicar qual interrupção, qual a função a ser executada quando ela ocorrer e qual a ação monitorada (LOW = nível lógico baixo; HIGH = nível lógico alto; CHANGE = alteração de nível lógico, RISING = transição de subida; e FALLING = transição de descida).

```
attachInterrupt(interrupt, function, [LOW,HIGH,CHANGE,RISING,FALLING])
detachInterrupt(interrupt)
```

A seguir é apresentado um código que ilustra a mudança de estado de uma saída, em função da mudança de uma entrada, através de interrupção.

```
int led = 13; //declarar LED
volatile int estado = LOW; //estado inicial = Baixo

void setup() {
  pinMode(led, OUTPUT); //configura pino LED como saída
  attachInterrupt(0, piscar, CHANGE); //configura interrupção 0...
                                     //...(pino 2) em sua mudança
}                                     //executando a função "piscar"

void loop() {
  digitalWrite(led, estado); //impõe nível lógico "estado" no pino LED
}

void piscar() {
  estado = !estado; //altera o estado da variável "estado"
}
```

CAPÍTULO 05

PROJETOS E DESAFIOS

Neste capítulo desenvolvemos uma sequência de projetos e desafios que abordam os conceitos de entradas e saídas digitais e analógicas e o uso de dispositivos sensores e atuadores.

Projeto 1 – Controle de um LED

O objetivo deste projeto é utilizar uma porta digital do Arduino para controlar o funcionamento de um Diodo Emissor de Luz (LED). Um nível 1 (HIGH) colocado no pino irá acender o LED, enquanto um nível 0 (LOW) vai apagar o LED.

Material necessário:

- 1 Arduino.
- 1 Resistor de 220 ohms (vermelho, vermelho, marrom) ou 330 ohms (laranja, laranja, marrom).
- 1 Led (qualquer cor).
- 1 Protoboard.
- Jumper cable.

Passo 1: Montagem do circuito

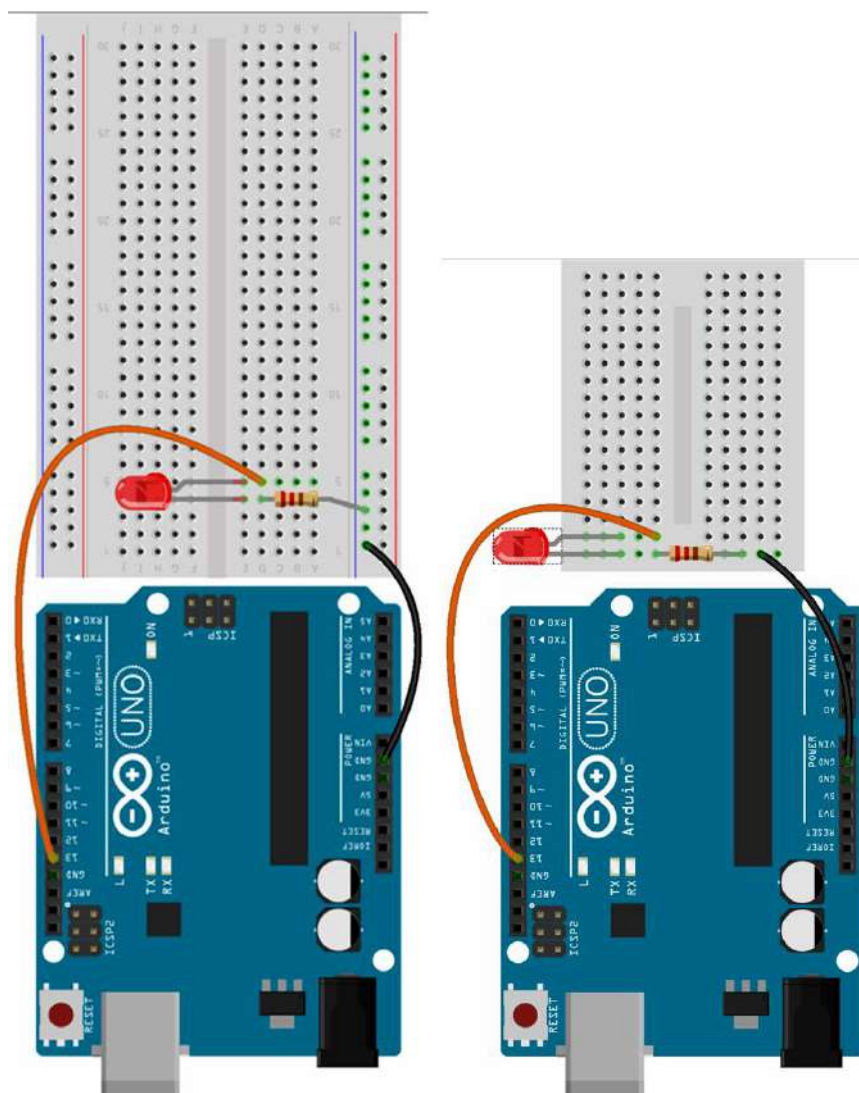


Figura 5.1 - Opções de Montagem do Projeto



Observe a Figura 5.1 e de acordo com o modelo de protoboard que estiver usando:

- Conecte o pino GND do Arduino à linha de alimentação negativa (preta ou azul) da protoboard.
- Coloque o resistor de 220 ohms (ou 330 ohms) entre a linha de alimentação negativa e qualquer outra linha da protoboard.

c. Coloque o LED com o Cátodo (lado chanfrado) conectado ao resistor.

d. Conecte o Ânodo do LED ao pino 13 do Arduino.

Passo 2: Programa

Inicie o ambiente de desenvolvimento do Arduino e digite o sketch (programa) a seguir:

```
int LED = 13; // Pino onde o LED foi conectado

void setup() {
  pinMode(LED, OUTPUT); // Definir pino como
                        // saída
}

void loop() {
  digitalWrite(LED, HIGH); // Colocar nível 1 no
                          // pino (liga o LED)
  delay(2000); // Aguardar por 2 segundos
  digitalWrite(LED, LOW); // Colocar nível 0 no
                        // pino (apaga o LED)
  delay(2000);
}
```

Passo 3: Compilação e transferência do programa para o Arduino

Observe a Figura 5.2 e após salvar o sketch (programa),

faça a compilação e, em seguida, conecte o Arduino à porta USB do computador. Finalizando, pressione o botão Carregar (Transferir) para fazer a transferência do sketch para o Arduino.

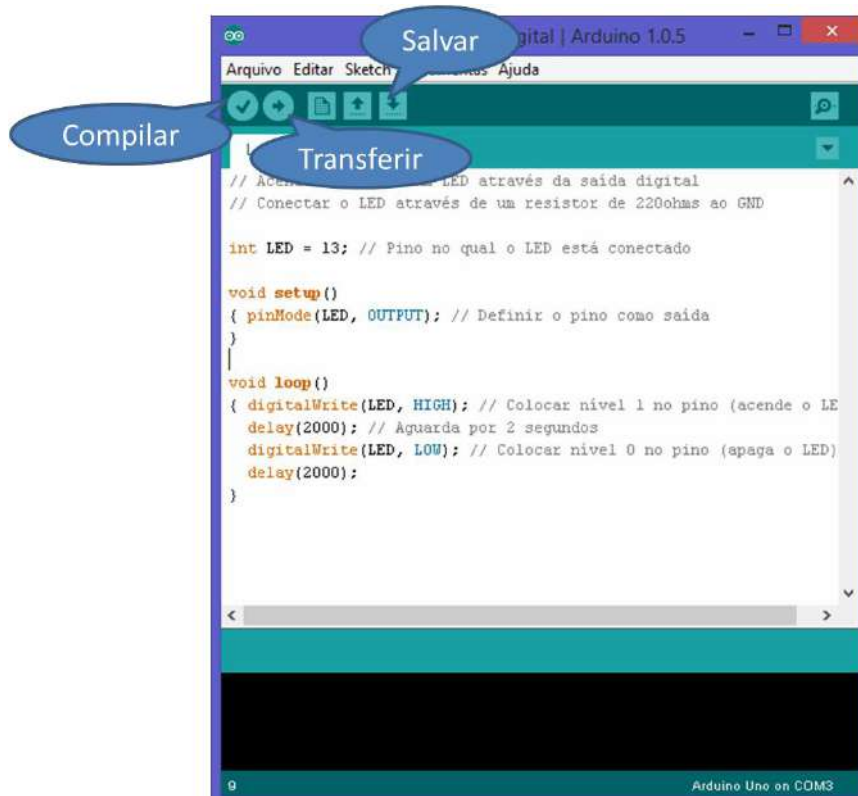


Figura 5.2 - Ambiente de Desenvolvimento do Arduino

Desafio 1 – Controle de Semáforo

a) Semáforo de Veículos

Objetivo:

Aplicando o conceito de Saídas Digitais, abordado no Projeto 1, reproduzir o funcionamento de um sinal de trânsito.

Material necessário:

- 1 Arduino.
- 3 Resistores de 220 ohms (vermelho, vermelho, marrom) ou de 330 ohms (laranja, laranja, marrom).
- 3 LEDs (1 vermelho, 1 verde e 1 amarelo).
- 1 Protoboard.
- Jumper cable.

Montagem do circuito

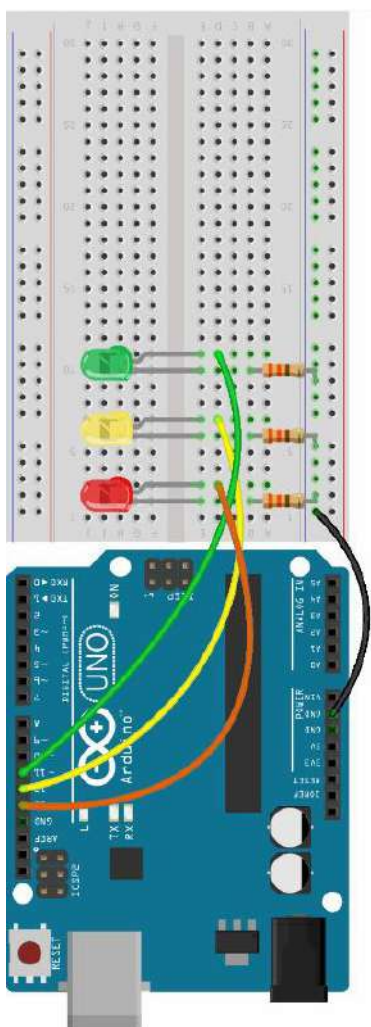


Figura 5.3: Semáforo de Veículos

Realize a montagem do circuito, conforme ilustra a Figura 5.3:

- Conecte o pino GND do Arduino à linha de alimentação negativa (azul) do protoboard.
- Coloque 5 resistores de 220 ohms (ou 330 ohms) entre com uma ligação na linha de alimentação negativa e qualquer outra linha do protoboard.

c. Coloque os 3 LEDs com o Cátodo (lado chanfrado) conectado em cada um dos resistores.

d. Conecte o Ânodo dos LEDs na seguinte ordem: pino 13 do Arduino em um LED vermelho, pino 12 do Arduino em um LED amarelo e pino 11 do Arduino em um LED verde.

b) Semáforo de Veículos e Pedestres



Objetivo:

Reproduzir um cenário similar ao de um semáforo de veículos e pedestres. Supondo o estado inicial do cenário com semáforo de veículos (VEÍCULO) sendo vermelho (PARE) e o semáforo de pedestres (PEDESTRE) sendo verde (SIGA), deve-se programar a sequência de luzes indicando os estados do semáforo de veículos sincronizado com os estados do semáforo de pedestres. Algumas especificações a serem seguidas:

- O sinal vermelho e sinal verde de VEÍCULO tem duração de 10 segundos cada.
- O sinal amarelo de VEÍCULO tem duração de 2 segundos.
- O sinal vermelho de PEDESTRE ficará acesso durante todo o tempo que o sinal vermelho e sinal amarelo de VEÍCULO estiverem acessos, impedindo a passagem de pedestres enquanto os carros transitam.

- O sinal verde de PEDESTRE ficará acesso durante todo o tempo que o sinal vermelho de VEÍCULO ficar acesso, indicando que os pedestres estão livres para atravessar.
- Antes transição do sinal verde para o vermelho de PEDESTRE, faltando 2 segundos para a transição, o sinal verde pisca rapidamente 2 vezes indicando aos pedestres que se tornará vermelho.

Material necessário:

- 1 Arduino.
- 5 Resistores de 220 ohms (vermelho, vermelho, marrom) ou de 330 ohms (laranja, laranja, marrom).
- 5 Leds (2 vermelhos, 2 verdes e 1 amarelo).
- 1 Protoboard.
- Jumper cable.

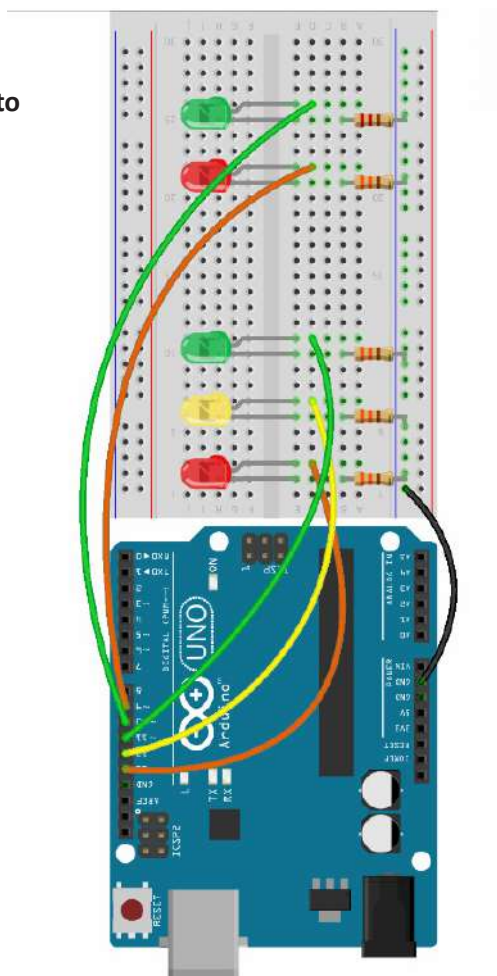
Montagem do circuito

Figura 5.4: Semáforo de Veículos e Pedestres

Conforme ilustra a Figura 5.4 realize os seguintes passos:

- a. Conecte o pino GND do Arduino à linha de alimentação negativa (azul) do protoboard.
- b. Coloque 5 resistores de 220 ohms (ou 330 ohms) entre com uma ligação na linha de alimentação negativa e qualquer outra linha do protoboard.
- c. Coloque 5 LEDs com o Cátodo (lado chanfrado) conectado em cada um dos resistores.
- d. Conecte o Ânodo dos LEDs na seguinte ordem: pino 13 do Arduino no LED vermelho (vermelho de VEÍCULO), pino 12 do Arduino no LED amarelo (amarelo de VEÍCULO), pino 11 do Arduino no LED verde (verde de VEÍCULO), pino 9 do Arduino no LED vermelho (vermelho de PEDESTRE) e pino 10 do Arduino no LED verde (verde de PEDESTRE).

Projeto 2 – Potenciômetro

O objetivo deste projeto é controlar a frequência de acender e apagar (frequência de pisca-pisca) e a intensidade da luminosidade de um LED. Nesse workshop teremos dois experimentos para alcançar esses objetivos. Um potenciômetro um resistor variável no formato de um botão giratório que fornece um valor analógico. Se girarmos o potenciômetro, alteramos a resistência em cada lado do contato elétrico que vai conectado ao terminal central do botão. Essa mudança implica em uma mudança no valor analógico de entrada. Quando o cursor for levado até o final da escala, teremos 0 volts e assim obtendo o valor 0 na entrada analógica. Quando giramos o cursor até o outro extremo da escala, teremos 5 volts e assim tendo o valor 1023 na entrada analógica. Outro conceito que podemos notar é a utilização dos pinos digitais com a marcação “~” (til) como, por exemplo, o pino digital “~9” usado no Programa N° 2.

Material necessário:

- 1 Arduino.
- 1 Potenciômetro.
- 1 Resistor de 220 ohms (vermelho, vermelho, marrom) ou 330 ohms (laranja, laranja, marrom) para o Led.
- 1 LED de qualquer cor.
- 1 Protoboard.
- Jumper cable.

Passo 1: Montagem do circuito

Conforme ilustra a Figura 5.5:

- a. Conecte o pino 5v do Arduino à linha de alimentação positiva (vermelha) do protoboard.
- b. Conecte o pino GND do Arduino à linha de alimentação negativa (preta) do protoboard.
- c. Conecte um LED utilizando um resistor de 220 ohms (ou 330 ohms).
- d. Conecte o LED no pino digital 13.
- e. Conecte o potenciômetro na protoboard com o botão de girar virado para você.
- f. Conecte o pino da esquerda do potenciômetro na linha de alimentação GND.
- g. Conecte o pino da direita do potenciômetro na linha de alimentação positiva.
- h. Conecte o pino do centro do potenciômetro no pino analógico A1 do Arduino.



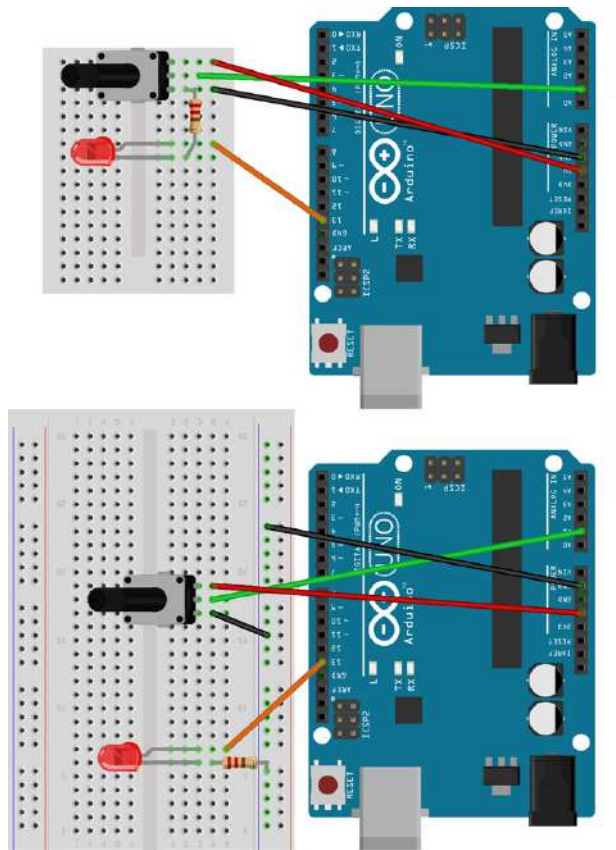


Figura 5.5 - Possibilidades de Montagem do Projeto

Passo 2: Programa Nº 1 – Controlando a frequência do pisca-pisca

Inicie o ambiente de desenvolvimento do Arduino e digite o sketch (programa) a seguir:

```
// Liga e desliga um LED na frequência
// determinada pelo potenciômetro.

// Pino de entrada ligado ao potenciômetro
int POT = A1;

// Pino conectado ao LED
int LED = 13;

// Variável que armazenará o valor do
// obtido do potenciômetro
int valor = 0;

void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  valor = analogRead(POT);
  digitalWrite(LED, HIGH);
  delay(valor);
  digitalWrite(LED, LOW);
  delay(valor);
}
```

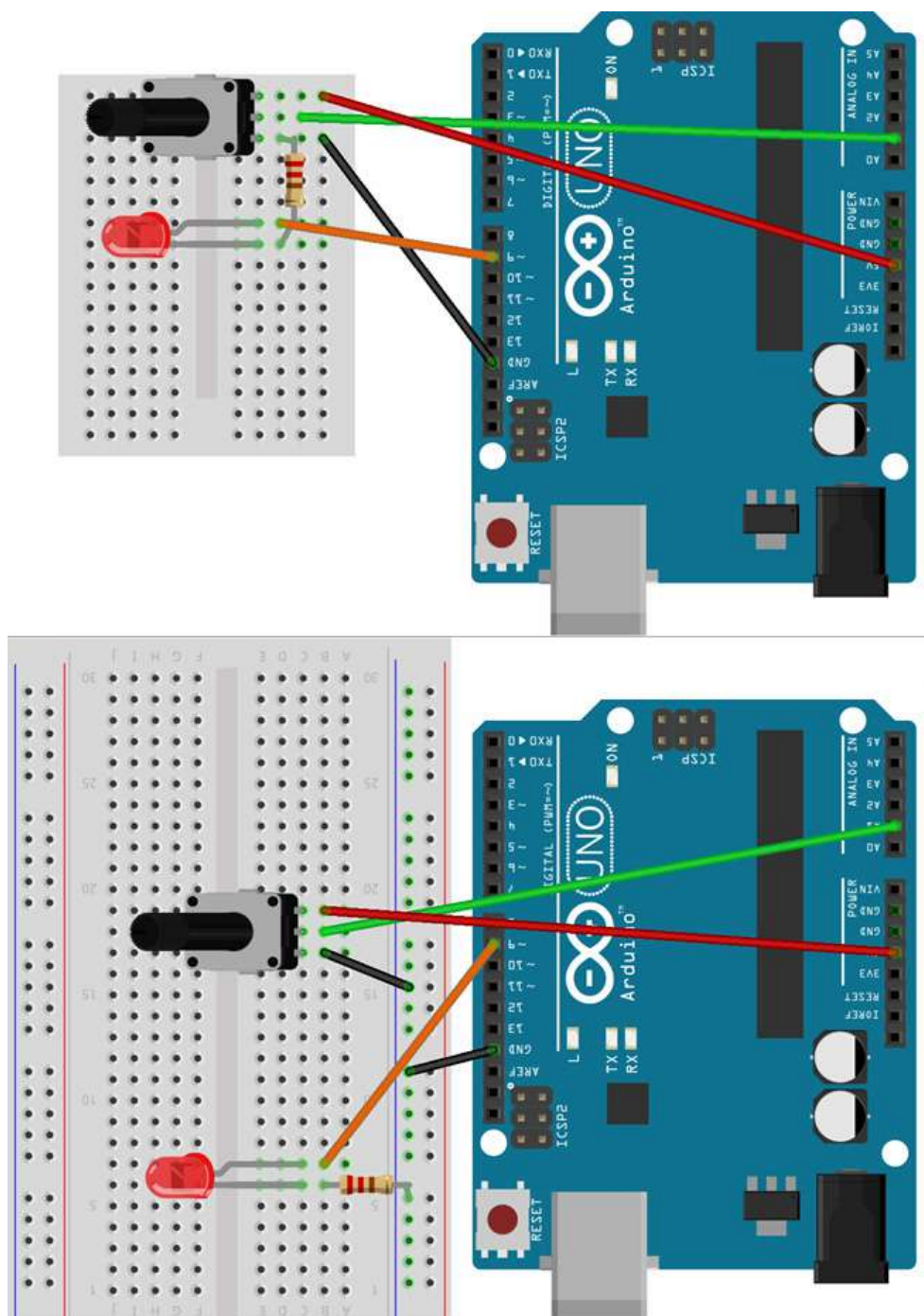
Passo 3: Montagem do circuito

Figura 3.6: Opções de Montagem do Projeto

Conforme ilustra a Figura 5.6:

- Conecte o pino 5v do Arduino à linha de alimentação positiva (vermelha) do protoboard.
- Conecte o pino GND do Arduino à linha de alimentação negativa (preta) do protoboard.
- Conecte um LED utilizando um resistor de 220 ohms (ou 330 ohms).
- Conecte o LED no pino digital 9.
- Conecte o potenciômetro na protoboard com o botão de girar virado para você.
- Conecte o pino da esquerda do potenciômetro na linha de alimentação GND.



- g. Conecte o pino da direita do potenciômetro na linha de alimentação positiva.
- h. Conecte o pino do centro do potenciômetro no pino analógico A1 do Arduino.

Passo 4: Programa N° 2 – Controle da intensidade da luminosidade

Inicie o ambiente de desenvolvimento do Arduino e digite o sketch (programa) a seguir:

```
// Controla a intensidade da luminosidade de um
// LED através de frequência determinada pelo
// potenciômetro.

int POT = A1;
int LED = 9;
int valor = 0;

void setup() {
  Serial.begin(9600);
  pinMode(LED, OUTPUT);
}

void loop() {
  valor = analogRead(POT);
  if(valor > 0) {
    // Acende o led com intensidade proporcional
    // ao valor obtido
    analogWrite(LED, (valor/4));

    // Exibe no Serial Monitor o valor obtido
    // do potenciômetro
    Serial.println(valor);
  }
}
```

Dicas - Como funciona a PWM?

A Modulação por Largura de Pulso (Pulse Width Modulation - PWM) é uma técnica que consiste em fornecer um sinal analógico através de meios digitais. A forma de onda do sinal digital consiste em uma onda quadrada que alterna seu estado em nível lógico alto e um nível lógico baixo (pode ser representado por ligado/

desligado ou pelo sistema binário 1 e 0).

A razão entre o período de pico e o período total da onda é chamada de Duty Cycle (Figura 5.7). Podemos, então, entender que para termos uma onda quadrada real (que possui picos e vales iguais) é necessário que o Duty Cycle seja de 50%, ou seja, 50% de pico e 50% de vale.

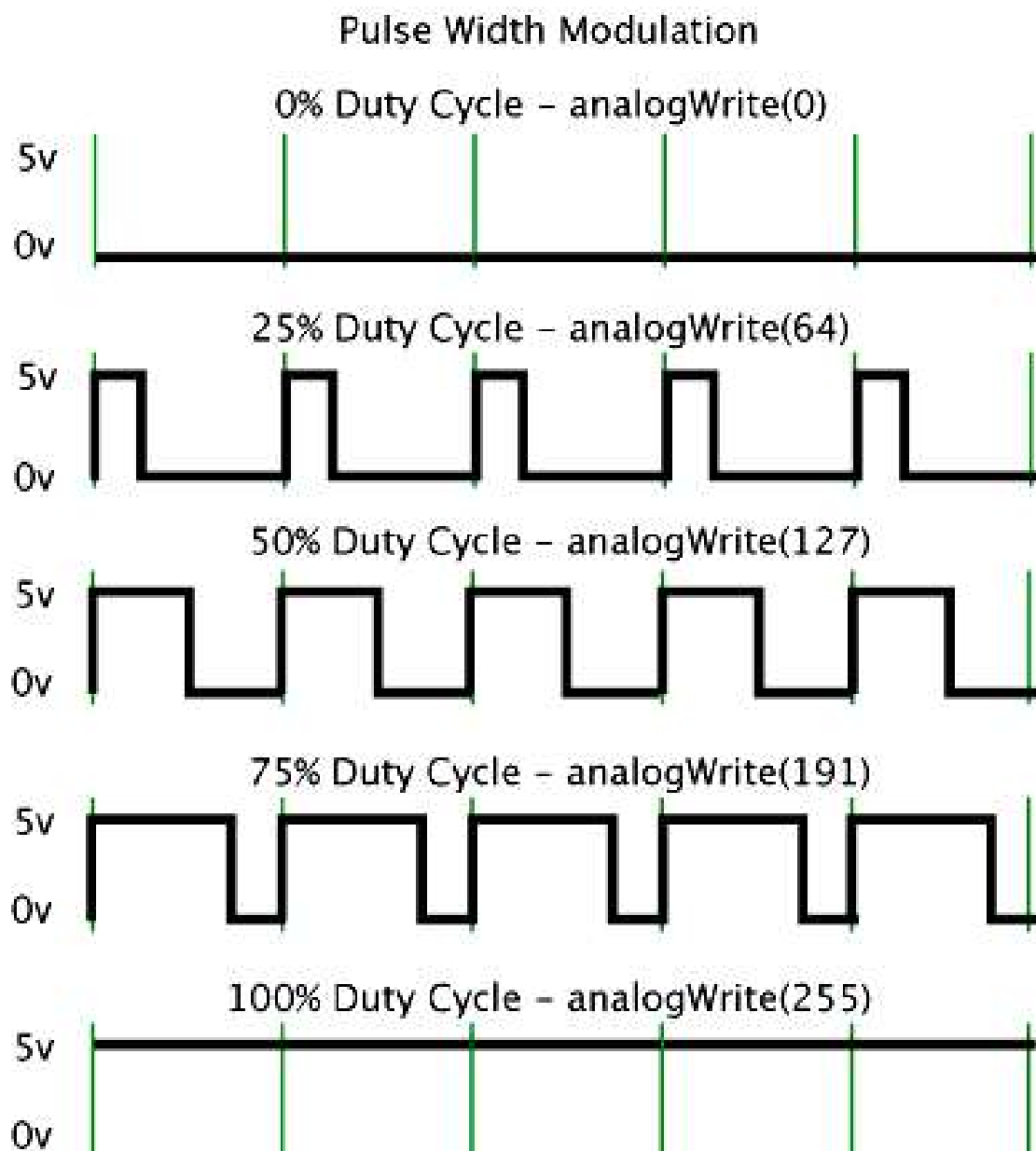


Figura 5.7: Razão de Ciclo

No Arduino UNO, as portas digitais que permitem PWM são as portas 3, 5, 6, 9, 10 e 11. Essas portas são facilmente

identificadas pelo símbolo “~” abaixo de cada porta.

CAPÍTULO 06

PROJETO 3 – LDR

O objetivo deste projeto é controlar o estado de um LED (aceso ou apagado) através da verificação de luminosidade do ambiente utilizando um sensor de luminosidade LDR. O LDR (Light Dependent Resistor) é um componente que varia a sua resistência conforme o nível de luminosidade que incide sobre ele. A resistência do LDR varia de forma inversamente proporcional à quantidade de luz incidente sobre ele. Quanto maior a luminosidade, menor será a resistência, por outro lado, no Arduino, maior será o valor presente na entrada analógica. Quanto menor for a luminosidade, maior será a resistência, ou seja, menor será o valor na entrada analógica do Arduino. Para essa experiência, vamos supor que o nível limite de luminosidade para que o LED se acenda como sendo um valor menor que 100 na entrada analógica. Para monitorar o valor gerado pelo LDR vamos estabelecer a comunicação serial entre o Arduino e o computador e, em seguida, utilizar o Serial Monitor da IDE do Arduino para monitorar.

Material necessário:

- 1 Arduino.
- 1 Resistor de 220 ohms (vermelho, vermelho, marrom) ou 330 ohms (laranja, laranja, marrom)2.
- 1 LED (qualquer cor) 2.
- 1 LDR1 2.
- 1 Resistor de 10k ohms (marrom, preto laranja) para o LDR1 2.
- 1 Protoboard2.
- Jumper cable.

1. Podem ser substituídos pelo módulo P13-LDR da GBK Robotics.
2. Podem ser substituídos pelo módulo P7-Sensor de Luminosidade da GBK Robotics.

Passo 1: Montagem do circuito

Conforme ilustra a Figura 5.1:

- a. Conecte o pino 5v do Arduino à linha de alimentação positiva (vermelha) da protoboard.
- b. Conecte o pino GND do Arduino à linha de alimentação negativa (preta) da protoboard.
- c. Coloque o resistor de 220 ohms (ou 330 ohms) entre a linha de alimentação negativa e qualquer outra linha da protoboard.
- d. Coloque o LED com o Cátodo (lado chanfrado) conectado ao resistor de 220 ohms (ou 330 ohms);
- e. Conecte o Ânodo do LED ao pino 13 do Arduino.
- f. Coloque o resistor de 10k ohms entre a linha de alimentação negativa e qualquer outra linha da protoboard.
- g. Conecte uma das extremidades do LDR na linha o resistor de 10k ohms.
- h. Conecte uma extremidade do jumper entre o LDR e o resistor. A outra extremidade conecte no pino analógico A0;
- i. Conecte a outra extremidade do LDR à linha de alimentação positiva (vermelha).

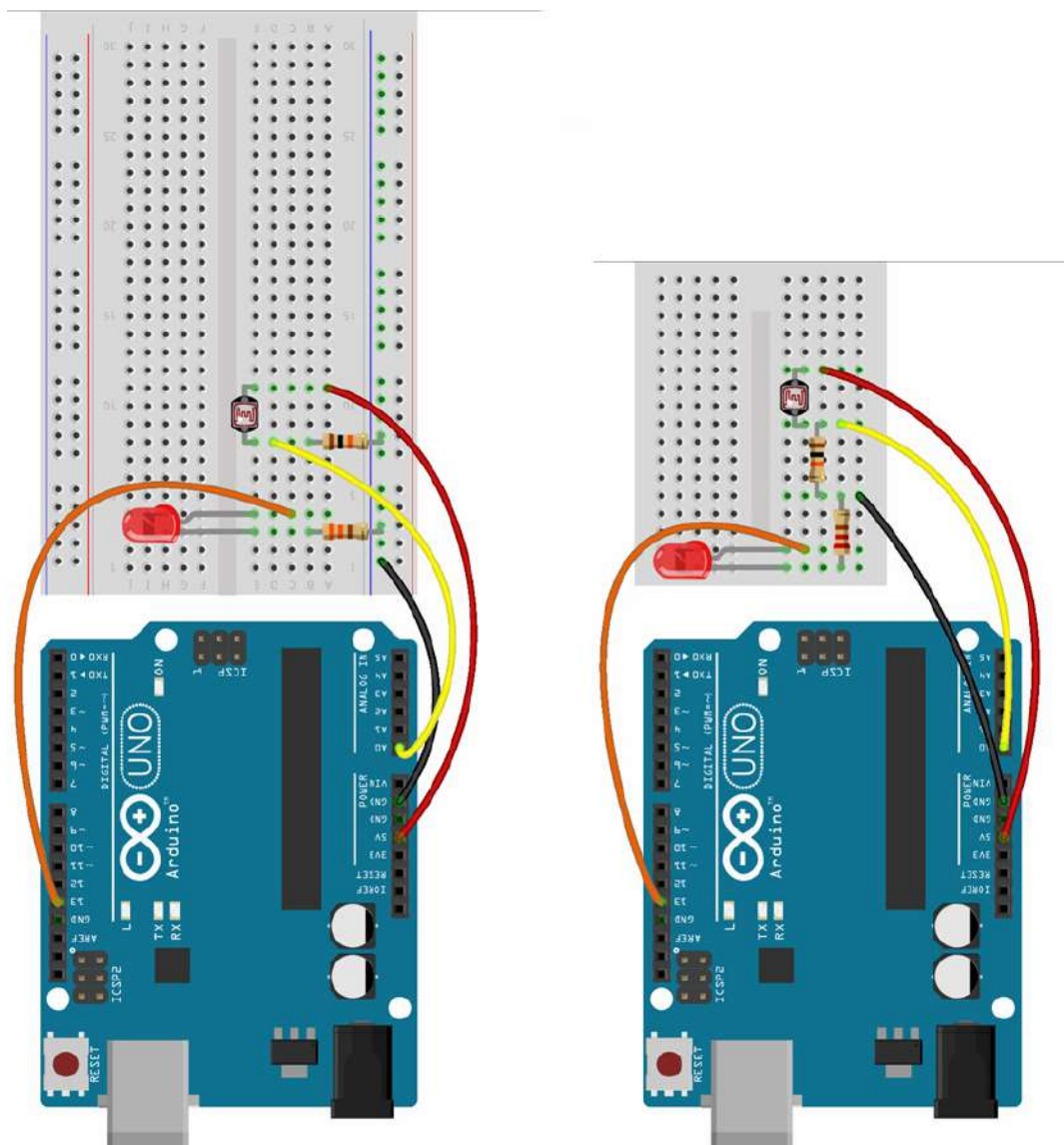


Figura 6.1: Disposição dos Componentes e Ligações

Passo 2: Programa

Inicie o ambiente de desenvolvimento do Arduino e digite o sketch (programa) a seguir:




```
// Pino no qual o LED está conectado
int LED = 13;

// Pino no qual o LDR está conectado
int LDR = A0;

// Variável que receberá o valor lido do LDR
int entrada = 0;

void setup() {
  // Iniciar e definir a velocidade de
  // comunicação da porta serial
  Serial.begin(9600);
  pinMode(LED, OUTPUT);
}

void loop() {
  entrada = analogRead(LDR);

  // Valor que será exibido no Serial Monitor do
  // ambiente de desenvolvimento do Arduino
  Serial.println(entrada);

  if (entrada < 500)
    digitalWrite(LED, HIGH); // Acender o LED
  else
    digitalWrite(LED, LOW); // Apagar o LED
  delay(100);
}
```

Passo 3: Serial Monitor

Clique no botão mostrado em destaque na Figura 6.1 para abrir a janela do Monitor Serial.

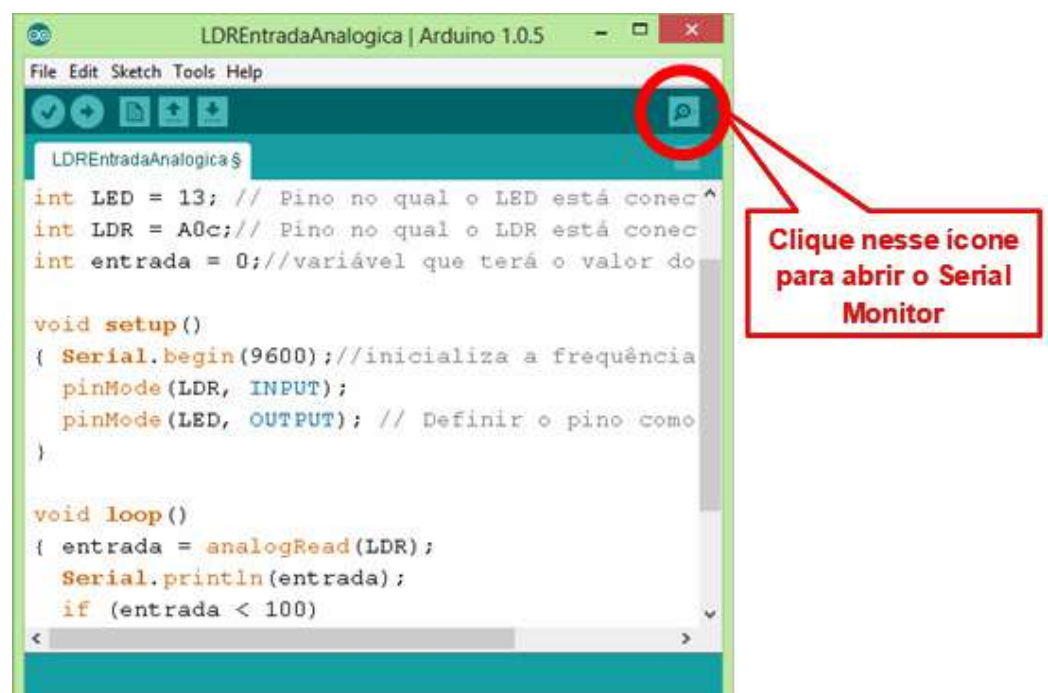


Figura 6.2: Botão para Abrir o Monitor Serial

Projeto 4 – Buzzer

O objetivo deste projeto é acionar um buzzer utilizando as funções `tone()` e `noTone()`. Buzzer é um dispositivo para geração de sinais sonoros (beeps), como aqueles encontrados em computadores. Para a emissão do som, o buzzer vibra através de um oscilador. Essa oscilação é determinada por uma frequência, que por sua vez define um som específico. Nesse experimento será usado 2 modos de uso da função `tone()` e uma pequena variação de sons representando notas musicais. A função `tone()` possui 2 sintaxes: `tone(pino, frequência)` e `tone(pino, frequência, duração)`, onde pino referencia qual é o pino que irá gerar a frequência (ligado ao positivo do buzzer), a frequência é definida em hertz e a duração (opcional) é em milissegundos. Caso opte pela sintaxe sem duração é necessário usar a função `noTone(pino)` para parar a frequência enviada pelo pino definido.

Material necessário:

- 1 Arduino.
- 1 Buzzer*.
- 1 Resistor de 220 ohms (vermelho, vermelho, marrom) ou 330 ohms (laranja, laranja, marrom) para o buzzer*.
- 1 Protoboard*.
- Jumper cable.

* Podem ser substituídos pelo módulo P15-Buzzer da GBK Robotics.

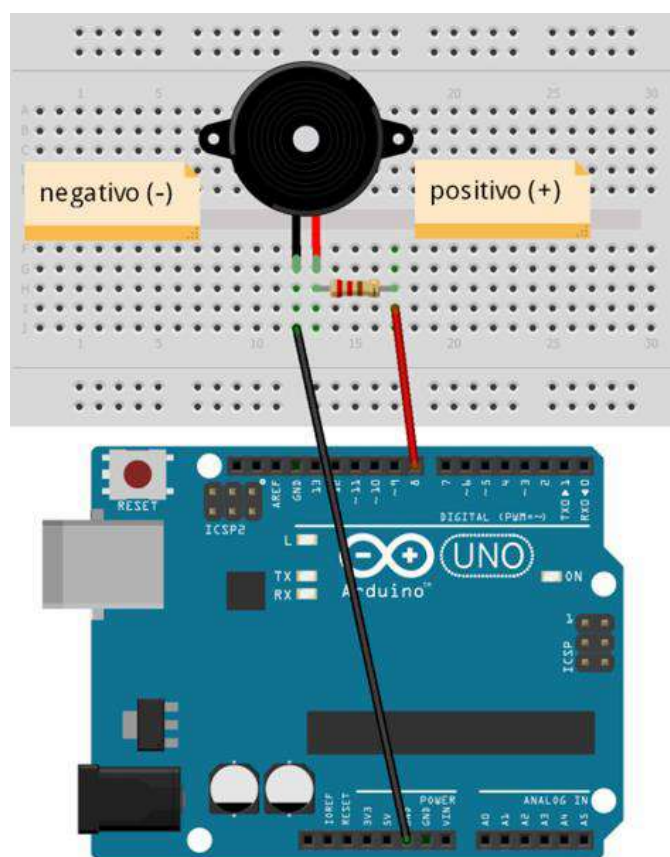


Figura 6.3 - Ligação do Buzzer ao Arduino

Conforme ilustra a Figura 6.3:

- Coloque o buzzer na protoboard.
- Conecte o pino GND do Arduino ao pino negativo do buzzer.

- Coloque o resistor de 220 ohms (ou 330 ohms) ligado ao pino positivo do buzzer.

Nesse resistor, conecte um jumper até a porta digital 8 do Arduino.





Passo 2: Programa N° 1 – Uso das funções tone(pino, frequência) e noTone(pino)

Inicie o ambiente de desenvolvimento do Arduino e digite o sketch (programa) a seguir:

```
// Pino ao qual o buzzer está conectado
int buzzer = 8;

void setup() {
  pinMode(buzzer, OUTPUT);
}

void loop() {
  tone(buzzer, 1200); //Define pino e frequência
  delay(500);
  noTone(buzzer); //Desliga o buzzer
  delay(500);
}
```

Passo 3: Programa N° 2 – Uso da função tone(pino, frequência, duração)

No ambiente de desenvolvimento do Arduino digite o sketch (programa) a seguir:


```
// Pino ao qual o buzzer está conectado
int buzzer = 8;

void setup() {
  pinMode(buzzer, OUTPUT);
}

void loop() {
  // Define pino, a frequência e duração
  tone(buzzer, 1200, 500);
  delay(1000);
}
```

Passo 4: Programa N° 3 – Notas musicais com buzzer

No ambiente de desenvolvimento do Arduino digite o sketch (programa) a seguir:

```
// Pino ao qual o buzzer está conectado
int buzzer = 8;

int numNotas = 10;
// Vetor contendo a frequência de cada nota
int notas[] = {261, 277, 294, 311, 330, 349, 370, 392, 415, 440};
// Notas: C, C#, D, D#, E, F, F#, G, G#, A
/*
C=Dó D=Ré E=Mi F=Fá G=Sol A=Lá B=Si
As notas sem o "#", são as notas naturais
(fundamentais).
Aqueles com o "#", são chamadas "notas
sustenidas" (por exemplo: C#=Dó Sustenido).
*/
void setup() {
  pinMode(buzzer, OUTPUT);
}

void loop() {
  for (int i = 0; i < numNotas; i++) {
    tone(buzzer, notas[i]);
    delay(500);
  }
  noTone(buzzer);
}
```

Projeto 5 – Botão

O objetivo deste projeto é utilizar um botão para acender, apagar e, posteriormente, também controlar a luminosidade de um LED.

Material necessário:

- 1 Arduino.
- 1 Resistor de 220 ohms (vermelho, vermelho,

marrom) ou 330 ohms (laranja, laranja, marrom) para o LED.

- 1 Resistor de 10k ohms (marrom, preto laranja) para o botão.
- 1 LED (qualquer cor) .
- 1 Protoboard.



- Jumper cable.

Passo 1: Montagem do circuito

Conforme o tipo de protoboard usado realize a montagem adotando, como referência, a Figura 6.5.

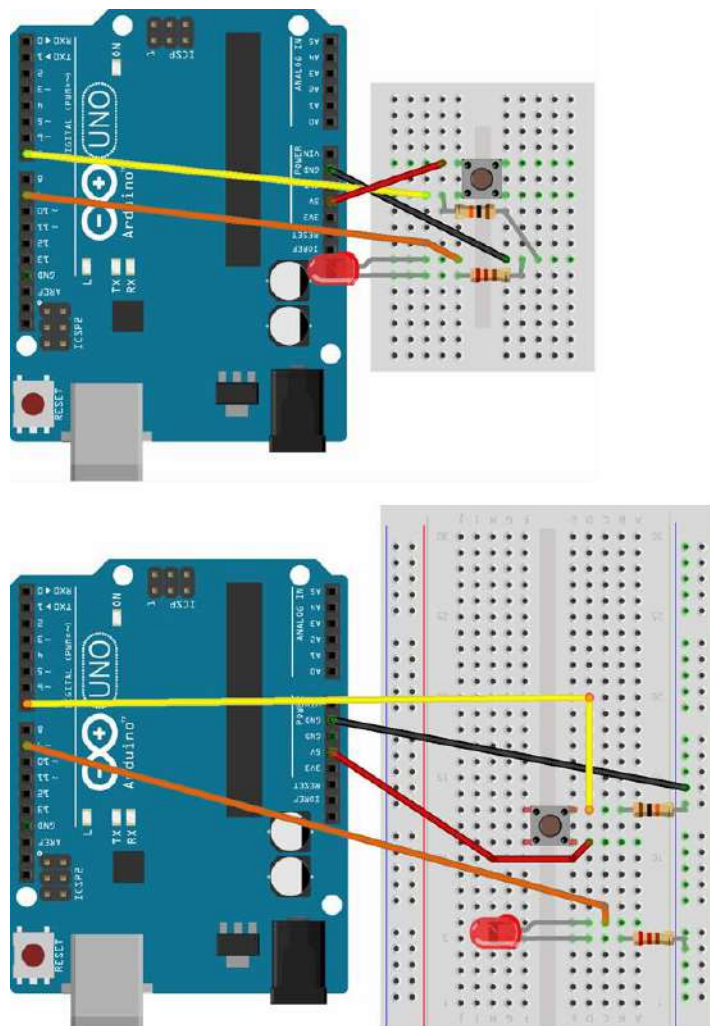


Figura 6.5 - Conexões Necessárias

Passo 2: Programa-Ligando e desligando um LED através do botão

Neste programa, enquanto o botão estiver pressionado o

LED ficará acesso, caso contrário, o LED fica apagado. Para implementá-lo, acesse o ambiente de desenvolvimento do Arduino e digite o sketch (programa) a seguir:

```
// Pino ao qual o LED está conectado
int LED = 9;
// Pino ao qual o Botão está conectado
int BOTAO = 7;
int valor;

void setup() {
  // Definir o pino como saída
  pinMode(LED, OUTPUT);
  // Definir o pino com entrada
  pinMode(BOTAO, INPUT);
}

void loop() {
  // Obtém LOW (botão não pressionado) ou
  // HIGH (botão pressionado)
  valor = digitalRead(BOTAO);

  digitalWrite(LED, valor);
  delay (500);
}
```

Passo 3: Programa-Ligando e desligando um LED através do botão (com troca de estado)

LED acenderá, a pressionar e soltar o botão novamente o LED vai apagar e assim sucessivamente. Desenvolva o programa acessando o ambiente de desenvolvimento do Arduino e digitando o seguinte sketch.

Neste outro programa ao pressionar e soltar o botão o

```
// Pino ao qual o LED está conectado
int LED = 9;
// Pino ao qual o Botão está conectado
int BOTAO = 7;
int valor;
int anterior = 0;
int estado = LOW;
void setup() {
  pinMode(LED, OUTPUT);
  pinMode(BOTAO, INPUT);
}

void loop() {
  valor = digitalRead(BOTAO);
  if (valor == HIGH && anterior == LOW) {
    estado = !estado;
  }
  digitalWrite(LED, estado);
  anterior = valor;
  delay (50);
}
```



Passo 4: Programa-Ligando, desligando e alterando a intensidade luminosa de um LED

Nesta implementação, através de um pino capaz de utilizar valores analógicos (PWM), poderemos ligar, desligar e obter uma variação de luminosidade. O LED começa com

seu estado “apagado”. Com um pressionar no botão, altera-se o estado do LED para “aceso”. Caso permaneça com o botão pressionado por mais de 5 segundos, poderá ser identificada uma variação de luminosidade. Crie o sketch a seguir no ambiente de desenvolvimento do Arduino.

```
// Pino ao qual o LED está conectado
int LED = 9;
// Pino ao qual o Botão está conectado
int BOTAO = 7;
int valor = LOW;
int valorAnterior = LOW;
// 0 = LED apagado, 1 = LED aceso
int estado = 0;
int brilho = 128;
unsigned long inicio;

void setup() {
  pinMode(LED, OUTPUT);
  pinMode(BOTAO, INPUT);
}

void loop() {
  valor = digitalRead(BOTAO);
  if ((valor == HIGH) && (valorAnterior == LOW)) {
    estado = 1 - estado;
    // Obtém a quantidade de milissegundos após
    // o Arduino ser inicializado
    inicio = millis();
    delay (10);
  }

  // Verifica se o botão está pressionado
  if ((valor == HIGH) && (valorAnterior == HIGH)) {
    // Verifica se o botão está pressionado por
    // mais de 0,5 segundos
    if (estado == 1 && (millis() - inicio) > 500) {
      brilho++;
      delay(10);
      if (brilho > 255)
        brilho = 0;
    }
  }
  valorAnterior = valor;

  if (estado == 1) {
    // Define o nível de luminosidade do LED
    analogWrite(LED, brilho);
  }
  else {
    analogWrite(LED, 0); // Apaga o LED
  }
}
```

Projeto 6 – LCD (Liquid Crystal Display 16x2)

O objetivo deste projeto é aprender a montagem de um display LCD 16x2 controlado pelo Arduino utilizando a biblioteca LiquidCrystal.h. Essa biblioteca possui funções que auxiliam nas configurações e tratamento dos dados a serem enviados ao LCD. A montagem do display deve ser de acordo com sua especificação (datasheet), onde cada um dos pinos possui uma função específica (ver no passo 1 – Montagem do circuito). Para ver todas as funções disponíveis na biblioteca LiquidCrystal.h acesse o site oficial da biblioteca, que está disponível em <http://arduino.cc/en/Reference/LiquidCrystal>.

Material necessário:

- 1 Arduino.
- 1 LCD 16x2.
- 1 Resistor de 220 ohms (vermelho, vermelho, marrom).
- 1 Protoboard.
- Jumper cable.

Passo 1: Montagem do circuito

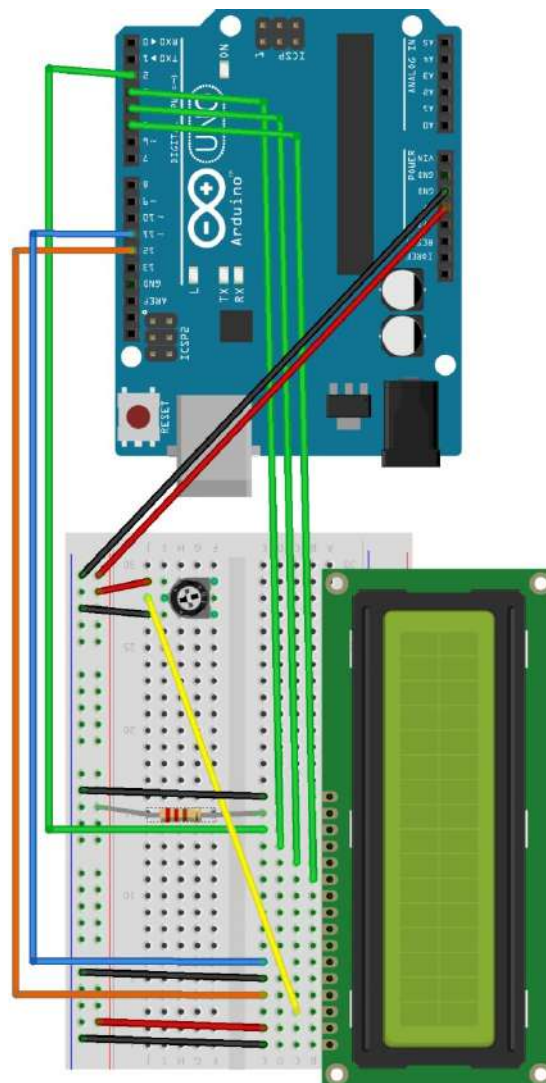


Figura 6.6 - Conexão do Display LCD ao Arduino

Conforme ilustra a Figura 3.15, realizar a seguinte sequência de montagem:

- Pino 1 do LCD ligado ao GND do Arduino.
- Pino 2 do LCD ligado ao 5V do Arduino.
- Pino 3 do LCD ligado ao pino central do potenciômetro (controle de contraste).
- Pino 4 do LCD ligado ao pino digital 12 do Arduino.
- Pino 5 do LCD ligado ao GND do Arduino.
- Pino 6 do LCD ligado ao pino digital 11 do Arduino.
- Pino 11 do LCD ligado ao pino digital 5 do Arduino.
- Pino 12 do LCD ligado ao pino digital 4 do Arduino.
- Pino 13 do LCD ligado ao pino digital 3 do Arduino.
- Pino 14 do LCD ligado ao pino digital 2 do Arduino.
- Pino 15 do LCD ligado ao 5V do Arduino com um resistor de 220 ohms (controle do brilho).
- Pino 16 do LCD ligado ao GND do Arduino.

Na tabela a seguir apresentamos as funções de cada um dos pinos do Display de Cristal Líquido:

Pino	Símbolo	Função
1	VSS	GND(Alimentação)
2	VDD	5V(Alimentação)
3	VO	Ajuste de Contraste
4	RS	Habilita/Desabilita Seletor de Registrador
5	R/W	Leitura/Escrita
6	E	Habilita Escrita no LCD
7	DB0	Dado
8	DB1	Dado
9	DB2	Dado
10	DB3	Dado
11	DB4	Dado
12	DB5	Dado
13	DB6	Dado
14	DB7	Dado
15	A	5V(Backlight)
16	K	GND(BackLight)

Passo 2: Programa N° 1 – Exibição simples de texto

Inicie o ambiente de desenvolvimento do Arduino e digite o sketch (programa) a seguir:




```

#include <LiquidCrystal.h>

// LCD - Modelo ACM 1602K
// Pin Sigla Função Conectar
// 1 Vss Ground GND
// 2 Vdd +5V VCC
// 3 Vo LCD contrast adjust Potenciômetro
// 4 RS Register select Arduino 12
// 5 R/W Read/write GND
// 6 E Enable Arduino 11
// 7 DB0 Data bit 0 NC
// 8 DB1 Data bit 1 NC
// 9 DB2 Data bit 2 NC
// 10 DB3 Data bit 3 NC
// 11 DB4 Data bit 4 Arduino 5
// 12 DB5 Data bit 5 Arduino 4
// 13 DB6 Data bit 6 Arduino 3
// 14 DB7 Data bit 7 Arduino 2
// + BL+ Alimentação (+) Resistor de 1k para VCC
// - BL- Alimentação (-) GND

#define TEMPO_ATUALIZACAO 500

LiquidCrystal lcd (12, 11, 5, 4, 3, 2);

void setup() {
  pinMode(12, OUTPUT);
  pinMode(11, OUTPUT);

  // Inicia o LCD com dimensões 16 x 2 (Colunas
  // x Linhas)
  lcd.begin (16, 2);
}

void loop() {
  lcd.clear();
  lcd.setCursor(0, 0); // Linha 0 e Coluna 0
  lcd.print("Ola");
  lcd.setCursor(0, 1); // Linha 1 e Coluna 0
  lcd.print("FATECINO");
  delay(TEMPO_ATUALIZACAO);
}

```

Passo 3: Programa N° 2 – Rolagem (scroll) do texto

Inicie o ambiente de desenvolvimento do Arduino e digite o sketch (programa) a seguir:



```

#include <LiquidCrystal.h>

// LCD - Modelo ACM 1602K
// Pin Sigla Função          Conectar
// 1 Vss Ground             GND
// 2 Vdd +5V                VCC
// 3 Vo LCD contrast adjust Potenciômetro
// 4 RS Register select     Arduino 12
// 5 R/W Read/write         GND
// 6 E Enable               Arduino 11
// 7 DB0 Data bit 0         NC
// 8 DB1 Data bit 1         NC
// 9 DB2 Data bit 2         NC
// 10 DB3 Data bit 3        NC
// 11 DB4 Data bit 4        Arduino 5
// 12 DB5 Data bit 5        Arduino 4
// 13 DB6 Data bit 6        Arduino 3
// 14 DB7 Data bit 7        Arduino 2
// + BL+ Alimentação (+)    Resistor de 1k
//                             para VCC
// - BL- Alimentação (-)    GND

#define TEMPO_ATUALIZACAO 500

LiquidCrystal lcd (12, 11, 5, 4, 3, 2);
int inicio = 0, tamanho = 1;
boolean alterar = false;

void setup() {
  pinMode(12, OUTPUT);
  pinMode(11, OUTPUT);

  // Inicia o LCD com dimensões 16 x 2 (Colunas
  // x Linhas)
  lcd.begin (16, 2);
}

void loop() {
  lcd.clear(); // Limpa o display de LCD
  String nome = "Fatecino-Clube de Arduino";
  if (tamanho < 16) {
    // Posiciona o cursor nas coordenadas
    // especificadas
    lcd.setCursor(16 - tamanho, 0);

    // Exibe no LCD
    lcd.print(nome.substring(inicio, tamanho));
    tamanho++;
  }
  else {
    if (!alterar) {
      alterar = !alterar;
      tamanho = 16;
      lcd.setCursor(0, 0);
    }
    lcd.print(nome.substring(inicio, inicio +
tamanho));
    inicio++;
  }
  if (inicio > nome.length()) {
    inicio = 0;
    tamanho = 1;
    alterar = !alterar;
  }
  delay(TEMPO_ATUALIZACAO);
}

```

Projeto 7 – Uso do Sensor de Temperatura

O objetivo deste projeto é enviar os dados de um sensor de temperatura (LM35 ou DHT11) para a saída serial.

Material necessário:

- 1 Arduino.
- 1 LM35 (ou DHT11).
- 1 Protoboard.
- Jumper cable.

Passo 1: Montagem do circuito (com o LM35)

Realize as conexões conforme ilustra a Figura 7.1.

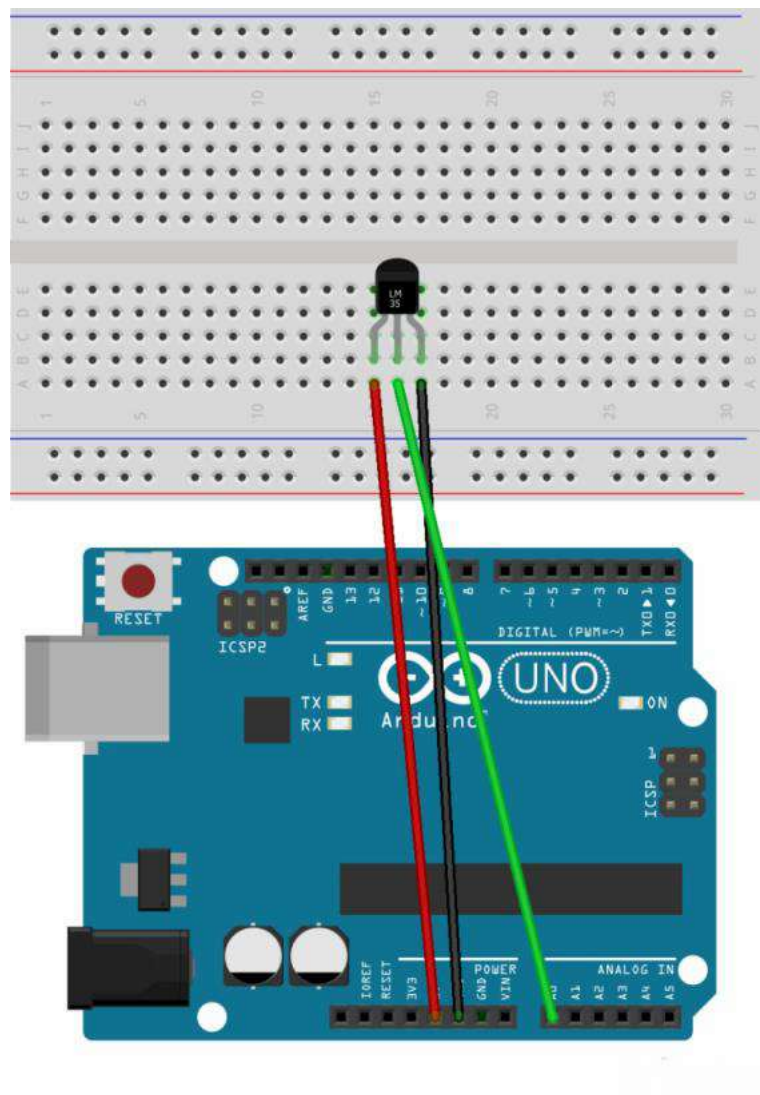


Figura 7.1 - Ligação do Arduino ao LM35



Passo 2: Programa para o LM35

Inicie o ambiente de desenvolvimento do Arduino e digite o sketch (programa) a seguir:

```
// Pino Analógico que vai ser ligado ao
// terminal 2 do LM35
const int LM35 = A0;

// Tempo de atualização em ms (milissegundos)
const int ATRASO = 5000;

// Base de conversão para Graus Celsius:
// ((5/1023) * 100)
const float BASE_CELSIUS =
    0.4887585532746823069403714565;

void setup() {
    Serial.begin(9600);
}

void loop() {
    Serial.print("Temperatura: ");
    Serial.print(lerTemperatura());
    Serial.println("\260C");
    delay(ATRASO);
}

float lerTemperatura() {
    return (analogRead(LM35) * BASE_CELSIUS);
}
```

Passo 3: Montagem do circuito (com o DH11)

Monte o projeto conforme mostra a Figura 7.1.

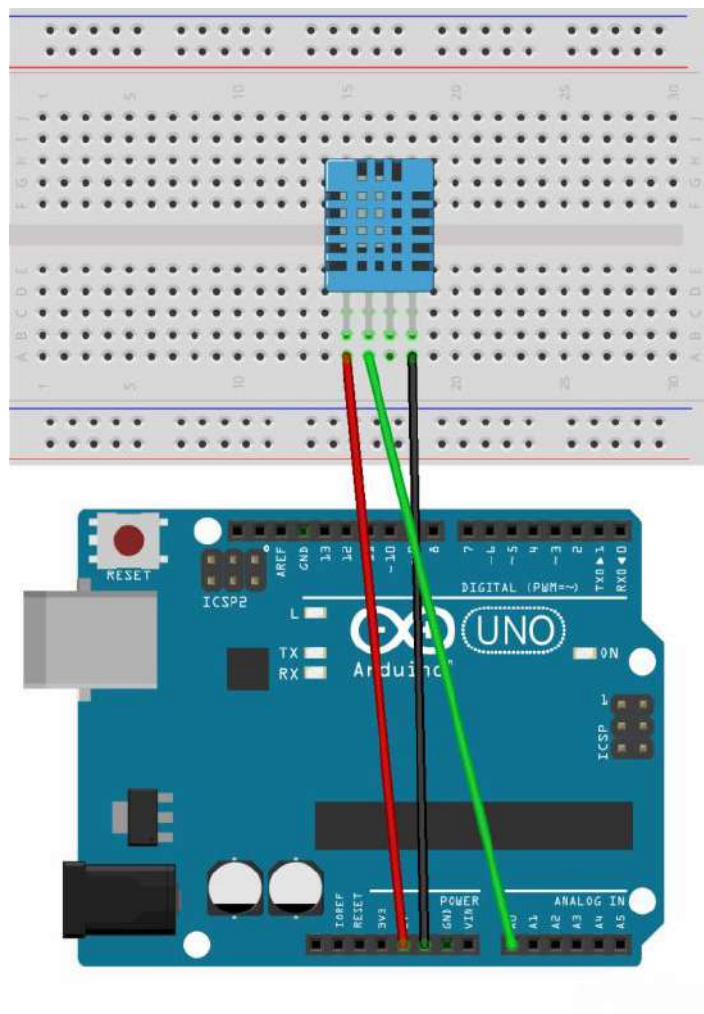


Figura 7.1 - Conexão do Arduino ao DHT11

Passo 4: Programa para o DHT11

Baixe a biblioteca a partir do link: <http://hobbyist.co.nz/sites/default/files/WeatherStation/DHT.zip>. Descompacte

o conteúdo do arquivo DHT.zip na pasta Arduino\libraries\ localizada na pasta Documentos. Em seguida, inicie o ambiente de desenvolvimento do Arduino e digite o sketch.



```

#include <dht.h>

// Pino analógico ligado ao terminal 2 do DHT11
const int DHT11 = A0;

//Tempo de atualização entre as leituras em ms
const int ATRASO = 2000;

dht sensor;

float temperatura, umidade;

void setup() {
  Serial.begin(9600);
}

void loop() {
  // Obtém os dados do sensor
  sensor.read11(DHT11);

  // Obtém a temperatura
  temperatura = sensor.temperature;

  // Obtém a umidade
  umidade = sensor.humidity;

  // Enviar a temperatura e a umidade através
  // da saída serial
  Serial.print("Temperatura: ");
  Serial.print(temperatura);
  Serial.print(';');
  Serial.print("\260C, Umidade: ");
  Serial.print(umidade);
  Serial.print(';');
  Serial.println("%");
  delay(ATRASO);
}

```

Passo 5: Conversão para Fahrenheit e Kelvin

Utilizando as fórmulas a seguir, alterar o programa para exibir a temperatura em Fahrenheit e Kelvin.

- $F = (C * 9) / 5 + 32$
- $K = C + 273.15$

Passo 6: Exibição da temperatura em um display de LCD

Utilizando como base o Projeto-6 (Uso do LCD), enviar os dados de temperatura para um display de LCD.

Dica: Como imprimir o símbolo de graus no display de LCD? Tente usar `lcd.write(B11011111);`



Projeto 8 – Termistor

O objetivo deste projeto é obter a temperatura ambiente através da leitura dos dados recebidos de um termistor. Há dois tipos de termistores:

- **Termistor PTC (Positive Temperature Coefficient):**
Este tipo de termistor tem o coeficiente de temperatura positivo, ou seja, a resistência aumenta com o aumento da temperatura.
- **Termistor NTC (Negative Temperature Coefficient):**
Já este é o inverso do anterior e seu coeficiente de temperatura é negativo. Com isto sua resistência diminui com o aumento da temperatura.

Material necessário:

- 1 Arduino.
- 1 Termistor NTC de 10k ohms*.
- 1 Resistor de 10k ohms (marrom, preto, laranja) para o termistor*.
- 1 Protoboard*.
- Jumper cable.
- Podem ser substituídos pelo módulo P10-Sensor de Temperatura com NTC da GBK Robotics.

Passo 1: Montagem do circuito

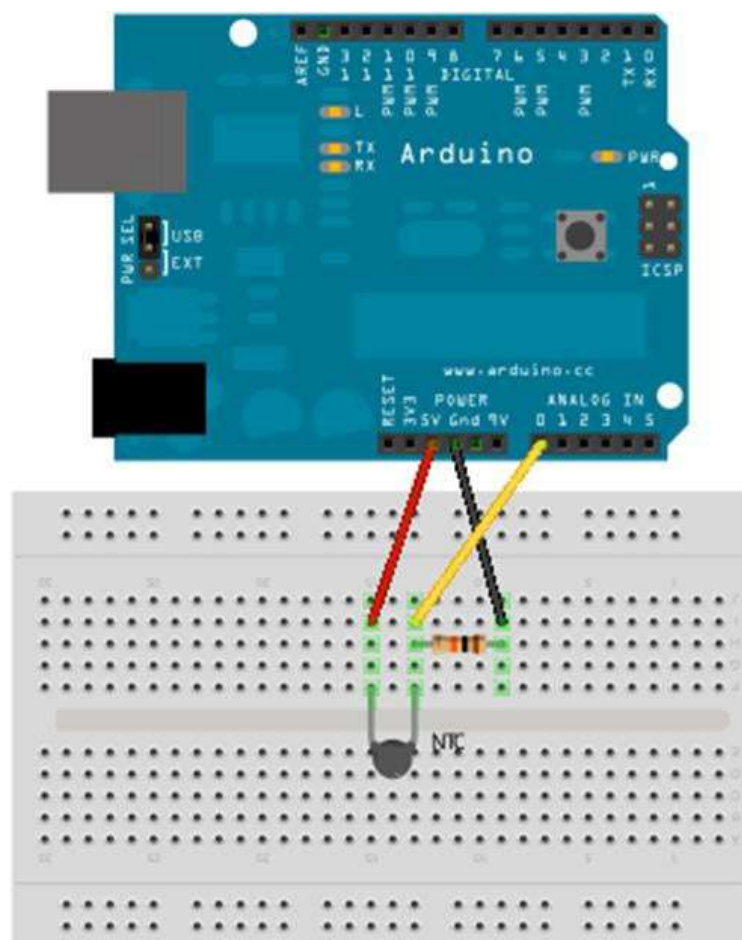


Figura 7.3 - Ligação do Termistor ao Arduino



Conforme ilustra a Figura 7.3:

- Coloque o termistor na protoboard.
- Conecte o pino 5V do Arduino a um dos terminais do termistor.
- Conecte o resistor de 10k ohms ao outro pino do termistor.
- Conecte o pino GND do Arduino ao outro pino do resistor de 10k ohms.

- Ligue o pino A0 do Arduino junto com o resistor de 10k ohms e o terminal do termistor.

Passo 2: Programa

A biblioteca Thermistor pode ser baixada através do link: http://www.fatecjd.edu.br/fatecino/arq_projetos/biblioteca-thermistor.zip. Após sua instalação, inicie o ambiente de desenvolvimento do Arduino e digite o sketch mostrado a seguir.

```
#include <Thermistor.h>

Thermistor termistor(A0);

void setup() {
  Serial.begin(9600);
}

void loop() {
  int temperatura = termistor.getTemp();
  Serial.print("A temperatura e: ");
  Serial.print(temperatura);
  Serial.println("°C");
  delay(1000);
}
```

Passo 3: Conversão para Fahrenheit e Kelvin

Utilizando as fórmulas a seguir, alterar o programa para exibir a temperatura em Fahrenheit e Kelvin.

- $F = (C * 9) / 5 + 32$
- $K = C + 273.15$

distantes entre 1 cm e 200 cm. Este sensor emite um sinal ultrassônico que reflete em um objeto e retorna ao sensor, permitindo deduzir a distância do objeto ao sensor tomando o tempo da trajetória do sinal. A velocidade do sinal no ar é de aproximadamente 340 m/s (velocidade do som). O sensor possui 4 pinos, sendo:

- VCC - Alimentação de 5V (+).
- TRIG - Pino de gatilho (trigger) .
- ECHO - Pino de eco (echo) .
- GND – Terra (-).

Projeto 9 – Sensor Ultrassônico (HC-SR04)

O objetivo deste projeto é utilizar o sensor ultrassônico HC-SR04 para medir distâncias entre o sensor e um objeto. O sensor HC-SR04 permite detectar objetos que lhe estão

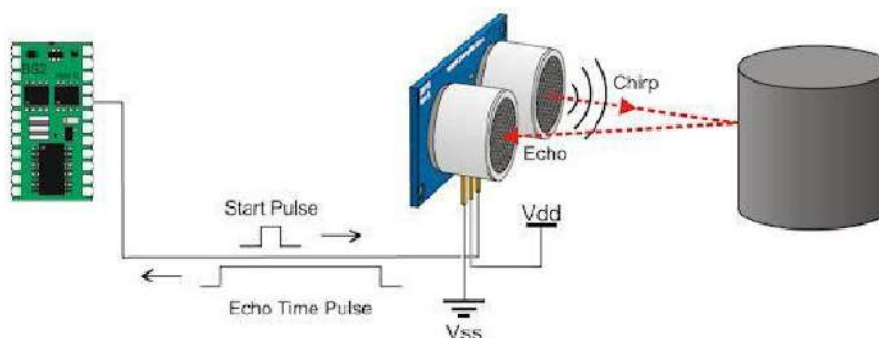


Figura 7.4 - Funcionamento do Sensor Ultrassônico

O pino ligado ao trigger (TRIG) normalmente deve estar em nível baixo. Para iniciar uma leitura de distância, o mesmo deve ser colocado em nível alto por 10 microsegundos e retornar para nível baixo em seguida. Neste momento, 8 pulsos de 40kHz são emitidos e no pino de eco (ECHO) será gerado um sinal em nível alto proporcional à distância do sensor ao objeto. Em seguida, basta verificar o tempo em que o pino ECHO permaneceu em nível alto e utilizar a fórmula de cálculo de distância (em centímetros): $\text{distância} = \text{duração} / 58$.

Material necessário:

- 1 Arduino.
- 1 sensor HC-SR04.
- 1 Protoboard.
- Jumper cable.

Passo 1: Montagem do circuito

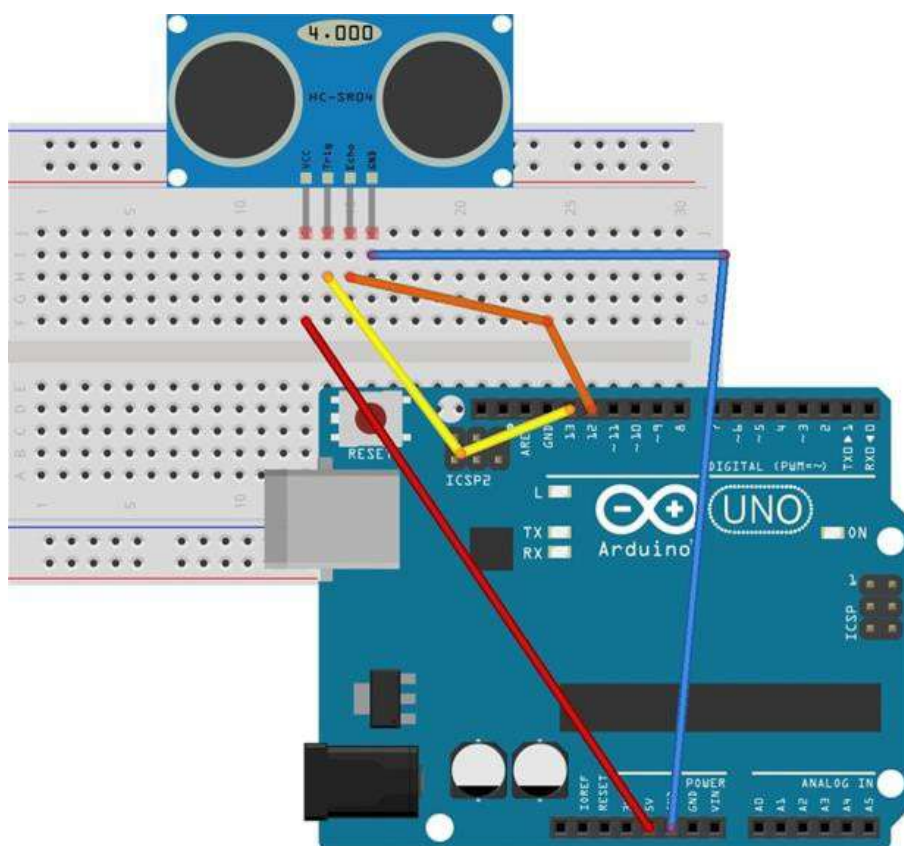


Figura 7.5 - Conexão do HC-SR04 ao Arduino

Conforme ilustra a Figura 7.5, realize as seguintes ligações:

- Pino GND do HC-SR04 ligado ao GND do Arduino.
- Pino VCC do HC-SR04 ligado ao 5V do Arduino.
- Pino TRIG do HC-SR04 ligado ao pino 13 do Arduino.

- Pino ECHO do HC-SR04 ligado ao pino 12 do Arduino.

Passo 2: Programa

Inicie o ambiente de desenvolvimento do Arduino e digite o sketch (programa) a seguir:


```
// Pino 12 irá receber o pulso do echo
int echoPino = 12;

// Pino 13 vai enviar o pulso para gerar o echo
int trigPino = 13;

long duracao = 0;
long distancia = 0;

void setup() {
    // Iniciar a porta serial com velocidade de
    // 9600 bits por segundo
    Serial.begin(9600);
    pinMode(echoPino, INPUT);
    pinMode(trigPino, OUTPUT);
}

void loop() {
    // Pino trigger com um pulso baixo LOW
    // (desligado)
    digitalWrite(trigPino, LOW);

    // Delay (atraso) de 10 microssegundos
    delayMicroseconds(10);

    // Pino trigger com pulso HIGH (ligado)
    digitalWrite(trigPino, HIGH);

    // Delay (atraso) de 10 microssegundos
    delayMicroseconds(10);

    // Pino trigger com um pulso baixo LOW
    // (desligado) novamente
    digitalWrite(trigPino, LOW);

    // A função pulseIn verifica o tempo que o
    // pino ECHO ficou HIGH
    // Calculando, desta forma, a duração do
    // tráfego do sinal
    duracao = pulseIn(echoPino, HIGH);

    // Cálculo: distância = duração / 58.
    distancia = duracao / 58;

    Serial.print("Distancia em cm: ");
    Serial.println(distancia);
    delay(100);
}
```

Passo 3: Exibição da distância em um display de LCD

Utilizando como base o Projeto-6 (Uso do LCD), enviar os dados da distância para um display de LCD, tanto em centímetros quanto em polegadas. Para o cálculo de polegadas utilize: $\text{distância} = \text{duração} / 37$.

Projeto 10 – Piezo Elétrico

O objetivo desse projeto é utilizar um piezo elétrico para identificar uma batida (pressão sobre o piezo) e acionar o toque em um buzzer. Um piezo elétrico possui a capacidade de converter uma oscilação mecânica (pressão sobre ele) em um sinal elétrico. O inverso desse princípio, ou seja, converter um sinal elétrico em oscilação mecânica é o que ocorre em um buzzer. A pressão sobre o piezo gera uma pequena corrente elétrica, por isso possuem polaridade, sendo o centro prateado polo positivo (ligado ao fio vermelho) e a borda dourada, o negativo (ligado ao fio preto). Essa corrente elétrica atinge um limiar, dependendo da pressão, e é a partir dele é que verificamos se houve ou não um toque nele. Esse limiar deve ser ajustado para definir a sensibilidade necessária

do toque e um valor analógico será lido pelo Arduino.

Material necessário:

- 1 Arduino.
- 1 Buzzer*.
- 1 Piezo elétrico.
- 1 Resistor de 1k ohms (marrom, preto, vermelho) .
- 1 Resistor de 220 ohms (vermelho, vermelho, marrom) ou 330 ohms (laranja, laranja, marrom)*.
- 1 Protoboard.
- Jumper cable.

* Podem ser substituídos pelo módulo P15-Buzzer da GBK Robotics.

Passo 1: Montagem do circuito

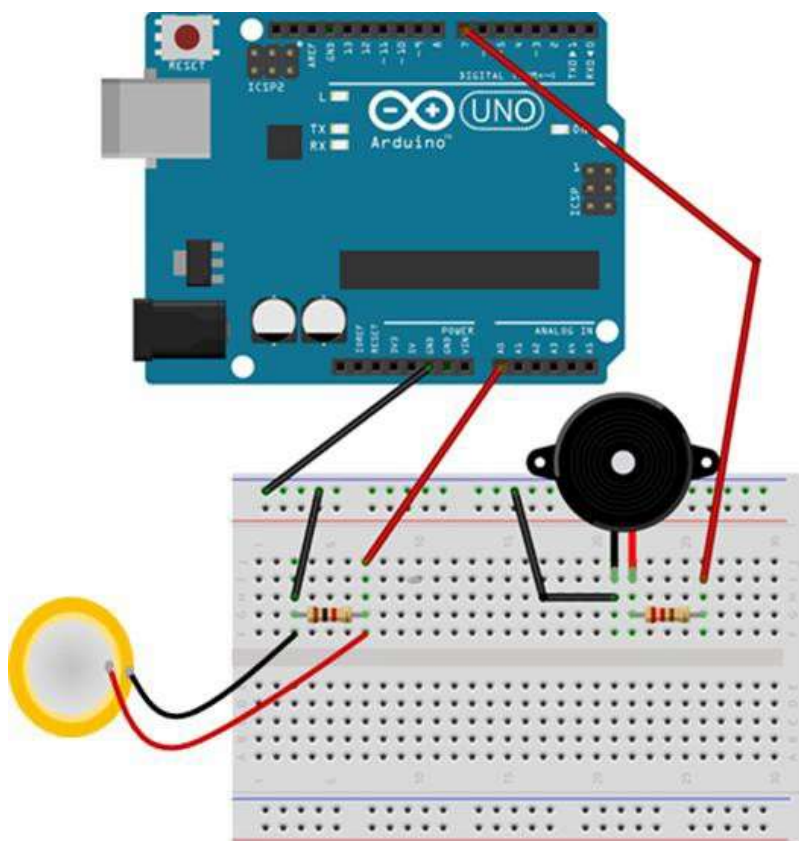


Figura 7.6 - Ligação do Arduino ao Buzzer e ao Piezo

Realize a sequência de montagem ilustrada pela Figura 7.6:

- Conecte o GND do Arduino na alimentação GND da protoboard.
- Conecte um resistor de 1k ohms no sentido horizontal da protoboard.
- Conecte os fios do piezo nas extremidades desse resistor.
- Conecte um jumper no GND da protoboard e na linha do negativo do piezo.
- Conecte um jumper no positivo do piezo e no pino

A0 do Arduino.

- Conecte o pino GND do Arduino ao pino negativo do buzzer já colocado na protoboard.
- Coloque o resistor de 220 ohms (ou 330 ohms) ligado ao pino positivo do buzzer.
- Nesse resistor, conecte um jumper até a porta digital 7 do Arduino.

Passo 2: Programa

Inicie o ambiente de desenvolvimento do Arduino e digite o sketch (programa) a seguir:

```
int BUZZER = 7 ;
int PIEZO = A0;
int LIMIAR = 50;      // Valor do limiar, que
                       // define se houve toque ou não!
int VALOR_PIEZO = A0; // Valor que será lido
                       // pelo piezo

void setup() {
  pinMode(BUZZER, OUTPUT);
  pinMode(BUZZER, INPUT);
  Serial.begin(9600);
}

void loop() {
  VALOR_PIEZO = analogRead(PIEZO);

  // Verifica se o valor lido pela pressão no
  // piezo ultrapassa o limiar
  if (VALOR_PIEZO >= LIMIAR) {
    // Ativa o buzzer e mostra uma mensagem no
    // Monitor Serial
    tone(BUZZER, 1600, 200);
    Serial.println("Bip!");
  }
  delay(100);
}
```

Projeto 11 – Relógio com LCD

O objetivo deste projeto é criar um relógio digital a partir de um módulo Real Time Clock (RTC) e um display LCD 16x2. Neste projeto usaremos as bibliotecas RTCLib.h e LiquidCrystal.h. A biblioteca RTCLib.h, disponível para download em <https://github.com/adafruit/RTCLib>, irá fornecer as funções necessárias para a utilização do módulo RTC DS-1307. Por outro lado, a biblioteca

LiquidCrystal.h possui funções que auxiliam nas configurações e tratamento dos dados a serem enviados ao LCD. A montagem do display deve ser de acordo com sua especificação (datasheet), onde cada um dos pinos apresenta uma função específica (ver no Passo 2 – Montagem do circuito). Para ver todas as funções disponíveis na biblioteca LiquidCrystal.h acesse o site oficial da biblioteca, que está disponível em <http://arduino.cc/en/Reference/LiquidCrystal>.



Material necessário:

- 1 Arduino.
 - 1 Real Time Clock (RTC DS-1307).
 - 1 LCD 16x2.
 - 1 Resistor de 220 ohms (vermelho, vermelho, marrom).
- 1 Protoboard.
 - Jumper cable.

Passo 1: Importar a Biblioteca RTCLib

A biblioteca RTCLib não faz parte da distribuição padrão da IDE do Arduino, desta forma, torna-se necessário importá-la antes de utilizar o RTC pela primeira vez. Para isso, no menu “Sketch”, escolha a opção “Importar Biblioteca” e, “Add Library”, conforme ilustrado pela Figura 7.7.

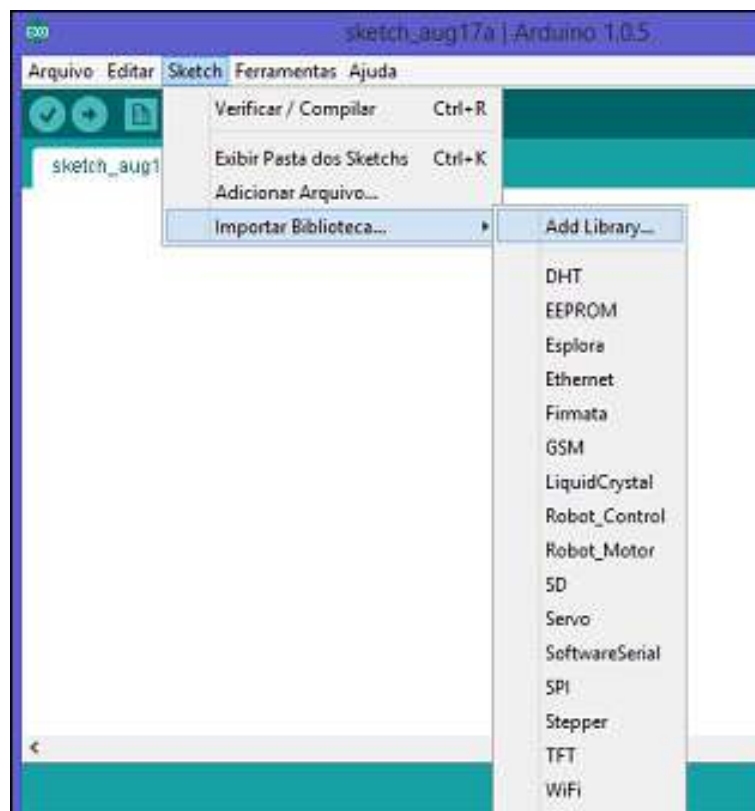


Figura 7.7 - Adicionar uma Biblioteca

Em seguida, observe a Figura 7.8 e selecione o arquivo compactado (ZIP) que contém a biblioteca a ser importada, ou seja:



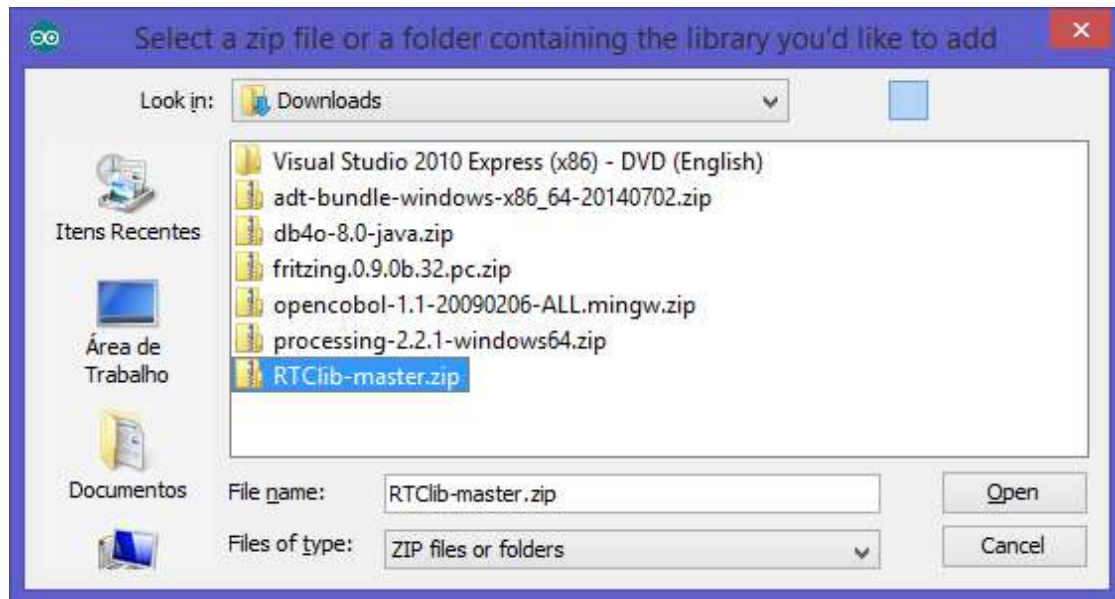


Figura 7.8 - Selecionar o Arquivo Zip ou a Pasta

Após importar a biblioteca a mesma estará disponível para uso dentro da opção do menu “Importar Biblioteca”, conforme podemos observar na Figura 7.9.

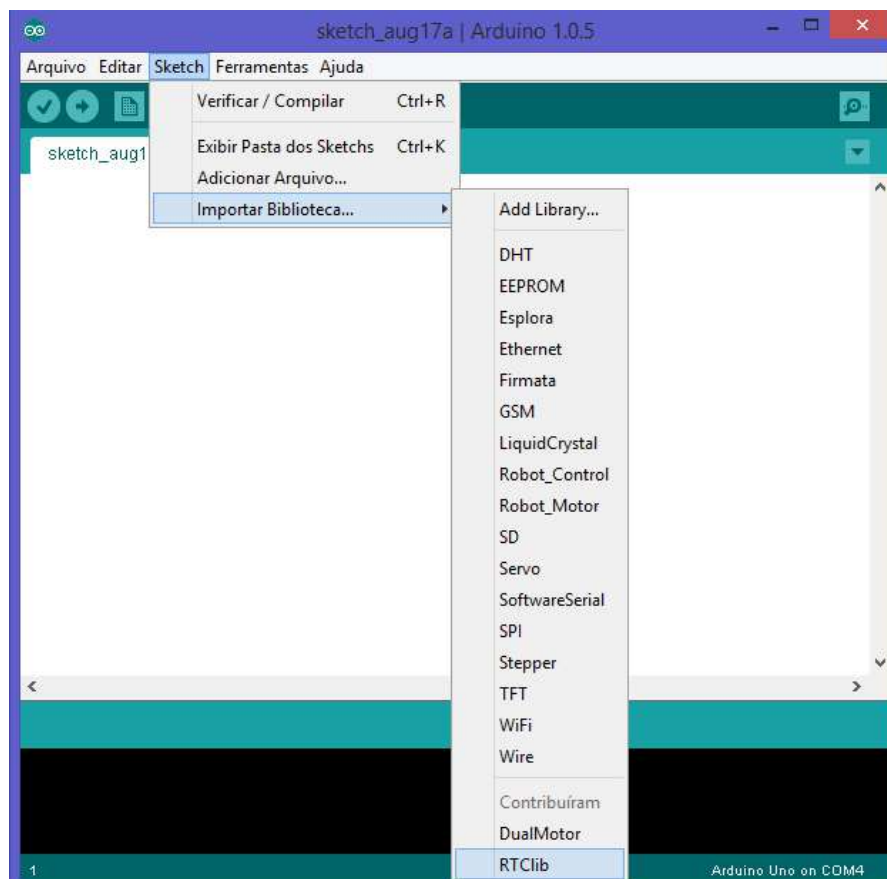


Figura 7.9 - Seleção da Biblioteca RTCLib

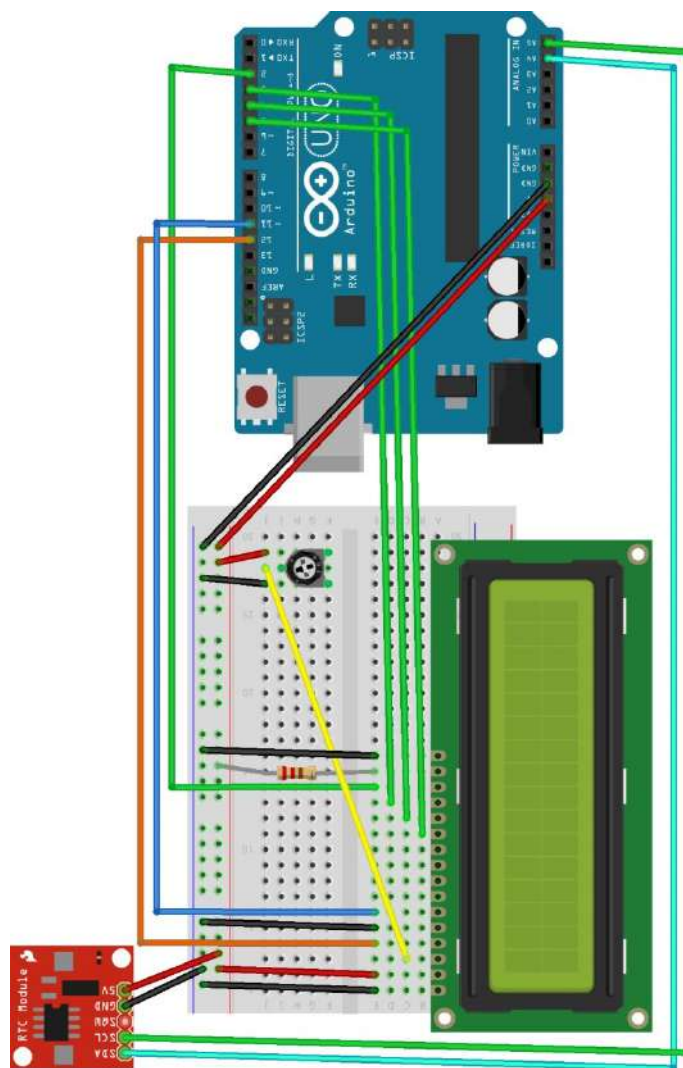
Passo 2: Montagem do circuito

Figura 7.10 - Ligação do Relógio em Tempo Real (RTC)

Com base na Figura 3.28, realizar esta sequência de montagem:

- Pino 1 do LCD ligado ao GND do Arduino.
- Pino 2 do LCD ligado ao 5V do Arduino.
- Pino 3 do LCD ligado ao pino central do potenciômetro (controle de contraste).
- Pino 4 do LCD ligado ao pino digital 12 do Arduino.
- Pino 5 do LCD ligado ao GND do Arduino.
- Pino 6 do LCD ligado ao pino digital 11 do Arduino.
- Pino 11 do LCD ligado ao pino digital 5 do Arduino.
- Pino 12 do LCD ligado ao pino digital 4 do Arduino.
- Pino 13 do LCD ligado ao pino digital 3 do Arduino.
- Pino 14 do LCD ligado ao pino digital 2 do Arduino.
- Pino 15 do LCD ligado ao 5V do Arduino com um resistor de 220 ohms (controle do brilho).
- Pino 16 do LCD ligado ao GND do Arduino.
- Pino SDA do RTC ligado ao pino analógico A4 do Arduino.
- Pino SCL do RTC ligado ao pino analógico A5 do Arduino.



do Arduino.

- Pino GND do RTC ligado ao pino GND do Arduino.
- Pino VCC do RTC ligado ao 5V do Arduino.

Passo 3: Programa

Inicie o ambiente de desenvolvimento do Arduino e digite o sketch (programa) a seguir:

```
#include <Wire.h>
#include <RTClib.h>
#include <LiquidCrystal.h>

#define TEMPO_ATUALIZACAO 1000

// RTC - Real Time Clock
// Conectar SCL (RTC) em A5 (Arduino) e
// SDA (RTC) em A4 (Arduino).

RTC_DS1307 RTC;

// LCD - Modelo ACM 1602K
// Pin Sigla Função                               Conectar
// 1  Vss   Ground                                GND
// 2  Vdd   +5V                                    VCC
// 3  Vo    LCD contrast adjust Potenciômetro
// 4  RS    Register select                      Arduino 12
// 5  R/W   Read/write                           GND
// 6  E     Enable                               Arduino 11
// 7  DB0   Data bit 0                           NC
// 8  DB1   Data bit 1                           NC
// 9  DB2   Data bit 2                           NC
// 10 DB3   Data bit 3                           NC
// 11 DB4   Data bit 4                           Arduino 5
// 12 DB5   Data bit 5                           Arduino 4
// 13 DB6   Data bit 6                           Arduino 3
// 14 DB7   Data bit 7                           Arduino 2
// +  BL+   Alimentação (+)                      Resistor de 1k
//                                     para VCC
// -  BL-   Alimentação (-)                      GND
```

```
LiquidCrystal lcd (12, 11, 5, 4, 3, 2);

int dia, mes, ano, hora, minuto, segundo,
dia_semana;
char semana[][4] = {"DOM", "SEG", "TER", "QUA",
"QUI", "SEX", "SAB"};

void setup () {
  // Inicializa comunicação serial
  Serial.begin(9600);

  // Inicializa o protocolo Wire
  Wire.begin();

  // Inicializa o módulo RTC
  RTC.begin();

  // Verifica se o modulo esta funcionando
  if (! RTC.isrunning()) {
    Serial.println("O RTC não está
executando!");
  }

  // Ajusta o relógio com a data e hora na qual
  // o programa foi compilado
  // RTC.adjust(DateTime(__DATE__, __TIME__));

  pinMode(12, OUTPUT);
  pinMode(11, OUTPUT);
  lcd.begin (16, 2);
}

void loop () {
  //Recupera data e hora atual
  DateTime now = RTC.now();
  dia = now.day();
  mes = now.month();
```



```
ano = now.year();
hora = now.hour();
minuto = now.minute();
segundo = now.second();
dia_semana = now.dayOfWeek();

lcd.clear();
if (dia < 10)
    lcd.print("0");
lcd.print(dia, DEC);
lcd.print("/");
if (mes < 10)
    lcd.print("0");
lcd.print(mes, DEC);
lcd.print("/");
lcd.print(now.year(), DEC);
lcd.setCursor(13, 0); // Coluna, Linha
lcd.print(semana[dia_semana]);
lcd.setCursor(0, 1); // Coluna, Linha
if (hora < 10)
    lcd.print("0");
lcd.print(hora, DEC);
lcd.print(":");
if (minuto < 10)
    lcd.print("0");
lcd.print(minuto, DEC);
lcd.print(":");
if (segundo < 10)
    lcd.print("0");
lcd.print(segundo, DEC);
delay(TEMPO_ATUALIZACAO);
}
```


CAPÍTULO 08

Projeto 12 – Display de Led de 7 Segmentos

O objetivo deste projeto é demonstrar a utilização de um display de led de 7 segmentos controlado diretamente a partir das portas digitais do Arduino.

Material necessário:

- 1 Arduino.
- 1 Display de Led de 7 Segmentos Cátodo ou Ânodo Comum (1 dígito)*.
- 1 Resistor de 220 ohms (vermelho, vermelho, marrom) ou 330 ohms (laranja, laranja, marrom)*.

marrom) ou 330 ohms (laranja, laranja, marrom)*.

- 1 Protoboard*.
- Jumper cable.
- Podem ser substituídos pelo módulo P11-Display Simples da GBK Robotics.

Passo 1: Displays de Led de 7 Segmentos

Os displays de led de sete segmentos (Figura 7.1) são bastante comuns e muitos utilizados para exibir, principalmente, informações numéricas. Podem ser de dois tipos:

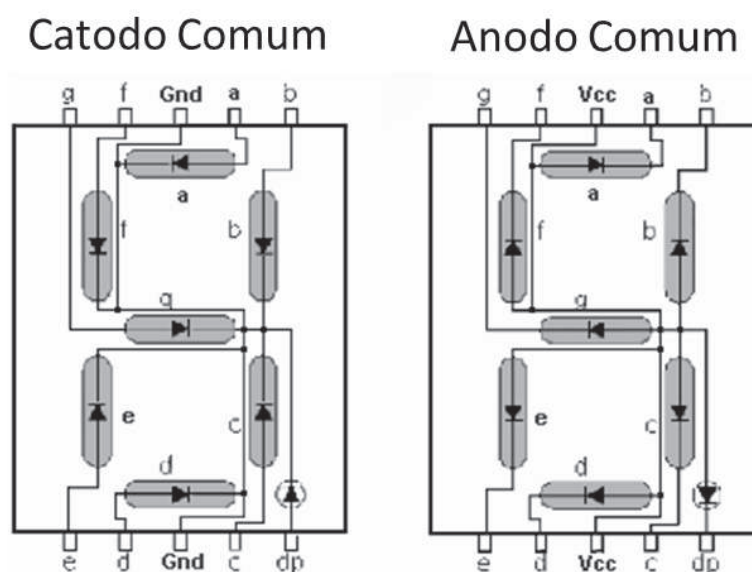


Figura 8.1 - Tipos de Displays de LEDs de 7 Segmentos



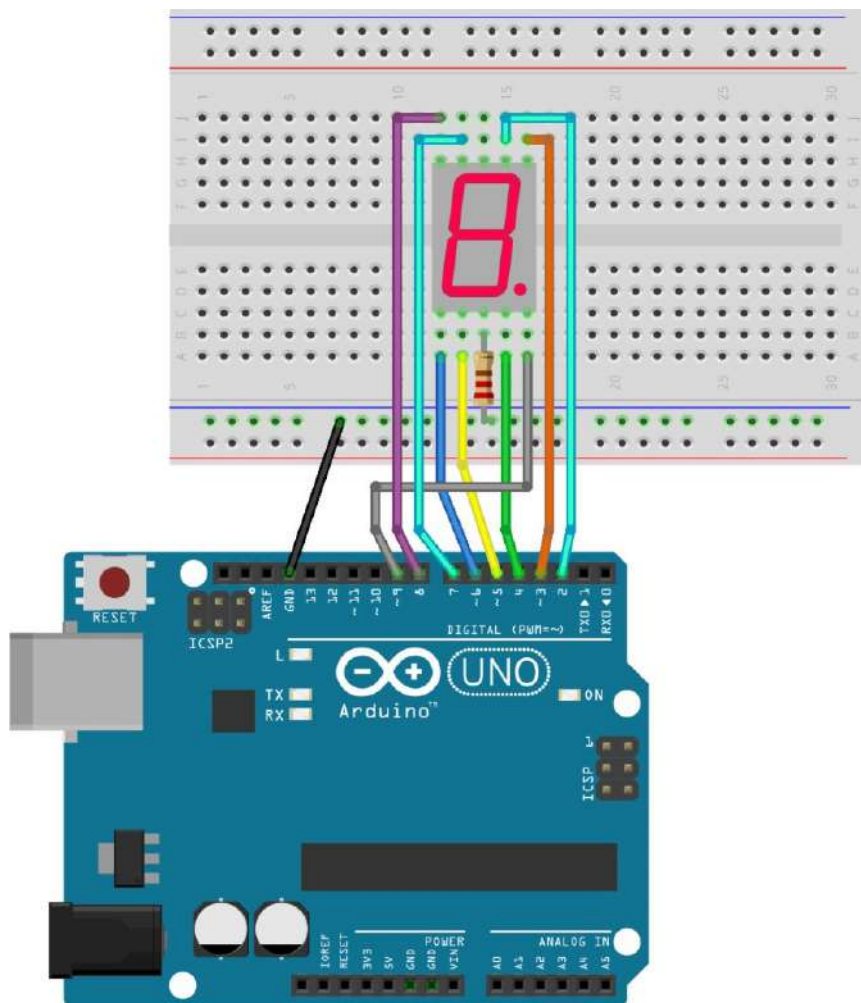


Figura 8.2 - Ligação do Display de 7 Segmentos

Adotando como referência a Figura 3.30, realizar a sequência de montagem:

- Pino 1 (segmento e) do display ligado ao 6 do Arduino.
- Pino 2 (segmento d) do display ligado ao 5 do Arduino.
- Pino 3 (Gnd, se Cátodo comum ou Vcc se Ânodo comum) do display ligado ao resistor de 220 ohms.
- Resistor de 220 ohms ligado ao Gnd (se Cátodo comum) ou Vcc (se Ânodo comum) do Arduino.
- Pino 4 (segmento c) do display ligado ao 4 do Arduino.
- Pino 5 (ponto decimal) do display ligado ao 9 do Arduino.
- Pino 6 (segmento b) do display ligado ao 3 do Arduino.
- Pino 7 (segmento a) do display ligado ao 2 do Arduino.
- Pino 9 (segmento f) do display ligado ao 7 do Arduino.
- Pino 10 (segmento g) do display ligado ao 8 do Arduino.

Passo 3: Programa N° 1 – Exibindo a Letra “A”

Inicie o ambiente de desenvolvimento do Arduino e digite o sketch (programa) a seguir:

```
int SEG_A = 2;
int SEG_B = 3;
int SEG_C = 4;
int SEG_D = 5;
int SEG_E = 6;
int SEG_F = 7;
int SEG_G = 8;
int PONTO = 9;
int ATRASO = 1000;

void setup() {
  for (int pino = SEG_A; pino <= PONTO; pino++)
  {
    pinMode(pino, OUTPUT);
    // Para displays de Cátodo comum:
    digitalWrite(pino, LOW);
    // Para displays de Ânodo comum:
    // digitalWrite(pino, HIGH);
  }
}

void loop() {
  // Para displays de Cátodo comum:
  digitalWrite(SEG_A, HIGH);
  digitalWrite(SEG_B, HIGH);
  digitalWrite(SEG_C, HIGH);
  digitalWrite(SEG_E, HIGH);
  digitalWrite(SEG_F, HIGH);
  digitalWrite(SEG_G, HIGH);

  // Para displays de Ânodo comum:
  // digitalWrite(SEG_A, LOW);
  // digitalWrite(SEG_B, LOW);
  // digitalWrite(SEG_C, LOW);
  // digitalWrite(SEG_E, LOW);
  // digitalWrite(SEG_F, LOW);
  // digitalWrite(SEG_G, LOW);

  delay(ATRASO);
}
```

Passo 4: Programa N° 2 – Animação Simples

Inicie o ambiente de desenvolvimento do Arduino e digite o sketch (programa) a seguir:




```

int SEG_A = 2;
int SEG_B = 3;
int SEG_C = 4;
int SEG_D = 5;
int SEG_E = 6;
int SEG_F = 7;
int SEG_G = 8;
int PONTO = 9;

int ATRASO = 150;

void setup() {
  for (int pino = SEG_A; pino <= PONTO; pino++)
  {
    pinMode(pino, OUTPUT);
    // Para displays de Cátodo comum:
    digitalWrite(pino, LOW);
    // Para displays de Ânodo comum:
    // digitalWrite(pino, HIGH);
  }
}

void loop() {
  for (int pino = SEG_A; pino < SEG_G; pino++) {
    // Para displays de Cátodo comum:
    digitalWrite(pino, HIGH);
    // Para displays de Ânodo comum:
    // digitalWrite(pino, LOW);
    if (pino > SEG_A) {
      // Para displays de Cátodo comum:
      digitalWrite(pino - 1, LOW);
      // Para displays de Ânodo comum:
      // digitalWrite(pino - 1, HIGH);
    }
    else {
      // Para displays de Cátodo comum:
      digitalWrite(SEG_F, LOW);
      // Para displays de Ânodo comum:
      // digitalWrite(SEG_F, HIGH);
    }
    delay(ATRASO);
  }
}

```

Passo 5: Programa N° 3 – Contagem Regressiva

Inicie o ambiente de desenvolvimento do Arduino e digite o sketch (programa) a seguir:



```

// Matriz com os dígitos de 0 a 9.
byte digitos[10][7] = {
  { 1,1,1,1,1,1,0 }, // = 0
  { 0,1,1,0,0,0,0 }, // = 1
  { 1,1,0,1,1,0,1 }, // = 2
  { 1,1,1,1,0,0,1 }, // = 3
  { 0,1,1,0,0,1,1 }, // = 4
  { 1,0,1,1,0,1,1 }, // = 5
  { 1,0,1,1,1,1,1 }, // = 6
  { 1,1,1,0,0,0,0 }, // = 7
  { 1,1,1,1,1,1,1 }, // = 8
  { 1,1,1,0,0,1,1 }  // = 9
};

void setup() {
  pinMode(2, OUTPUT);
  pinMode(3, OUTPUT);
  pinMode(4, OUTPUT);
  pinMode(5, OUTPUT);
  pinMode(6, OUTPUT);
  pinMode(7, OUTPUT);
  pinMode(8, OUTPUT);
  pinMode(9, OUTPUT);
  pontoDecimal(false);
}

void pontoDecimal(boolean ponto) {
  digitalWrite(9, ponto);
}

void escrever(int digito) {
  int pino = 2;
  for (int segmento = 0; segmento < 7;
segmento++) {
    // Para Cátodo Comum:
    digitalWrite(pino,
digitos[digito][segmento]);
    // Para Ânodo Comum - apenas inverter o
    // valor através do operador not (!):
    // digitalWrite(pino,
    // !digitos[digito][segmento]);
    pino++;
  }
  pontoDecimal(false);
}

void limpar() {
  byte pino = 2;

```



```

    for (int segmento = 0; segmento < 7;
segmento++) {
        // Para Cátodo Comum:
        digitalWrite(pino, LOW);
        // Para Ânodo Comum:
        // digitalWrite(pino, HIGH);
        pino++;
    }
}

void loop() {
    for (int cont = 9; cont >= 0; cont--) {
        escrever(cont);
        boolean ponto = true;
        for (int i = 0; i < 4; i++) {
            delay(250);
            pontoDecimal(ponto);
            ponto = !ponto;
        }
    }
    limpar();
    delay(1000);
}

```

Projeto 14 – Servo Motor

O objetivo destes projetos é mostrar o uso da biblioteca “Servo.h” (já existente na IDE Arduino) para a manipulação de servo motores. Um servo motor é um motor que tem sua rotação limitada através de uma angulação, podendo variar entre de 0º a 180º para indicar seu posicionamento. São muito utilizados em modelos de controle remoto, como em carrinhos, para girar o eixo de direção ou em barcos, para direcionamento do leme. Outras aplicações interessantes são em direcionamento automatizado de antenas e em articulações de braços robóticos.

A biblioteca “Servo.h” apresenta um conjunto de métodos, como o attach(), para definir qual é o pino está conectado, e o write(), para “escrever” o valor do ângulo

no motor. Para a utilização desses métodos da biblioteca é preciso criar um objeto do tipo “servo”, procedimento similar ao do experimento usando o display LCD e a biblioteca “LiquidCrystal.h”.

Material necessário:

- 1 Arduino.
- 1 Servo motor RC padrão.
- 1 Potenciômetro 10K (resistor variável).
- 1 Protoboard.
- Jumper cable.

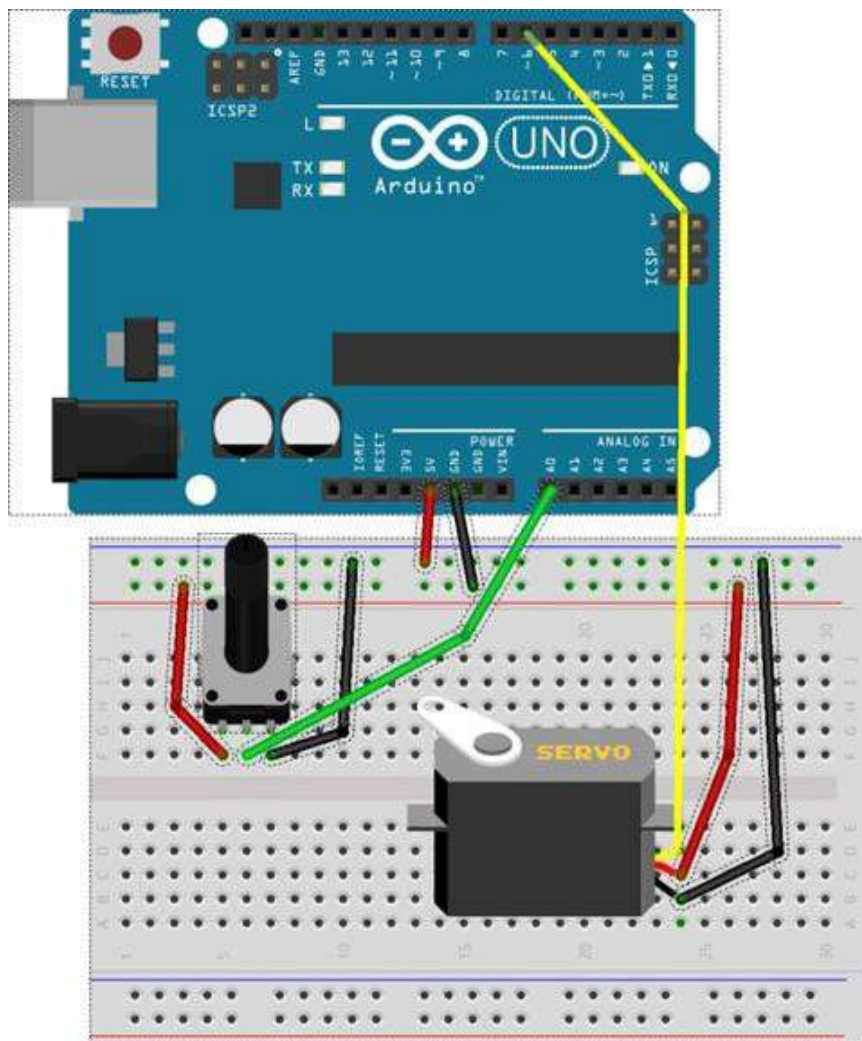


Figura 8.3 - Conexão do Servo Motor

Adotando, como referência, a Figura 3.34 realize as seguintes ligações:

- Cabo de alimentação GND do servo motor (cabo preto) na alimentação GND da protoboard.
- Cabo de alimentação de 5V do servo motor (vermelho ou outra cor, logo ao lado do preto) na alimentação 5V da protoboard.
- Cabo de sinal do servo motor (amarelo) no pino digital 6 do Arduino.
- Pino positivo do potenciômetro na alimentação 5V da protoboard.
- Pino negativo do potenciômetro na alimentação GND da protoboard.
- Pino do meio (sinal) do potenciômetro no pino analógico A0 do Arduino.

- Pino 5V do Arduino na trilha positiva da protoboard.

- Pino GND do Arduino na trilha negativa da protoboard.

Passo 2: Programa N° 1

Neste programa vamos utilizar o potenciômetro para variar a angulação do servo motor. Quanto maior o valor proveniente do sinal do potenciômetro maior o ângulo, e vice-versa. Como a variação do potenciômetro é de 0 a 1023 e o do servo motor é de 0 a 180, utilizamos a função `map()`, como o objetivo de converter o range (intervalo de variação) do potenciômetro com o do servo motor. A função `map()` é definida como:

`map(valor, deMin, deMax, paraMin, paraBaixo)`

Onde:

- **valor:** valor a ser convertido.



- **deMin:** valor mínimo do intervalo do valor.
 - **deMax:** valor máximo do intervalo do valor.
 - **paraMin:** valor mínimo do intervalo a ser convertido.
 - **paraMax:** valor máximo do intervalo a ser convertido.
- Basicamente a função `map()` faz uma “regra de 3”, para determinar o valor de um intervalo em outro intervalo. Em nosso programa usamos `map(valorPot, 0, 1023, 0, 180)`, e podemos entender como “o valor do potenciômetro que está no intervalo de 0 a 1023, será convertido em um valor equivalente no intervalo de 0 a 180”. Vamos implementar o sketch a seguir para demonstrar o uso do servo motor.

```
#include "Servo.h"

Servo servo;
int Pinpotenciometro = 0;
int PinservoMotor = 6;
int valorPot;
int valorMotor=0;

void setup() {
  servo.attach(PinservoMotor);
  Serial.begin(9600);
}

void loop() {
  valorPot = analogRead(Pinpotenciometro);
  valorMotor = map(valorPot, 0, 1023, 0, 180);
  servo.write (valorMotor);
  Serial.print(valorMotor);
  delay(20);
}
```

Passo 3: Programa N° 2

Mantenha a mesma montagem, apenas despreze o potenciômetro, já que não será usado. Nesse próximo programa, quando digitado a letra 'D' ou 'd' no Serial Monitor para fazer com que o servo motor diminua

sua angulação, de 15° em 15°. Caso digitado 'A' ou 'a' aumenta sua angulação. No Serial Monitor, digite a letra e aperte ENTER ou o botão “Enviar”. Acesse o ambiente de desenvolvimento do Arduino e digite o sketch (programa) a seguir para implementá-lo.

```

#include "Servo.h"

Servo servo;
int PinservoMotor = 6;
char tecla;
int valorMotor=0;

void setup() {
    servo.attach(PinservoMotor);
}

void loop() {
    tecla = Serial.read();
    if (tecla == 'D' || tecla == 'd') {
        valorMotor = valorMotor - 15;
        if (valorMotor >=180) {
            valorMotor = 180;
        }
    }
}
#include "Servo.h"

Servo servo;
int PinservoMotor = 6;
char tecla;
int valorMotor=0;

void setup() {
    servo.attach(PinservoMotor);
}

void loop() {
    tecla = Serial.read();
    if (tecla == 'D' || tecla == 'd') {
        valorMotor = valorMotor - 15;
        if (valorMotor >=180) {
            valorMotor = 180;
        }
    }
}

```

Projeto 15 – Sensor Óptico Reflexivo

Este projeto irá utilizar um Sensor Óptico Reflexivo TCRT5000 para implementar um interruptor de proximidade. Desta forma, não será necessário que a pessoa toque o sensor para acender ou apagar um Led. Esse tipo de circuito é muito útil quando desejamos manter a pessoa “isolada” do circuito elétrico evitando choques indesejáveis.

Um Sensor Óptico Reflexivo consiste em um diodo emissor (ou Led) de infravermelho, igual ao utilizado em controles remotos e um foto transistor que irá receber o sinal quando houver uma reflexão, ou seja, quando um obstáculo estiver à frente do sensor. No exemplo que iremos desenvolver vamos utilizar o modelo TCRT5000 (Figura 3.35), porém, o projeto pode ser facilmente alterado para utilizar outro modelo similar.



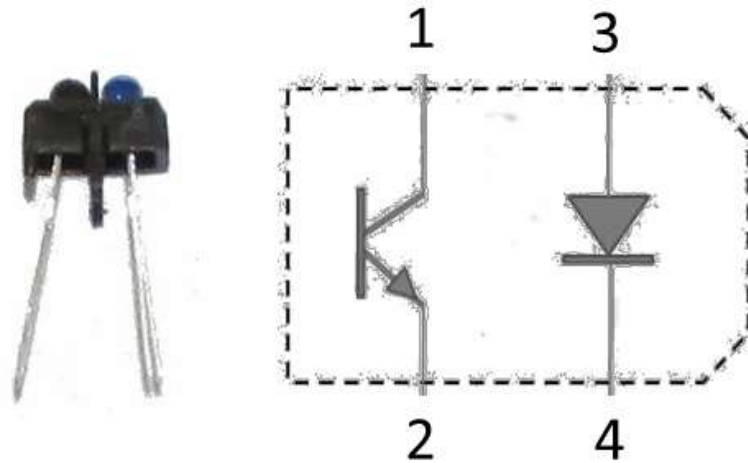


Figura 8.4 - Vista lateral e superior do Sensor Óptico Reflexivo TCRT5000

No quadro a seguir estão relacionados os pinos do sensor com as respectivas funções.

Pino	Nome	Função
1	Coletor (T+)	Coletor do fototransistor
2	Emissor (T-)	Emissor do fototransistor
3	Ânodo (D+)	Ânodo do led infravermelho
4	Cátodo (D-)	Cátodo do led intravermelho

Material necessário:

- 1 Arduino.
- 1 Sensor Óptico Reflexivo TCRT5000*.
- 1 Resistor de 220 ohms (vermelho, vermelho, marrom) ou de 330 ohms (laranja, laranja, marrom) para a ligação do LED.
- 1 Resistor de 220 ohms (vermelho, vermelho,

marrom) ou de 330 ohms (laranja, laranja, marrom) para ligação ao Sensor Óptico Reflexivo*.

- 1 Resistor de 10k ohms (marrom, preto, laranja)*.
- 1 Protoboard.
- Jumper cable.

* Podem ser substituídos pelo módulo P12-Sensor de Obstáculos da GBK Robotics.

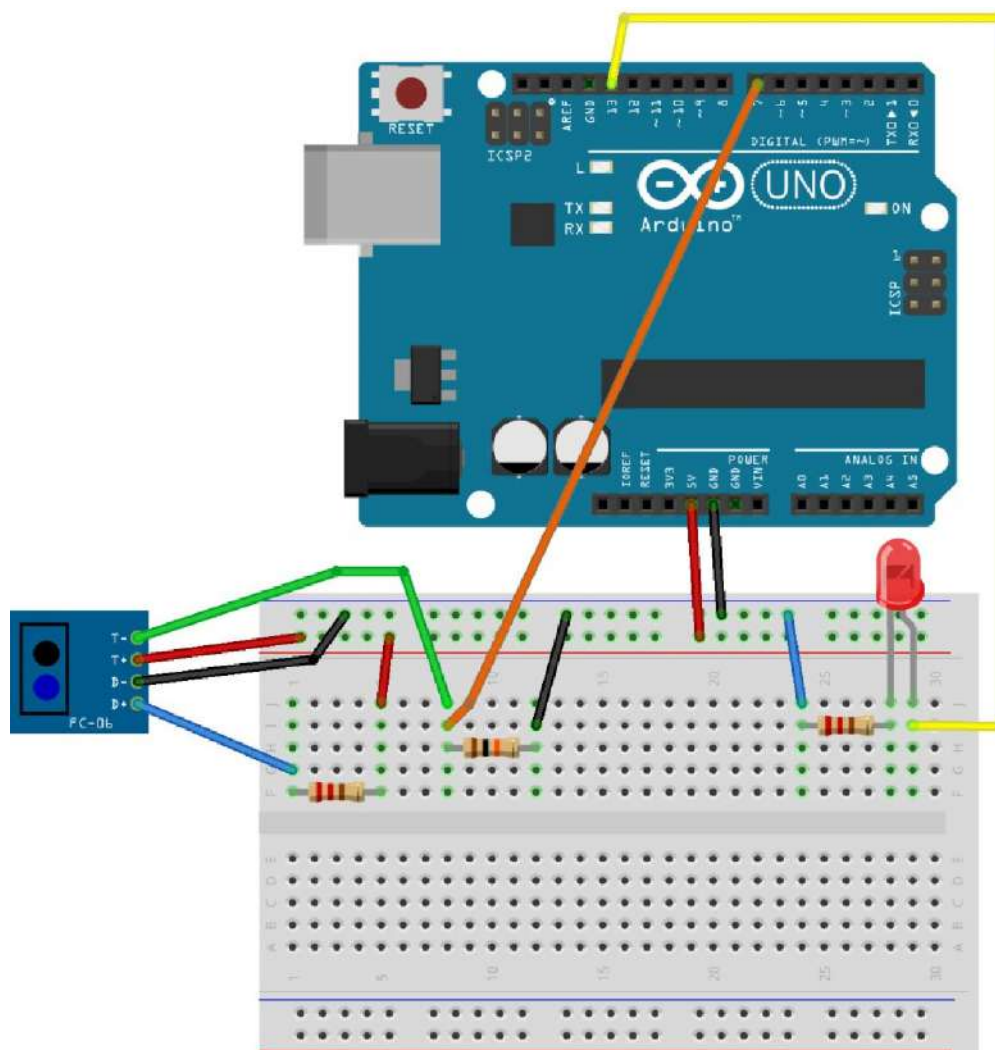


Figura 8.5 - Interruptor de proximidade

Adotando como referência a Figura 3.36 realize os seguintes passos para montar o hardware que será usado neste projeto:

- Inserir o sensor óptico reflexivo na protoboard.
- Conectar o pino 4 (D-) do sensor na linha de alimentação negativa (preta ou azul) da protoboard.
- Inserir um resistor de 220 ohms (ou 330 ohms) na protoboard e conecte um dos seus terminais no pino 3 (D+) do sensor.
- Conecte o outro terminal do resistor de 220 ohms (ou 330 ohms) na linha de alimentação positiva (vermelha) da protoboard.
- Insira o resistor de 10k ohms na protoboard e conecte um dos seus terminais ao pino 2 (T-) do sensor.
- Conecte o mesmo terminal do resistor de 10k ohms ao pino digital 7 do Arduino.
- Conecte o outro terminal do resistor de 10k ohms à linha de alimentação negativa (preta ou azul) da protoboard.
- Conecte o pino 1 (T+) do sensor a linha de alimentação positiva (vermelha) da protoboard.
- Insira o outro resistor de 220 ohms (ou 330 ohms) na protoboard e conecte um dos seus terminais na linha de alimentação negativa (preta ou azul) da protoboard.
- Insira na protoboard o led com o cátodo (lado chanfrado e que possui o terminal mais curto) conectado ao outro terminal do resistor de 220



ohms (ou 330 ohms).

alimentação positiva (vermelha) da protoboard.

- k. Conecte o ânodo do led ao pino digital 13 do Arduino.
- l. Conecte o pino 5 Volts do Arduino à linha de
- m. Conecte o pino Gnd do Arduino à linha de alimentação negativa (preta ou azul) da protoboard.



Figura 8.6 - Módulo P12-Sensor de Obstáculos

Este projeto pode ser montado substituindo o Sensor Óptico Reflexivo TCRT5000, um dos resistores de 220 ohms (ou 330 ohms) e o resistor 10k ohms pelo módulo P12- Sensor de Obstáculos (Figura 3.37), neste caso:

- a. Conecte o pino Gnd do Arduino à linha de alimentação negativa (preta ou azul) da protoboard.
- b. Conecte o pino 5+ do módulo P12 ao pino 5V do Arduino.
- c. Conecte o pino GND do módulo P12 à linha de alimentação negativa (preta ou azul) da protoboard.
- d. Ligue o pino 7 do Arduino ao pino A0 do módulo P12.
- e. Insira o resistor de 220 ohms (ou 330 ohms) na protoboard e conecte um dos seus terminais na linha de alimentação negativa (preta ou azul) da protoboard.
- f. Insira na protoboard o led com o cátodo (lado chanfrado e que possui o terminal mais curto) conectado ao outro terminal do resistor de 220 ohms (ou 330 ohms).
- g. Conecte o ânodo do led ao pino digital 13 do Arduino.

IMPORTANTE: Não há alterações no sketch (programa).

Programa N° 1: Obtendo o valor através da entrada digital

Após montar o circuito, entre no ambiente de desenvolvimento do Arduino e digite o sketch (programa) a seguir.

```
int LED = 13;
// Pino digital que irá receber o sinal do
// fototransistor
int SENSOR = 7;
int valor;

void setup() {
  pinMode(LED, OUTPUT);
  pinMode(SENSOR, INPUT);
}

void loop() {
  //Obter o valor do sensor, LOW ou HIGH
  valor = digitalRead(SENSOR);

  digitalWrite(LED, valor);
  delay (100);
}
```

Programa N° 2: Obtendo o valor através da entrada analógica

Entre no ambiente de desenvolvimento do Arduino e digite o sketch (programa) a seguir.

```
int LED = 13;
// Pino analógico que irá receber o sinal do
// fototransistor
int SENSOR = A0;

int valor;

void setup() {
  Serial.begin(9600);
  pinMode(LED, OUTPUT);
}

void loop() {
  // Obter o valor do sensor
  // Que consiste em um valor entre 0 e 1023
  valor = analogRead(SENSOR);

  Serial.print("Valor: ");
  Serial.println(valor);

  if (valor > 300)
    digitalWrite(LED, HIGH);
  else
    digitalWrite(LED, LOW);
  delay (500);
}
```



Neste exemplo, observe que precisamos definir um valor de limiar (300) para determinar se o LED será ou não aceso. Se necessário, ajuste esse valor para as condições do ambiente e também em relação à distância desejada.

Projeto 16 – Teclado com Divisor de Tensão

Neste projeto será aplicado o conceito de divisor de tensão para obter o valor de vários botões através de uma única porta analógica. A aplicação deste conceito permite “poupar” o uso de portas do Arduino em projetos que requerem a utilização de muitos botões.

Material necessário:

- 1 Arduino.
- 3 Interruptores táteis (push button).
- 3 Resistores de 10 kohms (marrom, preto, laranja).
- 1 Protoboard.
- Jumper cable.

Montagem do circuito

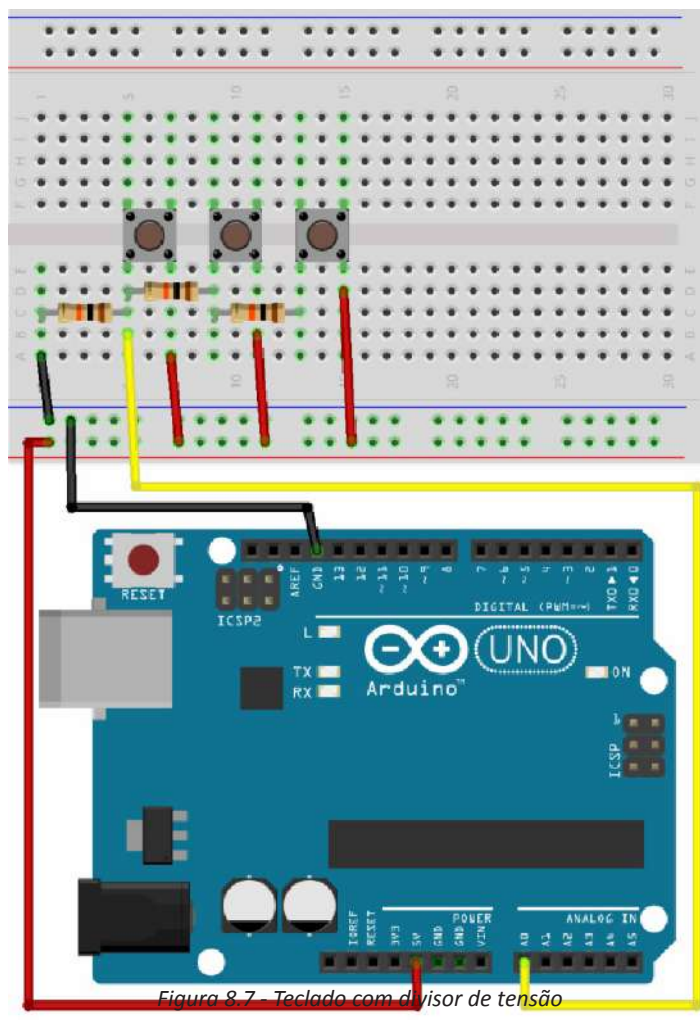


Figura 8.7 – Teclado com divisor de tensão

Adotando como referência a Figura 8.7 realize a montagem do circuito que será usado neste projeto.

Programa

No ambiente de desenvolvimento do Arduino e digite o sketch (programa) a seguir.

```

const int TECLADO = A0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  long valor = 0;

  for(int i = 0; i < 20; i++) {
    valor += analogRead(TECLADO);
  }
  valor /= 20;

  if (valor > 0) {
    Serial.print("Teclado = ");
    Serial.print(valor);
    if (valor > 1020)
      Serial.println(", Tecla 1");
    else if (valor > 505 && valor < 515)
      Serial.println(", Tecla 2");
    else if (valor > 335 && valor < 345)
      Serial.println(", Tecla 3");
    else
      Serial.println("");
  }
  delay(200);
}

```

Projeto 22 – Sensor de Presença

Neste projeto apresentamos o sensor de presença (Figura 8.8), este tipo de sensor utiliza infravermelho para

detectar algum movimento, no módulo pode-se ajustar a sensibilidade e o tempo que o sinal será enviado ao Arduino, ao se detectar algum movimento o sensor envia o sinal 1 (HIGH) para o Arduino.



Figura 8.8 - Sensor de Presença (PIR)

Material necessário:

- 1 Arduino.
- 1 Protoboard.
- Jumper cable.
- Resistores de 220 ohms à um 1k ohms para os LEDs.
- Leds (qualquer cor).
- 1 Modulo Sensor de Movimento Presença (PIR).

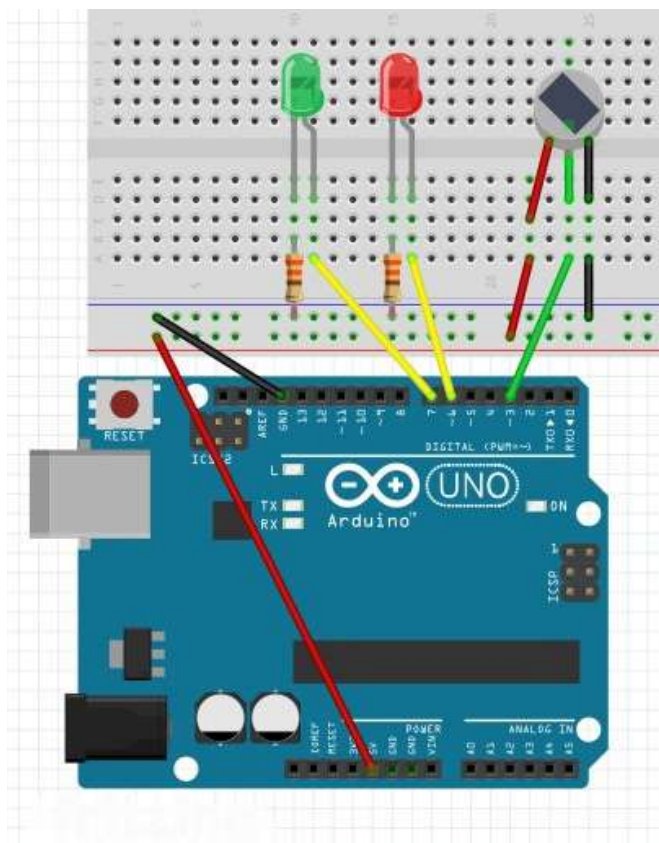


Figura 8.9 - Ligação do Sensor e Outros Componentes ao Arduino

Adotando como referência a Figura 3.52 realize a montagem do circuito que será usado neste projeto.

Programa

Implemente o programa a seguir no ambiente de desenvolvimento do Arduino.

```
int ledMovimento = 6;
int ledParado = 7;
int pinoPIR = 3; //Pino ligado ao sensor PIR
int acionamento;

void setup() {
  pinMode(ledMovimento, OUTPUT);
  pinMode(ledParado, OUTPUT);
  // Define pino onde o sensor foi ligado como
  // entrada
  pinMode(pinoPIR, INPUT);
  Serial.begin(9600);
}

void loop() {
  acionamento = digitalRead(pinoPIR);
  Serial.print("Acionamento: ");
  Serial.print(acionamento);
  if (acionamento == LOW) {
    // Sem movimento, LED verde é ligado
    digitalWrite(ledMovimento, LOW);
    digitalWrite(ledParado, HIGH);
    Serial.println(" - Sem Movimento");
  }
  else {
    // Em caso de movimento, ligar o LED
    // vermelho
    digitalWrite(ledMovimento, HIGH);
    digitalWrite(ledParado, LOW);
    Serial.println(" - Movimento Detectado");
  }
}
```

Desafio 7 – Sensor de Presença com Temporizador

Neste desafio utilize os conhecimentos adquiridos no Projeto 22 - Sensor de Presença, Projeto 21 - Utilizando Entradas com INPUT_PULLUP e Projeto 3 - LDR para criar um sensor de presença que acenda uma lâmpada de

uma área externa que será representando por um LED, onde este LED apenas acenderá quando estiver escuro e for detectado algum movimento ou for pressionado um botão. O LED deverá ficar acesso por 10 segundos antes de desligar.

